

IOC Intelligence Platform

Complete Documentation

Compiled 2026-08-09

Table of Contents

1. [README](#)
2. [Documentation Gaps & Known Limitations](#)
3. [Quickstart](#)
4. [User Guide](#)
5. [FAQ](#)
6. [Glossary](#)
7. [SOC Analyst Guide](#)
8. [Threat Intelligence Guide](#)
9. [IOC Types](#)
10. [Admin Guide](#)
11. [Configuration](#)
12. [Deployment](#)
13. [Developer Guide](#)
14. [API Documentation](#)
15. [Data Model](#)
16. [Architecture](#)
17. [Providers](#)
18. [AI Engine](#)
19. [Testing](#)
20. [Security](#)
21. [Troubleshooting](#)
22. [Changelog](#)

README

IOC Intelligence Platform

Enterprise threat-intelligence workbench. A single IOC search fans out to every configured intelligence provider in parallel, results are correlated and deduplicated, an AI backend summarizes each provider and produces a final evidence-based assessment (verdict, risk score, MITRE ATT&CK mappings, detection rules, recommended actions), and everything streams live into one SOC dashboard over Server-Sent Events.

The AI backend defaults to a locally-hosted Ollama server (`AI_BACKEND=ollama` , no API key, no cloud cost) and can be switched to AWS Bedrock, Google Gemini, or Anthropic's direct API via `.env` — see `backend/app/ai/service.py` .

What's actually built

- **One-search investigation pipeline.** `POST /api/v1/lookup/stream` detects the IOC type, fans it out to every provider that supports that type (concurrently, over a shared HTTP client, with Redis caching + retry/timeout handling), streams each provider's result and AI summary as they arrive, then runs a deterministic correlation pass and a final AI-generated assessment. See [docs/ARCHITECTURE.md](#) for the full request lifecycle.
- **15+ intelligence connectors**, one plugin interface: VirusTotal, AbuseIPDB, AlienVault OTX, URLhaus, ThreatFox, MalwareBazaar, crt.sh, NVD, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Hybrid Analysis, Spamhaus, PhishTank, Censys, plus an OSINT crawler-as-provider (GitHub, Reddit, RSS, Pastebin). Providers without a configured API key report `not_configured` rather than failing the lookup. Full detail in [docs/PROVIDERS.md](#).
- **Deterministic correlation + evidence ledger.** A pure, no-I/O correlation engine builds a relationship graph (IP resolutions, related hashes/URLs, certificates, ASN, malware families, threat actors, MITRE techniques, CVEs) and a factual evidence ledger that every AI-generated explanation must cite by ID — invented citations are stripped server-side. See [docs/DATA_MODEL.md](#) and [docs/THREAT_INTELLIGENCE_GUIDE.md](#). Note: despite an internal model docstring describing edges as "mirrored into Neo4j," Postgres is currently the sole store for the correlation graph — Neo4j and OpenSearch are provisioned in `docker-compose.yml` but have no code path calling into them today (**not implemented**).
- **On-demand AI analysis and hunting.** Evidence-grounded endpoints for "why is this malicious," "what is this," false-positive checks, verdict challenges, next actions, intelligence gaps, score explanations, a freeform copilot Q&A, and multi-format detection-rule generation (Sigma, Splunk SPL, Sentinel KQL, Elastic, QRadar AQL, Chronicle YARA-L, Suricata, Snort, Zeek). See [docs/AI_ENGINE.md](#) and [docs/API_DOCUMENTATION.md](#).
- **Auth/RBAC, basket, and case management.** JWT-based auth with three roles (`admin` , `analyst` , `viewer`); a per-analyst basket for scratch comparisons; shared, team-wide case management with notes and IOC attachment. Server-side PDF/CSV export, admin/user-management APIs, and MFA are **not implemented** — see [docs/DOCUMENTATION_GAPS.md](#) for the full, audited list of what's stubbed, partial, or missing.
- **Background OSINT refresh.** Celery beat runs an hourly job that re-crawls OSINT sources for recently-looked-up IOCs and warms the shared Redis provider cache — separate from, and not a substitute for, the interactive lookup path.

Quick start

```
# install Ollama (https://ollama.com) and pull a model that fits your GPU's
# VRAM, e.g.:
ollama pull llama3.2:3b

cp .env.example .env
# edit .env: add provider API keys you have, and set OLLAMA_MODEL to match
# whatever you pulled above (see `ollama list`)
docker compose up --build
```

Frontend: <http://localhost:3000> Backend API docs: <http://localhost:8000/docs>

No seed admin account exists — the **first** user you register (via the frontend `/register` page or `POST /api/v1/auth/register`) automatically becomes `admin` ; everyone after that defaults to `analyst` . For the full zero-to-first-lookup walkthrough (including the exact `curl` commands and what each dashboard panel does as it streams in), see [docs/QUICKSTART.md](#).

Documentation

Full index and known gaps: [docs/DOCUMENTATION_GAPS.md](#) is the honest counterpart to every guide below — read it alongside any of these to know what's real vs. aspirational.

New user

- [docs/QUICKSTART.md](#) — zero-to-first-lookup walkthrough.
- [docs/USER_GUIDE.md](#) — every dashboard panel and what it does.
- [docs/FAQ.md](#) / [docs/GLOSSARY.md](#) — common questions and terminology.

SOC analyst

- [docs/SOC_ANALYST_GUIDE.md](#) — day-to-day investigation SOP: triage, pivoting, basket, cases, hunting.

- [docs/THREAT_INTELLIGENCE_GUIDE.md](#) — evidence quality, confidence scoring, and trust calibration.
- [docs/IOC_TYPES.md](#) — every supported IOC type, detection behavior, and which providers accept it.

Administrator

- [docs/ADMIN_GUIDE.md](#) — role assignment, enabling providers, provider health checks.
- [docs/CONFIGURATION.md](#) — every `.env` / settings variable explained.
- [docs/DEPLOYMENT.md](#) — Docker Compose and Kubernetes deployment reference.

Developer

- [docs/DEVELOPER_GUIDE.md](#) — local dev setup, code layout, adding a provider or route.
- [docs/API_DOCUMENTATION.md](#) — full REST/SSE reference (canonical; supersedes the older `docs/API.md`).
- [docs/DATA_MODEL.md](#) — schema, ERD, and migration history.
- [docs/ARCHITECTURE.md](#) — system design, component and sequence diagrams.
- [docs/PROVIDERS.md](#) / [docs/AI_ENGINE.md](#) — connector-by-connector and AI-prompting detail.
- [docs/TESTING.md](#) — how to run the test suite and known local-run gotchas.

Security

- [docs/SECURITY.md](#) — auth/JWT model, RBAC, rate limiting, secrets handling, and known gaps to close before production.

Other references

- [docs/TROUBLESHOOTING.md](#) — symptom → cause → fix.
- [docs/CHANGELOG.md](#) — snapshot of what's implemented as of the current source tree.
- [docs/API.md](#) / [docs/INSTALL.md](#) — earlier drafts, kept for history; superseded by `API_DOCUMENTATION.md` / `QUICKSTART.md` + `DEPLOYMENT.md` respectively.

Scope of this build

This is a full vertical-slice scaffold:

- **Every module exists:** frontend, backend, plugin-based provider connectors, correlation engine, OSINT crawler, AI service, auth/RBAC, background workers, Postgres/Redis/Neo4j/OpenSearch, Docker Compose, K8s manifests, tests.
- **Real, working connectors** for free/no-key sources: VirusTotal (public API), AbuseIPDB, AlienVault OTX, URLHaus, ThreatFox, MalwareBazaar, crt.sh, NVD, CISA KEV, MITRE ATT&CK/CAPEC/CWE, WHOIS/RDAP.
- **Stubbed plugins** (implement the same `BaseProvider` interface) for Hybrid Analysis, Spamhaus, PhishTank, Censys. Spamhaus and PhishTank work out of the box with no key (`configured = True` unconditionally; PhishTank accepts an optional key for higher rate limits). Hybrid Analysis and Censys compute `configured` from `.env` — add `HYBRID_ANALYSIS_API_KEY`, or both `CENSYS_PERSONAL_ACCESS_TOKEN` and `CENSYS_ORGANIZATION_ID`, to activate them. Adding a new provider is a two-line change in `app/providers/registry.py`.
- **End-to-end depth** on IPv4/IPv6, Domain, URL, and file-hash IOC types (detection → parallel fetch → correlation → AI summaries → final verdict → dashboard). Other IOC types in the enum route through the same pipeline; they light up automatically as matching providers/plugins are added.
- **Known, documented gaps:** server-side PDF/CSV export, admin/user-management UI, MFA, password reset, and a Neo4j/OpenSearch graph/search mirror are all provisioned or scaffolded but **not implemented** — see [docs/DOCUMENTATION_GAPS.md](#) for the full audit.

Documentation Gaps & Known Limitations

Documentation Gaps & Known Limitations

This document is the honest counterpart to the rest of `docs/`. Every other guide in this set ([README.md](#), [USER_GUIDE.md](#), [SOC_ANALYST_GUIDE.md](#), [THREAT_INTELLIGENCE_GUIDE.md](#), [ADMIN_GUIDE.md](#), [DEVELOPER_GUIDE.md](#), [API_DOCUMENTATION.md](#), [PROVIDERS.md](#), [AI_ENGINE.md](#), [ARCHITECTURE.md](#)) describes what the platform **does**. This document describes what it **does not** do yet, what is only partially built, and what could not be verified from source code alone.

Everything below was checked directly against the backend/frontend source (routes, components, `lib/api.ts`, provider connectors, test suite) as of this audit. Anything not confirmed in source is labeled **NOT IMPLEMENTED**, **PARTIAL**, or **UNDETERMINED** rather than assumed.

1. Frontend pages/views that do not exist

The full frontend route tree is: `/`, `/login`, `/register`, `/lookup/new`, `/lookup/[id]`, `/basket`, `/cases`, `/cases/[id]`. Nothing else exists under `frontend/app/`.

Feature	Status	Notes
Threat Actor profile page	NOT IMPLEMENTED	Threat-actor data appears only as an evidence type (<code>threat_actor_association</code> in <code>EvidencePanel</code>) and as free-text fields inside a <code>FinalAssessment</code> . There is no page to browse/search actors across lookups.
Malware family profile page	NOT IMPLEMENTED	Same as above — malware family shows up as an evidence type (<code>malware_association</code>) and a basket-comparison column, not a dedicated browsable entity page.
Campaign view	NOT IMPLEMENTED	<code>campaign_association</code> exists only as an evidence type on a single lookup; no cross-lookup campaign aggregation page.
Infrastructure clustering	NOT IMPLEMENTED	<code>RelationshipGraph</code> renders the correlation graph (nodes/edges) for one lookup only , from <code>GET /lookup/{id}</code> correlation data produced during the SSE run. There is no engine that clusters infrastructure across multiple, unrelated lookups.
Watchlists	NOT IMPLEMENTED	No route, component, or API client function references a watchlist concept anywhere.
Bulk IOC analysis / upload	NOT IMPLEMENTED	The only lookup entry points are the single-value search box (<code>app/page.tsx</code>) and <code>/lookup/new?value=<one value></code> . No file-upload or multi-value submission UI or endpoint exists.
IOC graph diff / compare-over-time	NOT IMPLEMENTED (partial adjacent feature exists)	The Basket's "Compare Selected" (<code>compareBasketIOCs</code> , <code>frontend/app/basket/page.tsx</code>) compares multiple different IOCs' latest lookups side by side in a table. There is no feature that re-runs the same IOC over time and diffs the two results, and no lookup-history/versioning view.
Dedicated ATT&CK navigator page	NOT IMPLEMENTED (partial adjacent feature exists)	<code>MitreMatrix</code> renders ATT&CK tactic/technique columns, but only for the mappings attached to one lookup's <code>FinalAssessment</code> . There is no standalone/global ATT&CK matrix browsing page independent of a lookup.
Admin user-management UI	NOT IMPLEMENTED	No admin route exists in <code>frontend/app/</code> . The backend clearly has a permission system (<code>require_permission('case:read')</code> , <code>'case:create'</code> , <code>'case:write'</code> , <code>'case:close'</code> , <code>'basket:manage'</code> , <code>'lookup:read'</code> , <code>'lookup:create'</code> , <code>'analysis:generate'</code> , <code>'copilot:query'</code> , <code>'evidence:read'</code> , <code>'hunting:generate'</code> — all grepped from <code>backend/app/api/routes/*.py</code>), and <code>register/page.tsx</code> states "The first account created becomes an admin," implying roles exist server-side. There is no frontend surface to view/create/deactivate users or assign/revoke permissions.

2. Backend/API gaps

Feature	Status	Notes
PDF export	NOT IMPLEMENTED	<code>frontend/components/dashboard/ExportMenu.tsx</code> calls <code>POST /api/v1/lookup/{lookupId}/export?format=pdf</code> directly. No <code>export</code> route exists anywhere in <code>backend/app/api/routes/*.py</code> (confirmed by <code>grep</code>). The call always 404s; the UI catches this and shows "Export format not yet available."
CSV export	NOT IMPLEMENTED	Same missing route, <code>format=csv</code> . Only client-side Markdown and JSON export actually work (built entirely in-browser from the already-fetched <code>FinalAssessment</code> , no backend call).
ATT&CK ingestion beyond <code>mitre_attack.py</code>	NOT IMPLEMENTED	<code>backend/app/providers/mitre_attack.py</code> fetches the public STIX bundle and answers single <code>MITRE_TECHNIQUE</code> IOC lookups (with a 1-hour in-memory cache). There is no ingestion pipeline that loads the full ATT&CK framework into the database, no technique/tactic browsing API, and no scheduled sync job beyond that per-process cache.
Per-provider rate limiting / circuit breakers	NOT IMPLEMENTED	<code>ProviderStatus.RATE_LIMITED</code> exists and is reachable (HTTP 429/403/509 mapped in <code>base.py</code>), but no connector proactively self-throttles. The only place <code>core/cache.py</code> 's <code>RateLimiter</code> is actually instantiated is the per-user lookup-creation limiter in <code>backend/app/api/routes/lookup.py</code> (<code>lookup_create:{user.id}</code> , 10/60s default) — nothing per-provider, and no exponential-backoff/circuit-breaker logic inside individual connector files (all <code>retry/timeout</code> logic lives in <code>orchestrator.py</code>).
MFA (multi-factor authentication)	NOT IMPLEMENTED (<i>frontend-confirmed; backend not directly inspected for this audit — see §4</i>)	<code>frontend/app/login/page.tsx</code> is a plain email+password form. <code>frontend/lib/api.ts</code> exposes only <code>login</code> , <code>register</code> , and <code>refreshAccessToken</code> for auth — no MFA-related function, route, or UI element anywhere in the frontend.
Password reset / "forgot password"	NOT IMPLEMENTED (<i>same caveat as above</i>)	No forgot-password link, route, or API client function exists anywhere in <code>frontend/app/login/</code> or <code>frontend/lib/api.ts</code> .

3. Testing gaps

See [TESTING.md](#) for how to actually run what exists. Gaps:

Gap	Status	Notes
Frontend automated tests	NOT IMPLEMENTED	<code>frontend/package.json</code> wires up <code>"test": "vitest run"</code> and lists <code>vitest@2.1.1</code> as a dependency, but there are zero <code>*.test.*</code> / <code>*.spec.*</code> files anywhere under <code>frontend/app</code> , <code>frontend/components</code> , or <code>frontend/lib</code> , and no <code>vitest.config.*</code> . Running <code>npm test</code> executes vitest with nothing to collect.
Shared pytest fixtures (<code>conftest.py</code>)	NOT IMPLEMENTED	No <code>conftest.py</code> exists anywhere in the backend. <code>test_lookup_flow.py</code> and <code>test_lookup_stream_persistence.py</code> each independently reimplement very similar Postgres/Redis-override and connection-pool-disposal fixtures instead of sharing one.
Auth endpoint tests	NOT IMPLEMENTED	No test file exercises <code>POST /api/v1/auth/register</code> or <code>/login</code> as HTTP requests. Auth is only exercised indirectly, by constructing <code>User</code> rows and tokens directly inside <code>test_lookup_stream_persistence.py</code> 's fixtures.
Pytest config file	NOT IMPLEMENTED	No <code>pytest.ini</code> , <code>pyproject.toml</code> , <code>setup.cfg</code> , or <code>tox.ini</code> exists. Tests run with pytest defaults; inferred invocation (from <code>.pytest_cache</code> node IDs) is <code>cd backend && pytest</code> (or <code>pytest app/tests</code>), run from the <code>backend/</code> directory.

Known local test-run issues (not code bugs in the app itself)

- bcrypt/passlib version mismatch:** `backend/app/auth/security.py` builds a `passlib CryptContext(schemes=["bcrypt"])`. In an environment with `bcrypt>=4.1` (reproduced against `backend/.venv_test`, which has `bcrypt==5.0.0`), `passlib`'s `bcrypt` handler reads `bcrypt.__about__.__version__`, which no longer exists, and its self-test then hits `bcrypt 5.x`'s stricter 72-byte password check — both raise. The main `backend/.venv` pins `bcrypt==4.0.1` (matching `requirements.txt`) and does not exhibit this. **Two separate backend venvs exist (`.venv`, `.venv_test`) with drifted dependency versions**, and it is unclear from any repo doc/comment which one is meant to be the canonical test environment.
- "Event loop is closed" under `pytest-asyncio`:** documented directly in test fixture docstrings (`test_lookup_flow.py`, `test_lookup_stream_persistence.py`). Redis and Postgres connection pools are opened lazily and bound to whichever `asyncio` event loop was running at first use; since `pytest-asyncio` gives each test function its own loop, reusing a pool across tests raises `RuntimeError: Event loop is closed`. Both integration files work around this with an `autouse` fixture that disposes the pools after every test.
- Two integration tests require live infrastructure and self-skip otherwise: `test_lookup_flow.py` needs Redis at `localhost:6379` (`docker compose up -d redis`); `test_lookup_stream_persistence.py` needs Postgres at `localhost:5433` and Redis at `localhost:6379` (`docker compose up -d postgres redis`).

4. Features that exist but could be mistaken for something they're not

- "Ask AI" popup (`AskAiPanel.tsx`) is not the platform's AI engine.** It builds a prompt client-side (`frontend/lib/aiPrompt.ts`) and opens `https://gemini.google.com/app` in a popup for the analyst to paste into manually, because Google blocks iframing Gemini. This is a copy/paste convenience against the analyst's *own* Gemini account — there is no server-side Gemini call. The platform's actual AI features (WHY/Score Explanation/Copilot/etc., documented in [AI_ENGINE.md](#)) are separate and go through `backend/app/ai/analysis_service.py` and `hunting_service.py`.
- PivotPanel's "What should I do next?" and "What don't we know?" are manual, not automatic.** They only call `getNextActions()` / `getIntelligenceGaps()` on explicit button click, unlike the rest of the lookup page which streams in automatically over SSE. The Recommended Pivots list itself (`backend/app/evidence/pivot.py`) is a deterministic sort, not AI-generated.

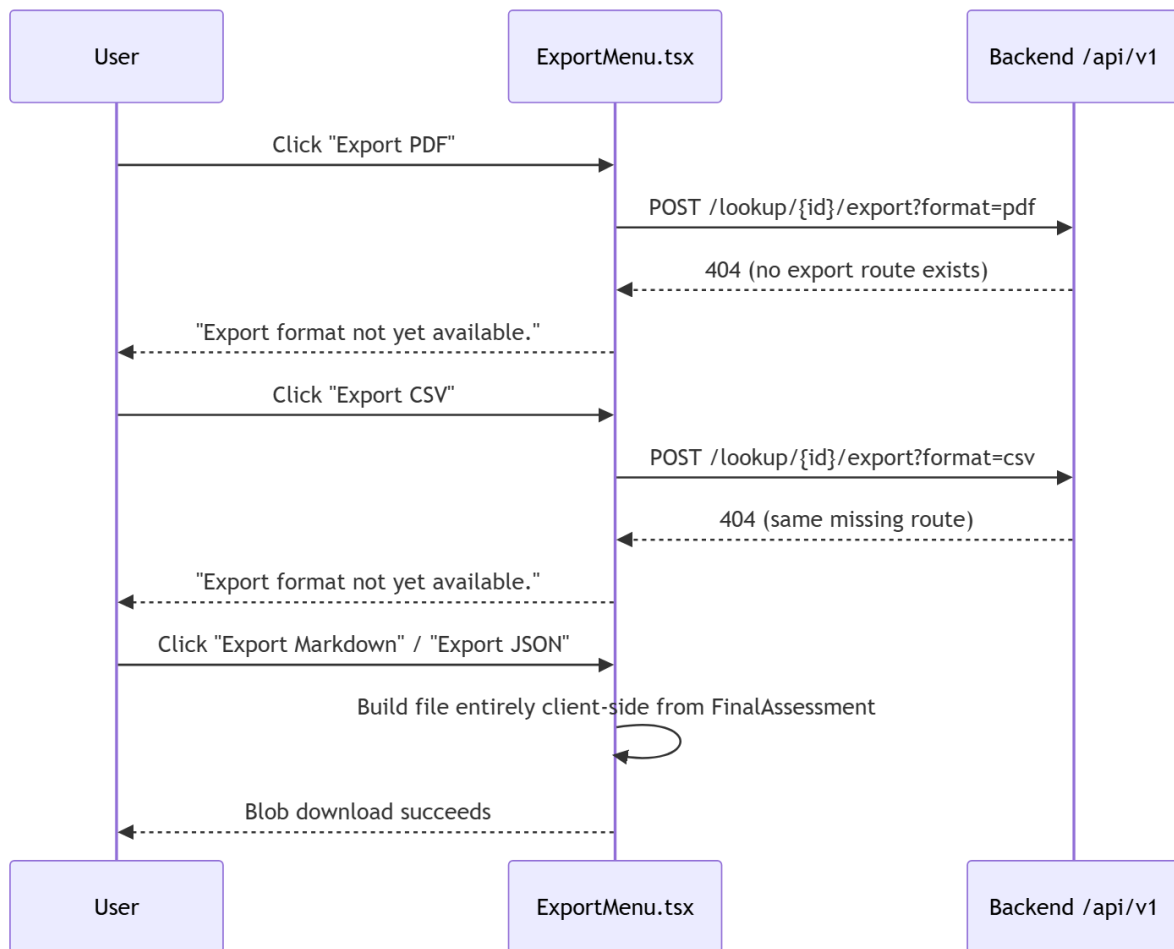
- `frontend/app/lookup/new/page.tsx` **carries a stale docstring**. The module comment describes itself as a "composition root" with placeholder components "until those land," but every real dashboard component (`ThreatScoreGauge` , `ProviderCardGrid` , `FinalAssessmentPanel` , `RelationshipGraph` , `MitreMatrix` , etc.) is already wired in. The comment predates the current code and should not be read as a to-do list.
- `frontend/app/lookup/[id]/page.tsx` **never shows a correlation graph on reload**. Its own docstring states `GET /api/v1/lookup/{id}` does not return correlation edges, so `correlation` is hardcoded to `null` on that page. The relationship graph is only populated during the live SSE run at `/lookup/new` , not when revisiting a completed lookup by ID.

5. Undeterminable / out of scope for a source-only audit

These could not be confirmed or denied from the code alone:

Item	Why it's undetermined
Whether currently configured provider API keys (VirusTotal, AbuseIPDB, OTX, abuse.ch, Censys, Hybrid Analysis, PhishTank) are free-tier or paid-tier	Each connector docstring only states the <i>free</i> -tier limits it was written against (e.g. VirusTotal "4 req/min, 500/day"); none of the connector code branches on a paid-vs-free flag. Actual account-level tier/pricing is external to the codebase and wasn't verified against any live provider account. See PROVIDERS.md .
External provider rate limits/pricing in general	Same reasoning — these are third-party services (VirusTotal, AbuseIPDB, OTX, URLhaus/ThreatFox/MalwareBazaar, Censys, Hybrid Analysis, PhishTank, NVD, CISA KEV, MITRE, crt.sh, Spamhaus). Only what each connector's docstring claims was checked; no external pricing pages were consulted.
Backend auth implementation details (MFA, password reset, session/token lifetime edge cases)	No dedicated research pass covered <code>backend/app/auth/</code> route/config internals for this document. The MFA/password-reset conclusions above are based on the absence of any corresponding frontend UI or API client call , not on a direct reading of the backend auth routes. Refer to SECURITY.md and API_DOCUMENTATION.md for what was verified there.
Whether <code>get_provider_health()</code> 's <code>configured</code> field updates without a process restart	Each provider computes <code>self.configured</code> once, from <code>get_settings()</code> , at module-import time (e.g. <code>virustotal_provider = VirusTotalProvider()</code> runs at import). Whether adding a key to <code>.env</code> after backend startup retroactively flips <code>configured</code> without a restart was not independently tested.
Why two backend virtualenvs (<code>.venv</code> , <code>.venv_test</code>) exist with drifted versions	No comment, README, or config file explains this split; <code>bcrypt</code> , <code>asynpcpg</code> , and <code>pytest</code> / <code>pytest-asyncio</code> versions differ between them and from <code>requirements.txt</code> 's pinned versions.
Whether <code>GET /api/v1/lookup</code> (list) is used anywhere in the frontend	<code>frontend/lib/api.ts</code> exports <code>listLookups()</code> , but no page read during this audit appears to call it — it may be unused frontend API surface, not confirmed either way.
Exact prompt content built by <code>frontend/lib/aiPrompt.ts</code>	Referenced by <code>AskAiPanel.tsx</code> but not read in full during this audit.

6. Why the export gap matters (concrete flow)



Practical guidance: until a server-side `/lookup/{id}/export` route is added, document PDF/CSV export as unavailable in any user-facing guide, and point analysts to the working Markdown/JSON export buttons instead.

7. Summary table

Area	Verdict
Threat Actor / Malware / Campaign dedicated pages	NOT IMPLEMENTED
Infrastructure clustering across lookups	NOT IMPLEMENTED
Watchlists	NOT IMPLEMENTED
Bulk IOC analysis/upload	NOT IMPLEMENTED
IOC diff/compare over time (same IOC, different runs)	NOT IMPLEMENTED (cross-IOC basket compare exists instead)
Dedicated/global ATT&CK navigator	NOT IMPLEMENTED (per-lookup MitreMatrix exists instead)
Admin user-management UI	NOT IMPLEMENTED (backend permission strings exist, no UI)
Server-side PDF export	NOT IMPLEMENTED
Server-side CSV export	NOT IMPLEMENTED
Full ATT&CK framework ingestion	NOT IMPLEMENTED (single-technique-lookup connector only)
Per-provider rate limiting/circuit breakers	NOT IMPLEMENTED
MFA	NOT IMPLEMENTED (frontend-confirmed)
Password reset	NOT IMPLEMENTED (frontend-confirmed)
Frontend automated tests	NOT IMPLEMENTED (vitest configured, unused)
Backend <code>conftest.py</code> / shared fixtures	NOT IMPLEMENTED
Backend auth endpoint tests	NOT IMPLEMENTED
Server-side Gemini/AI "second opinion" integration	NOT IMPLEMENTED (client-side popup workflow only)

Quickstart

Quickstart

Zero-to-working walkthrough: bring the stack up, register the first (admin) account, log in, and run one real IOC investigation.

For everyday analyst workflows once you're running, see [SOC_ANALYST_GUIDE.md](#) and [USER_GUIDE.md](#). This page only covers first boot.

Prerequisites

Requirement	Why
Docker Desktop (Compose v2)	Runs Postgres, Redis, Neo4j, OpenSearch, the FastAPI backend, Celery worker + beat, and the Next.js frontend — all defined in <code>docker-compose.yml</code> .
Ollama installed on the host (not in Docker) with a model pulled	Default AI backend (<code>AI_BACKEND=ollama</code>). The backend container reaches it at <code>http://host.docker.internal:11434</code> . Without it, provider/AI summaries degrade gracefully but the rest of the pipeline still works.
A terminal with <code>curl</code> (optional)	Used below to confirm health and to demonstrate the register API call.

Pull a small model that fits your GPU's VRAM before starting the stack:

```
ollama pull llama3.2:3b
```

1. Configure environment

```
cp .env.example .env
```

Edit `.env`:

- `JWT_SECRET_KEY` — replace the placeholder with a long random string (signs access/refresh tokens). Use `<configure securely>` as a reminder if you're not setting it yet.
- `OLLAMA_MODEL` — set to whatever you pulled (`ollama list` to check the exact tag). Leave `AI_BACKEND=ollama` and `OLLAMA_BASE_URL` at their defaults for local Docker Desktop use.
- Provider API keys (`VIRUSTOTAL_API_KEY`, `ABUSEIPDB_API_KEY`, `OTX_API_KEY`, `NVD_API_KEY`, `ABUSECH_AUTH_KEY`) are optional — leave blank and those providers report `not_configured`. See [PROVIDERS.md](#).

2. Start the stack

```
docker compose up --build
```

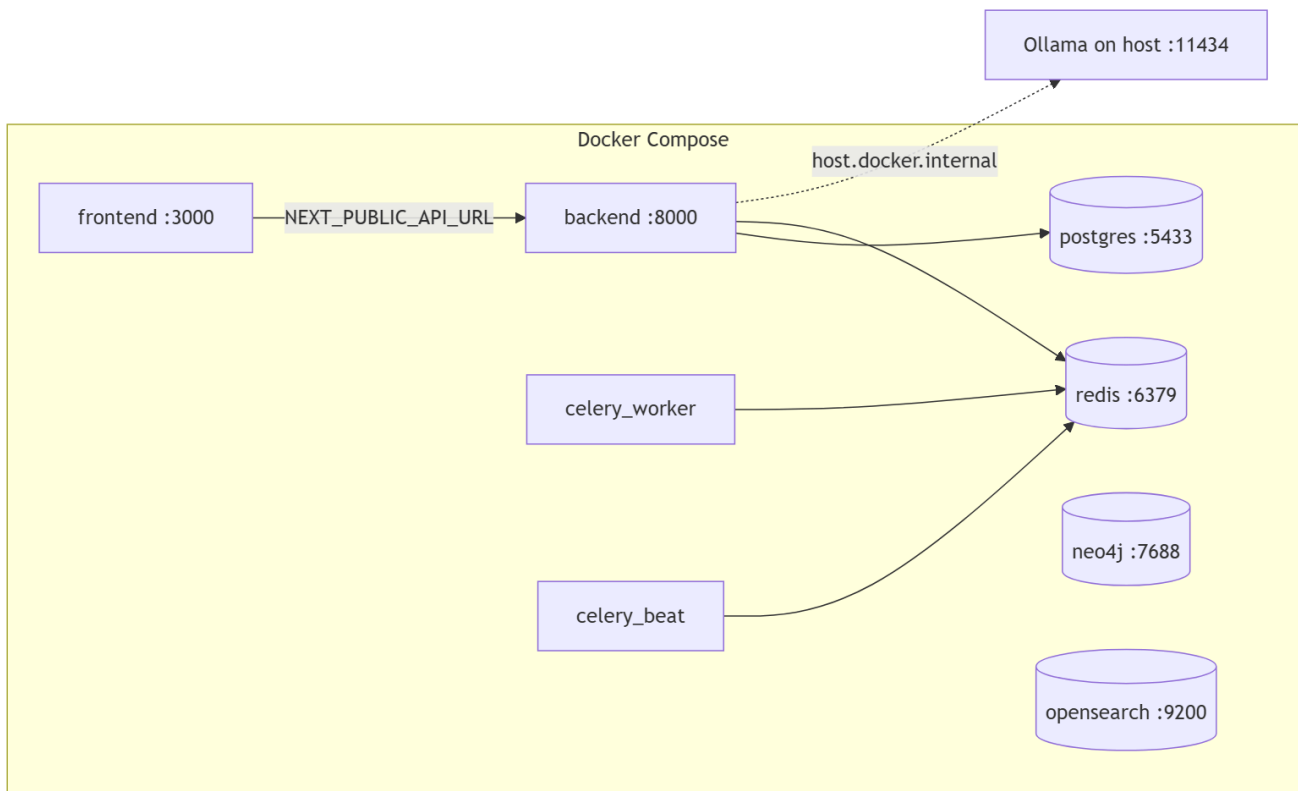
This starts: `postgres`, `redis`, `neo4j`, `opensearch`, `backend`, `celery_worker`, `celery_beat`, `frontend`.

The backend container's command runs migrations automatically before starting the API — there is no separate migration step:

```
sh -c "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload"
```

Wait for logs to settle, then confirm the backend is up:

```
curl http://localhost:8000/health
```



3. Open the app

<http://localhost:3000>

Also available:

URL	What
http://localhost:3000	Frontend (Next.js)
http://localhost:8000/docs	Backend interactive API docs (Swagger UI)
http://localhost:8000/health	Backend health check

4. Register the first account (becomes admin)

The platform ships with **no seed admin user**. Register via the frontend `/register` page, or via `curl`:

```
curl -X POST http://localhost:8000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"you@example.com","password":"<your-password>","full_name":"Your Name"}'
```

The **first user ever created** (i.e. the `users` table is empty at registration time) is automatically granted the `admin` role. Every registration after that defaults to `analyst`. This is enforced server-side in `backend/app/api/routes/auth.py` — the frontend register page shows the same rule: "The first account created becomes an admin."

Password rule: the frontend enforces an 8-character minimum client-side only — the backend itself does not currently validate password length or strength. See [SECURITY.md](#).

There is no email verification and no password-reset flow in this build — choose a password you'll remember.

5. Log in

Use the frontend `/login` page (email + password), or:

```
curl -X POST http://localhost:8000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"you@example.com","password":"<your-password>"}'
```

Returns { "access_token", "refresh_token", "token_type": "bearer" }. The frontend stores both tokens in localStorage (access_token, refresh_token) and refreshes the access token transparently on a 401. Access tokens expire in 30 minutes by default (ACCESS_TOKEN_EXPIRE_MINUTES); refresh tokens last 7 days.

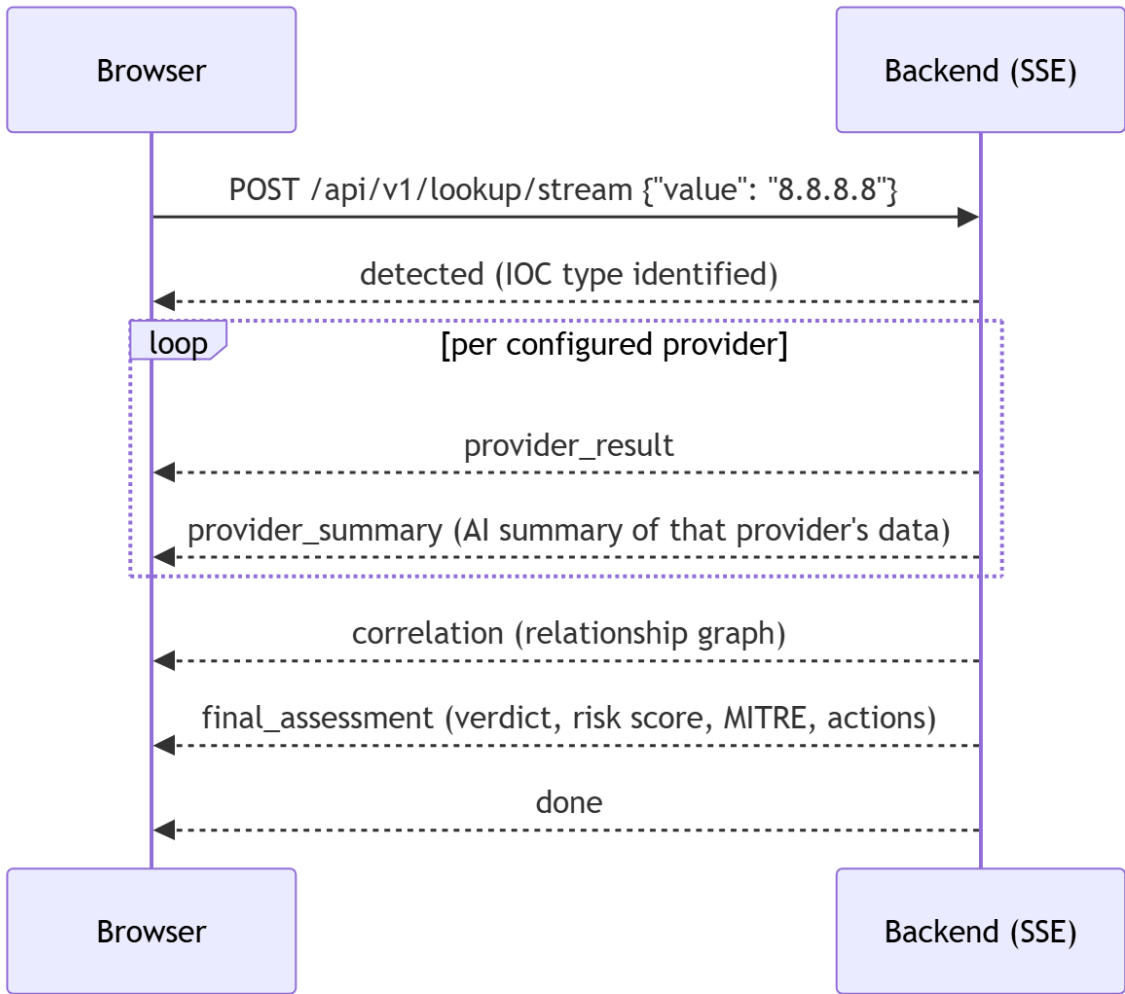
6. Run one investigation

On the home page, search box placeholder reads "IP, domain, URL, hash, CVE, threat actor, YARA rule..." with example chips you can click: 8.8.8.8, malicious-example.com, CVE-2024-3400, T1059.

Use a safe, well-known value for this first run:

8.8.8.8

Submit it. If you're not logged in, you'll be bounced to /login first and returned to the lookup afterward. Once submitted, the frontend opens a streaming connection (POST /api/v1/lookup/stream) and the page fills in live as results arrive, in this exact order:



7. Reading the results screen (high level)

The lookup page has a main column and a sidebar. As events stream in:

Area	Fills in when	Shows
Threat score gauge	<code>final_assessment</code>	Overall risk score (0–100), colored by severity.
Provider card grid	each <code>provider_result</code> / <code>provider_summary</code>	One card per provider (VirusTotal, AbuseIPDB, OTX, etc.), raw data plus an AI-written summary per provider.
Relationship graph	<code>correlation</code>	Force-directed graph of this IOC and everything it correlates with.
MITRE matrix	<code>final_assessment</code>	ATT&CK techniques implicated, grouped by tactic.
Detection rules / Recommended actions	<code>final_assessment</code>	Sigma/YARA/Splunk/etc. rule drafts and suggested next steps.
Sidebar: Provider progress tracker	streaming	Live count of providers responded vs. expected.
Sidebar: Export menu, Add to Basket / Case	always	Investigation actions — see USER_GUIDE.md .

Once the stream reaches `done`, additional panels unlock: verdict explanation ("WHY?", "What is this?", false-positive check, etc.), the evidence panel ("Show Receipts"), pivots, the hunting center, and an in-page AI copilot you can ask follow-up questions.

Full detail on every panel, what each button does, and how to interpret a verdict is in [USER_GUIDE.md](#). Analyst workflows (triage, pivoting, cases, basket) are in [SOC_ANALYST_GUIDE.md](#).

Next steps

- [USER_GUIDE.md](#) — full walkthrough of every dashboard panel.
- [SOC_ANALYST_GUIDE.md](#) — day-to-day analyst workflows (basket, cases, hunting, pivoting).
- [PROVIDERS.md](#) — which intelligence providers are wired up, which need API keys, which are stubs.
- [CONFIGURATION.md](#) — every `.env` variable explained.
- [TROUBLESHOOTING.md](#) — Ollama not responding, providers stuck `not_configured`, 401/403s, etc.

User Guide

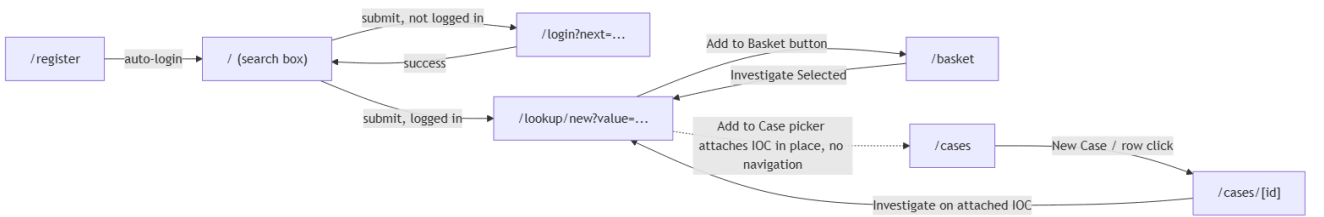
User Guide

This guide walks through every real page and dashboard component in the IOC Intelligence Platform frontend (`frontend/app/`), what each button does, and which backend API call or SSE event feeds each piece of data. It reflects the actual shipped UI only. For role/permission details see [ADMIN_GUIDE.md](#); for the underlying REST/SSE contract see [API.md](#); for what each dashboard panel's AI content is grounded in see [AI_ENGINE.md](#).

There are exactly eight routes in the app. Nothing else exists:

Route	Purpose
<code>/</code>	Home search box
<code>/login</code>	Sign in
<code>/register</code>	Create account
<code>/lookup/new?value=...</code>	Live SSE investigation view
<code>/lookup/[id]</code>	Completed-lookup detail view (no live stream)
<code>/basket</code>	IOC Basket (per-analyst scratch space)
<code>/cases</code>	Case list + create
<code>/cases/[id]</code>	Case detail

There is **no** threat-actor page, malware page, campaign page, watchlist page, or bulk-upload page. If you're looking for one of those, see the "Not implemented / planned" section at the bottom of this guide, and [CHANGELOG.md](#) / [DOCUMENTATION_GAPS.md](#) for status.



1. Sign up and sign in

`/register`

Form fields: full name (optional), email, password (`minLength=8`). Submitting calls `register()` then `login()` then routes to `/`. On-screen copy states: **"The first account created becomes an admin."** This matches the backend: `POST /api/v1/auth/register` makes the very first ever registered user `role: admin`; every subsequent registration defaults to `role: analyst` (see [ADMIN_GUIDE.md](#)).

`/login`

Email + password form. Calls `login(email, password)` against `POST /api/v1/auth/login`, stores `access_token` / `refresh_token` in `localStorage`, and redirects to the `next` query param (defaults to `/`).

Session handling notes:

- Access tokens expire (backend `ACCESS_TOKEN_EXPIRE_MINUTES`, default 30 minutes). The frontend's `authedFetch()` helper transparently retries once via `POST /api/v1/auth/refresh` on a 401 before giving up.
- `isLoggedIn()` / `logout()` are pure `localStorage` checks — no network call is made just to check login state.

2. Home page (`/`)

A single search box. Placeholder text: *"IP, domain, URL, hash, CVE, threat actor, YARA rule..."*. Below it are four example chips you can click to fill the box: `8.8.8.8`, `malicious-example.com`, `CVE-2024-3400`, `T1059`.

Note: the placeholder text advertises "threat actor" and "YARA rule" as example input types, but the pipeline's real depth (detection → parallel provider fetch → correlation → AI summaries → verdict) is documented as end-to-end only for IPv4/IPv6, domain, URL, and file-hash IOC types. Other `Ioctype`

enum values (including threat actor names and YARA rules as literal search input) route through the same pipeline but light up only as matching providers exist — see [IOC_TYPES.md](#) and [PROVIDERS.md](#) for current provider coverage.

Submitting the form:

- If not logged in, redirects to `/login?next=/lookup/new?value=<value>`.
- If logged in, navigates straight to `/lookup/new?value=<value>`.

Top-right corner shows **Sign in** / **Register** (logged out) or **Sign out** (logged in), driven by `isLoggedIn()` / `logout()`.

3. Live investigation (`/lookup/new?value=...`)

This is the main SOC workbench screen. On mount it:

1. Reads the `?value=` query param (redirects to `/login?next=...` first if you aren't logged in).
2. Opens a streaming POST to `/api/v1/lookup/stream` via `streamLookup()` (a manual `fetch` + stream reader, not `EventSource`, specifically because `EventSource` cannot send an `Authorization` header and this endpoint requires a bearer token).
3. Renders panels incrementally as SSE events arrive.

The page key is set to the raw `?value=` param, so pivoting to a new IOC (e.g. via the top search bar) forces a full remount instead of reusing stale state from the previous IOC.

SSE event sequence

Per the backend docstring, the exact order is:

```
detected → N × provider_result → N × provider_summary → correlation → final_assessment → done
```

(or an `error` event on failure, at any point). Any event name other than these seven is silently ignored by the frontend.

SSE event	Frontend handler	Drives
<code>detected</code>	<code>onDetected</code>	IOC type/value confirmation
<code>provider_result</code>	<code>onProviderResult</code>	<code>ProviderCardGrid</code> , <code>ProviderProgressTracker</code>
<code>provider_summary</code>	<code>onProviderSummary</code>	AI summary section inside each <code>ProviderCard</code>
<code>correlation</code>	<code>onCorrelation</code>	<code>RelationshipGraph</code>
<code>final_assessment</code>	<code>onFinalAssessment</code>	<code>ThreatScoreGauge</code> , <code>FinalAssessmentPanel</code> , <code>MitreMatrix</code> , <code>DetectionRulesPanel</code> , <code>RecommendedActionsPanel</code>
<code>done</code>	<code>onDone</code>	unlocks the "post-completion" cluster (below)
<code>error</code>	<code>onError</code>	error banner

Provider rows and correlation edges are persisted to Postgres incrementally during the stream, not just at the very end.

Layout

Main column (always mounted, panels self-render loading/empty states until their data arrives):

- `ThreatScoreGauge`
- `ProviderCardGrid`
- `FinalAssessmentPanel`
- `RelationshipGraph`
- `MitreMatrix`
- `DetectionRulesPanel`
- `RecommendedActionsPanel`

Once the stream reaches `done` (`isDone`), four more panels mount:

- `VerdictAnalysisPanel`
- `EvidencePanel`
- `PivotPanel`
- `HuntingCenterPanel`
- `InvestigationCopilot`

Sidebar:

- `ProviderProgressTracker`

- `ExportMenu`
- `InvestigationActions`
- `AskAiPanel`
- A debug **"Event log"** card showing the raw SSE events as they arrive (useful for troubleshooting a stuck lookup).

Dashboard components, in detail

ThreatScoreGauge

Plain-SVG semicircular gauge for `final_assessment.risk.overall_risk_score`. Shows a pulsing skeleton with caption **"Awaiting final assessment..."** until the `final_assessment` SSE event arrives (it's always the last substantive event before `done`).

ProviderCardGrid / ProviderCard

Grid of one card per provider, `ok`-status results sorted before non-`ok` ones. Each card generically renders the provider's raw `data` fields (primitives inline, small arrays as pills, larger nested objects collapsed into a "Raw data" `<details>` JSON dump). A special-cased renderer shows OSINT search hits (title/url/snippet/source/published date) for the `internet_intelligence` provider's `osint_findings` field. Below the raw data, an AI summary section (what it knows, reputation, confidence, interesting findings, relationships, unique observations) renders once that provider's `provider_summary` SSE event arrives; shows "Generating AI summary..." while waiting, or "No AI summary" if the provider's status wasn't `ok`.

ProviderProgressTracker (sidebar)

Purely derived from the `provider_result` events accumulated so far — it does not fetch anything itself. Because the SSE stream doesn't announce the full provider roster up front, the gap between how many results have arrived and the expected total is rendered as anonymous spinning "Provider N" rows until real names arrive. Category icons: threat intel, sandbox, passive DNS, certificate intel, WHOIS, vulnerability, OSINT.

RelationshipGraph

Renders the `correlation` event's `{nodes, edges}` payload as a force-graph (client-side only, via `react-force-graph-2d`). Node/edge colors follow the IOC type family (network/infra, hash/file, attribution, vuln/technique, host artifact, other) using the app's own theme colors. Includes a **"View as list"** toggle that renders the same graph as an accessible HTML table for keyboard/screen-reader use.

FinalAssessmentPanel

Five tabs: **Executive Summary, Technical Summary, Threat Assessment, Relationships, Risk & Verdict**.

- Executive tab: `executive_summary`, verdict badge, `verdict_rationale`.
- Threat tab: `threat_assessment`, agreeing/disagreeing provider pills, numbered `supporting_evidence` list.
- Risk tab: overall risk score, confidence score, severity, reputation, malicious probability, analyst confidence. Shows a skeleton placeholder until `final_assessment` arrives.

MitreMatrix

Groups `final_assessment.mitre_mappings` by tactic into columns. Each technique cell links out to `attack.mitre.org`. Mappings the AI inferred (rather than a provider explicitly asserting) are marked with an **"AI-inferred"** badge; hovering shows the rationale in a tooltip.

DetectionRulesPanel

Tabbed view of `final_assessment.detection_rules`, grouped by rule format (preferred tab order: Sigma, YARA, Splunk SPL, Sentinel KQL, Elastic, QRadar AQL, Suricata, Snort, Zeek). Each rule has a **Copy** button. Shows "No detection logic generated for this IOC type." if empty.

RecommendedActionsPanel

Three columns: **Recommended Actions, Investigation Priorities, Incident Response** — sourced directly from `final_assessment.recommended_actions` / `.investigation_priorities` / `.incident_response_recommendations`.

VerdictAnalysisPanel (post- `done`)

Six on-demand modes, each a button, fetched only when clicked (not automatic) and cached per-session once fetched:

Button label	Backend call
WHY?	<code>POST .../analysis/why</code>
What is this?	<code>POST .../analysis/what-is-this</code>
Score Explanation	<code>POST .../analysis/score-explanation</code>
Intelligence Conflicts	<code>POST .../analysis/disagreement</code>
False Positive Check	<code>POST .../analysis/false-positive</code>
Challenge This Verdict	<code>POST .../analysis/challenge</code>

Every result includes a "Show receipts" link wired to `EvidencePanel`.

EvidencePanel (post- done) — "Show Receipts"

Fetches `GET .../analysis/evidence`. This is **deterministic, non-AI** evidence built server-side from provider summaries and correlation edges — never from an AI call. Evidence rows are categorized as: detection, reputation, relationship, malware association, threat actor association, campaign association, MITRE technique, infrastructure, other. Supports filtering by type, and can auto-expand/scroll to specific rows when a "Show receipts (N)" link elsewhere on the page is clicked.

PivotPanel (post- done)

Three sub-sections:

- **Recommended Pivots** — a pure deterministic sort over real correlation edges (`GET .../pivots`), never AI-generated. Rows show IOC value/type, relationship, number of corroborating providers, confidence %, and a relevance badge (high/medium/low). Clicking a row navigates to `/lookup/new?value=<that IOC>`.
- **"What should I do next?"** — an explicit **"Ask AI"** button, not automatic. Calls `POST .../analysis/next-actions`.
- **"What don't we know?"** — also gated behind an explicit **"Ask AI"** button. Calls `POST .../analysis/gaps`.

HuntingCenterPanel (post- done)

- **"Hunt This IOC"** button calls `POST .../hunt` (formats: sigma, splunk_spl, sentinel_kql, elastic, qradar_aql, chronicle_yara_l, suricata, snort, zeek). Renders tabbed exact-match queries plus expansion targets and broader queries. Expansion targets are explicitly "grounded, never invented" per the component's own documentation.
- **"Create Detection"** section: choose a format from a dropdown (same list plus YARA) and click to call `POST .../detection?format=...`, rendering title, rule body, objective, data source, logic explanation, false-positive considerations, severity, and MITRE technique IDs.

InvestigationCopilot (post- done)

Free-form Q&A against everything this lookup already knows — calls `POST .../analysis/copilot` with `{question, notes}` where `notes` is the prior Q/A pairs from the same session (so context carries across turns). Suggested starter questions: "Why is this suspicious?", "What changed since last week?", "Which provider has the strongest evidence?", "Is this likely a false positive?", "What should I investigate next?". Every answer includes a "Show receipts" link and clickable follow-up-question chips.

Sidebar components

ExportMenu

Four buttons:

- **Export PDF** and **Export CSV** — call `POST .../export?format=pdf / ...?format=csv`. **This backend route does not exist** (see "Not implemented" below). The UI handles the resulting 404 gracefully and shows "Export format not yet available."
- **Export Markdown** — fully client-side, builds a Markdown document from the already-fetched assessment (executive/technical summary, threat assessment, verdict rationale, supporting evidence, MITRE mappings, recommended actions, investigation priorities, IR recommendations, detection rules). Downloads as `ioc-assessment-{lookupId}.md`. **Works.**
- **Export JSON** — client-side `JSON.stringify` of the assessment object. Downloads as `ioc-assessment-{lookupId}.json`. **Works.**

InvestigationActions

Two buttons:

- **Add to Basket** — calls `addToBasket(iocValue)` → `POST /api/v1/basket`.
- **Add to Case** — opens a case picker (lazy-loads your case list via `GET /api/v1/cases`); clicking a case calls `POST /api/v1/cases/{caseId}/iocs`.

AskAiPanel — "Ask AI (Gemini second opinion)"

Not a backend AI call. This builds a prompt client-side and opens `https://gemini.google.com/app` in a small popup window, copying the prompt to your clipboard so you can paste it into your own Gemini account manually. Google blocks embedding `gemini.google.com` in an iframe, so a true in-page embed isn't possible — this is a manual copy/paste workflow, not a server integration.

WorkspaceNav

Persistent nav bar linking to `/basket` (with a live item-count badge, refetched on route change) and `/cases`, so you can move from IOC → basket → case → hunting without leaving the workbench.

TopSearchBar

Persistent pivot search box present on every lookup page. Submitting a new value pushes a new `?value=` onto `/lookup/new`; because it's the same route, the page uses a `key={value}` trick to force React to fully remount rather than silently reuse the previous IOC's state.

4. Completed lookup view (/lookup/{id})

Same dashboard component layout as /lookup/new (all the panels above), but fed from a single `GET /api/v1/lookup/{id}` fetch on page load instead of a live SSE stream. Two important differences from the live view:

- **No correlation graph data.** `GET /api/v1/lookup/{id}` does not return correlation edges, so the `RelationshipGraph` on this page is always given a null/empty correlation payload.
- **Backfilled provider-result fields.** This endpoint's `provider_results` rows omit `ioc_value` / `ioc_type` / `fetched_at` / `from_cache`; the frontend fills in defaults (`fetched_at: 0` , `from_cache: false`) since the shared `ProviderResult` type requires them.
- The post-completion panel cluster (VerdictAnalysisPanel, EvidencePanel, PivotPanel, HuntingCenterPanel, InvestigationCopilot) is gated on `status === 'completed'` rather than an `isDone` streaming flag — it shows as soon as the fetched lookup's status is `completed` , with no waiting.

Use this view to revisit a past lookup (e.g. from a case's attached-IOC list, or a basket "Investigate" link that previously ran) without re-querying every provider.

5. IOC Basket (/basket)

Per-analyst scratch space for collecting IOCs across an investigation, then acting on the set. **Private to you** — the backend scopes basket rows to `owner_id == current_user` , unlike lookups and cases which are shared across the whole SOC team.

- **Investigate Selected** — for every checked item, opens `/lookup/new?value=<ioc>` in a **new browser tab** (`window.open`). Disabled until at least one item is checked.
- **Compare Selected** — requires at least 2 checked items that already have a completed lookup (`latest_lookup_id` set). Calls `POST /api/v1/basket/compare` with those lookup IDs. Renders a comparison table: IOC, Verdict, Risk, ASN, Malware, Threat Actors — with the row matching the response's `most_dangerous_ioc_value` highlighted, plus a narrative summary sentence below the table.
- **Clear Basket** — `DELETE /api/v1/basket` , empties the whole basket. Disabled when the basket is empty.
- Per-row **Investigate** button navigates to `/lookup/new?value=<ioc>` in the same tab; a trash-can icon button removes that single item (`DELETE /api/v1/basket/{itemId}`).
- Each row shows the IOC value, type, whether it has a completed lookup yet ("has completed lookup" / "not yet investigated"), and any note attached when it was added.

Items are added from the lookup pages via `InvestigationActions` → **Add to Basket** (there is no way to add directly from the basket page itself).

6. Cases (/cases , /cases/{id})

Cases are **shared across the whole SOC team** (anyone with case-read access sees every case) — unlike the basket, which is private per analyst.

/cases — list and create

- Lists all cases via `GET /api/v1/cases` (id, title, severity, status, tags, created date), each with a status badge and severity badge.
- **New Case** button toggles a form: title (required), description (optional), and a severity dropdown (Low / Medium / High / Critical, default Medium). Submitting calls `POST /api/v1/cases` , then routes to the new case's detail page.

/cases/{id} — case detail

Fetched via `GET /api/v1/cases/{id}` on load.

- Title, description, and a **status** dropdown covering all six statuses: `open` , `investigating` , `contained` , `resolved` , `false_positive` , `closed` . Changing it calls `PATCH /api/v1/cases/{id}` with `{status}` .
- Severity badge and any tags, shown read-only on this page.
- **IOCs** card: every IOC attached to the case, each with an **Investigate** button that opens `/lookup/new?value=<ioc>` . IOCs get attached here via the **Add to Case** button on a lookup page (`InvestigationActions`), not from a control on this page directly.
- **Analyst Notes** card: chronological note list (body + timestamp) plus an **Add Note** form. Submitting calls `addCaseNote(caseId, noteText)` → `POST /api/v1/cases/{id}/notes` with just `{body}` — the note UI does not expose the `anchor_type` / `anchor_ref` fields that the backend endpoint otherwise supports.

Not implemented / planned

The following are explicitly **NOT IMPLEMENTED** in the current build. Do not expect them; they are called out here so they aren't mistaken for a bug:

- **Server-side PDF/CSV export.** `ExportMenu`'s "Export PDF" and "Export CSV" buttons call `POST /api/v1/lookup/{id}/export?format=pdf|csv` , but no such backend route exists anywhere in `backend/app/api/routes/` . The button click will always fail with a 404, surfaced in the UI as *"Export format not yet available."* Only client-side **Markdown** and **JSON** export actually work today. A `lookup:export` permission string is even pre-defined for admin/analyst roles, suggesting this was planned but never wired up. See [CHANGELOG.md](#) / [DOCUMENTATION_GAPS.md](#).

- **No dedicated threat-actor, malware-family, or campaign pages.** These entities appear only as fields/labels inside lookup results (e.g. the basket comparison table's "Threat Actors"/"Malware" columns, or MITRE/ relationship data in a lookup's dashboard) — there is no standalone page to browse or manage them.
- **No watchlist / monitoring page.** There is no way to subscribe to an IOC for ongoing monitoring or alerting from the UI.
- **No bulk-upload page.** IOCs are looked up one at a time via the search box or added one at a time to the basket; there is no CSV/bulk-import flow.
- **No server-side AI "second opinion."** `AskAiPanel`'s "Ask AI (Gemini second opinion)" button does not call any backend endpoint — it only builds a prompt in the browser and opens Google's own Gemini web app for you to paste into manually, using your own Gemini account.
- **No admin/user-management UI.** The backend defines `provider:manage`, `user:manage`, and `audit:read` permissions for the admin role, but no route or page in this build actually uses them — there is no in-app way to manage users or view an audit log. See [ADMIN_GUIDE.md](#).

For anything not covered in this guide, check [DOCUMENTATION_GAPS.md](#) before assuming it's a defect.

FAQ

FAQ

Answers to the questions that come up most once you're actually running lookups. Every answer here is grounded in the current source code — where something looks like a feature but isn't wired up, it's labeled **NOT IMPLEMENTED** rather than glossed over.

See also: [TROUBLESHOOTING.md](#) for step-by-step fixes, [ARCHITECTURE.md](#) for the full request lifecycle, [SECURITY.md](#) for RBAC/auth details, and [PROVIDERS.md](#) for per-provider specifics.

1. Why does a provider show `not_configured` instead of a result?

Every connector sets `requires_key` (usually `True`) and computes a `configured` flag once, at process start, from `app/core/config.py` settings — e.g. `self.configured = bool(get_settings().virustotal_api_key)`. `BaseProvider.run()` checks this **before** calling `fetch()`:

```
requires_key=True and configured=False
→ ProviderStatus.NOT_CONFIGURED
error_message = "<name> is not configured (missing API key/credentials)."
```

No network call is made at all. Set the corresponding env var in `.env` and restart the backend — `configured` is computed once at import time, so a key added after startup does not take effect without a restart. Full var list in [PROVIDERS.md](#) and [CONFIGURATION.md](#).

Providers that need **no** key (`requires_key=False`, always `configured=True`): crt.sh, NVD (key optional, just raises the rate ceiling), CISA KEV, MITRE ATT&CK, WHOIS/RDAP, PhishTank, Spamhaus, and the internet-intelligence crawler.

2. Why is a provider's status `no_data` instead of `error`?

`no_data` means the provider was reachable and answered successfully, but had nothing to say about this specific IOC — this is a deliberate, provider-specific mapping, not a generic fallback:

Provider	<code>no_data</code> trigger
VirusTotal	HTTP 404 on the object lookup
AbuseIPDB	HTTP 200 with an empty <code>data</code> payload
OTX	HTTP 200 with <code>pulse_count == 0</code>
ThreatFox / MalwareBazaar	<code>query_status == "ok"</code> but zero entries returned (explicitly <i>not</i> treated as an error — "ok with zero entries is a genuine empty result")
URLhaus	non- "ok" <code>query_status</code> , via the shared <code>abuse.ch</code> status mapper
PhishTank	<code>results.in_database == False</code>

By contrast, an `abuse.ch` **auth failure** (`no_api_key`, `invalid_api_key`, `unauthorized`) is deliberately mapped to `ERROR`, not `NO_DATA` — because the UI renders `NO_DATA` as "nothing malicious found," and a misconfigured key must never look like a clean verdict (`backend/app/providers/abusech.py`).

3. Why did a provider come back `rate_limited`?

`BaseProvider.run()` catches `httpx.HTTPStatusError` and maps status codes **429, 403, or 509** to `ProviderStatus.RATE_LIMITED` (509 is PhishTank's documented over-limit code). The OSINT crawler's own GitHub source module separately raises the same signal on HTTP 403 or 429, per a comment noting GitHub's Search API uses 403 for rate limiting, not only for auth failures. Any other HTTP error status becomes `ERROR`.

No connector self-throttles proactively — there is no per-provider `RateLimiter` instance anywhere in `app/providers/*.py`. The orchestrator's Redis-backed `RateLimiter` class is used for exactly one thing platform-wide: per-user lookup creation (`lookup_create:<user_id>`, 10 calls / 60s by default), not per-provider throttling. The OSINT crawler is the exception — see Q13.

4. What's the difference between `overall_risk_score`, `confidence_score`, and `malicious_probability`?

All three are floats on a **0–100 scale**, never a 0–1 fraction (`app/ai/schemas.py`'s `RiskAssessment`):

Field	Meaning
<code>overall_risk_score</code>	0 (no risk) to 100 (maximum risk)
<code>confidence_score</code>	How confident the assessment is, as a percentage
<code>malicious_probability</code>	Probability the IOC is malicious, as a percentage (e.g. <code>50</code> , not <code>0.5</code>)

They answer different questions: risk score is "how bad is this," confidence is "how sure is the model," and malicious probability feeds the verdict check in Q6. A `field_validator ( _reject_0_to_1_scale )` rescales any value $0 < v < 1$ by multiplying by 100 rather than rejecting it — this exists because smaller local models (the comment cites `llama3.2:3b`) have been observed literally returning `0.5` when asked for "50% confidence."

5. Why do the AI provider summary and the raw provider data seem to disagree?

`summarize_provider()` (`app/ai/service.py`) is grounded **only** in that one provider's own `result.data` — it never sees other providers' output. If `result.status != OK` or `result.data` is empty, the AI backend isn't even called: a degraded `ProviderSummary` is returned directly (`reputation="unknown"` , `detection_status="no data"` , `threat_level="none"` , `confidence="low"`). Every system prompt instructs the model to say "no data"/"not observed" rather than infer, so a summary that looks sparse or literal is the anti-hallucination control working as intended, not a bug.

6. Why did the final assessment "fail" / come back as a generic fallback?

`generate_final_assessment()` returns a static fallback `FinalAssessment` (all narrative fields state generation failed, `overall_risk_score=0` , `final_verdict="unknown"`) whenever the underlying call raises — and that includes **schema validation failures**, not just network errors. The most common cause is `_verdict_must_agree_with_risk` , a model validator that enforces:

- `final_verdict` in {malicious, highly_malicious} requires `malicious_probability >= 30`
- `final_verdict` in {benign, likely_benign} requires `malicious_probability <= 50`

If the model's stated verdict contradicts its own probability number, the `ValueError` raised here **is** the enforcement mechanism — there is no repair/retry step. `app/ai/schemas.py` 's module docstring claims the AI service "retries once with the validation error appended to the prompt," but **no such retry logic exists** in `service.py` , `analysis_service.py` , or `hunting_service.py` — every one of them goes straight from a validation or call exception to a static fallback on the first failure.

7. Why is a MITRE ATT&CK mapping marked ungrounded (or why don't ungrounded ones get removed)?

`_ground_final_assessment()` builds `grounded_technique_ids` from the real correlation graph — specifically, every `CorrelationEdge` where `relationship == "uses_technique"` . For each `MitreMapping` the model returned, it overwrites `mapping.grounded` to:

```
mapping.technique_id.upper() in grounded_technique_ids
```

This **relabels** `grounded` ; it does **not** delete or hide mappings the model invented that aren't backed by a real edge — you can still see an AI-proposed technique with `grounded=false` in the UI. That's deliberate: the model may reasonably infer a technique from context even when no provider explicitly surfaced it, and the platform prefers to show it labeled honestly over silently dropping it.

Separately: `MitreMapping.technique_id` has **no regex pattern= constraint** on purpose. Ollama's grammar compiler (llama.cpp's json-schema-to-grammar) fails to compile a `pattern` constraint into a decoding grammar and returns HTTP 400 "failed to parse grammar" — which would break every `generate_final_assessment()` call when running on Ollama, because the whole schema is compiled into one grammar and a single unsupported constraint fails the entire thing. A malformed `technique_id` isn't a safety issue since grounding only checks ID membership, not format.

8. Why do two providers disagree on whether an IOC is malicious?

The correlation engine tracks this explicitly rather than resolving it. `provider_agreement` (a `dict[str, list[str]]`) buckets providers by the verdict they reported: `result.data.get("verdict")` or `result.data.get("reputation")` , lowercased, mapping that string to the list of `provider_id` s that asserted it. If VirusTotal says `malicious` and Spamhaus says `clean` , both buckets appear side-by-side in `provider_agreement` — the correlation engine (a pure function, no I/O) does not pick a winner. `generate_final_assessment` sees this and is expected to adjudicate; its `agreeing_providers` / `disagreeing_providers` lists are grounded by filtering to only provider IDs that actually appear in real correlation edges/agreement data (`_ground_final_assessment`) — an AI-invented provider name in either list is silently dropped.

Disagreement is common because providers answer fundamentally different questions: Censys always reports `verdict="unknown"` (it surfaces exposure data, not a malicious/clean call), Spamhaus reports a valid `OK` "clean" result for a genuine DNSBL miss, and detection-ratio scanners like VirusTotal can be split even internally.

9. Why does confidence differ from the risk/threat score?

They measure different things and can move independently:

- **Risk/threat score** (`overall_risk_score` , `severity`) — how dangerous the AI judges this IOC to be, given the evidence.
- **Confidence** (`confidence_score` , `analyst_confidence`) — how much evidence backed that judgment.

A single corroborated hit from one authoritative source (e.g. CISA KEV, which sets `verdict="malicious"` unconditionally for any CVE in its catalog) can justify high risk with only moderate confidence, while five providers unanimously returning `unknown` / `no data` can justify high confidence in a *low*-risk read. The two numbers are validated independently (`RiskAssessment` 's `overall_risk_score` and `confidence_score` are separate 0–100 fields) and are not derived from each other.

10. Why is a correlation edge's confidence higher than any single provider reported?

`...CORROBORATION_BONUS_PER_PROVIDER = 0.15` in `app/correlation/engine.py`: when two or more distinct providers assert the exact same (source, target, relationship) edge, the engine takes the **max** of their base confidences and adds $0.15 \times (\text{distinct_provider_count} - 1)$, capped at `1.0`. Base confidence per field ranges from `0.9` (e.g. `resolved_ips`, `asn`) down to `0.5` (`malware_families`, `threat_actors`, `campaigns`) — see the full table in [ARCHITECTURE.md](#). So an edge that starts at `0.5` base confidence but is corroborated by three providers can reach $0.5 + 0.15 \times 2 = 0.8$ — higher than any one provider's contribution on its own. This is the literal mechanism behind "Correlation Engine (corroborated)" labels you'll see as an evidence source.

11. Why does the evidence panel show a confidence number different from the correlation graph edge?

The correlation engine's edge confidence is on a **0–1** scale internally. When `build_evidence_from_correlation()` (`app/evidence/builder.py`) turns an edge into an `EvidenceItem`, it converts: `confidence = round(edge.confidence`

- `, 100, 1)` — so a `0.8` edge becomes an `80.0` evidence-item confidence. This is intentional: `EvidenceItem.confidence` is documented as "**0-100 scale, matches RiskAssessment**" specifically so evidence and risk numbers are directly comparable in the UI without a mental conversion.

12. Why is a pivot suggestion ranked "high" relevance even though its confidence number looks low?

`rank_pivots()` (`app/evidence/pivot.py`) bands relevance using the **raw 0–1** edge confidence, not the $\times 100$ value shown in the `confidence` field of the emitted pivot dict:

```
"high" if provider_count > 1 or edge.confidence >= 0.85
"medium" elif edge.confidence >= 0.6
"low" otherwise
```

So a pivot corroborated by two providers is "high" relevance even at, say, `edge.confidence = 0.55` (→ displayed `confidence: 55.0`) — the multi-provider corroboration alone qualifies it, independent of the raw confidence number sitting next to it.

13. Why does the OSINT crawler sometimes report `rate_limited` with an empty result?

`InternetIntelligenceCollector.fetch()` fans out to 4 sources concurrently (GitHub, Reddit, RSS news feeds, Pastebin/psbdlmp.ws search). Only **GitHub** and **Reddit** implement rate-limit backoff (`AsyncMinIntervalLimiter`, a process-local — not Redis-backed — spacer: 6.0s minimum between GitHub calls, 1.1s for Reddit) and can raise `SourceRateLimitedError`. RSS feeds and Pastebin search have no limiter and never raise that error — any failure there just returns an empty list silently.

Collector-level status logic:

```
merged findings non-empty      → OK
merged empty, any source rate-limited → RATE_LIMITED
merged empty, nothing rate-limited → NO_DATA
```

The `RATE_LIMITED` branch exists specifically so a rate-limited crawl run doesn't look identical to "genuinely nothing found" in the UI. In practice this path can only be triggered by GitHub or Reddit throttling, since the other two sources never raise it.

14. Why does the "Export PDF" / "Export CSV" button do nothing (or show "Export format not yet available")?

Because there is no server-side export endpoint. **NOT IMPLEMENTED.** `frontend/components/dashboard/ExportMenu.tsx` calls:

```
POST /api/v1/lookup/{lookup_id}/export?format=pdf
POST /api/v1/lookup/{lookup_id}/export?format=csv
```

...but no `/export` route exists anywhere under `backend/app/api/routes/`. The request 404s, and the frontend explicitly catches that:

```
if (res.status === 404) {
  showMessage("Export format not yet available.");
  return;
}
```

Client-side **JSON** and **Markdown** export both work today — they build the file entirely in the browser from the already-fetched `FinalAssessment` object (`handleExportJson`, `buildMarkdown` in the same file) and never hit the network. Use those two if you need to get a result out of the platform right now.

Relatedly: the `lookup:export` permission string exists in `ROLE_PERMISSIONS` (`backend/app/models/user.py`, granted to `admin` and `analyst`, not `viewer`) but is **never referenced by** `require_permission()` **anywhere** — there's no route to gate, so today it's a declared-but-unused permission. See [SECURITY.md](#).

15. Why does `GET /auth/me` show a blank name?

NOT IMPLEMENTED (bug, not a config issue): the handler in `backend/app/api/routes/auth.py` hardcodes `full_name=""` in its response regardless of what's actually stored on the user record. The real `full_name` you registered with is in the database — `/me` just doesn't return it. See [SECURITY.md](#).

16. Why can't I log out everywhere / revoke a stolen token?

NOT IMPLEMENTED. `/auth/refresh` issues a new access/refresh token pair on every call, but there is no server-side token store or denylist. A refresh token stays valid until it naturally expires (`refresh_token_expire_days`, default 7 days) regardless of client-side logout, which only clears `localStorage`. There is no MFA, no password reset, and no email verification either — see [SECURITY.md](#).

17. Why is the relationship graph missing the labels I expected on nodes?

`GraphNode.labels` (`app/correlation/engine.py`) defaults to an empty list and **nothing in `correlate()` ever populates it** — every constructed `GraphNode` (the seed node and every target node) omits the `labels` kwarg entirely. It's a real field with no producer yet; don't expect anything there today.

18. Why does `correlation_edges` in Postgres say it's "mirrored into Neo4j," but I don't see anything in Neo4j?

Because that mirroring is **NOT IMPLEMENTED**. `neo4j_uri`, `neo4j_user`, and `neo4j_password` exist as settings (`backend/app/core/config.py`), and code comments in `engine.py` and `CorrelationEdgeRecord`'s docstring both describe edges as "mirrored into Neo4j for graph traversal" — but there is no Neo4j driver import, session, or query/write call anywhere in the backend. Postgres's `correlation_edges` table is the sole, actual store for graph edges; the relationship graph the frontend renders is read entirely from there (directly, or via `correlation_from_records()` for previously-completed lookups). The same applies to `OpenSearch` (`opensearch_url` is configured but never referenced by the correlation/evidence subsystem). See [ARCHITECTURE.md](#) and [DATA_MODEL.md](#).

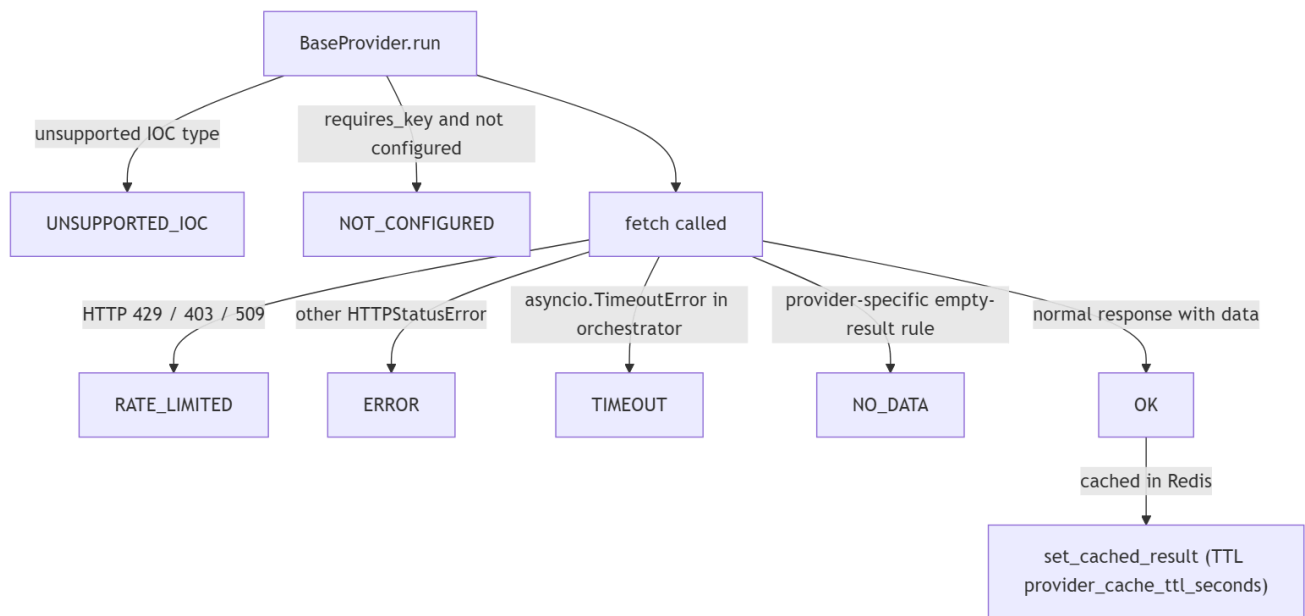
19. Why does switching `AI_BACKEND` change nothing until I restart?

`_get_ai_client()` (`app/ai/service.py`) reads `settings.ai_backend` and picks one of four client modules — `ollama` (default/fallback for any unrecognized value), `anthropic`, `gemini`, `bedrock` — each of which computes its own `is_configured` flag once from settings at construction time (e.g. Anthropic: `bool(settings.anthropic_api_key)`; Ollama: `bool(base_url and model)`, true by default since both have non-empty defaults). If the selected backend's `is_configured` is `False`, the request raises `RuntimeError("AI backend '<name>' is not configured...")` rather than silently falling back to another backend. Changing `.env` requires a backend restart to take effect, same as provider API keys (Q1).

20. Why did an AI call fail instead of retrying?

Only `bedrock_client.py` has any retry policy, and it's not custom code — it's `botocore`'s `BotoConfig(retries={"max_attempts": 3, "mode": "adaptive"})`, applied at client construction. `ollama_client.py`, `anthropic_client.py`, and `gemini_client.py` each make **exactly one** HTTP attempt in `call_claude_json()` and raise a `RuntimeError` on any failure — connection error, timeout, non-2xx status, empty/non-JSON content, or a parsed-but-wrong-shape response. There is no retry-with-error-appended loop anywhere despite a docstring in `app/ai/schemas.py` claiming one exists (see Q6). Every caller (`service.py`, `analysis_service.py`, `hunting_service.py`) catches the resulting exception and returns a static, clearly-labeled fallback object instead of surfacing a raw error to the UI.

Quick reference: status/verdict decoder ring



Only `OK` results are cached, feed the correlation engine, and get an AI provider summary. Everything else degrades gracefully and is visible in the provider card / `GET /api/v1/providers/health`.

Glossary

Glossary

Definitions for terms this platform actually uses in its code, database schema, and UI — enum values, status codes, permission strings, and domain terms — with source references. This is not a generic security glossary; a term only appears here if it is grep-backed in `backend/app/` or `frontend/`.

See also: [IOC_TYPES.md](#) for IOC-type detection details, [PROVIDERS.md](#) for provider-specific behavior, [DATA_MODEL.md](#) for full table schemas, and [AI_ENGINE.md](#) for how the AI layer consumes several of these enums.

IOC (Indicator of Compromise)

A piece of data (hash, IP, domain, CVE, ATT&CK technique ID, etc.) submitted for lookup. The platform's `IOCType` enum (`backend/app/ioc/types.py:5-38`) defines these values:

`ipv4`, `ipv6`, `domain`, `url`, `hostname`, `email`, `md5`, `sha1`, `sha256`, `sha512`, `tls_certificate`, `ja3`, `ja4`, `asn`, `cidr`, `malware_family`, `threat_actor`, `campaign`, `cve`, `cwe`, `capec`, `mitre_technique`, `file_name`, `registry_key`, `process_name`, `mutex`, `windows_service`, `file_path`, `user_agent`, `crypto_wallet`, `yara_rule`, `sigma_rule`, `unknown`.

Two derived sets are used elsewhere in the codebase:

- `HASH_TYPES` (`types.py:42`) = `{md5, sha1, sha256, sha512}` — used by connectors that accept any hash type.
- `NETWORK_TYPES` (`types.py:44-53`) = `{ipv4, ipv6, domain, url, hostname, asn, cidr}` — used to seed the correlation graph and to classify evidence as infrastructure.

Classification is performed exclusively server-side by `detect_ioc_type()` (`backend/app/ioc/detector.py:61`); see [IOC_TYPES.md](#) for the full detection order. Several enum members exist but can **never** actually be returned by the detector today — see [Detector-only vs. reachable IOC types](#) below.

Detector-only vs. reachable IOC types

Type	Reachable via <code>detect_ioc_type()</code> ?	Notes
<code>ja3</code>	NOT IMPLEMENTED	Regex exists (<code>_JA3_RE</code> , <code>detector.py:15</code>) but is never called; 32-hex strings always resolve to <code>md5</code> instead.
<code>mutex</code>	NOT IMPLEMENTED	Regex exists (<code>_MUTEX_HINT_RE</code> , <code>detector.py:31</code>) but is never referenced.
<code>windows_service</code>	NOT IMPLEMENTED	No regex or logic anywhere in <code>detector.py</code> .
<code>process_name</code>	NOT IMPLEMENTED	The one code branch that returns it (<code>detector.py:157</code>) is unreachable dead code guarded by a contradictory condition.
<code>hostname</code>	NOT IMPLEMENTED (by the detector)	No public-suffix-list check exists; all bare dotted strings resolve to <code>domain</code> . <code>hostname</code> is produced elsewhere (e.g. <code>providers/otx.py</code>) but never by <code>detect_ioc_type()</code> itself.
<code>tls_certificate</code> , <code>malware_family</code> , <code>threat_actor</code> , <code>campaign</code>	NOT IMPLEMENTED (by the detector)	Never returned by <code>detect_ioc_type()</code> ; produced by other subsystems (correlation engine, crawler, workers, <code>cr.sh</code> 's <code>supported_types</code>).

The only override mechanism is `ioc_type_hint` on `LookupCreateRequest` / `BasketAddRequest` (`backend/app/schemas/lookup.py:11`, `backend/app/schemas/basket.py:10`) — a generic `Optional[IOCType]`, not a type-specific hint (e.g. there is no dedicated "this is JA3, not MD5" signal).

Verdict

The `Verdict` enum (`backend/app/models/lookup.py:22-34`) is the final classification stored on `IOCLookup.final_verdict` and produced by the AI's `FinalAssessment.final_verdict` field (`backend/app/ai/schemas.py:158`). Twelve values:

Value	Frontend color (<code>frontend/lib/utls.ts</code>)	Meaning
<code>highly_malicious</code>	destructive (red)	Confirmed high-confidence malicious
<code>malicious</code>	destructive (red)	Malicious
<code>suspicious</code>	warning (yellow)	Ambiguous / partially corroborated malicious signal
<code>unknown</code>	muted (default)	Insufficient data to judge
<code>likely_benign</code>	success (green)	Probably benign
<code>benign</code>	success (green)	Confirmed benign
<code>scanner</code>	accent	Known internet-scanning infrastructure
<code>tor_exit_node</code>	accent	Tor exit node
<code>vpn</code>	accent	VPN infrastructure
<code>cdn</code>	accent	CDN infrastructure
<code>cloud_infrastructure</code>	accent	Generic cloud provider infrastructure
<code>dormant_infrastructure</code>	accent	Infrastructure with no current active signal

`malicious` and `highly_malicious` are grouped as `_MALICIOUS_VERDICTS`; `benign` and `likely_benign` are grouped as `_BENIGN_VERDICTS` (`backend/app/ai/schemas.py:28-29`). A Pydantic validator (`FinalAssessment._verdict_must_agree_with_risk` , `schemas.py:161-182`) rejects an AI response where `final_verdict` is in `_MALICIOUS_VERDICTS` but `malicious_probability < 30`, or in `_BENIGN_VERDICTS` but `malicious_probability > 50` — a sanity check against small local models that emit an internally contradictory assessment.

The badge-color switch (`frontend/lib/utls.ts:8-26` , `components/dashboard/ProviderCard.tsx`) is a hardcoded string match on these exact lowercase values; any off-spec string silently falls through to the muted default style, which is why `FinalAssessment` 's AI-facing fields use real `Literal` /enum types rather than free text (`backend/app/ai/schemas.py:10-20`).

Reputation, Threat Level, Confidence (AI schema literals)

Three `Literal` types constrain the AI's structured output (`backend/app/ai/schemas.py:31-33`), used on both `ProviderSummary` (per-provider) and `RiskAssessment` (final):

Literal	Values
<code>_Reputation</code>	<code>malicious</code> , <code>suspicious</code> , <code>clean</code> , <code>unknown</code> , <code>no data</code>
<code>_ThreatLevel</code> (severity)	<code>none</code> , <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>
<code>_Confidence</code>	<code>low</code> , <code>medium</code> , <code>high</code>

Confidence bands

Two distinct 0–100 numeric confidence conventions exist in the platform — do not conflate them:

1. **AI assessment confidence** — `RiskAssessment.confidence_score` (`ai/schemas.py:105-109`) and `overall_risk_score` / `malicious_probability` are all `float` fields on a **0–100** scale (explicitly *not* 0–1). A validator (`_reject_0_to_1_scale` , `ai/schemas.py:119-131`) rescales any value strictly between 0 and 1 by $\times 100$, because small local models (observed: `llama3.2:3b`) sometimes emit a 0–1 probability despite the 0–100 field description.
2. **Deterministic evidence confidence** — `EvidenceRecord.confidence` (`backend/app/evidence/builder.py`) is also 0–100, "same scale as `RiskAssessment` ... so evidence and risk numbers are directly comparable in the UI" (`builder.py:1-10` docstring). Two sources feed it:
 - Provider-summary evidence: `_CONFIDENCE_LEVEL_TO_SCORE` maps the `_Confidence` literal to a score — `low` $\rightarrow$ `30.0` , `medium` $\rightarrow$ `60.0` , `high` $\rightarrow$ `90.0` (`builder.py:21`); an unrecognized value defaults to `50.0` .
 - Correlation-edge evidence: `confidence = round(edge.confidence * 100, 1)` — the correlation engine's internal 0–1 edge confidence (see [Correlation confidence](#) below), converted to the 0–100 scale.

The **pivot ranking** endpoint uses yet another convention on the *same* correlation-edge confidence: `rank_pivots()` bands relevance as `high` if `provider_count > 1` or `edge.confidence >= 0.85` , `medium` if `edge.confidence >= 0.6` , else `low` — evaluated on the **raw 0–1 engine value**, even though the pivot dict it emits also carries a `confidence` key already converted to 0–100 (`backend/app/evidence/pivot.py:45-47`). Don't assume the 0.85/0.6 thresholds apply to the 0–100 value in the same object.

ProviderStatus

The `ProviderStatus` enum (`backend/app/providers/base.py:31-38`) — the outcome of a single provider's `run()` call for one IOC lookup:

Value	Meaning	Set by
ok	Provider returned usable data	Connector <code>fetch()</code> on success
error	Unexpected failure (non-retryable exception, or HTTP error not otherwise mapped)	<code>BaseProvider.run()</code> (<code>base.py:126-151</code>), orchestrator on exhausted retries
timeout	Provider call exceeded <code>provider_timeout_seconds</code>	Orchestrator (<code>orchestrator.py:59-78</code>), <code>error_message="Timed out after {N}s"</code>
rate_limited	Provider signaled its own rate limit	<code>BaseProvider.run()</code> maps HTTP 429, 403, 509 → <code>rate_limited</code> (509 is PhishTank's documented over-limit code, <code>base.py:129</code>)
not_configured	Provider requires an API key/credential that isn't set	<code>BaseProvider.run()</code> (<code>base.py:112-123</code>) when <code>requires_key=True</code> and <code>configured=False</code>
unsupported_ioc	The IOC type isn't in this provider's <code>supported_types</code>	<code>BaseProvider.run()</code> short-circuit (<code>base.py:101-111</code>), before any network call
no_data	Provider reached out successfully but had nothing on this IOC	Connector-specific (e.g. VirusTotal 404, AbuselPDB empty <code>data</code> , <code>abuse.ch</code> "ok" status with zero entries)

Only `ok` results are cached in Redis (`orchestrator.py:80-83`) and only `ok` results contribute nodes/edges to the correlation graph (`correlation/engine.py:106`) or top-level evidence records (`evidence/builder.py:75-76`). See [PROVIDERS.md](#) for the per-connector mapping rules (e.g. why `abuse.ch` auth failures are deliberately `error` , not `no_data` — a misconfigured key must never look like a clean verdict, per `abusech.py:5-9`).

EvidenceType

The `EvidenceType` enum (`backend/app/models/evidence.py:19-28`) — the classification of a single persisted `EvidenceItem` row. All 9 values are actually producible by `backend/app/evidence/builder.py` :

Value	Produced by	Path
detection	<code>build_evidence_from_providers()</code>	Top-level per-provider record, when <code>summary.reputation</code> is <code>unknown</code> or <code>no_data</code>
reputation	<code>build_evidence_from_providers()</code>	Top-level per-provider record, for any other reputation value
other	<code>build_evidence_from_providers()</code>	One record per entry in <code>summary.interesting_findings</code>
malware_association	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>malware_family</code>
threat_actor_association	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>threat_actor</code>
campaign_association	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>campaign</code>
mitre_technique	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>mitre_technique</code>
infrastructure	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is in <code>NETWORK_TYPES</code> (see IOC) plus <code>tls_certificate</code> / <code>asn</code>
relationship	<code>build_evidence_from_correlation()</code>	Fallback for any correlation-edge target type not covered above (e.g. <code>cve</code>)

`build_evidence()` (`builder.py:142-147`) simply concatenates the provider-path and correlation-path results — no other `EvidenceType` values exist. Evidence is deterministic: the module docstring states explicitly "with NO AI involved" (`builder.py:1-10`).

Correlation edge confidence and corroboration

The correlation engine (`backend/app/correlation/engine.py`) assigns each `GraphEdge` a 0–1 `confidence` float, from a fixed base-confidence table keyed by the source data field (`_FIELD_BASE_CONFIDENCE` , `engine.py:77-90`):

Field	Base confidence
<code>resolved_ips</code> , <code>resolved_domains</code> , <code>asn</code>	0.9
<code>certificates</code>	0.85
<code>related_urls</code> , <code>related_hashes</code> , <code>cves</code>	0.8
<code>mitre_techniques</code>	0.75
<code>related_domains</code>	0.7
<code>malware_families</code> , <code>threat_actors</code> , <code>campaigns</code>	0.5
(any other field)	1.0 if the provider's category is <code>threat_intel</code> , else 0.7

When multiple providers independently assert the *same* (source, target, relationship) edge, confidence is boosted: `boosted = min(1.0, max_base_confidence + 0.15 * (distinct_providers - 1))` (`_CORROBORATION_BONUS_PER_PROVIDER = 0.15` , `engine.py:94, 154-175`). The provenance string

(comma-joined provider IDs) is what `rank_pivots()` and `build_evidence_from_correlation()` later split on to decide corroboration and to label the evidence source as "Correlation Engine (corroborated)" vs. a single provider ID (`evidence/builder.py:122-123`, `evidence/pivot.py:35`).

Only `correlate()` results built from providers with `status == ok` contribute edges at all — a failed provider contributes zero nodes/edges (`engine.py:106`).

ATT&CK / MITRE

References the [MITRE ATT&CK](#) framework. Two IOC-facing concepts:

- **mitre_technique** — an `IOType` value matched by `detect_ioc_type()` via `^T\d{4}(\.\d{3})?$` (case-insensitive), matching both a bare technique ID (`T1059`) and a subtechnique (`T1059.001`) (`backend/app/ioc/detector.py:84-85`).
- **MITRE ATT&CK provider** — `provider_id="mitre_attack"` (`backend/app/providers/mitre_attack.py`), category `threat_intel`, `supported_types={mitre_technique}`, free/no-key, fetches the ~30MB public STIX enterprise-attack bundle from `https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/enterprise-attack.json`, cached in-process for up to 3600s.

The AI schema's `_MitreTactic` literal (`backend/app/ai/schemas.py:41-45`) constrains `MitreMapping.tactic` to the 14 exact ATT&CK `phase_name` slugs (e.g. `initial-access`, `command-and-control`) so the frontend's exact-string-equality tactic grouping (`MitreMatrix.tsx`) never splits one tactic into two differently-cased buckets. `MitreMapping.grounded` (`ai/schemas.py:85-92`) is `true` only if the technique was explicitly surfaced by a provider (e.g. the MITRE ATT&CK provider's `mitre_techniques` field); the AI must set it `false` for a technique it inferred itself rather than one grounded in provider data.

Related, separate `IOType` values that are **not** produced by `detect_ioc_type()`'s MITRE matching but exist for adjacent standards: `cve` (`^CVE-\d{4}-\d{4,7}$`), `cwe` (`^CWE-\d{1,5}$`), `capec` (`^CAPEC-\d{1,5}$`) — checked in the order `CVE` → `CAPEC` → `CWE` → `mitre_technique` (`detector.py:78-85`).

JA3 / JA4

TLS client-fingerprint identifiers.

- **JA4**: matched by `detect_ioc_type()` via `^[a-z0-9]{10}_[a-fA-F0-9]{12}_[a-fA-F0-9]{12}$` (case-insensitive) (`_JA4_RE`, `backend/app/ioc/detector.py:112-113`). The 10-character prefix encodes protocol(1) + version(2) + SN(1) + cipher_count(2) + ext_count(2) + ALPN(2) — e.g. `q13i0207h3_55b375c5d22e_cd85d2d88918`. This check runs **before** the MD5 check specifically because JA4 has its own distinguishable shape.
- **JA3: NOT IMPLEMENTED** as a distinguishable classification. `IOType.JA3` exists as an enum value and a regex (`_JA3_RE`, `detector.py:15`, identical shape to MD5: `^[a-fA-F0-9]{32}$`) exists in a comment describing a "caller hint" disambiguation scheme, but that regex is never invoked anywhere in `detector.py`. Every 32-hex-character string resolves to `md5`, never `ja3`, and no such hint mechanism exists in the reviewed request schemas (the only override, `ioc_type_hint`, is a generic `Optional[IOType]`, not JA3-specific).

ASN

Autonomous System Number. `IOType.ASN` is matched by `detect_ioc_type()` via `^(?:AS|ASN)\s?(\d+)$` (case-insensitive) (`_ASN_RE`, `backend/app/ioc/detector.py:100-101`) — **requires** an explicit `AS / ASN` prefix. This was a deliberate hardening: without the required prefix, a bare hex hash beginning with the letter `a` followed by digits (e.g. `a11111111111111111111111111111111`) would otherwise satisfy a looser ASN pattern before the hash regexes ran; a regression test pins this case to resolve as `md5`, not `asn` (`backend/app/tests/unit/test_ioc_detector.py:64`).

`asn` is part of `NETWORK_TYPES` (`backend/app/ioc/types.py:44-53`) and is supported by the WHOIS/RDAP provider (`whois_rdap.py`, via `rdap.org`) and by the correlation engine's `_RELATIONSHIP_EXTRACTORS` table as the `belongs_to_asn` relationship, with a 0.9 base confidence (`correlation/engine.py:57-90`).

Roles and permissions (ROLE_PERMISSIONS)

Three roles, defined by `Role` (`backend/app/models/user.py:11-14`): `admin`, `analyst`, `viewer`. The permission matrix `ROLE_PERMISSIONS` (`backend/app/models/user.py:31-44`) is a `dict[Role, set[str]]` consumed by `require_permission(permission)` (`backend/app/auth/rbac.py`), a FastAPI dependency factory that 403s with `"Role '{role}' lacks permission '{permission}'"` if the current user's role's set doesn't contain the requested permission string.

Permission string	admin	analyst	viewer	Gates
lookup:create	✓	✓		POST /lookup/stream
lookup:read	✓	✓	✓	GET /lookup/{id} , GET /lookup , GET /lookup/{id}/pivots , GET /providers/health
lookup:export	✓	✓		(lookup export)
provider:manage	✓			(provider administration)
user:manage	✓			(user administration)
audit:read	✓			(audit log access)
evidence:read	✓	✓	✓	Evidence retrieval
analysis:generate	✓	✓		AI analysis generation
hunting:generate	✓	✓		Hunting-query generation
copilot:query	✓	✓		Copilot queries
basket:manage	✓	✓		POST /basket
case:create	✓	✓		Case creation
case:read	✓	✓	✓	Case retrieval
case:write	✓	✓		Case editing
case:close	✓	✓		Case closure

`viewer` is strictly read-only: `{lookup:read, evidence:read, case:read}` — it cannot create lookups, manage baskets, or touch cases beyond reading them (`user.py:43`). The first user ever registered via `POST /auth/register` is assigned `admin`; every subsequent registration defaults to `analyst` (per [SECURITY.md](#)). Full permission-string-to-route mapping is in [API_DOCUMENTATION.md](#) / [SECURITY.md](#).

Other platform-specific terms

Term	Definition	Source
Basket	A saved collection of IOCs a user is tracking, added via <code>POST /basket</code> , gated by the <code>basket:manage</code> permission.	<code>backend/app/api/routes/basket.py</code> , <code>backend/app/schemas/basket.py</code>
Case	A grouped investigation record with associated IOCs, notes, and reports; cascade-deletes its children on removal.	<code>backend/app/models/case.py</code> (per DATA_MODEL.md)
Pivot	A one-click suggested next-IOC-to-investigate, derived purely from correlation edges touching the seed IOC — deliberately not AI-generated ("a pure sort over real correlation edges can never hallucinate a pivot target," <code>backend/app/evidence/pivot.py:1-5</code>). Returned by <code>GET /api/v1/lookup/{lookup_id}/pivots</code> .	<code>backend/app/evidence/pivot.py</code> , <code>backend/app/api/routes/pivot.py</code>
Corroborating providers	The count of distinct provider IDs (parsed from an edge's comma-joined <code>provenance</code> string) that independently asserted the same correlation edge; drives both the pivot <code>relevance</code> band and the correlation-engine confidence boost.	<code>backend/app/evidence/pivot.py:35</code> , <code>backend/app/correlation/engine.py:94</code>
Grounded (AI mapping)	A boolean on <code>MitreMapping</code> indicating whether an ATT&CK technique the AI cited was explicitly present in provider data (<code>true</code>) vs. inferred by the model itself (<code>false</code>). Checked downstream by <code>_ground_final_assessment()</code> , which sets <code>grounded=False</code> on any technique ID that doesn't match a real correlation-graph technique.	<code>backend/app/ai/schemas.py:85-92</code> , <code>backend/app/ai/service.py</code>
Provider category	One of <code>threat_intel</code> , <code>sandbox</code> , <code>passive_dns</code> , <code>certificate_intel</code> , <code>whois</code> , <code>vulnerability</code> , <code>osint</code> (<code>ProviderCategory</code> enum).	<code>backend/app/providers/base.py:21-28</code>
from_cache	Boolean on <code>ProviderResult</code> indicating the result was served from the Redis provider cache (keyed by SHA256 of the IOC value) rather than a live network call.	<code>backend/app/providers/base.py:41-73</code> , <code>backend/app/core/cache.py:24-39</code>

Terms with NO code/UI presence (excluded deliberately)

The following would be expected in a generic threat-intel glossary but are **NOT IMPLEMENTED** in this platform and are therefore not defined above as live concepts: Neo4j graph storage (config vars and docstrings reference it, but no driver import, session, or write/query call exists anywhere in `backend/`), OpenSearch, STIX/TAXII ingestion, and per-provider circuit breakers (all retry/backoff logic lives centrally in `backend/app/providers/orchestrator.py`, not in individual connectors). See [PROVIDERS.md](#) and [ARCHITECTURE.md](#) for details.

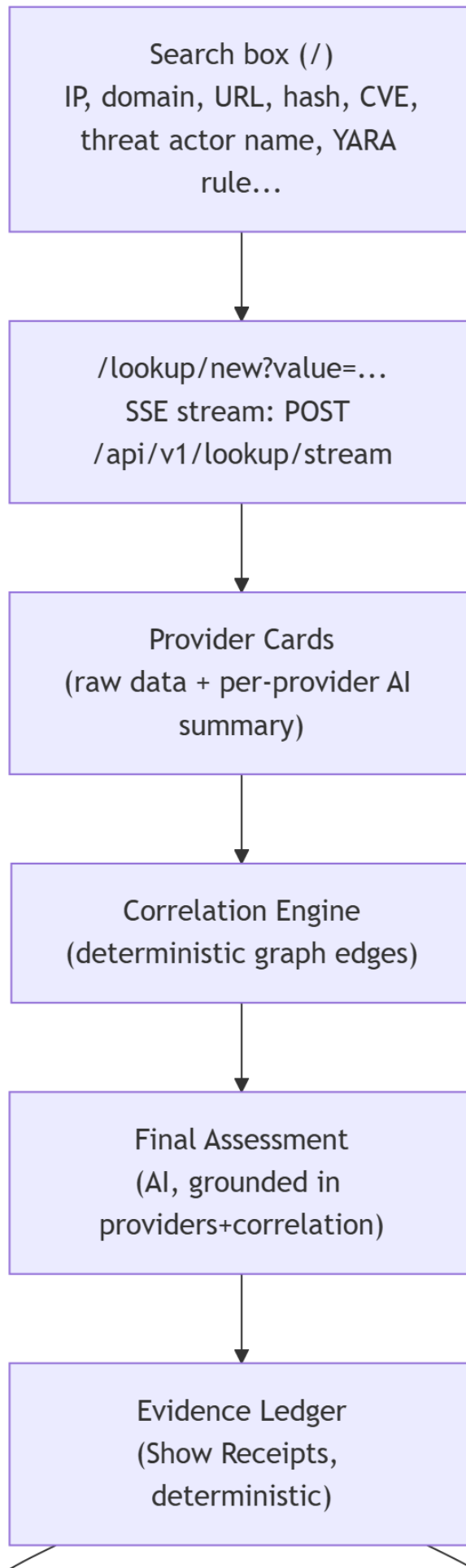
SOC Analyst Guide

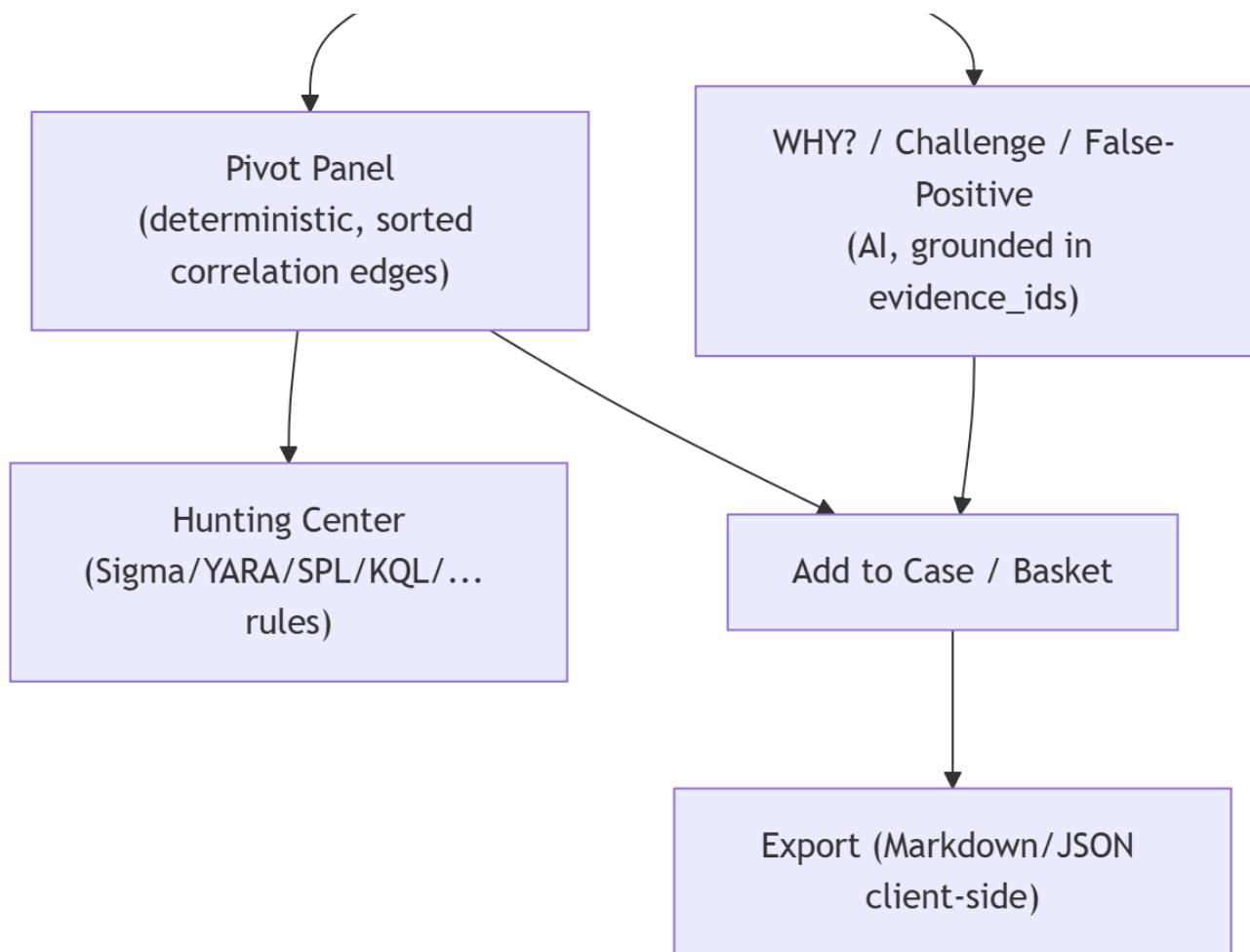
SOC Analyst Guide

A practical, end-to-end SOP for investigating an indicator of compromise (IOC) in the platform using only features that actually exist in the codebase today. For API/route details see [API.md](#); for system design see [ARCHITECTURE.md](#).

Scope note: this platform has **no dedicated Threat Actor, Malware Family, or Campaign pages**. Attribution data (actor/family/campaign names) surfaces only *inside* a lookup's provider cards, evidence ledger, and correlation graph — there is nothing to browse under `/actors`, `/malware`, or `/campaigns` because those routes/pages do not exist.

1. The investigation loop, at a glance





Everything left of "Export" is real and working. The single gap is noted in [Step 9](#).

2. Step 1 — Submit an IOC

1. Sign in at `/login` (or `/register` — the **first account ever created becomes admin**; every later registration defaults to `analyst`).
2. From the home page (`/`), type a value into the search box. Supported example inputs shown as chips: `8.8.8.8`, `malicious-example.com`, `CVE-2024-3400`, `T1059`.
3. Submitting navigates to `/lookup/new?value=<value>`, which opens a Server-Sent Events (SSE) stream:

```
POST /api/v1/lookup/stream
Authorization: Bearer <access_token>
Content-Type: application/json

{ "value": "8.8.8.8" }
```

Requires permission `lookup:create`. Rate-limited per user to **10 calls per 60 seconds** (defaults `lookup_rate_limit_max_calls=10`, `lookup_rate_limit_window_seconds=60`) — a `429` means you've hit the limit, not an error in your input.

4. If the platform cannot classify your input, it returns `422 "Could not determine IOC type; pass ioc_type_hint."` — retry with an explicit `ioc_type_hint` (one of the `IOCType` enum values, e.g. `ipv4`, `domain`, `sha256`, `cve`, `threat_actor`).

The SSE event sequence, in order, is always:

```
detected → N × provider_result → N × provider_summary → correlation → final_assessment → done
```

(or a single `error` event on failure). The UI's "Event log" debug card in the sidebar shows this raw stream if you need to see exactly what arrived and when.

Re-visiting a completed lookup later via `/lookup/{id}` re-fetches the same data with a plain `GET /api/v1/lookup/{id}` (no SSE) — but note that endpoint does **not** return correlation edges, so the Relationship Graph on that page is always empty; only the original `/lookup/new` stream view shows the live graph payload.

3. Step 2 — Read the provider cards

Each `provider_result` SSE event renders as a card (`ProviderCardGrid` / `ProviderCard`). Cards with `status: "ok"` sort first. Each card shows:

- The provider's **raw returned data** (fields rendered generically; small arrays as pills, large arrays comma-joined, nested objects behind a collapsible "Raw data" JSON dump).
- An **AI summary** (`ProviderSummary` , one per provider, generated independently per provider and grounded *only* in that provider's own payload — never in another provider's data): `what_it_knows` , `reputation` (`malicious` | `suspicious` | `clean` | `unknown` | `no data`), `threat_level` (`none` | `low` | `medium` | `high` | `critical`), `confidence` (`low` | `medium` | `high`), plus `interesting_findings` , `relationships` , `unique_observations` , and optional `caveats` .
- Non-`ok` providers show "No AI summary" — a failed/errored provider contributes **zero** correlation nodes/edges and zero evidence.

Use `GET /api/v1/providers/health` (permission `lookup:read`) if a provider looks suspiciously absent — see [PROVIDERS.md](#) for the provider roster.

4. Step 3 — Correlation graph (deterministic)

After every provider has responded, the backend runs a **pure, no-I/O** correlation function once (`app/correlation/engine.py`) over all `status: ok` provider results and streams a single `correlation` event containing `nodes` and `edges` . This is rendered as `RelationshipGraph` (force-graph, with an accessible "View as list" table toggle).

Key mechanics to know when judging an edge's strength:

Concept	Mechanic
Base confidence	Per-field defaults, e.g. <code>resolved_ips</code> / <code>resolved_domains</code> / <code>asn</code> = 0.9, <code>related_urls</code> / <code>related_hashes</code> / <code>cves</code> = 0.8, <code>malware_families</code> / <code>threat_actors</code> / <code>campaigns</code> = 0.5
Corroboration bonus	+0.15 per additional distinct provider asserting the exact same (source, target, relationship) edge, capped at 1.0
Provenance	Comma-separated list of provider IDs that asserted the edge — this is what the corroboration count is derived from

An edge with `provenance` listing 3 providers is materially stronger evidence than one with a single provider, even if their raw confidence numbers look similar — check provenance, not just the confidence percent.

Correlation edges are persisted to Postgres (`correlation_edges` table) so the [Pivot Panel](#) and case/hunt tooling can rebuild the graph after the stream ends, **without** re-running correlation. (A Neo4j mirror of this graph is referenced in code comments as an intent but is **NOT IMPLEMENTED** — there is no Neo4j driver call anywhere in the codebase; Postgres is the only real store for edges.)

5. Step 4 — Final Assessment: Threat Score vs Confidence Score vs Evidence Quality

The `final_assessment` SSE event carries a `FinalAssessment` object, including a `risk: RiskAssessment` block. **All three of these numbers are 0–100 scales, not 0–1 probabilities** — the schema explicitly rejects and auto-rescales a value under 1 (a defensive fix for local models that default to 0–1 despite instructions). Do not confuse them:

Field	What it actually measures	Where in UI
<code>overall_risk_score</code> (0–100)	Threat Score. How dangerous the AI judges this IOC to be, given everything it saw. Higher = more dangerous.	Threat Score Gauge (top of page); "Risk & Verdict" tab in Final Assessment
<code>confidence_score</code> (0–100)	Confidence Score. How confident the AI is <i>in its own assessment</i> — a measure of certainty, independent of how bad the IOC is. A high threat score with a low confidence score means "probably bad, but the evidence is thin."	"Risk & Verdict" tab
<code>malicious_probability</code> (0–100)	The AI's estimated probability this IOC is malicious specifically (distinct from severity).	"Risk & Verdict" tab
<code>severity</code>	<code>none</code> <code>low</code> <code>medium</code> <code>high</code> <code>critical</code>	Risk & Verdict tab
<code>reputation</code>	<code>malicious</code> <code>suspicious</code> <code>clean</code> <code>unknown</code> <code>no data</code>	Risk & Verdict tab
<code>analyst_confidence</code>	<code>low</code> <code>medium</code> <code>high</code> — a categorical companion to <code>confidence_score</code>	Risk & Verdict tab

"Evidence Quality" is not a single field the AI emits — there is no `evidence_quality` score in the schema. Judge it yourself from the **Evidence Ledger** (next section): how many evidence records exist, how many distinct providers corroborate each claim, and each record's own `confidence` (0–100). Treat `confidence_score` as the AI's self-reported certainty, and the evidence ledger's per-record confidence + corroboration counts as the *actual, auditable* evidence quality.

`final_verdict` (`highly_malicious` | `malicious` | `suspicious` | `unknown` | `likely_benign` | `benign` | `scanner` | `tor_exit_node` | `vpn` | `cdn` | `cloud_infrastructure` | `dormant_infrastructure`) is validated server-side to not flatly contradict `malicious_probability` (e.g. a `malicious` verdict with probability < 30 is rejected and the AI call is retried/degraded) — so if you see a malicious verdict on this platform, the probability number backing it was at least self-consistent at generation time.

The Final Assessment panel has 5 tabs: **Executive Summary**, **Technical Summary**, **Threat Assessment** (with `agreeing_providers` / `disagreeing_providers` pill lists and a numbered `supporting_evidence` list), **Relationships**, **Risk & Verdict**.

6. Step 5 — Evidence Ledger and "Show Receipts"

The Evidence tab (`EvidencePanel`, backed by `GET /api/v1/lookup/{id}/analysis/evidence`, permission `evidence:read`) is **deterministically built** by `app/evidence/builder.py` — "No AI involved in this endpoint." Treat it as the audit trail underneath every AI claim.

Every evidence record has an `evidence_type`, one of:

```
detection | reputation | relationship | malware_association |
threat_actor_association | campaign_association | mitre_technique |
infrastructure | other
```

Two sources populate the ledger:

1. **Per-provider records** — one per `ok` provider result: `detection` (if reputation was `unknown` / `no data`) or `reputation` (otherwise), plus one `other` record per `interesting_finding` the provider's AI summary surfaced.
2. **Per-correlation-edge records** — one per correlation edge, confidence = `edge.confidence × 100` (converted to the 0–100 evidence scale). `source_label` reads `"Correlation Engine (corroborated)"` when more than one provider asserted that edge, otherwise the single provider's ID, or plain `"Correlation Engine"` if provenance is empty.

"Show Receipts" links (`ShowReceiptsLink`) appear next to AI-generated claims throughout the UI. Clicking one scrolls to and highlights the exact evidence rows an AI answer cited. If a claim has no receipts, the link reads *"No supporting evidence cited"* in italics — treat any AI claim with zero receipts as unverified, not as fact.

The grounding mechanism (why you can trust a citation)

This is the core anti-hallucination control and is worth understanding precisely (`app/ai/analysis_service.py`):

1. Every AI explanation endpoint below is prompted with the real, numbered evidence ledger (`[id=<uuid>]` (`type`, `confidence=NN`) `label: claim`) and instructed: *"Every reason you give MUST cite the evidence_ids of the specific evidence records... NEVER invent an evidence_id that isn't in the ledger."*
2. After the AI responds, the backend recursively walks the validated response and **strips any `evidence_id` that does not exist in the real ledger** (`_strip_invalid_evidence_ids`) — a fabricated ID is silently removed before you ever see it, not flagged, just deleted from the citation list.
3. If the AI call fails outright, the endpoint returns a safe fallback object (e.g. `"AI explanation unavailable (generation error)."`) rather than propagating an error to you.

What this means in practice: if an AI answer cites `evidence_ids`, every one of those IDs is a real, clickable row in the ledger you can go inspect via Show Receipts. If an AI answer cites *zero* `evidence_ids`, either nothing in the ledger supported the claim, or the model's citations were all invalid and got stripped — either way, treat an uncited claim with the same skepticism you'd give an uncorroborated provider.

The same grounding pattern applies to MITRE ATT&CK technique mappings in the Final Assessment: each `mitre_mappings[]` entry has a `grounded` boolean, `true` only if that technique ID was explicitly surfaced by a provider's own data (e.g. a MITRE ATT&CK provider's `mitre_techniques` field); the UI shows an **"AI-inferred" badge** on any mapping where `grounded=false`, meaning it's the model's own inference, not a provider-observed fact.

7. Step 6 — Ask the AI cluster (all grounded, all on-demand)

The **Verdict Analysis** panel exposes six explicitly-triggered AI modes (none run automatically — you click a button):

Mode	Endpoint	What it answers
WHY?	POST .../analysis/why	Step-by-step reasons for the verdict, each citing <code>evidence_ids</code>
What is this?	POST .../analysis/what-is-this	Plain-language + technical explanation of what the IOC is
Score Explanation	POST .../analysis/score-explanation	Breaks the risk score into evidence-backed components
Intelligence Conflicts	POST .../analysis/disagreement	Where providers agree/disagree, and what's missing
False Positive Check	POST .../analysis/false-positive	Checks for shared/legitimate infra (CDN, cloud, NAT, VPN, scanner, crawler...) as a false-positive explanation
Challenge This Verdict	POST .../analysis/challenge	Red-teams the platform's own verdict: supporting vs. contradictory evidence, alternative benign explanation, and an honest <code>final_confidence</code> that may be <i>lower</i> than the original score implied

All require permission `analysis:generate` and a **completed** lookup (`409` if the lookup isn't `status=completed` yet). Each result is cached client-side per mode for the session and comes with Show Receipts wiring.

The **Investigation Copilot** (`POST .../analysis/copilot` , permission `copilot:query`) answers free-text questions grounded in this lookup's evidence ledger, correlation edges, and your running notes — it never requires re-pasting context. Suggested starters: *"Why is this suspicious?"*, *"What changed since last week?"*, *"Which provider has the strongest evidence?"*, *"Is this likely a false positive?"*, *"What should I investigate next?"*. It explicitly answers "only from the supplied context" and says so plainly if the context doesn't contain an answer, rather than guessing.

Separate from all of the above: the sidebar's "Ask AI (Gemini second opinion)" button is a **manual workflow**, not a platform AI call. It copies a client-built prompt to your clipboard and opens `gemini.google.com` in a popup for you to paste into with your own Google account. No backend endpoint is involved and no grounding/receipt mechanism applies to whatever Gemini says there — treat it as an unaffiliated second opinion, not part of this platform's evidence chain.

Two more AI features live inside the **Pivot Panel** (see next section) and are also click-triggered, not automatic: *"What should I do next?"* (`getNextActions`) and *"What don't we know?"* (`getIntelligenceGaps`).

8. Step 7 — Pivot, hunt, and escalate

Pivot Panel (deterministic — never AI)

`GET /api/v1/lookup/{id}/pivots` (permission `lookup:read`) is a **pure sort over real correlation edges**, deliberately not AI-generated so it "can never hallucinate a pivot target." It only surfaces edges that directly touch your seed IOC (one-hop only — deeper relationships belong in the graph view). Each row shows `ioc_value`, `ioc_type`, `relationship`, `corroborating_providers` count, `confidence` (0–100), and a `relevance` band:

- **high** — more than 1 corroborating provider, OR raw edge confidence ≥ 0.85
- **medium** — raw edge confidence ≥ 0.6
- **low** — everything else

Rows are sorted by corroborating-provider count first, confidence second. Click a row to pivot: it navigates to `/lookup/new?value=<pivot value>` and starts a fresh investigation.

Hunting Center

"Hunt This IOC" (`POST .../hunt` , permission `hunting:generate`) generates exact-match hunting queries plus grounded expansion targets/broader queries across formats: `sigma`, `splunk_spl`, `sentinel_ql`, `elastic`, `qradar_aql`, `chronicle_yara_l`, `suricata`, `snort`, `zeek` . "Create Detection" (`POST .../detection` , same permission) generates a single named detection rule (title, rule body, objective, data source, logic explanation, false-positive considerations, severity, and MITRE technique IDs) for a chosen format — the format selector here additionally offers `yara` .

Basket and Cases

- **IOC Basket** (`/basket`) is a **per-analyst private scratch space** (`basket:manage` permission) for collecting IOCs mid-investigation. "Investigate Selected" opens each item's lookup in a new tab; "Compare Selected" (needs ≥ 2 completed items) calls `POST /api/v1/basket/compare` for a side-by-side table (IOC, Verdict, Risk, ASN, Malware, Threat Actors) plus an AI narrative naming the `most_dangerous_ioc_value` .
- **Cases** (`/cases`) are **shared across the whole SOC team** — anyone with `case:read` sees every case, unlike the private basket. From a lookup page, "Add to Case" attaches the current IOC via `POST /api/v1/cases/{id}/iocs` . Case statuses: `open`, `investigating`, `contained`, `resolved`, `false_positive`, `closed` . Severities: `low`, `medium`, `high`, `critical` .

9. Export: what works and what does not

Export	Mechanism	Status
Markdown	Built entirely client-side from the already-fetched Final Assessment (executive/technical summary, threat assessment, verdict rationale, evidence, MITRE mappings, actions, IR recommendations, detection rules)	Works
JSON	<code>JSON.stringify()</code> of the in-browser assessment object, downloaded as a Blob	Works
PDF	Calls <code>POST /api/v1/lookup/{id}/export?format=pdf</code>	NOT IMPLEMENTED — no such backend route exists anywhere in <code>backend/app/api/routes/</code> . The button will always fail and the UI shows <i>"Export format not yet available."</i>
CSV	Calls <code>POST /api/v1/lookup/{id}/export?format=csv</code>	NOT IMPLEMENTED — same missing route, same fallback message

There is also a `lookup:export` permission string defined in the role table for `admin / analyst`, but it is never checked by any route — it is configured but currently unused, presumably reserved for the missing export endpoint. **Do not rely on server-side PDF/CSV export for reporting today** — use Markdown or JSON, or copy content out of the UI manually.

10. Quick reference: permissions you'll hit as an analyst

Action	Permission required
Start a lookup	<code>lookup:create</code>
View a lookup / list lookups / pivots / provider health	<code>lookup:read</code>
View evidence ledger	<code>evidence:read</code>
Run any "Ask AI" explanation (WHY?, Challenge, etc.) or Copilot	<code>analysis:generate</code> / <code>copilot:query</code>
Generate hunting queries / detection rules	<code>hunting:generate</code>
Manage your basket	<code>basket:manage</code>
Read / create / edit / close cases	<code>case:read</code> / <code>case:create</code> / <code>case:write</code> / <code>case:close</code>

`viewer` role has only `lookup:read`, `evidence:read`, `case:read` — a viewer can read everything above but cannot start lookups, run AI explanations, hunt, or touch the basket/cases. See [ARCHITECTURE.md](#) for role/permission internals.

11. Things that do not exist (so you stop looking for them)

- No dedicated Threat Actor, Malware Family, Campaign, or Watchlist pages — this data only appears inline within a lookup's evidence/correlation data.
- No bulk-upload / batch-lookup page.
- No server-side PDF/CSV export (see [Section 9](#)).
- No Neo4j-backed graph traversal — the "Neo4j mirror" mentioned in some backend code comments was never implemented; the correlation graph you see is rebuilt from Postgres every time.
- The "Ask AI (Gemini second opinion)" button is not a platform feature in the same sense as everything else in this guide — it's a clipboard + popup convenience, with no grounding guarantee.

Threat Intelligence Guide

Threat Intelligence Guide: Evidence Quality & Trust Calibration

This is the analyst-facing reference for **how much to trust a piece of evidence** produced by the IOC Intelligence Platform. It covers the `EvidenceType` taxonomy, the exact confidence-scoring math (correlation-edge confidence and corroboration bonus, verbatim from source), how provider disagreement is surfaced, and precisely what `grounded: false` means on a MITRE ATT&CK mapping.

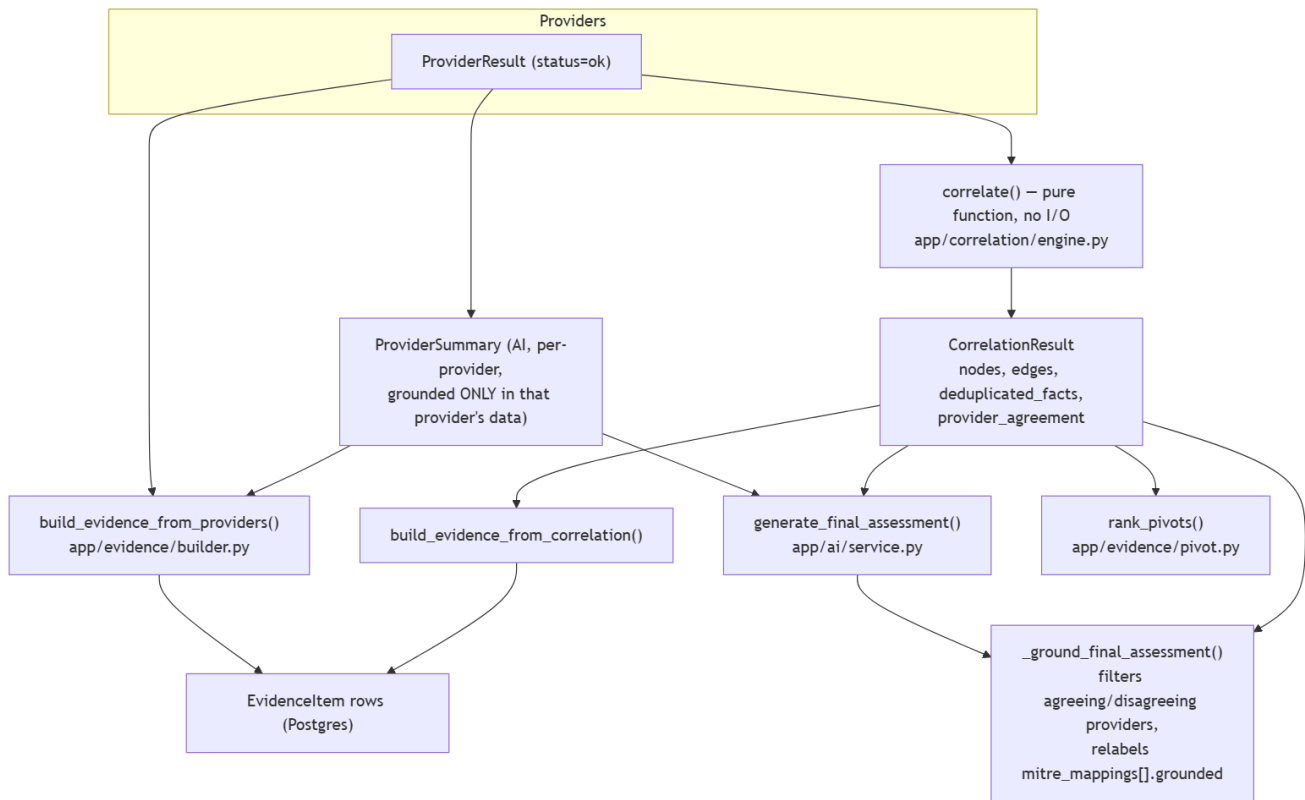
Source of truth: `backend/app/correlation/engine.py`, `backend/app/evidence/builder.py`, `backend/app/evidence/pivot.py`, `backend/app/models/evidence.py`, `backend/app/ai/service.py`, `backend/app/ai/schemas.py`, `backend/app/ai/analysis_service.py`.

See also: [PROVIDERS.md](#) for what each provider's `data` fields mean, [AI_ENGINE.md](#) for the two-step AI summarization/assessment flow this document's grounding logic feeds into, [ARCHITECTURE.md](#) for the end-to-end pipeline, [DATA_MODEL.md](#) for the `evidence_items` / `correlation_edges` table schemas, and [API_DOCUMENTATION.md](#) for the SSE event payloads referenced below.

1. Why this document exists

Every claim the platform shows an analyst — a reputation verdict, a "related to malware family X" link, a MITRE technique mapping — has a different evidentiary basis. Some claims come straight from a provider's raw response. Some are inferred by deterministic correlation logic across multiple providers. Some are generated by an LLM and then checked against real data before display. This document exists so an analyst can look at any evidence item, correlation edge, or MITRE mapping and know **exactly which of those three buckets it came from**, and how much corroboration backs it.

2. Evidence assembly pipeline



Two independent things are built from the same `CorrelationResult`:

1. **EvidenceItem rows** — persisted, analyst-visible facts with a 0–100 confidence score (`build_evidence()`, §3–§5 below).
2. **The final AI assessment's grounding pass** — `agreeing_providers`, `disagreeing_providers`, and `mitre_mappings[].grounded` are cross-checked against the correlation graph before the `final_assessment` SSE event is emitted (§6–§7 below).

Only providers whose `ProviderResult.status == "ok"` contribute anything to either path (`backend/app/correlation/engine.py:106`). A provider that returned `error`, `timeout`, `rate_limited`, `not_configured`, `unsupported_ioc`, or `no_data` contributes **zero** nodes, edges, deduplicated facts, or evidence — its silence is not evidence of anything, and the platform does not synthesize a placeholder claim for it.

3. The `EvidenceType` taxonomy

Enum: `EvidenceType` (`backend/app/models/evidence.py:19-28`), persisted on the `evidence_items` table's `evidence_type` column. All nine values are actually producible by `backend/app/evidence/builder.py` — there is no dead enum value.

Value	Produced by	Condition
<code>detection</code>	<code>build_evidence_from_providers()</code>	Per <code>ok</code> provider result with a matching <code>ProviderSummary</code> , when <code>summary.reputation</code> is "unknown" or "no data"
<code>reputation</code>	<code>build_evidence_from_providers()</code>	Same as above, when <code>summary.reputation</code> is anything else (<code>malicious</code> , <code>suspicious</code> , <code>clean</code>)
<code>other</code>	<code>build_evidence_from_providers()</code>	One record per entry in <code>summary.interesting_findings</code>
<code>malware_association</code>	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>malware_family</code>
<code>threat_actor_association</code>	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>threat_actor</code>
<code>campaign_association</code>	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>campaign</code>
<code>mitre_technique</code>	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is <code>mitre_technique</code>
<code>infrastructure</code>	<code>build_evidence_from_correlation()</code>	Correlation edge whose target type is a network type (<code>app/ioc/types.py</code> <code>NETWORK_TYPES</code>) or <code>tls_certificate</code> / <code>asn</code>
<code>relationship</code>	<code>build_evidence_from_correlation()</code>	Fallback for any correlation-edge target type not matched above (e.g. <code>cve</code>)

Dispatch order for the correlation-edge path (`_relationship_evidence_type()`, `backend/app/evidence/builder.py:51-62`) is checked **in this exact sequence** — `malware` → `threat actor` → `campaign` → `MITRE technique` → `infrastructure` → `relationship` fallback. A target type could in principle match more than one bucket; the first match in this order wins.

`EvidenceRecord` fields

`backend/app/evidence/builder.py:33-48` — mirrors the `EvidenceItem` table minus `lookup_id` (assigned once the parent `IOCLookup` row exists):

```
evidence_type, source_label, provider_id, claim, interpretation, confidence,
related_ioc_type, related_ioc_value, source_url, observed_at, raw_data
```

Claim text by source

Path	claim format	source_label
Provider (<code>detection</code> / <code>reputation</code>)	"{provider_name} reports reputation={reputation}, threat_level={threat_level} ({detection_status})"	provider's own name
Correlation edge	"{source_value} {relationship.replace('_', ' ')} {target_value}"	"Correlation Engine (corroborated)" if more than one provider id appears in the edge's provenance, else the single provider id, else "Correlation Engine" if provenance is empty

On a correlation-edge evidence record, `provider_id` is set only when the edge's provenance contains **exactly one** provider ID; if it was corroborated by more than one provider, `provider_id` is left `None` (the attribution lives in `source_label` / `raw_data.provenance` instead) (`backend/app/evidence/builder.py:129`).

`build_evidence(provider_results, provider_summaries, correlation)` (`backend/app/evidence/builder.py:142-147`) is simply the concatenation of the two functions above — no other evidence is generated anywhere else in the codebase.

Coverage gap to know about: the provider path only fires when a `ProviderSummary` exists for that `provider_id` in `summaries_by_id`; if no matching summary was found (e.g. its persisted `AISummaryRecord` row failed to re-validate on reload), that provider's `ok` result is **silently skipped** — no evidence record, no error surfaced (`backend/app/evidence/builder.py:77-79`).

4. Confidence scale — two systems, one final number

The platform runs two different confidence calculations that ultimately share the same 0–100 display scale as `RiskAssessment` (`app/ai/schemas.py`) "so evidence and risk numbers are directly comparable in the UI" (`backend/app/evidence/builder.py:1-10`).

4a. Provider-summary confidence (qualitative → 0–100)

`ProviderSummary.confidence` is a qualitative `low` / `medium` / `high` label set by the AI when it summarizes a single provider's raw data. The evidence builder converts it via a fixed lookup table — `_CONFIDENCE_LEVEL_TO_SCORE` (`backend/app/evidence/builder.py:21`):

```
_CONFIDENCE_LEVEL_TO_SCORE = {"low": 30.0, "medium": 60.0, "high": 90.0}
```

An unrecognized or missing confidence level defaults to `50.0` (`builder.py:81`).

4b. Correlation-edge confidence (0–1 internal → ×100 display)

Correlation edges carry an internal `confidence: float` on a **0–1** scale (`GraphEdge`, `backend/app/correlation/engine.py:33-39`). When an edge becomes an `EvidenceRecord`, its confidence is rescaled: `confidence = round(edge.confidence * 100, 1)` (`backend/app/evidence/builder.py:132`). This 0–1 → 0–100 edge-confidence model is described in detail in §5.

5. Correlation engine confidence model (verbatim)

Base confidence per field

`_FIELD_BASE_CONFIDENCE` (`backend/app/correlation/engine.py:77-90`) — the starting confidence for a correlation edge, keyed by which `ProviderResult.data` field produced it:

data field	Correlation relationship	Base confidence (0–1)
<code>resolved_ips</code>	<code>resolves_to</code> → <code>IOType.IPV4</code>	0.9
<code>resolved_domains</code>	<code>resolves_to</code> → <code>IOType.DOMAIN</code>	0.9
<code>asn</code>	<code>belongs_to_asn</code> → <code>IOType.ASN</code>	0.9
<code>certificates</code>	<code>serves_certificate</code> → <code>IOType.TLS_CERTIFICATE</code>	0.85
<code>related_urls</code>	<code>hosts</code> → <code>IOType.URL</code>	0.8
<code>related_hashes</code>	<code>delivers</code> → <code>IOType.SHA256</code>	0.8
<code>cves</code>	<code>exploits</code> → <code>IOType.CVE</code>	0.8
<code>mitre_techniques</code>	<code>uses_technique</code> → <code>IOType.MITRE_TECHNIQUE</code>	0.75
<code>related_domains</code>	<code>related_to</code> → <code>IOType.DOMAIN</code>	0.7
<code>malware_families</code>	<code>associated_with</code> → <code>IOType.MALWARE_FAMILY</code>	0.5
<code>threat_actors</code>	<code>attributed_to</code> → <code>IOType.THREAT_ACTOR</code>	0.5
<code>campaigns</code>	<code>part_of_campaign</code> → <code>IOType.CAMPAIGN</code>	0.5

Read the table as a trust ranking: DNS/network-topology facts (resolution, ASN, certificates) start at 0.7–0.9 because they’re closer to raw technical fact; attribution-style facts (malware family, threat actor, campaign) start at only 0.5 because naming conventions and attribution are inherently fuzzier across providers.

Fallback rule for any field not in the table above (`backend/app/correlation/engine.py:125-127`):

```
base_confidence = _FIELD_BASE_CONFIDENCE.get(
    field_name,
    1.0 if result.category.value == "threat_intel" else 0.7,
)
```

i.e. an unmapped field from a `threat_intel`-category provider defaults to full confidence (1.0); from any other category, 0.7.

Corroboration bonus

```
_CORROBORATION_BONUS_PER_PROVIDER = 0.15
```

(`backend/app/correlation/engine.py:94`) — “Per additional distinct provider corroborating the exact same (source, target, relationship) edge, beyond the first.”

Merge logic when two providers assert the **same** (source, target, relationship) edge (`backend/app/correlation/engine.py:154-175`):

- 1. `base_confidences[dedup_key] = max(existing_base, new_edge.confidence)` — the base is the **maximum**, not the sum, of the base confidences contributed by each provider for that field.
- 2. `boosted = min(1.0, base + 0.15 * (len(distinct_providers) - 1))` — capped at 1.0.
- 3. `provenance` strings are concatenated with commas (`existing.provenance + "," + edge.provenance`).

Worked example: three distinct providers each report the same IOC is `associated_with` the same malware family (base confidence 0.5 each, from the table above):

```
boosted = min(1.0, 0.5 + 0.15 * (3 - 1)) = min(1.0, 0.80) = 0.80
```

That edge, when written to `EvidenceItem.confidence`, becomes `round(0.80 * 100, 1) = 80.0`.

Known double-counting caveat: the merge step has *no* uniqueness check on `provenance` itself — only the confidence-bonus provider set is deduplicated. If a single provider emits the same edge via two different `data` fields, its provider ID can appear twice in the comma-joined `provenance` string. `rank_pivots()`'s `corroborating_providers` count (§8) is computed as `len([p for p in edge.provenance.split(",") if p])` — a **raw split-and-count with no deduplication** — so it can be inflated relative to the true number of distinct corroborating providers in this edge case.

6. How provider disagreement is surfaced

`provider_agreement` bucketing

Inside `correlate()` (`backend/app/correlation/engine.py:140-142`), for every `ok` provider result:

```
verdict = result.data.get("verdict") or result.data.get("reputation")
if verdict:
    provider_agreement.setdefault(str(verdict).lower(), []).append(result.provider_id)
```

`verdict` is checked before `reputation`. The result is a dict like `{"malicious": ["virustotal", "abuseipdb"], "clean": ["otx"]}` — one bucket per distinct verdict string, each holding the provider IDs that reported it.

This dict is **not** streamed to the frontend directly. The SSE `correlation` event (see [API DOCUMENTATION.md](#)) only carries `nodes` and `edges` — the `provider_agreement` dict (and `deduplicated_facts`) are consumed internally as prompt context for `generate_final_assessment()`.

`agreeing_providers` / `disagreeing_providers`

The AI populates `FinalAssessment.agreeing_providers` and `.disagreeing_providers` (`backend/app/ai/schemas.py`) from that same context. Before the `final_assessment` SSE event is emitted, `_ground_final_assessment()` (`backend/app/ai/service.py:122-150`) filters both lists:

- Builds `real_provider_ids` = the union of every provider ID appearing in `correlation.edges`' provenance strings, and every provider ID appearing anywhere in `correlation.provider_agreement`'s value lists.
- Filters `assessment.agreeing_providers` / `.disagreeing_providers` to only the IDs present in `real_provider_ids`. **Exception:** if `real_provider_ids` is empty, the model's claimed list is passed through unfiltered (nothing real to check it against).

So a provider name appearing in `disagreeing_providers` on the final assessment is guaranteed to be a provider ID that actually contributed a correlation edge or a verdict/reputation value for this lookup — it cannot be a hallucinated provider name (as long as at least one real provider contributed *something* to the correlation graph).

Verdict/risk consistency is enforced by validation failure, not repair

`FinalAssessment` has a `model_validator` (`_verdict_must_agree_with_risk`, `backend/app/ai/schemas.py:161-182`, `mode="after"`):

- `final_verdict` in `{malicious, highly_malicious}` but `risk.malicious_probability < 30` → raises.
- `final_verdict` in `{benign, likely_benign}` but `risk.malicious_probability > 50` → raises.

This raise is caught by the generic `except Exception` in `generate_final_assessment()` (`backend/app/ai/service.py:243`) — **the schema validation failure itself is the enforcement mechanism**, not a repair step. A contradictory verdict/risk pair from the AI does not get fixed up; the entire assessment falls back to a degraded `FinalAssessment` stating generation failed, with `final_verdict: "unknown"`, all risk fields zeroed, and `verdict_rationale` citing the exception. There is no second attempt (see [AI_ENGINE.md](#) for the broader no-retry behavior of the AI clients).

7. MITRE ATT&CK mapping: what `grounded: false` actually means

`MitreMapping` (`backend/app/ai/schemas.py:61-92`) fields:

```
technique_id, technique_name, tactic (Literal of 14 ATT&CK tactic slugs),
kill_chain_stage (optional), rationale, grounded: bool = True
```

The AI is prompted to set `grounded=false` on a mapping it inferred itself (e.g. "this reputation pattern is consistent with T1071") rather than one a provider explicitly surfaced. **In practice this initial model-set value is irrelevant** — `_ground_final_assessment()` (`backend/app/ai/service.py:122-150`) always overwrites it:

- Builds `grounded_technique_ids` from every correlation edge whose `relationship == "uses_technique"`, taking `edge.target.split(":", 1)[1].upper()` (the technique ID half of the `mitre_technique:<id>` node id).
- For every mapping in `assessment.mitre_mappings`:

```
mapping.grounded = mapping.technique_id.upper() in grounded_technique_ids
```

What this means for an analyst reading a MITRE mapping:

grounded	Meaning
true	This exact technique ID appears as a <code>uses_technique</code> correlation edge target for this IOC — i.e. some provider's raw data actually reported this technique (e.g. OTX pulse tags, MITRE-tagged threat-intel fields), not just the AI's own inference.
false	The AI proposed this technique mapping, but no provider result for this lookup produced a matching <code>uses_technique</code> edge. It may still be analytically reasonable (the AI is instructed never to fabricate outright), but it is unverified against real provider data for this specific lookup.

Grounding **relabels, it does not remove**. An ungrounded mapping is never stripped from `mitre_mappings` — it is still returned to the frontend, just flagged. Treat `grounded: false` mappings as analyst-review items, not as confirmed technique usage.

Why `technique_id` has no format validation (and why that's safe)

`backend/app/ai/schemas.py:62-71` deliberately omits a `pattern=` regex constraint on `technique_id`. Reason (verbatim intent): Ollama's grammar compiler (`llama.cpp json-schema-to-grammar`) fails to compile a `pattern` constraint into a decoding grammar and returns HTTP 400 for the **entire** `generate_final_assessment()` call, not just the offending field — a single unsupported constraint anywhere in the schema breaks every AI backend routed through Ollama. This is judged safe to skip because grounding (above) already checks `technique_id` by **exact membership** against real correlation-graph technique IDs, not by format — a malformed ID simply fails the membership check and is marked `grounded: false`; it was never a security-relevant validation gap. Cloud backends (Claude/Gemini/Bedrock) don't compile schemas into decoding grammars, so the field's description alone still guides them.

8. Grounding beyond MITRE: evidence IDs and expansion targets

The same "never let the AI invent something that isn't in the real ledger" pattern recurs in two other AI-generated artifacts derived from evidence:

- **AI explanations** (`backend/app/ai/analysis_service.py`) — functions like `explain_why_malicious`, `explain_disagreement`, `assess_false_positive`, `challenge_verdict`, etc. are prompted with the real `EvidenceItem` ledger (`_evidence_block()`) and asked to cite `evidence_ids`. After the AI responds, `_strip_invalid_evidence_ids()` (`backend/app/ai/analysis_service.py:55-71`) recursively walks the validated response and filters every field literally named `evidence_ids` down to IDs that actually exist in the real ledger (`real_ids = {str(item.id) for item in evidence}`) — an invented evidence ID is silently dropped, never shown to the analyst as if real.
- **Hunting packages** (`backend/app/ai/hunting_service.py`) — `generate_hunting_package()`'s `expansion_targets` are grounded differently: filtered to only targets whose `related_ioc_value` (case-insensitive) matches a real node value in the correlation graph (`real_values = {node.value.lower() for node in correlation.nodes}`). This module does **not** use `_strip_invalid_evidence_ids` — it has no evidence-ledger dependency, only the correlation graph. `generate_detection_rule()` has no grounding step at all (single seed IOC, nothing to invent against).

Both mechanisms fail closed: on any exception, the AI backend returns the caller's pre-defined fallback object rather than partially-grounded output.

9. Pivot relevance banding (uses the same corroboration signal)

`rank_pivots()` (`backend/app/evidence/pivot.py:22-52`) powers `GET /api/v1/lookup/{id}/pivots` and is explicitly **not** AI-generated — "a pure sort over real correlation edges can never hallucinate a pivot target." It reuses the same provenance/confidence signals described above:

```
provider_count = len([p for p in edge.provenance.split(",") if p]) # see double-counting caveat, §5
relevance = (
    "high" if provider_count > 1 or edge.confidence >= 0.85
    else "medium" if edge.confidence >= 0.6
    else "low"
)
```

Note the thresholds (`0.85`, `0.6`) apply to the raw **0–1** `edge.confidence`, even though the pivot dict's own `confidence` output field is the `x100` display value (`round(edge.confidence * 100, 1)`). Results are sorted by (`corroborating_providers`, `confidence`) descending, and only edges directly touching the seed IOC are surfaced (no multi-hop pivots). See [SOC_ANALYST_GUIDE.md](#) for the analyst workflow this feeds.

10. Trust-calibration checklist for analysts

- **No evidence ≠ clean**. A provider with status `not_configured`, `error`, `timeout`, or `unsupported_ioc` contributes nothing — it is absent from evidence, correlation, and provider agreement entirely. Only `no_data` (and DNSBL "not listed") are genuine negative findings.
- **detection / reputation evidence requires both an ok result and a matching AI summary**. If the AI summary row for a provider couldn't be reloaded/re-validated, that provider's evidence is silently absent — check `provider_results` directly (via `GET /api/v1/lookup/{id}`) if a provider you expected to see is missing from the evidence list.
- **Attribution-style edges (malware family, threat actor, campaign) start at 0.5 base confidence** — lower than network/DNS facts (0.7–0.9) — reflecting that naming/attribution is less standardized across providers than resolution data.
- **Corroboration only compounds, it never multiplies risk**. Three providers agreeing on a malware-family link caps out well below the ceiling unless enough distinct providers stack the `+0.15` bonus; `min(1.0, ...)` prevents runaway confidence from many providers repeating one fact.

- `agreeing_providers` / `disagreeing_providers` **are grounded**, but grounding only checks that the provider ID is *real* for this lookup, not that the AI's characterization of what it agreed or disagreed on is itself correct — read `verdict_rationale` and the correlation edges directly for the underlying facts.
 - `grounded: false` **on a MITRE mapping is a flag for review, not an error** — the mapping is still shown; it means no provider explicitly surfaced that technique for this IOC.
 - **The correlation SSE event's nodes / edges are not persisted verbatim.** `GET /api/v1/lookup/{id}` does not return correlation nodes/edges at all; only the derived `CorrelationEdgeRecord` rows (flat DB rows, used to rebuild a `CorrelationResult` for `pivots` / `analysis` / `hunting` routes) and `EvidenceItem` rows persist long-term.
-

11. NOT IMPLEMENTED

- **Neo4j graph mirroring.** `engine.py`'s module docstring and the `CorrelationEdgeRecord` model docstring both describe correlation edges as "mirrored into Neo4j for graph traversal," and `neo4j_uri` / `neo4j_user` / `neo4j_password` settings exist (`backend/app/core/config.py:31-33`). There is no Neo4j driver import, session, or write/query call anywhere in the codebase. Postgres (`correlation_edges` table) is the **sole** persisted store for correlation edges today.
 - **`GraphNode.labels`.** The `GraphNode` dataclass has a `labels: list[str]` field (`backend/app/correlation/engine.py:30`), but no code path in `correlate()` ever populates it — every node built has `labels: []`. Do not expect node labels in any correlation output.
 - **Per-provider proactive rate limiting.** `ProviderStatus.RATE_LIMITED` exists and is reachable (HTTP 429/403/509), but no connector self-throttles in advance — all retry/timeout logic lives solely in the orchestrator (see [AI_ENGINE.md](#) / [PROVIDERS.md](#)). Not an evidence-quality gap per se, but relevant context: a `rate_limited` result contributes no evidence, same as any other non-`ok` status.
-

Glossary

See [GLOSSARY.md](#) for platform-wide term definitions. Terms specific to this document: **base confidence**, **corroboration bonus**, **grounding**, **provider agreement**, **dedup key** (`(source, target, relationship)` tuple used to merge duplicate correlation edges).

IOC Types

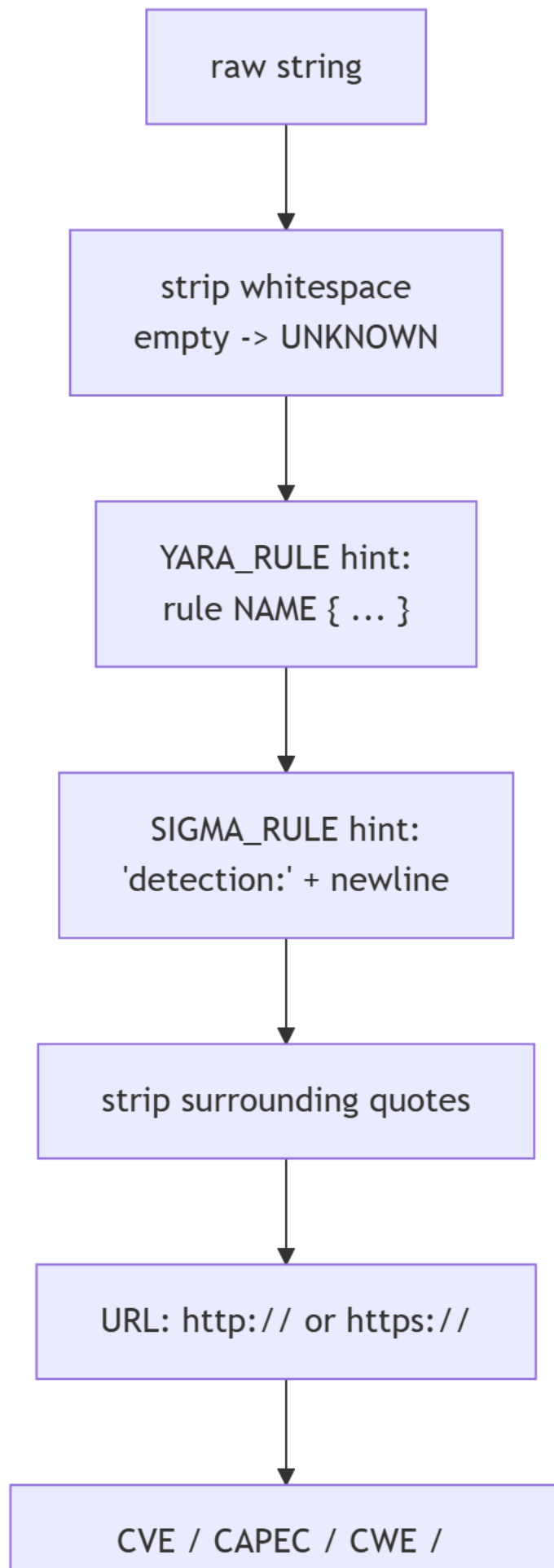
IOC Types

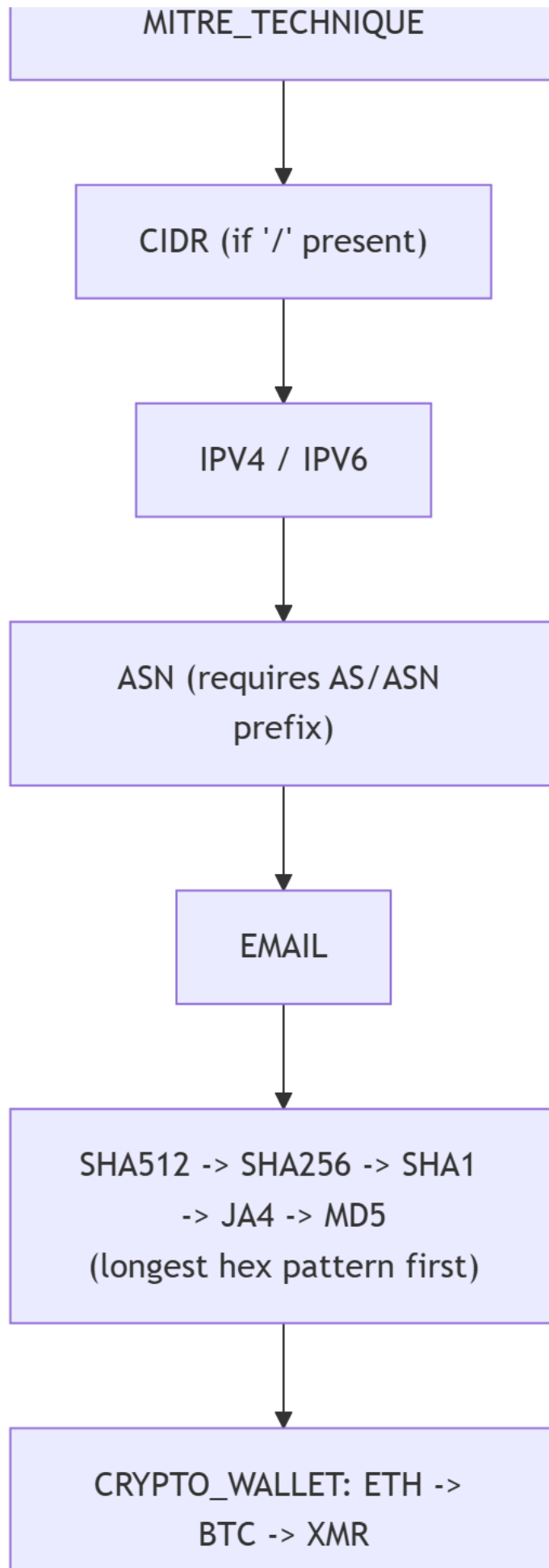
This document is the authoritative reference for every `IOCType` enum value in the platform: its wire format, how (and whether) `detect_ioc_type()` can actually produce it, a safe example value, and which provider connectors accept it.

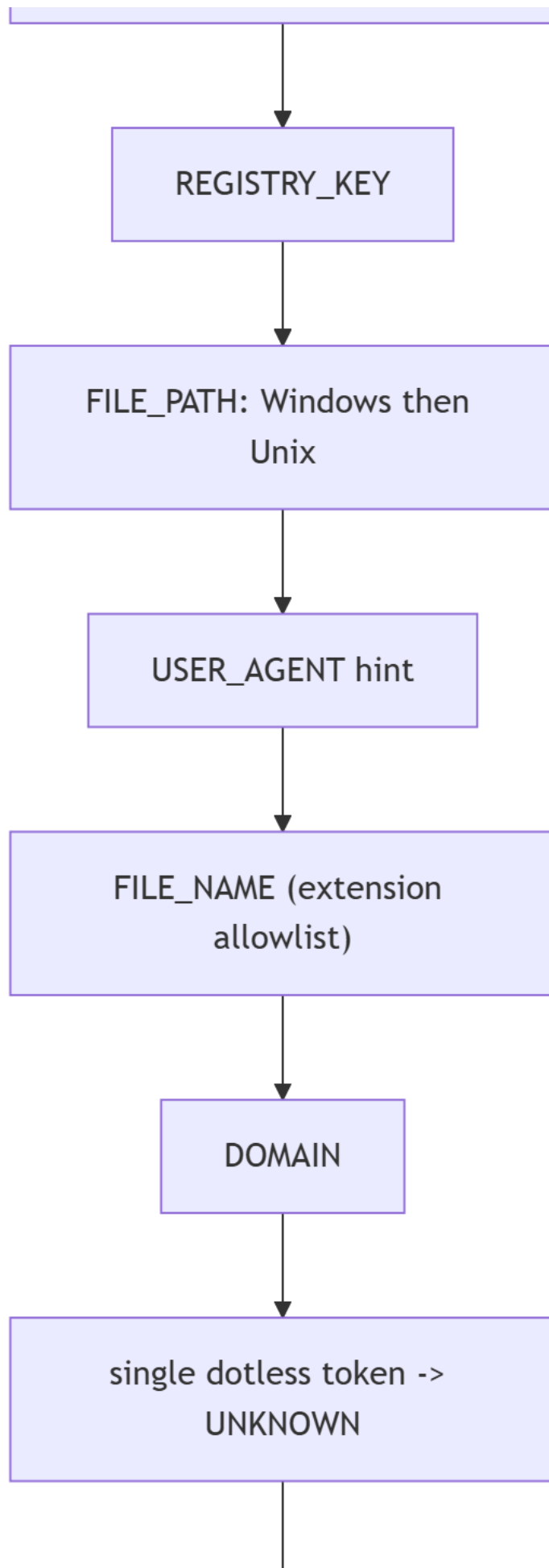
Source of truth: `backend/app/ioc/types.py` (enum + type-groups) and `backend/app/ioc/detector.py` (`detect_ioc_type()`), the sole classification entry point). See also [PROVIDERS.md](#) for full connector details and [ARCHITECTURE.md](#) for how detection fits into the lookup pipeline.

How classification works

`detect_ioc_type(raw: str) -> IOCType` (`backend/app/ioc/detector.py:61`) is the **only** place IOC type is inferred from a raw string. There is no client-side detection: the frontend (`frontend/app/lookup/new/page.tsx:112`) simply displays the `ioc_type` the backend already resolved via the SSE `detected` event.







UNKNOWN (final fallback)

Design intent (module docstring, `detector.py:1-5`): "Order matters -- more specific patterns (hashes, CVEs, IPs) are checked before generic fallbacks (hostname/file_name) so ambiguous strings resolve correctly."

Callers and override

Both IOC-submission endpoints run the same pattern:

```
ioc_type = payload.ioc_type_hint or detect_ioc_type(ioc_value)
if ioc_type == IOCType.UNKNOWN:
    raise HTTPException(status_code=422, detail="Could not determine IOC type; pass ioc_type_hint.")
```

Endpoint	File	Permission	Request field
POST /lookup/stream	backend/app/api/routes/lookup.py:45-74	lookup:create	LookupCreateRequest.ioc_type_hint: Optional[IOCType] (backend/app/schemas/lookup.py:11)
POST /basket	backend/app/api/routes/basket.py:45-54	basket:manage	BasketAddRequest.ioc_type_hint: Optional[IOCType] (backend/app/schemas/basket.py:10)

`ioc_type_hint` is a generic enum override -- pass any `IOCType` value to force classification (e.g. to disambiguate MD5 vs. JA3, or to force HOSTNAME instead of DOMAIN). There is **no** dedicated "sandbox context" or "TLS context" hint parameter despite a code comment implying one; it is **NOT IMPLEMENTED**.

Type groups

backend/app/ioc/types.py :

```
HASH_TYPES = {IOCType.MD5, IOCType.SHA1, IOCType.SHA256, IOCType.SHA512}

NETWORK_TYPES = {
    IOCType.IPV4, IOCType.IPV6, IOCType.DOMAIN, IOCType.URL,
    IOCType.HOSTNAME, IOCType.ASN, IOCType.CIDR,
}
```

`HASH_TYPES` is used by connectors that accept any hash algorithm (e.g. MalwareBazaar, VirusTotal, OTX). `NETWORK_TYPES` seeds the correlation graph (see [ARCHITECTURE.md](#)).

Reference table (all 33 enum values)

Legend for **Detectable?**: Yes = `detect_ioc_type()` can return this value from an unhinted raw string. No (hint/other subsystem only) = the value only ever appears via `ioc_type_hint`, correlation edges, or other code that isn't the detector.

#	Enum value	Wire value	Detectable?	Detection position
1	IPV4	ipv4	Yes	Step 10 (<code>ipaddress.ip_address</code>)
2	IPV6	ipv6	Yes	Step 10 (<code>ipaddress.ip_address</code>)
3	DOMAIN	domain	Yes	Step 26 (<code>_DOMAIN_RE</code>)
4	URL	url	Yes	Step 4 (<code>http(s)://</code> prefix)
5	HOSTNAME	hostname	No (hint/other subsystem only)	Never returned by the detector
6	EMAIL	email	Yes	Step 12 (<code>_EMAIL_RE</code>)
7	MD5	md5	Yes	Step 17 (<code>_MD5_RE</code> , default for 32-hex)
8	SHA1	sha1	Yes	Step 15 (<code>_SHA1_RE</code>)
9	SHA256	sha256	Yes	Step 14 (<code>_SHA256_RE</code>)
10	SHA512	sha512	Yes	Step 13 (<code>_SHA512_RE</code> , checked first among hashes)
11	TLS_CERTIFICATE	tls_certificate	No (hint/other subsystem only)	Never returned by the detector
12	JA3	ja3	No (hint/other subsystem only)	Regex defined but unused -- unreachable
13	JA4	ja4	Yes	Step 16 (<code>_JA4_RE</code>)
14	ASN	asn	Yes	Step 11 (<code>_ASN_RE</code>)
15	CIDR	cidr	Yes	Step 9 (<code>ipaddress.ip_network</code>)
16	MALWARE_FAMILY	malware_family	No (hint/other subsystem only)	Never returned by the detector
17	THREAT_ACTOR	threat_actor	No (hint/other subsystem only)	Never returned by the detector
18	CAMPAIGN	campaign	No (hint/other subsystem only)	Never returned by the detector
19	CVE	cve	Yes	Step 5 (<code>_CVE_RE</code>)
20	CWE	cwe	Yes	Step 7 (<code>_CWE_RE</code>)
21	CAPEC	capec	Yes	Step 6 (<code>_CAPEC_RE</code> , checked before CWE)
22	MITRE_TECHNIQUE	mitre_technique	Yes	Step 8 (<code>_MITRE_TECHNIQUE_RE</code>)
23	FILE_NAME	file_name	Yes	Step 25 (extension allowlist)
24	REGISTRY_KEY	registry_key	Yes	Step 21 (<code>_REGISTRY_KEY_RE</code>)
25	PROCESS_NAME	process_name	No (dead code)	Step 27 guard is unreachable
26	MUTEX	mutex	No (dead code)	Regex defined but unused -- unreachable
27	WINDOWS_SERVICE	windows_service	No (not implemented)	No regex/logic exists at all
28	FILE_PATH	file_path	Yes	Steps 22-23 (Windows/Unix path regexes)
29	USER_AGENT	user_agent	Yes	Step 24 (<code>_USER_AGENT_HINT_RE</code>)
30	CRYPTO_WALLET	crypto_wallet	Yes	Steps 18-20 (ETH, BTC, XMR regexes -- all collapse to this one value)
31	YARA_RULE	yara_rule	Yes	Step 1 (<code>_YARA_HINT_RE</code> , checked first, multiline)
32	SIGMA_RULE	sigma_rule	Yes	Step 2 (<code>_SIGMA_HINT_RE</code> , checked second, multiline)
33	UNKNOWN	unknown	Yes (fallback)	Final fallback / empty input

Table numbering above uses the enum's declaration order (33 members, `backend/app/ioc/types.py:6-38`); "Detection position" step numbers reference the sequence documented in the next section, which does not map 1:1 to enum order since `YARA_RULE` / `SIGMA_RULE` are checked first regardless of declaration position.

Detailed entries

Each entry gives: format/regex, detection notes (including documented historical bugs, called out explicitly), a safe example, and providers whose `supported_types` include it.

IPV4 / IPV6

- **Format:** any string `ipaddress.ip_address()` accepts.
- **Detection:** step 10, after CIDR is ruled out. `ip.version == 6` selects IPV6, else IPV4.
- **Example:** `8.8.8.8` (IPV4), `2001:4860:4860::8888` (IPV6, both are Google DNS, safe to publish).

- **Providers:** AbuseIPDB, OTX, ThreatFox, URLhaus, VirusTotal, WHOIS/RDAP, Censys (stub), Spamhaus (stub, IPV4 only), Internet Intelligence Collector (IPV4 only -- IPV6 excluded from crawler search per `backend/app/crawler/collector.py:36-44`).

DOMAIN

- **Format:** `_DOMAIN_RE = r'^(?=[1,253}$)(?:[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.)+[a-zA-Z]{2,63}$'`.
- **Detection:** step 26, only reached if the string contains a `.`, contains no space, and did not match the `FILE_NAME` extension allowlist first (see `FILE_NAME` below for why order matters -- `shell.php` must resolve to `FILE_NAME`, not `DOMAIN`).
- **No public-suffix-list (PSL) check exists.** The detector cannot distinguish a registrable domain from a hostname with a subdomain; both collapse to `DOMAIN` by design (comment, `detector.py:144-147`).
- **Example:** `example.com`
- **Providers:** OTX, ThreatFox, URLhaus, VirusTotal, WHOIS/RDAP, crt.sh, Spamhaus (stub), Internet Intelligence Collector.

URL

- **Format:** any string whose lowercased form starts with `http://` or `https://`.
- **Detection:** step 4 -- checked immediately after quote-stripping and before every other regex, so URLs never get misclassified as domains/paths.
- **Example:** `http://example.com/path`
- **Providers:** ThreatFox, URLhaus, VirusTotal, PhishTank (stub).

HOSTNAME -- NOT IMPLEMENTED via detector

- **Format:** same dotted-label shape as `DOMAIN`.
- **Detection:** `detect_ioc_type()` never returns `HOSTNAME` -- there is no PSL, so every bare dotted string resolves to `DOMAIN` (see `DOMAIN` above). `HOSTNAME` is only reachable by passing `ioc_type_hint=hostname` explicitly.
- **Used elsewhere:** OTX provider maps it to OTX's own "hostname" indicator type (`backend/app/providers/otx.py:20,29,141`) and it is included in `NETWORK_TYPES` (`types.py:44-53`) for correlation-graph seeding.
- **Example:** `internal-host.example.com` (only usable via explicit hint).
- **Providers:** OTX (accepts it if hinted).

EMAIL

- **Format:** `_EMAIL_RE = r'^[^\s]+@[^\s]+\.[^\s]+$'`.
- **Detection:** step 12, checked after ASN and before the hash regexes.
- **Example:** `analyst@example.com`
- **Providers:** none of the reviewed connectors declare `EMAIL` in `supported_types`.

MD5 / SHA1 / SHA256 / SHA512

- **Format:** hex strings of exactly 32 / 40 / 64 / 128 characters, case-insensitive.
- **Detection order (longest-first):** SHA512 (step 13) -> SHA256 (14) -> SHA1 (15) -> JA4 (16) -> MD5 (17). The module comment explains this is intentional: *"more specific patterns (hashes, CVEs, IPs) are checked before generic fallbacks"* -- and among same-shaped hex strings, the longest, most specific pattern wins first.
- **Historical edge case ("fixed"), ASN-vs-hash ordering:** a hex hash that happens to start with the letter `a` (e.g. `a11111111111111111111111111111111`, 33 chars) must **not** be misclassified as an ASN just because it superficially looks like `A<digits>`. The `_ASN_RE` pattern requires an explicit `AS / ASN` prefix precisely to prevent this (`detector.py:23-26`), and this exact case is pinned by `backend/app/tests/unit/test_ioc_detector.py:62-64`, which asserts the string resolves to `MD5`.
- **32-hex ambiguity (MD5 vs JA3 vs mutex):** MD5, JA3 fingerprints, and 32-character mutex-like tokens are all indistinguishable 32-hex-character strings. The detector **defaults to MD5** for all of them (`detector.py:114-117`). The code comment claims "callers with sandbox/TLS context can override via hint," but no such JA3-specific hint mechanism exists anywhere in the reviewed API schemas -- `ioc_type_hint` is a generic override, not an automatic disambiguator. See JA3 entry below.
- **Examples:** MD5 `826f75224ddb4979721c1240f5b423b5`; SHA1 `7053eba4b69b1898302218aae3a3a49982b319e4`; SHA256 `71e88b019795fb624c5382163cba1e6bcea14d90713c6aedc2f9359d6a31af15`; SHA512 `c52153a0a10bb13dfc184d8229a6f7b0d906279fa66a95eabf5c269dff6481f5d0ca52a673c4a254d00cc89c4f166b4b44d4a15853168a8a310ba9e64a70df58` (all fabricated/placeholder hex, not real malware hashes).
- **Providers (all via HASH_TYPES):** MalwareBazaar (any hash), VirusTotal (any hash), OTX (any hash). ThreatFox declares `MD5` and `SHA256` explicitly (not the full `HASH_TYPES` set). Hybrid Analysis (stub) declares **SHA256 -only**, not the full hash set -- called out explicitly in `backend/app/providers/stubs/hybrid_analysis.py:10`.

TLS_CERTIFICATE -- NOT IMPLEMENTED via detector

- **Detection:** never returned by `detect_ioc_type()`. Produced only as a correlation-graph edge type (`backend/app/correlation/engine.py:63`) when `crt.sh` returns certificate data.
- **Example:** n/a (not a user-submittable classification without a hint).
- **Providers:** `crt.sh` declares `TLS_CERTIFICATE` in `supported_types` alongside `DOMAIN` (`backend/app/providers/crtsh.py:30`).

JA3 -- NOT IMPLEMENTED

- **Format:** identical 32-hex-character shape to MD5 (`_JA3_RE` , `detector.py:15`).
- **Detection:** the regex is defined but **never referenced** anywhere else in `detector.py` -- `detect_ioc_type()` can never return `JA3` ; any 32-hex string always resolves to `MD5` instead (see MD5 above). This is an explicit gap, not a hint-driven feature: no JA3-specific override mechanism exists in the reviewed code.
- **Example:** n/a -- not reachable without a hint, and no code path sets the hint to JA3.
- **Providers:** none declare `JA3` in `supported_types` .

JA4

- **Format:** `_JA4_RE = r'^[a-z0-9]{10}_[a-fA-F0-9]{12}_[a-fA-F0-9]{12}$'` (case-insensitive). The 10-character prefix encodes: protocol(1) + version(2) + SNI(1) + cipher_count(2) + ext_count(2) + ALPN(2).
- **Historical bug ("fixed"), prefix length:** the JA4 prefix is correctly 10 characters, not 8. A prior version of this regex apparently implied an 8-char prefix; this is now pinned by `backend/app/tests/unit/test_ioc_detector.py:60-61` with the inline comment "*JA4 fingerprint (10-char prefix, not 8)*", asserting `q13i0207h3_55b375c5d22e_cd85d2d88918` resolves to `JA4` .
- **Detection order:** step 16, checked **before** MD5, because JA4's underscore-delimited shape is distinct enough to disambiguate from the 32-hex MD5/JA3 pattern once checked first.
- **Example:** `q13i0207h3_55b375c5d22e_cd85d2d88918`
- **Providers:** none of the reviewed connectors declare `JA4` in `supported_types` .

ASN

- **Format:** `_ASN_RE = r'^(?:AS|ASN)\s?(\d+)$'` (case-insensitive) -- requires an explicit `AS` or `ASN` prefix (optionally followed by a space) then digits.
- **Detection:** step 11. See the MD5 entry above for the paired historical bug fix -- this regex's explicit-prefix requirement is exactly what prevents hex hashes starting with `a` / `A` from being misread as an ASN.
- **Alternate spellings accepted:** `AS15169` , `ASN15169` , `AS 15169` all match (confirmed by `test_ioc_detector.py` 's `test_detect_ioc_type_asn_alternate_spelling`).
- **Example:** `AS15169` (Google's real public ASN, safe to reference).
- **Providers:** WHOIS/RDAP.

CIDR

- **Format:** any string containing `/` that `ipaddress.ip_network(value, strict=False)` accepts.
- **Detection:** step 9 -- only attempted if `/` is present in the stripped string; a `ValueError` is silently swallowed and falls through to later checks (so a non-network string containing `/` , e.g. a Unix path, is not derailed).
- **Example:** `192.168.0.0/24`
- **Providers:** none of the reviewed connectors declare `CIDR` in `supported_types` (it is used for correlation-graph seeding via `NETWORK_TYPES` , not direct provider lookups).

MALWARE_FAMILY / THREAT_ACTOR / CAMPAIGN -- NOT IMPLEMENTED via detector

- **Detection:** none of these three are ever returned by `detect_ioc_type()` . Free-text names for malware families, threat actors, or campaigns are lexically indistinguishable from each other and from process/mutex/service names, so the detector's single-token fallback (step 27) deliberately punts to `UNKNOWN` rather than guessing (comment, `detector.py:151-155`).
- **Produced elsewhere:** correlation-graph edges (`backend/app/correlation/engine.py:65-67`) and the crawler's supported-search-type allowlist (`backend/app/crawler/collector.py:39-41` , `backend/app/workers/tasks.py:41-43`).
- **Example:** `Emotet` (malware family), `APT29` (threat actor), `SolarWinds Compromise` (campaign) -- all only usable via `ioc_type_hint` .
- **Providers:** Internet Intelligence Collector (`internet_intelligence`) declares all three in `supported_types` (`backend/app/crawler/collector.py:36-44`) since free-text search on names is its actual use case, unlike raw network atoms.

CVE

- **Format:** `_CVE_RE = r'^CVE-\d{4}-\d{4,7}$'` (case-insensitive).
- **Detection:** step 5, checked early alongside CWE/CAPEC/MITRE_TECHNIQUE, before IP/hash logic.
- **Example:** `CVE-2021-44228` (the real, publicly known Log4Shell CVE ID -- safe to reference).
- **Providers:** NVD, CISA KEV. Internet Intelligence Collector also declares `CVE` in `supported_types` .

CWE

- **Format:** `_CWE_RE = r'^CWE-\d{1,5}$'` (case-insensitive).
- **Detection:** step 7 -- checked **after** CAPEC (see CAPEC entry for why ordering matters here).
- **Example:** `CWE-79` (real, public "Cross-site Scripting" weakness ID).
- **Providers:** none of the reviewed connectors declare `CWE` in `supported_types` .

CAPEC

- **Format:** `_CAPEC_RE = r'^CAPEC-\d{1,5}$'` (case-insensitive).
- **Detection:** step 6 -- checked **before** `CWE` (`detector.py:80-83`). Both prefixes are textually distinct (`CAPEC-` vs `CWE-`) so this ordering has no practical ambiguity today, but the code checks CAPEC first regardless.
- **Example:** `CAPEC-66` (real, public "SQL Injection" attack pattern ID).
- **Providers:** none of the reviewed connectors declare `CAPEC` in `supported_types`.

MITRE_TECHNIQUE

- **Format:** `_MITRE_TECHNIQUE_RE = r'^T\d{4}(\.\d{3})?$',` (case-insensitive) -- matches both a bare technique (`T1059`) and a sub-technique (`T1059.001`).
- **Detection:** step 8, checked immediately after `CWE`.
- **Example:** `T1059` or `T1059.001` (real, public ATT&CK technique IDs for "Command and Scripting Interpreter").
- **Providers:** MITRE ATT&CK provider (`mitre_attack`) declares this as its sole supported type (`backend/app/providers/mitre_attack.py:74`).

FILE_NAME

- **Format:** the string contains a `.`, and the substring after the final `.` (lowercased) is in the `_FILE_NAME_EXTENSIONS` frozenset.
- **Detection:** step 25 -- checked **before** the `DOMAIN` regex specifically because the domain pattern alone cannot distinguish `shell.php` from `shell.pl` (a real ccTLD); extension match wins so filenames are not misread as domains (comment, `detector.py:46-49`).
- **Full extension allowlist (49 entries, detector.py:50-58):** `exe, dll, sys, scr, bat, ps1, vbs, js, jar, bin, elf, apk, docm, xlsx, doc, docx, xls, xlsx, ppt, pptx, pdf, txt, rtf, csv, log, ini, cfg, conf, php, php3, php4, php5, phtml, asp, aspx, jsp, jsp, cgi, zip, rar, 7z, tar, gz, iso, msi, dmg, dat, tmp, bak, md, py, json, xml, yaml, yml`.
- **Notably absent from the allowlist:** `pl, sh, rb, go, c, cpp, html, css, ico, png, jpg`. A string like `shell.pl` therefore does **not** hit `FILE_NAME` and instead falls through to the `DOMAIN` regex (since `.pl` is a real ccTLD).
- **Examples pinned by unit tests:** `malware.exe, shell.php, invoice.doc, readme.txt, cmd.aspx, notes.md` -- all resolve to `FILE_NAME`.
- **Providers:** Internet Intelligence Collector declares `FILE_NAME` in `supported_types`.

REGISTRY_KEY

- **Format:** `_REGISTRY_KEY_RE = r'^(HKLM|HKCU|HKCR|HKU|HKCC|HKEY_[A-Z_]+)\'` (case-insensitive).
- **Detection:** step 21, checked after crypto-wallet patterns and before file-path patterns.
- **Example:** `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`
- **Providers:** none of the reviewed connectors declare `REGISTRY_KEY` in `supported_types`.

PROCESS_NAME -- NOT IMPLEMENTED (dead code)

- **Format:** intended to catch single dotless tokens ending in a Windows executable-like extension pattern, per the code at `detector.py:156-157`.
- **Detection: unreachable.** The enclosing guard at `detector.py:150` requires `"." not in stripped`, but the inner check at line 156 searches for a literal `.exe / .dll / .sys` suffix -- which by definition requires a `.` to be present. These two conditions contradict each other, so the `PROCESS_NAME` branch can never execute; every single dotless token falls through to `UNKNOWN` instead.
- **Example:** n/a -- not reachable without a hint.
- **Providers:** none.

MUTEX -- NOT IMPLEMENTED

- **Format:** a regex exists, `_MUTEX_HINT_RE = r'^(Global\\|Local\\)?[A-Za-z0-9\-\.\.]{2,}$'` (`detector.py:31`), but it is **never referenced anywhere else** in the module.
- **Detection:** `detect_ioc_type()` cannot return `MUTEX` under any input.
- **Example:** n/a -- not reachable without a hint (e.g. `Global\SomeMutexName` would need `ioc_type_hint=mutex` set explicitly).
- **Providers:** none.

WINDOWS_SERVICE -- NOT IMPLEMENTED

- **Detection:** no regex and no logic anywhere in `detector.py` references this enum value at all. It exists only as an enum member (`types.py:32`).
- **Example:** n/a.
- **Providers:** none.

FILE_PATH

- **Format (Windows):** `_WINDOWS_PATH_RE = r'^[a-zA-Z]:\\^\\\\'` (drive letter or UNC path).
- **Format (Unix):** `_UNIX_PATH_RE = r'^[\\w\-. /]+$'`, **and** the final `/` -delimited segment must not contain a `.` (otherwise it's treated as a dotted filename and falls through to `FILE_NAME / DOMAIN` checks instead).
- **Detection order:** steps 22-23, Windows path checked before Unix path.
- **Dead code called out:** the inner extension sniff at `detector.py:130` (`re.search(r'\.(exe|dll|sys|scr|bat|ps1)$', ...)`) has **no behavioral effect** -- both the `if` branch and the unconditional fallback return the identical `IOCType.FILE_PATH`. It looks like it should route differently based on extension but does not.

- **Example:** `C:\Windows\System32\evil.exe` (Windows), `/usr/local/bin/dropper` (Unix, no dotted final segment).
- **Providers:** none of the reviewed connectors declare `FILE_PATH` in `supported_types`.

USER_AGENT

- **Format:** `_USER_AGENT_HINT_RE` searches (not anchors) for any of `Mozilla/`, `AppleWebKit/`, `Gecko/`, `Chrome/`, `Safari/`, `Edg/`, `curl/`, `python-requests/` (case-insensitive).
- **Detection:** step 24, checked before `FILE_NAME/DOMAIN` so a UA string isn't misread as a dotted hostname.
- **Example:** `Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36`
- **Providers:** none of the reviewed connectors declare `USER_AGENT` in `supported_types`.

CRYPTO_WALLET

- **Format:** three distinct address shapes all collapse to this **one** enum value (there are no separate BTC/ETH/XMR enum members):
 - ETH: `_ETH_RE = r'^0x[a-fA-F0-9]{40}$'` -- checked first (step 18).
 - BTC: `_BTC_RE = r'^(bc1|[13])[a-zA-HJ-NP-Z0-9]{25,62}$'` -- checked second (step 19).
 - XMR: `_XMR_RE = r'^4[0-9AB][1-9A-HJ-NP-Za-km-z]{93}$'` -- checked third (step 20).
- **Example:** BTC `1BvBMSEYstWetqTFn5Au4m4GFg7xJaNVN2`; ETH `0xa1b2c3d4e5f60718293a4b5c6d7e8f9012345678` (fabricated placeholder, not a real wallet).
- **Providers:** none of the reviewed connectors declare `CRYPTO_WALLET` in `supported_types`.

YARA_RULE

- **Format:** `_YARA_HINT_RE = r'\brule\s+\w\s*\{'` (case-insensitive), matched with `.search()` against the **raw, unstripped-of-quotes** value.
- **Detection:** step 1 -- checked before every other rule, including before quote-stripping, because it is a multiline structural hint rather than a single-token pattern.
- **Example:**

```
rule ExampleRule {
    condition: true
}
```

- **Providers:** none of the reviewed connectors declare `YARA_RULE` in `supported_types`.

SIGMA_RULE

- **Format:** `_SIGMA_HINT_RE = r'\bdetection:\s*\n'` (case-insensitive), matched with `.search()`.
- **Detection:** step 2 -- checked immediately after `YARA_RULE`, also before quote-stripping.
- **Example:**

```
title: Example Sigma Rule
detection:
  selection:
    EventID: 4688
```

- **Providers:** none of the reviewed connectors declare `SIGMA_RULE` in `supported_types`.

UNKNOWN

- **Format:** `n/a` -- the catch-all.
- **Detection:** returned for empty/whitespace-only input (immediately, before any regex runs) and as the final fallback (step 33) for any string that matches none of the above, including single-token dotless strings (malware family names, process names, mutex names, service names -- all "lexically indistinguishable," per the code comment).
- **API behavior:** both `POST /lookup/stream` and `POST /basket` reject `UNKNOWN` with `HTTPException(422, "Could not determine IOC type; pass ioc_type_hint.")` -- an analyst must supply `ioc_type_hint` to submit an otherwise-unclassifiable value.
- **Providers:** `n/a` -- lookups never reach provider fan-out with this type.

Quick example: forcing a hint

Since `detect_ioc_type()` cannot classify names like malware families, threat actors, campaigns, or `MUTEX/WINDOWS_SERVICE/PROCESS_NAME/JA3/HOSTNAME` strings, submit them with an explicit `ioc_type_hint`:

```
curl -X POST http://localhost:8000/lookup/stream \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <your-token-here>" \
-d '{"value": "Emotet", "ioc_type_hint": "malware_family"}'
```

Without the hint, `detect_ioc_type("Emotet")` falls through every regex to the single-token fallback (step 27) and returns `UNKNOWN`, which the endpoint rejects with HTTP 422.

Testing

Unit tests pinning classification behavior (including both documented bug fixes) live at `backend/app/tests/unit/test_ioc_detector.py`. See [TESTING.md](#) for how to run the suite.

See also

- [PROVIDERS.md](#) -- per-connector `supported_types`, configuration, and stub status.
- [ARCHITECTURE.md](#) -- how detected IOC type flows through the lookup pipeline, provider orchestrator, and correlation engine.
- [API_DOCUMENTATION.md](#) -- full request/response schemas for `/lookup/stream` and `/basket`.
- [DATA_MODEL.md](#) -- how `ioc_type` is persisted on `IOCLookup` / `BasketItem`.

Admin Guide

Admin Guide

Practical, mechanics-only reference for the three things an operator actually does on this platform: assign/change a user's role, enable a provider connector, and check provider health. This platform has **no admin web UI** — every task below is either a `.env` + Docker Compose operation, a direct database edit, or a REST call. Where a capability looks like it should exist (user management, audit log, provider configuration UI) but doesn't, that's called out explicitly as **NOT IMPLEMENTED** rather than guessed at.

Related docs: [SECURITY.md](#) (full auth/RBAC model), [CONFIGURATION.md](#) (every environment variable), [PROVIDERS.md](#) (per-connector behavior), [DEPLOYMENT.md](#) (container topology), [DATA_MODEL.md](#) (`users` table schema), [API_DOCUMENTATION.md](#) (endpoint reference).

1. Users and roles

1.1 How accounts are created

There is no seed admin account and no invite/admin-created-user flow. Every account is created by the same public endpoint:

```
curl -X POST http://localhost:8000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"analyst@example.com","password":"<configure securely>","full_name":"Jane Analyst"}'
```

`POST /api/v1/auth/register` (`backend/app/api/routes/auth.py:20-39`): the **first user ever inserted into the `users` table** (checked via `SELECT` returning no rows) is automatically assigned `Role.ADMIN`. Every registration after that defaults to `Role.ANALYST`. There is no flag, invite code, or config setting to request a different role at signup time — role assignment at registration is entirely determined by whether the table was empty.

1.2 Roles and permissions

Three roles exist (`backend/app/models/user.py`): `admin`, `analyst`, `viewer`. `ROLE_PERMISSIONS` is a static in-code dict — there is no per-user permission override and no custom-role support.

Permission	admin	analyst	viewer
lookup:create	✓	✓	✗
lookup:read	✓	✓	✓
lookup:export	✓	✓	✗
evidence:read	✓	✓	✓
analysis:generate	✓	✓	✗
hunting:generate	✓	✓	✗
copilot:query	✓	✓	✗
basket:manage	✓	✓	✗
case:create / case:write / case:close	✓	✓	✗
case:read	✓	✓	✓
provider:manage	✓	✗	✗
user:manage	✓	✗	✗
audit:read	✓	✗	✗

NOT IMPLEMENTED: `provider:manage`, `user:manage`, and `audit:read` are defined in `ROLE_PERMISSIONS` and are enforceable by `require_permission()`, but **no route in `backend/app/api/routes/` calls `require_permission()` with any of these three strings**. There is no in-app provider-configuration screen, no user-management endpoint, and no audit-log endpoint. Being "admin" today only matters for the routes that actually gate on the other 12 shared permissions — admin and analyst have identical route access in practice, aside from whatever you build yourself against these three unused permission strings.

1.3 Changing a user's role — no UI, no API; direct DB edit only

There is no admin panel, no `PATCH /users/{id}`, and no CLI command to promote or demote a user. The only way to change `role` or deactivate an account (`is_active`) after registration is to edit the `users` row directly. Two equivalent ways to do that:

Option A — Python/ORM, run inside the running `backend` container (recommended: it serializes the `Role` enum correctly for you and reuses the app's own session factory, `app/core/db.py`'s `new_session()`):

```

docker compose exec backend python -c "
import asyncio
from sqlalchemy import select
from app.core.db import new_session
from app.models.user import User, Role

async def main():
    async with new_session() as db:
        result = await db.execute(select(User).where(User.email == 'analyst@example.com'))
        user = result.scalar_one()
        user.role = Role.ADMIN # or Role.ANALYST / Role.VIEWER
        await db.commit()
        print(f'{user.email} is now {user.role.value}')

asyncio.run(main())
"

```

Option B — raw SQL against Postgres directly. The `postgres` service in `docker-compose.yml` runs with `POSTGRES_USER=ioc`, `POSTGRES_PASSWORD=ioc`, `POSTGRES_DB=ioc_intel`:

```

docker compose exec postgres psql -U ioc -d ioc_intel \
-c "UPDATE users SET role = 'ADMIN' WHERE email = 'analyst@example.com';"

```

Enum-label gotcha: the `role` column is a native Postgres enum (`CREATE TYPE role AS ENUM ('ADMIN', 'ANALYST', 'VIEWER')`), from `backend/alembic/versions/660d2aa3bc20_initial_schema.py`). SQLAlchemy stored the Python enum **member names** (`ADMIN / ANALYST / VIEWER` , uppercase) as the column's allowed labels — not the lowercase `.value` strings (`admin / analyst / viewer`) that appear in JWTs and API responses. Raw SQL against this column **must use the uppercase label** (`'ADMIN'`), or Postgres will reject it as an invalid enum value. The ORM approach in Option A avoids this entirely since `Role.ADMIN` serializes itself correctly either way.

Deactivating an account instead of changing its role uses the same mechanism, just a different column:

```

docker compose exec backend python -c "
import asyncio
from sqlalchemy import select
from app.core.db import new_session
from app.models.user import User

async def main():
    async with new_session() as db:
        result = await db.execute(select(User).where(User.email == 'analyst@example.com'))
        user = result.scalar_one()
        user.is_active = False
        await db.commit()

asyncio.run(main())
"

```

A deactivated user gets `403 Account disabled` on `/auth/login` and `401 Could not validate credentials` on every other authenticated route (checked in `get_current_user`, `backend/app/auth/rbac.py:28-47`) — their existing access/refresh tokens stop working immediately on the next call, since every authenticated request re-queries the `users` row.

There is no listing endpoint either — to find a user's row to edit, you need their email address (there's no `GET /users` to browse the table). If you don't already know it, query it directly:

```

docker compose exec postgres psql -U ioc -d ioc_intel \
-c "SELECT id, email, role, is_active, created_at FROM users ORDER BY created_at;"

```

1.4 What's NOT implemented for user management

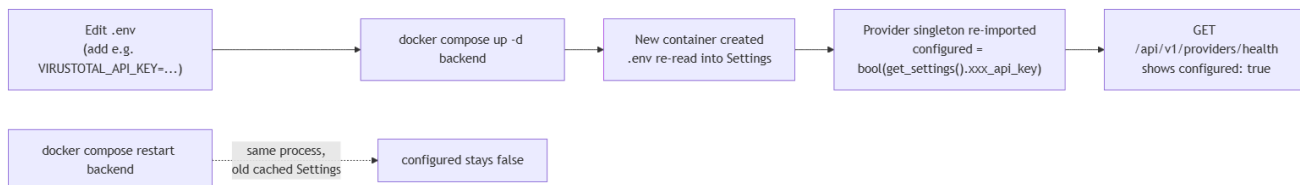
- No admin UI or API to list, promote, demote, deactivate, or delete users.
- No password-reset / forgot-password flow (no endpoint, no DB field, no frontend link).
- No email verification.
- No MFA / 2FA / TOTP.
- No account-lockout or brute-force throttling on `/auth/login` (only the unrelated per-user *lookup-creation* rate limit exists — see §3.4).
- No audit log of who changed what (`audit:read` permission exists but is unused).

2. Enabling a provider

Provider connectors read their credentials exclusively from `Settings` (`backend/app/core/config.py`), which loads from process environment variables, then `.env`, then hardcoded defaults. Enabling a provider is always: **add the key to `.env`, then recreate the `backend` container — do not just restart it.**

2.1 Why `up -d`, not `restart`

`get_settings()` is `@lru_cache`d — one `Settings` instance lives for the whole process lifetime. Each provider computes its own `configured` flag once, at import time, from that cached `Settings` instance (e.g. `configured = bool(get_settings().virustotal_api_key)` in `virustotal.py`). A running container never re-reads `.env`, and `docker compose restart backend` does not recreate the container or re-inject environment variables — only `docker compose up -d` does that when the service's config (including `.env`) has changed.



```
# 1. Edit .env in the repo root
#   VIRUSTOTAL_API_KEY=<your-key-here>

# 2. Recreate (not restart) the backend container
docker compose up -d backend

# 3. Confirm it picked up the key
curl -s http://localhost:8000/api/v1/providers/health | grep -A5 '"provider_id": "virustotal"'
```

2.2 Providers that need no key at all (always `configured: true`)

provider_id	Name	Category
crtsh	crt.sh	Certificate intel
nvd	NIST NVD	Vulnerability (works unauthenticated; key only raises the rate limit)
cisa_kev	CISA Known Exploited Vulnerabilities	Vulnerability
mitre_attack	MITRE ATT&CK	Threat intel (technique lookups)
whois_rdap	WHOIS/RDAP	WHOIS
spamhaus	Spamhaus DBL/ZEN	Threat intel (DNS-based, no HTTP call)
phishtank	PhishTank	Threat intel (works unauthenticated; key only raises the rate limit)

Nothing to do for these — they're already usable out of the box.

2.3 Providers that require a key to activate

provider_id	Name	Env var(s)	Notes
virustotal	VirusTotal	VIRUSTOTAL_API_KEY	Free-tier limits: 4 req/min, 500/day
abuseipdb	AbuseIPDB	ABUSEIPDB_API_KEY	Free-tier: 1000 checks/day
otx	AlienVault OTX	OTX_API_KEY	Free OTX account key
urlhaus	URLhaus	ABUSECH_AUTH_KEY	Shared abuse.ch key
threatfox	ThreatFox	ABUSECH_AUTH_KEY	Same key as URLhaus/MalwareBazaar
malwarebazaar	MalwareBazaar	ABUSECH_AUTH_KEY	Same key as URLhaus/ThreatFox
hybrid_analysis	Hybrid Analysis (Falcon Sandbox)	HYBRID_ANALYSIS_API_KEY	SHA256 hashes only — MD5/SHA1/URL support was dropped when the connector was rewritten around a deprecated endpoint
censys	Censys	CENSYS_PERSONAL_ACCESS_TOKEN and CENSYS_ORGANIZATION_ID	Both must be set — the connector treats the pair as one unit

`ABUSECH_AUTH_KEY` is a single shared credential across three connectors (URLhaus, ThreatFox, MalwareBazaar) — set it once to enable all three.

```
# .env
ABUSECH_AUTH_KEY=<your-abuse.ch-key>
HYBRID_ANALYSIS_API_KEY=<your-key-here>
CENSYS_PERSONAL_ACCESS_TOKEN=<your-token-here>
CENSYS_ORGANIZATION_ID=<your-org-id>
```

Then:

```
docker compose up -d backend
```

A 16th entry, `internet_intelligence` (the OSINT crawler wrapped as a provider, registered from `app.crawler`), also appears in `/providers/health`; its configuration surface was not verified in this pass — see [PROVIDERS.md](#) for anything further documented on it.

2.4 Full connector reference

For request/response shapes, verdict logic, and every quirk per connector (e.g. ThreatFox's "ok-with-zero-results is NO_DATA, not OK" override, Spamhaus's pure-DNS lookup, crt.sh's malformed-JSON tolerance), see [PROVIDERS.md](#).

3. Checking provider health

3.1 Endpoint

```
GET /api/v1/providers/health
```

Gated by `require_permission("lookup:read")` — **any authenticated user** can call this, not just admins, since all three roles (`admin`, `analyst`, `viewer`) hold `lookup:read`.

```
# Log in to get a token
TOKEN=$(curl -s -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"you@example.com","password":"<configure securely>"}' | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# Check provider health
curl -s http://localhost:8000/api/v1/providers/health \
  -H "Authorization: Bearer $TOKEN"
```

Response is a JSON array, one object per registered provider (`backend/app/providers/registry.py:53-64`):

```
[
  {
    "provider_id": "virustotal",
    "provider_name": "VirusTotal",
    "category": "threat_intel",
    "configured": false,
    "requires_key": true,
    "supported_types": ["domain", "ipv4", "ipv6", "md5", "sha1", "sha256", "sha512", "url"]
  }
]
```

Field	Meaning
<code>configured</code>	<code>true</code> if the required key(s)/credentials are set. This does not confirm the key is valid — only that a value is present. An invalid key still shows <code>configured: true</code> ; you'll only see the failure as a <code>status: "error"</code> on an actual lookup (see below).
<code>requires_key</code>	Whether this connector needs credentials at all (<code>false</code> for crt.sh, NVD, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, PhishTank).
<code>supported_types</code>	IOC types this connector can be queried with — feeds the "how many providers apply to this IOC" count shown in the UI.

This endpoint only reports configuration presence. It does not test connectivity to the upstream API. To confirm a key actually works, run a real lookup against an IOC type that provider supports and check that provider's entry in the lookup's per-provider results for `status: "ok"` rather than `"error"` or `"not_configured"`. `not_configured` specifically means `requires_key` is true and the key is still blank/missing at the process level (see §2.1 on why that can lag a `.env` edit).

3.2 Other health/monitoring endpoints

Endpoint	Auth	Purpose
GET /health	None	Liveness check: { "status": "ok", "service": "IOC Intelligence Platform" }
GET /metrics	None	Prometheus metrics via <code>prometheus-fastapi-instrumentator</code> . NOT IMPLEMENTED: no Prometheus/Grafana server is bundled to scrape or display this — bring your own.
GET /docs	None	FastAPI/Swagger interactive API docs

3.3 Other admin-tunable settings (env vars, all require `up -d`)

Variable	Default	Effect
PROVIDER_TIMEOUT_SECONDS	20	Per-attempt timeout the orchestrator applies to every provider call
PROVIDER_MAX_RETRIES	2	Retries on connect/read/pool timeouts only (not on 4xx/5xx)
PROVIDER_CACHE_TTL_SECONDS	3600	How long a successful (<code>status: ok</code>) provider result is cached in Redis before a fresh call is made for the same IOC

3.4 Lookup-creation rate limit

Distinct from provider health — this throttles how often *a user* can start a new lookup, enforced in `backend/app/api/routes/lookup.py` via the Redis-backed `RateLimiter` in `app/core/cache.py` (key `lookup_create:{user.id}`):

Variable	Default	Effect
LOOKUP_RATE_LIMIT_MAX_CALLS	10	Max lookup-creation calls per window, per user
LOOKUP_RATE_LIMIT_WINDOW_SECONDS	60	Window length in seconds

NOT IMPLEMENTED: no per-provider rate limiter exists — this Redis `RateLimiter` class is only ever instantiated for the lookup-creation endpoint, not inside any individual provider connector. Individual connectors rely solely on the orchestrator's shared timeout/retry policy and on upstream APIs' own 429/403/509 responses (mapped to `RATE_LIMITED`).

4. Security-relevant settings an admin should set before going beyond localhost

These are documented in full in [CONFIGURATION.md](#) and [SECURITY.md](#); the ones that matter operationally:

Variable	Default	Why it matters
JWT_SECRET_KEY	change-me-in-production	HMAC signing key for every access/refresh token. A leaked or default key lets anyone forge a valid token for any role, including admin. Must be changed before any non-local deployment.
ACCESS_TOKEN_EXPIRE_MINUTES	30	Access token lifetime
REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh token lifetime. There is no server-side revocation list — a valid, unexpired refresh token can mint new access tokens indefinitely even after a client-side logout, since logout only clears <code>localStorage</code> in the browser.
DEBUG	true	Controls CORS: <code>true</code> allows only <code>http://localhost:3000</code> ; <code>false</code> allows no origins (<code>app/main.py</code>) — there's no configurable CORS allow-list beyond this binary switch.

Changing any of these follows the same rule as §2.1: edit `.env`, then `docker compose up -d backend`.

5. Summary: what this guide is, and isn't

Task	Real mechanism
Create the first (admin) account	<code>POST /api/v1/auth/register</code> on an empty <code>users</code> table
Change a user's role	Direct DB edit (ORM script or <code>psql</code>) — no UI/API
Deactivate a user	Direct DB edit (<code>is_active = false</code>) — no UI/API
Enable a provider	Add key to <code>.env</code> , then <code>docker compose up -d backend</code>
Check provider health	<code>GET /api/v1/providers/health</code> (any logged-in role)
View an audit log	NOT IMPLEMENTED — <code>audit:read</code> permission exists, no route uses it
Manage providers via UI	NOT IMPLEMENTED — <code>provider:manage</code> permission exists, no route uses it

For anything this guide doesn't cover, check [DOCUMENTATION_GAPS.md](#) before assuming a feature is missing — it may simply not have been in scope for this document.

Configuration

Configuration Reference

Every setting the backend reads lives in exactly one place: `backend/app/core/config.py`, a `pydantic-settings` `Settings` class. It loads values from process environment variables first, falling back to a `.env` file in the repo root (`env_file=".env"`), and finally to the Python defaults shown below. There is no other settings module, no per-provider config file, and no database table of settings — this is the single source of truth every provider connector and the AI service reads from.

Related docs: [ARCHITECTURE.md](#) for how these settings are consumed by the request lifecycle, [INSTALL.md](#) for the step-by-step local setup that produces a working `.env`, and [API.md](#) for the endpoints these settings gate.

How settings are loaded

flowchart LR; A["Process environment"] -->|highest precedence| D["Settings instance"]; B["\".env file (repo root)\""] -->|env_file=\".env\"| D; C["Field default in config.py"] -->|fallback| D; D --> E["get_settings() -- lru_cache singleton"]; E --> F["Every provider / AI client / route reads from here"];

- **Env var naming:** `pydantic-settings` matches environment variable names to `Settings` fields **case-insensitively** by default (no `case_sensitive` override is set in `model_config`), but by convention every var in `.env.example`, `docker-compose.yml`, and the Kubernetes manifests is written in `UPPER_SNAKE_CASE` matching the field name (e.g. field `jwt_secret_key` ↔ env var `JWT_SECRET_KEY`). Use the uppercase form.
- **Unknown keys are ignored** (`extra="ignore"`), so extra vars in `.env` (e.g. `GITHUB_TOKEN`, read directly via `os.getenv` in `app/crawler/sources/github.py`, not through `Settings`) don't cause startup errors.
- **`get_settings()` is @lru_cache d** — one `Settings` instance per process lifetime. This is the mechanical reason restart semantics matter (see below): a running container never re-reads `.env`.

Restart semantics: `docker compose restart` **VS** `up -d`

`docker compose restart <service>` restarts the existing container process but does **not** re-read `.env` or `environment:` values — Compose only merges those in when a container is created. `docker compose up -d <service>` recreates the container (picking up new `.env` / `environment:` values) if its config changed, or is a no-op if nothing changed. **After editing `.env`, always use `docker compose up -d <service>`, never `restart`.** This is called out explicitly in [INSTALL.md](#)'s troubleshooting section for the "provider shows `not_configured`" case. Every table below marks which is required.

App

Variable	Purpose	Default	Required?	Restart-required
<code>APP_NAME</code>	Display name used in the FastAPI title and <code>/health</code> response.	IOC Intelligence Platform	No	<code>up -d</code>
<code>ENVIRONMENT</code>	Free-text environment label (<code>development</code> , <code>production</code> , ...). Not read for branching logic beyond documentation/observability today.	<code>development</code>	No	<code>up -d</code>
<code>DEBUG</code>	Controls CORS: when <code>true</code> , <code>allow_origins=["http://localhost:3000"]</code> ; when <code>false</code> , <code>allow_origins=[]</code> (see <code>app/main.py</code>). Set to <code>false</code> in production.	<code>true</code>	No	<code>up -d</code>
<code>API_V1_PREFIX</code>	URL prefix every router (<code>auth</code> , <code>lookup</code> , <code>providers</code> , <code>analysis</code> , <code>hunting</code> , <code>pivot</code> , <code>basket</code> , <code>cases</code>) is mounted under.	<code>/api/v1</code>	No	<code>up -d</code>

Auth / Security

Variable	Purpose	Default	Required?	Restart-required
JWT_SECRET_KEY	HMAC signing key for access/refresh JWTs (<code>app/auth/security.py</code>). Must be changed from the default before any non-local use — a leaked/default key lets an attacker forge tokens for any role.	<code>change-me-in-production</code>	Yes (production)	<code>up -d</code>
JWT_ALGORITHM	JWT signing algorithm passed to <code>python-jose</code> .	HS256	No	<code>up -d</code>
ACCESS_TOKEN_EXPIRE_MINUTES	Access token lifetime in minutes.	30	No	<code>up -d</code>
REFRESH_TOKEN_EXPIRE_DAYS	Refresh token lifetime in days.	7	No	<code>up -d</code>

See [ARCHITECTURE.md](#) and [API.md](#) for the role/permission model (`admin` , `analyst` , `viewer`) these tokens carry.

Datastores

Variable	Purpose	Default	Required?	Restart-required
DATABASE_URL	Async Postgres connection string (SQLAlchemy + <code>asyncpg</code>), system of record for users, lookups, provider results, AI summaries, correlation edges. <code>docker-compose.yml</code> overrides this per-service to <code>postgresql+asyncpg://ioc:ioc@postgres:5432/ioc_intel</code> .	<code>postgresql+asyncpg://ioc:ioc@postgres:5432/ioc_intel</code>	No (Docker Compose) / Yes (bare-metal)	<code>up -d</code>
REDIS_URL	Provider-result cache (TTL <code>provider_cache_ttl_seconds</code>) and fixed-window rate limiter backend (<code>app/core/cache.py</code>).	<code>redis://redis:6379/0</code>	No	<code>up -d</code>
NEO4J_URI	Bolt URI for the graph store intended for correlation-edge traversal. Declared in <code>Settings</code> and provisioned in <code>docker-compose.yml</code> /K8s manifests; no connector code currently opens a <code>neo4j</code> driver session against it — Postgres (<code>correlation_edges</code> table) is the durable source of truth today.	<code>bolt://neo4j:7687</code>	No	<code>up -d</code>
NEO4J_USER	Neo4j username.	<code>neo4j</code>	No	<code>up -d</code>
NEO4J_PASSWORD	Neo4j password — must match the <code>NEO4J_AUTH</code> value set on the <code>neo4j</code> service in <code>docker-compose.yml</code> (<code>neo4j/changeme-neo4j</code> by default).	<code>changeme-neo4j</code>	No	<code>up -d</code>
OPENSEARCH_URL	Endpoint for the full-text/log search datastore. Declared in <code>Settings</code> and provisioned as core infrastructure (<code>opensearch-py</code> is a dependency); no route or service currently issues queries against it.	<code>http://opensearch:9200</code>	No	<code>up -d</code>

Datastore roles are documented in more detail in [ARCHITECTURE.md](#).

Celery (broker/backend, also datastore-adjacent)

Variable	Purpose	Default	Required?	Restart-required
CELERY_BROKER_URL	Redis DB used as the Celery task broker.	<code>redis://redis:6379/1</code>	No	<code>up -d</code>
CELERY_RESULT_BACKEND	Redis DB used as the Celery result backend.	<code>redis://redis:6379/2</code>	No	<code>up -d</code>

Celery runs only the hourly OSINT crawl (`crawl-osint-sources-hourly`) and any future background/report jobs — the interactive lookup path does not use Celery (see [ARCHITECTURE.md](#)).

AI backends

`AI_BACKEND` selects one of four interchangeable clients, all exposing the same `call_claude_json()` method (`app/ai/service.py:_get_ai_client()`).

Variable	Purpose	Default	Required?	Restart-required
AI_BACKEND	Selects the active AI client: <code>ollama</code> (default), <code>bedrock</code> , <code>gemini</code> , or <code>anthropic</code> .	<code>ollama</code>	No	<code>up -d</code>

Ollama (default — local, no API key)

Variable	Purpose	Default	Required?	Restart-required
OLLAMA_BASE_URL	Base URL of a locally-hosted Ollama server. <code>host.docker.internal</code> resolves to the Docker host from inside the backend container (Docker Desktop Windows/Mac).	<code>http://host.docker.internal:11434</code>	Only if <code>AI_BACKEND=ollama</code>	<code>up -d</code>
OLLAMA_MODEL	Model tag to request (must match what <code>ollama list</code> shows after <code>ollama pull <model></code>).	<code>llama3.2:3b</code>	Only if <code>AI_BACKEND=ollama</code>	<code>up -d</code>
OLLAMA_MAX_TOKENS	<code>num_predict</code> cap passed to Ollama's <code>/api/chat</code> .	<code>8192</code>	No	<code>up -d</code>

AWS Bedrock (Claude)

Variable	Purpose	Default	Required?	Restart-required
BEDROCK_API_KEY	Bearer token from IAM → Security credentials → "Generate API key" (preferred auth). Set into <code>AWS_BEARER_TOKEN_BEDROCK</code> at client-init time by <code>app/ai/bedrock_client.py</code> .	None	Only if <code>AI_BACKEND=bedrock</code> and not using access-key auth	<code>up -d</code>
AWS_ACCESS_KEY_ID	Classic SigV4 fallback, only used if <code>BEDROCK_API_KEY</code> is blank.	None	Only if <code>AI_BACKEND=bedrock</code> and no <code>BEDROCK_API_KEY</code>	<code>up -d</code>
AWS_SECRET_ACCESS_KEY	Paired with <code>AWS_ACCESS_KEY_ID</code> .	None	Same as above	<code>up -d</code>
AWS_REGION	Region for the Bedrock Converse API call.	<code>us-east-1</code>	No	<code>up -d</code>
BEDROCK_MODEL_ID	Bedrock model ID or cross-region inference profile (e.g. the <code>global.</code> prefix is required for Claude Sonnet 4.5 from regions with no in-region/Geo profile, such as <code>ap-southeast-1</code>).	<code>global.anthropic.claude-sonnet-4-5-20250929-v1:0</code>	Only if <code>AI_BACKEND=bedrock</code>	<code>up -d</code>
BEDROCK_MAX_TOKENS	Max output tokens for the Converse API call.	<code>4096</code>	No	<code>up -d</code>

`AccessDeniedException` means the IAM principal is valid but hasn't been granted model access in **Bedrock** → **Model access** for that region — see [INSTALL.md](#).

Google Gemini

Variable	Purpose	Default	Required?	Restart-required
GEMINI_API_KEY	API key with quota for <code>generateContent</code> .	None	Only if <code>AI_BACKEND=gemini</code>	<code>up -d</code>
GEMINI_MODEL_ID	Gemini model ID.	<code>gemini-2.0-flash</code>	No	<code>up -d</code>
GEMINI_MAX_TOKENS	Max output tokens.	<code>8192</code>	No	<code>up -d</code>

Anthropic direct API

Variable	Purpose	Default	Required?	Restart-required
ANTHROPIC_API_KEY	<code>sk-ant-...</code> key with credit, sent to <code>api.anthropic.com/v1/messages</code> .	None	Only if <code>AI_BACKEND=anthropic</code>	<code>up -d</code>
ANTHROPIC_MODEL_ID	Anthropic model ID.	<code>claude-sonnet-4-5-20250929</code>	No	<code>up -d</code>
ANTHROPIC_MAX_TOKENS	Max output tokens.	<code>8192</code>	No	<code>up -d</code>

If a backend's `is_configured` check fails (missing key/URL), `_get_ai_client()` raises before making any HTTP call, degrading provider/final summaries to "AI summarization unavailable" rather than reaching the UI unvalidated — see [ARCHITECTURE.md](#).

Providers

Every free-tier connector computes its own `configured` flag from one of these settings in `__init__`; a provider whose key is unset returns `ProviderStatus.NOT_CONFIGURED` for every lookup rather than failing (see `app/providers/base.py`).

Variable	Purpose	Default	Required?	Restart-required
VIRUSTOTAL_API_KEY	VirusTotal public API v3 (free tier: 4 req/min, 500/day). Header <code>x-apikey</code> .	None	No (provider reports <code>not_configured</code> if unset)	up -d
ABUSEIPDB_API_KEY	AbuseIPDB v2 API (free tier: 1000 checks/day). Header <code>Key</code> .	None	No	up -d
OTX_API_KEY	AlienVault OTX. Header <code>X-OTX-API-KEY</code> .	None	No	up -d
NVD_API_KEY	NIST NVD CVE API v2.0. Works unauthenticated (~5 req/30s); setting this raises the rate limit (~50 req/30s) via the <code>apiKey</code> header.	None	No	up -d
ABUSECH_AUTH_KEY	Single abuse.ch Auth-Key shared across three connectors — URLHaus, ThreatFox, MalwareBazaar — sent via the <code>Auth-Key</code> header.	None	No	up -d

Stub / paid connectors

These implement the same `BaseProvider` interface as the free-tier connectors above; add the corresponding key(s) to activate.

Variable	Purpose	Default	Required?	Restart-required
HYBRID_ANALYSIS_API_KEY	Hybrid Analysis (CrowdStrike Falcon Sandbox), SHA256-only lookups via <code>GET /api/v2/overview/{sha256}/summary</code> . Header <code>api-key</code> .	None	No	up -d
CENSYS_PERSONAL_ACCESS_TOKEN	Censys Platform API Bearer token.	None	No — required together with <code>CENSYS_ORGANIZATION_ID</code> for <code>configured</code> to be <code>true</code>	up -d
CENSYS_ORGANIZATION_ID	Organization ID paired with the PAT above, sent as <code>X-Organization-ID</code> .	None	No — see above	up -d
PHISHTANK_API_KEY	Optional <code>app_key</code> for PhishTank's <code>checkurl</code> API — PhishTank works unauthenticated (<code>requires_key = False</code> , always <code>configured = True</code>); setting this only raises rate limits.	None	No	up -d

Two provider connectors need **no** key at all and are always `configured = True`: **Spamhaus** (DBL/ZEN DNSBL, queried via plain DNS, no HTTP/API key) and **NVD** (`requires_key = False` , key only raises rate limit). **crt.sh** (`configured = True`) and the **Internet Intelligence Collector** OSINT crawler (`requires_key = False`) are likewise always active — see Crawler below. Full connector inventory and how to add a new provider are documented in [ARCHITECTURE.md](#) and [PROVIDERS.md](#).

Not part of `Settings / .env`, mentioned for completeness: `GITHUB_TOKEN` is read directly via `os.getenv("GITHUB_TOKEN")` in `app/crawler/sources/github.py` (not through `Settings`, intentionally — the platform must work fully without it). Setting it raises the GitHub Search API's unauthenticated 10 req/min cap to 30 req/min.

Crawler / OSINT

Consumed by the OSINT crawler provider (`InternetIntelligenceCollector` in `app/crawler/collector.py`) and its four source modules (GitHub, Reddit, RSS security news, best-effort paste-dump search).

Variable	Purpose	Default	Required?	Restart-required
CRAWLER_USER_AGENT	<code>User-Agent</code> header sent by every crawler source request.	<code>IOC-Intel-Platform/1.0 (+https://github.com/your-org/ioc-intel-platform)</code>	No	up -d
CRAWLER_REQUEST_TIMEOUT_SECONDS	Per-request HTTP timeout for each crawler source.	15	No	up -d
CRAWLER_MAX_RESULTS_PER_SOURCE	Cap on findings kept per source before merge/dedupe; the collector's overall cap is this value × 4 sources.	5	No	up -d

Provider execution

Applies to every provider connector uniformly via the orchestrator (`app/providers/orchestrator.py`), not configurable per-provider.

Variable	Purpose	Default	Required?	Restart-required
PROVIDER_TIMEOUT_SECONDS	<code>asyncio.wait_for</code> timeout wrapped around each provider's <code>run()</code> call; exceeding it yields <code>ProviderStatus.TIMEOUT</code> .	20	No	up -d
PROVIDER_MAX_RETRIES	Extra attempts (via <code>tenacity</code> , exponential backoff) for connection-level failures (<code>ConnectError</code> , <code>ReadTimeout</code> , <code>PoolTimeout</code>) only — HTTP error responses are not retried.	2	No	up -d
PROVIDER_CACHE_TTL_SECONDS	TTL (seconds) for caching a provider's <code>OK</code> result in Redis, keyed by <code>provider_id</code> + <code>ioc_type</code> + a SHA-256 digest of the IOC value.	3600	No	up -d

Rate limiting

Per-user fixed-window limiter on the lookup-creation endpoint (`POST /api/v1/lookup/stream`) only, enforced via Redis so it holds across all backend workers (`app/core/cache.py:RateLimiter` , keyed `rate_limit:lookup_create:<user_id>`).

Variable	Purpose	Default	Required?	Restart-required
LOOKUP_RATE_LIMIT_MAX_CALLS	Max lookup-stream requests allowed per user per window.	10	No	up -d
LOOKUP_RATE_LIMIT_WINDOW_SECONDS	Window length in seconds.	60	No	up -d

Exceeding the limit returns `HTTP 429` with a message stating the exact `max_calls / window_seconds` in effect.

Not part of `Settings` (other configuration surfaces)

These are real, working configuration values in the repo, but they are set outside `backend/app/core/config.py` and therefore outside `.env` — included here so this document is a complete map of “how to configure the platform,” not just of the `Settings` class.

Variable	Where it's set	Purpose
NEXT_PUBLIC_API_URL	<code>docker-compose.yml</code> (<code>frontend</code> service <code>environment:</code>), baked into the Next.js build	Base URL the frontend's <code>lib/api.ts</code> calls for REST/SSE. Change and rebuild the frontend container if the backend isn't reachable at <code>http://localhost:8000</code> from the browser.
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	<code>docker-compose.yml</code> (<code>postgres</code> service <code>environment:</code>)	Postgres container's own bootstrap credentials — must stay consistent with the credentials embedded in <code>DATABASE_URL</code> .
NEO4J_AUTH / NEO4J_PLUGINS	<code>docker-compose.yml</code> (<code>neo4j</code> service <code>environment:</code>)	Neo4j container bootstrap auth (<code>neo4j/changeme-neo4j</code>) and the <code>apoc</code> plugin.
DISABLE_SECURITY_PLUGIN , OPENSEARCH_JAVA_OPTS , <code>discovery.type</code>	<code>docker-compose.yml</code> (<code>opensearch</code> service <code>environment:</code>)	Single-node OpenSearch container tuning; security plugin disabled for local dev.

For Kubernetes, the equivalent split is a `ConfigMap` (non-secret hostnames/ports/URLs) plus a `Secret` (everything with a credential) — see [k8s/README.md](#) and `k8s/base/configmap.yaml` / `k8s/base/secret.example.yaml`.

Example `.env`

```
# --- Security ---
JWT_SECRET_KEY=<configure securely>

# --- Ollama (default AI backend) ---
AI_BACKEND=ollama
OLLAMA_BASE_URL=http://host.docker.internal:11434
OLLAMA_MODEL=llama3.2:3b

# --- Free-tier providers (leave blank to skip) ---
VIRUSTOTAL_API_KEY=<your-key-here>
ABUSEIPDB_API_KEY=<your-key-here>
OTX_API_KEY=<your-key-here>
NVD_API_KEY=
ABUSECH_AUTH_KEY=<your-key-here>
```

Copy the full template from the repo root and fill in what you have:

```
cp .env.example .env
```

Then bring the stack up (first run builds images and applies Alembic migrations automatically via the `backend` service's command):

```
docker compose up --build
```

After changing any value in `.env` on an already-running stack, recreate just the affected service(s) rather than restarting:

```
docker compose up -d backend
```

Deployment

Deployment

Two deployment paths exist in this repo: **Docker Compose** (the only path that is fully wired up end-to-end and is what the rest of the docs assume) and a set of **Kubernetes manifests** under `k8s/base/` (a direct, hand-written translation of the compose file — not templated with Helm, and not wired to any CI/CD). There is **no CI/CD pipeline** in this repo (no `.github/workflows/`, no `Makefile`, no other pipeline config) — building and pushing images, and running `kubect1 apply`, is a manual/operator step. TLS termination is likewise **NOT IMPLEMENTED** anywhere in the repo; it's explicitly left to the operator (see below).

For the environment variables referenced throughout this doc, see [CONFIGURATION.md](#). For the auth/JWT and secrets-handling model, see [SECURITY.md](#). For system design, see [ARCHITECTURE.md](#).

Deployment options

Path	Status	Use case
Docker Compose (<code>docker-compose.yml</code>)	Fully working, dev-mode commands (<code>--reload</code> , <code>npm run dev</code> , bind-mounted source)	Local development, the only path exercised by the test suite / docs
Kubernetes (<code>k8s/base/</code>)	Manifests exist and are internally consistent; not production-hardened out of the box (see gaps below)	Reference starting point for a real cluster deployment
CI/CD pipeline	NOT IMPLEMENTED	N/A — build/push/apply is manual
TLS termination	NOT IMPLEMENTED (commented out in <code>k8s/base/ingress.yml</code>)	Operator must supply cert-manager config or their own certs

1. Docker Compose

Services

All defined in `docker-compose.yml` at the repo root:

Service	Image / build	Ports (host:container)	Healthcheck	Depends on
postgres	postgres:16-alpine	5433:5432	pg_isready -U ioc (5s interval, 10 retries)	—
redis	redis:7-alpine	6379:6379	redis-cli ping (5s interval, 10 retries)	—
neo4j	neo4j:5-community (+ apoc plugin)	7475:7474 (HTTP), 7688:7687 (Bolt)	none defined	—
opensearch	opensearchproject/opensearch:2.17.0 , single-node, security plugin disabled	9200:9200	none defined	—
backend	built from <code>backend/Dockerfile</code>	8000:8000	none defined (k8s manifests probe <code>/health</code> ; compose does not)	postgres (healthy), redis (healthy)
celery_worker	same build as <code>backend</code>	— (no exposed ports)	none	postgres (healthy), redis (healthy)
celery_beat	same build as <code>backend</code>	— (no exposed ports)	none	redis (healthy)
frontend	built from <code>frontend/Dockerfile</code>	3000:3000	none	backend (started, not healthy)

Named volumes: `postgres_data` , `neo4j_data` , `opensearch_data` . There is no automated backup of these volumes — **NOT IMPLEMENTED**; back them up with whatever volume-snapshot tooling your host provides.

Note the non-default host ports: Postgres is `5433` and Neo4j is `7475 / 7688` on the host side (container-internal ports are the standard `5432 / 7474 / 7687`) — this avoids clashing with any natively-installed Postgres/Neo4j on the dev machine.

Startup command

```
cp .env.example .env
# edit .env: JWT_SECRET_KEY, provider API keys, OLLAMA_MODEL, etc. -- see CONFIGURATION.md

docker compose up --build
```

The `backend` container's command runs migrations automatically before starting the API — there is no separate migration step to run by hand:

```
sh -c "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload"
```

`celery_worker` runs `celery -A app.workers.celery_app worker --loglevel=info`; `celery_beat` runs `celery -A app.workers.celery_app beat --loglevel=info` (this is the schedule source for the hourly OSINT crawl — see ARCHITECTURE.md). `frontend` runs `npm run dev` against the bind-mounted source tree.

AI backend dependency (Ollama)

The default AI backend (`AI_BACKEND=ollama`) expects **Ollama running natively on the host**, not in a container — the `backend` service is pointed at it via `OLLAMA_BASE_URL=http://host.docker.internal:11434` (resolves to the host machine from inside the container on Docker Desktop for Windows/Mac). This is intentional: a large model's mmap load was observed to be dramatically slower through Docker Desktop's WSL2 volume-mount layer than direct NTFS access on the host. Install Ollama on the host and `ollama pull <model>` before starting the stack, or switch `AI_BACKEND` to `bedrock`, `gemini`, or `anthropic` in `.env` (see CONFIGURATION.md / ARCHITECTURE.md for the client details).

First-run bootstrap

No seed admin user exists. Register the first user via the API — the first user ever registered is automatically granted the `admin` role, every subsequent registration defaults to `analyst`:

```
curl -X POST http://localhost:8000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"you@example.com","password":"<configure securely>","full_name":"Your Name"}'
```

Then obtain a token pair via `POST /api/v1/auth/login`.

Verifying the deployment

```
curl http://localhost:8000/health
# {"status": "ok", "service": "IOC Intelligence Platform"}
```

- Frontend: <http://localhost:3000>
- Backend Swagger UI: <http://localhost:8000/docs>
- Backend Prometheus metrics: <http://localhost:8000/metrics> (mounted via `prometheus-fastapi-instrumentator`; no Prometheus/Grafana server is bundled or configured to scrape it — **NOT IMPLEMENTED**, bring your own if you want dashboards)

Picking up a changed `.env` value (e.g. adding a provider API key) requires recreating the container, not just restarting it:

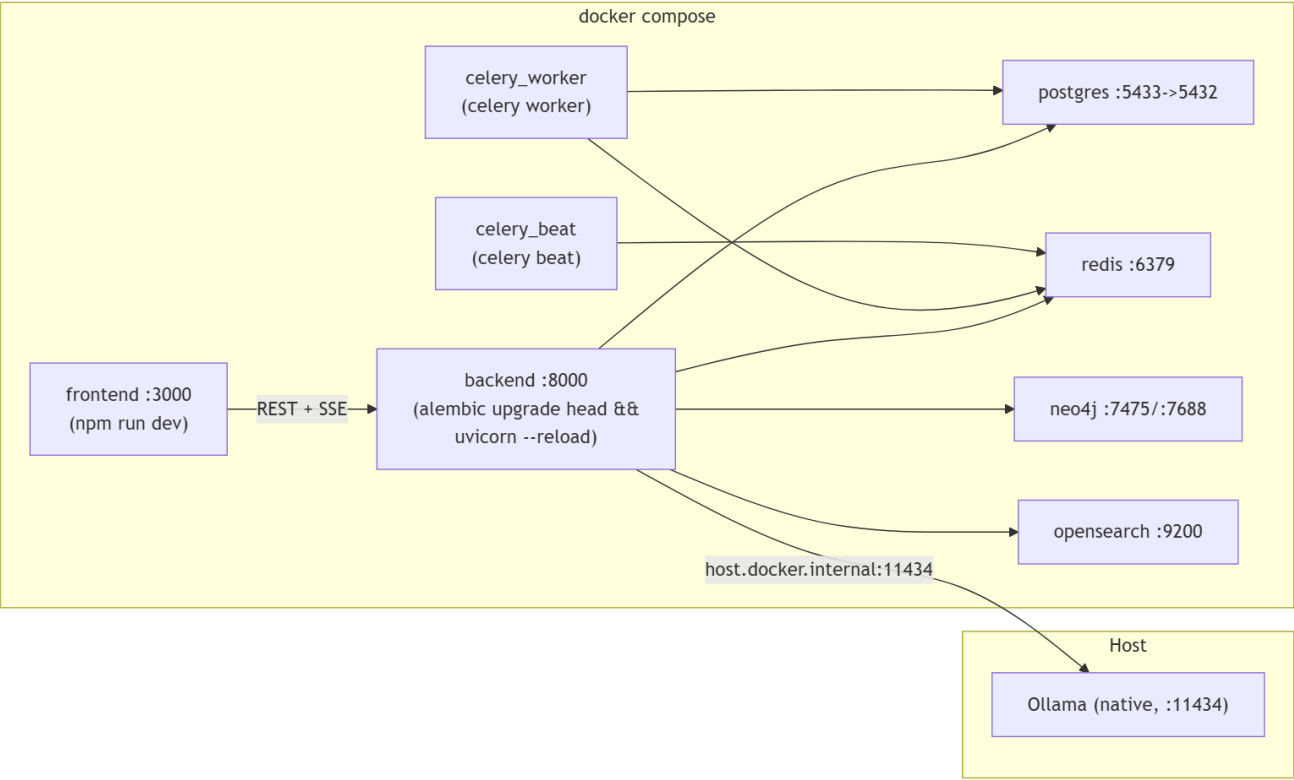
```
docker compose up -d backend
```

Applying a config/secret change without a rebuild

`docker compose restart backend` is **not** sufficient for `.env` changes — Compose only re-reads `.env` when a container is created. Use `docker compose up -d backend` instead (see above).

Not production-hardened as shipped

The compose file runs dev-mode commands throughout: `uvicorn --reload` with the backend source bind-mounted (`./backend:/app`), `npm run dev` with the frontend source bind-mounted, Postgres/Neo4j/OpenSearch with default or hardcoded credentials (`ioc/ioc`, `neo4j/changeme-neo4j`), and OpenSearch's security plugin explicitly disabled. Treat this compose file as a development/demo environment; harden credentials, disable `--reload`/bind mounts, and re-enable OpenSearch security before exposing it beyond localhost.



2. Kubernetes

`k8s/base/` is a plain-YAML, kustomize-compatible set of manifests — no Helm, no templating — mirroring the compose services one-for-one. Documented in full in `k8s/README.md`; summarized here.

Layout

k8s/base/	
namespace.yaml	# ioc-intel-platform namespace
configmap.yaml	# non-secret config (hosts, ports, URLs)
secret.example.yaml	# placeholder only -- NOT applied, NOT referenced by kustomization.yaml
postgres-statefulset.yaml	# + headless Service + PVC (via volumeClaimTemplates)
redis-deployment.yaml	# + Service
neo4j-statefulset.yaml	# + headless Service + PVC
opensearch-statefulset.yaml	# + headless Service + PVC
backend-deployment.yaml	# + Service, /health probes, resources
celery-worker-deployment.yaml	# same image as backend, runs the Celery worker
celery-beat-deployment.yaml	# same image as backend, runs Celery beat (1 replica, singleton)
frontend-deployment.yaml	# + Service
ingress.yaml	# /api -> backend, / -> frontend
kustomization.yaml	# ties it all together

ConfigMap vs. Secret split

Object	Contents
ioc-intel-config (ConfigMap)	Non-sensitive: POSTGRES_HOST / PORT / DB, REDIS_URL, CELERY_BROKER_URL, CELERY_RESULT_BACKEND, NEO4J_URI, NEO4J_PLUGINS, OPENSEARCH_URL / JAVA_OPTS / discovery settings, AWS_REGION, BEDROCK_MODEL_ID, BEDROCK_MAX_TOKENS, ENVIRONMENT, DEBUG, NEXT_PUBLIC_API_URL
ioc-intel-secrets (Secret, not committed with real values)	POSTGRES_USER, POSTGRES_PASSWORD, DATABASE_URL (embeds the Postgres password), NEO4J_USER, NEO4J_PASSWORD, JWT_SECRET_KEY, AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY, and every provider API key from .env.example (VirusTotal, AbuselPDB, OTX, NVD, abuse.ch, Hybrid Analysis, Censys, PhishTank)

backend, celery-worker, and celery-beat all consume both via `envFrom (configMapRef + secretRef)`, mirroring how compose merges service-level environment: With `env_file: .env`.

`k8s/base/secret.example.yaml` documents the Secret's required keys with empty-string placeholders; it is **intentionally not listed** in `kustomization.yaml`'s `resources:` so it can never be applied by accident, and is safe to commit as-is. Never fill it in and commit the filled-in copy.

Creating the namespace and the real Secret

```
kubectl create namespace ioc-intel-platform --dry-run=client -o yaml | kubectl apply -f -

# from your local .env (must also include POSTGRES_USER, POSTGRES_PASSWORD,
# DATABASE_URL, NEO4J_USER, NEO4J_PASSWORD -- these are compose service-level
# environment: values, not in .env.example, so add them to .env first)
kubectl create secret generic ioc-intel-secrets \
  --namespace ioc-intel-platform \
  --from-env-file=.env
```

Alternatively, copy `secret.example.yaml`, fill in real values, and `kubectl apply -f` it directly (do not add it back into `kustomization.yaml`, do not commit the filled-in copy).

Applying

```
kubectl apply -k k8s/base

kubectl -n ioc-intel-platform get pods
kubectl -n ioc-intel-platform rollout status deployment/backend
```

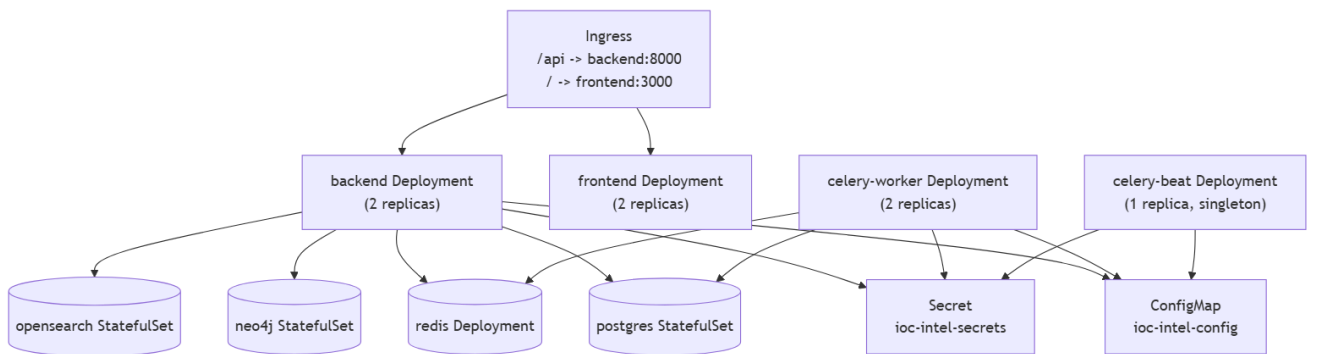
Preview rendered manifests without applying:

```
kubectl kustomize k8s/base
```

Probes and replicas

Workload	Replicas	Probe
backend	2	HTTP GET /health on :8000 (readiness: 10s initial delay/10s period; liveness: 20s/15s, both 6 failure threshold)
frontend	2	TCP :3000 (readiness 10s/10s, liveness 20s/15s, 6 failure threshold)
celery-worker	2	none defined
celery-beat	1 (kept as a singleton — running more than one replica would duplicate scheduled task dispatches)	none defined
postgres	1 (StatefulSet)	exec pg_isready -U ioc
neo4j	1 (StatefulSet)	TCP :7687
opensearch	1 (StatefulSet)	TCP :9200
redis	1	exec redis-cli ping

PVC sizes (`volumeClaimTemplates`): Postgres 5Gi, Neo4j 5Gi, OpenSearch 10Gi — starting points only; size for your real data volume and set `storageClassName` if your cluster's default isn't what you want.



Gaps to close before using this for real

- **Images:** the Deployments reference `ioc-intel-platform/backend:latest` and `ioc-intel-platform/frontend:latest`, which must be built and pushed to a registry your cluster can pull from, then the manifests updated to point at that registry and a pinned tag (not `latest`).
 - `backend/Dockerfile` (`python:3.12-slim`, `pip install -r requirements.txt`, `CMD uvicorn app.main:app --host 0.0.0.0 --port 8000`) is usable as-is for the k8s image — the Deployment's own `command` overrides `CMD` anyway to add `alembic upgrade head` first.

- `frontend/Dockerfile` (`node:20-alpine` , `npm install` , `CMD npm run dev`) is a **dev-mode Dockerfile only**. `frontend-deployment.yaml` runs `command: ["npm", "start"]` , which requires a production `next build` output (`.next`) that this Dockerfile never produces (it only ever runs `npm run dev`). Building a real production image (`npm run build && npm start` , per `k8s/README.md`) requires a **separate/modified Dockerfile — NOT IMPLEMENTED** in this repo as of this writing.
- **Ingress** (`ingress.yaml`): uses the legacy `kubernetes.io/ingress.class: "nginx"` annotation and a placeholder host `ioc-intel.example.com` . Update the host, and switch to `spec.ingressClassName: nginx` if your ingress-nginx version expects that instead.
- **TLS**: intentionally commented out in `ingress.yaml` . **NOT IMPLEMENTED** — fill in a real `tls:` block plus a `cert-manager ClusterIssuer` annotation (`cert-manager.io/cluster-issuer: ...`), or your own pre-provisioned secret, before exposing this outside a trusted network.
- **CI/CD: NOT IMPLEMENTED**. There is no pipeline that builds images, runs tests, or applies these manifests — every step above (`docker build` , `docker push` , `kubectl apply`) is manual.

Health and observability

Endpoint	Purpose
GET <code>/health</code>	Plain liveness check (<code>{"status": "ok", "service": ...}</code>), outside <code>/api/v1</code> . Used by the k8s readiness/liveness probes; not checked by a compose healthcheck.
GET <code>/docs</code>	FastAPI/Swagger UI (auto-generated).
GET <code>/metrics</code>	Prometheus-format metrics via <code>prometheus-fastapi-instrumentator</code> . No Prometheus server, scrape config, Grafana dashboards, or alerting are provided in this repo — NOT IMPLEMENTED , bring your own if needed.
GET <code>/api/v1/providers/health</code>	Application-level check of which providers are <code>configured</code> (has an API key) vs. not — not an infra health check.

Structured logging is configured via `structlog` (JSON renderer) in `backend/app/main.py` ; there is no shipped log-aggregation config (ELK/Loki/etc.) — left to the operator.

Related docs

- [CONFIGURATION.md](#) — full environment variable reference
- [SECURITY.md](#) — JWT/auth model, secrets handling, RBAC
- [ARCHITECTURE.md](#) — system design, request lifecycle, datastore roles
- [PROVIDERS.md](#) — which provider API keys are real vs. stub connectors
- [TESTING.md](#) — running the test suite, including the Docker-dependent integration tests
- [TROUBLESHOOTING.md](#) — common runtime issues (AI backend not responding, `not_configured` providers, 401/403s)

Developer Guide

Developer Guide

How to get the platform running locally, how the backend is organized, and the two most common extension points: adding a provider connector and adding an API route.

Source of truth for everything below: `backend/app/*`, `backend/alembic/*`, `docker-compose.yml`, `backend/requirements.txt`, `frontend/package.json`.

Related docs: [ARCHITECTURE.md](#) (system design), [DATA_MODEL.md](#) (schema/ERD), [PROVIDERS.md](#) (every connector in depth), [API_DOCUMENTATION.md](#) (full route reference), [TESTING.md](#) (test suite + Windows gotchas), [CONFIGURATION.md](#) (every setting), [AI_ENGINE.md](#) (AI backend abstraction).

1. Repo layout

```
ioc-intel-platform/
├── backend/                # FastAPI app, Alembic migrations, tests
│   ├── app/
│   ├── alembic/
│   └── requirements.txt
├── frontend/              # Next.js App Router UI
├── k8s/                   # Kubernetes manifests
├── docker-compose.yml     # postgres, redis, neo4j, opensearch, backend, celery_worker, celery_beat, frontend
├── .env.example
└── docs/
```

backend/app/ subpackages

Directory	Contents
api/routes/	FastAPI routers: <code>analysis.py</code> , <code>auth.py</code> , <code>basket.py</code> , <code>cases.py</code> , <code>hunting.py</code> , <code>lookup.py</code> , <code>pivot.py</code> , <code>providers.py</code> . <code>api/routes/__init__.py</code> is empty — <code>main.py</code> imports each router module directly.
models/	SQLAlchemy models: <code>base.py</code> (mixins), <code>user.py</code> , <code>lookup.py</code> , <code>evidence.py</code> , <code>basket.py</code> , <code>case.py</code> . <code>models/__init__.py</code> imports every module so <code>Base.metadata</code> is fully populated for Alembic.
schemas/	Pydantic request/response schemas for the routes: <code>auth.py</code> , <code>basket.py</code> , <code>case.py</code> , <code>lookup.py</code> .
providers/	One module per intelligence connector, <code>base.py</code> (the <code>BaseProvider</code> contract), <code>orchestrator.py</code> (fan-out/cache/retry), <code>registry.py</code> (the single list of registered providers), <code>stubs/</code> (connectors gated behind API keys the reference deployment doesn't have configured).
ai/	AI backend abstraction: <code>service.py</code> (core per-provider + final-assessment prompting), <code>analysis_service.py</code> (WHY/Challenge/Copilot/etc.), <code>hunting_service.py</code> (hunting queries/detection rules), <code>schemas.py</code> / <code>analysis_schemas.py</code> (Pydantic output contracts), <code>schema_utils.py</code> , and one thin client per backend: <code>ollama_client.py</code> , <code>bedrock_client.py</code> , <code>gemini_client.py</code> , <code>anthropic_client.py</code> .
correlation/	<code>engine.py</code> — pure function (<code>correlate</code>) turning a batch of <code>ProviderResult</code> s into graph nodes/edges. No I/O.
evidence/	<code>builder.py</code> (deterministic <code>EvidenceItem</code> extraction from provider data + correlation edges, no AI), <code>loaders.py</code> (reconstructs in-memory graph/evidence structures from persisted rows for non-streaming routes), <code>pivot.py</code> (pure pivot-ranking logic behind <code>GET /lookup/{id}/pivots</code>).
crawler/	The OSINT crawler wrapped as a <code>BaseProvider</code> (<code>collector.py</code> → <code>internet_intelligence_provider</code>), plus <code>sources/</code> (<code>github.py</code> , <code>reddit.py</code> , <code>rss_news.py</code> , <code>pastebin_search.py</code> , <code>rate_limit.py</code> , <code>errors.py</code>).
ioc/	<code>types.py</code> (the canonical <code>IOCType</code> enum, 32 members, plus <code>HASH_TYPES</code>), <code>detector.py</code> (<code>detect_ioc_type</code>).
auth/	<code>security.py</code> (password hashing, JWT), <code>rbac.py</code> (<code>get_current_user</code> , <code>require_permission</code> dependency factory).
core/	<code>config.py</code> (<code>Settings</code> / <code>get_settings()</code>), <code>db.py</code> (async engine/session), <code>cache.py</code> (Redis client, <code>RateLimiter</code>).
workers/	<code>celery_app.py</code> , <code>tasks.py</code> — the hourly OSINT crawl (<code>run_osint_crawl</code>). Lookups themselves run in-process (SSE), not through Celery.
tests/	<code>unit/</code> (9 files, no infra needed), <code>integration/</code> (3 files, 2 need Docker services). See TESTING.md .

2. Running the dev environment

Prerequisites

- Docker (for `docker compose up`).
- [Ollama](#) installed **natively on the host** (not in Docker) with a model pulled, e.g. `ollama pull llama3.2:3b`. The backend container reaches it via `OLLAMA_BASE_URL=http://host.docker.internal:11434` — see `docker-compose.yml` 's comment on why Ollama is intentionally run outside Docker (large

model mmap load is much slower through Docker Desktop's WSL2 volume layer on Windows/Mac).

First run

```
cp .env.example .env
# edit .env: set JWT_SECRET_KEY, add any provider API keys you have,
# and set OLLAMA_MODEL to whatever `ollama list` shows for the model you pulled

docker compose up --build
```

This brings up (per `docker-compose.yml`): `postgres` (host port 5433 → container 5432), `redis` (6379), `neo4j` (7475 / 7688 — provisioned but unused by app code, see [ARCHITECTURE.md](#)), `opensearch` (9200 — also unused by app code), `backend` (8000), `celery_worker`, `celery_beat`, and `frontend` (3000).

The `backend` container's start command runs migrations before serving:

```
alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

- Frontend: `http://localhost:3000`
- Backend Swagger UI: `http://localhost:8000/docs`
- Backend OpenAPI JSON: `http://localhost:8000/api/v1/openapi.json`
- Health check: `GET http://localhost:8000/health → {"status": "ok", "service": "..."}`
- Metrics: `GET http://localhost:8000/metrics` (Prometheus, via `prometheus-fastapi-instrumentator`)

Running only infra (for local, non-Docker backend dev)

```
docker compose up -d postgres redis
```

Then run the backend directly from `backend/` with your own Python environment (see `backend/requirements.txt` for pinned versions: FastAPI 0.115.0, SQLAlchemy 2.0.35, asyncpg 0.29.0, Alembic 1.13.2, etc.).

Database migrations

```
cd backend
alembic upgrade head
```

The migration history has exactly two revisions today (`backend/alembic/versions/`):

Order	Revision	Down-revision	Creates
1	660d2aa3bc20	None	<code>users</code> , <code>ioc_lookups</code> , <code>ai_summaries</code> , <code>correlation_edges</code> , <code>provider_results</code>
2 (HEAD)	a6d3ad2bb63c	660d2aa3bc20	<code>cases</code> , <code>basket_items</code> , <code>case_iocs</code> , <code>case_notes</code> , <code>case_reports</code> , <code>evidence_items</code>

The engine URL comes from `Settings.database_url` (`backend/app/core/config.py`, default `postgresql+asyncpg://ioc:ioc@postgres:5432/ioc_intel`), read by `alembic/env.py` via `get_settings().database_url` — override with a `.env` file or `DATABASE_URL` env var. Full schema detail: [DATA_MODEL.md](#).

To generate a new migration after changing a model (standard Alembic workflow; the repo relies on `app/models/__init__.py` importing every model module so `Base.metadata` is complete):

```
cd backend
alembic revision --autogenerate -m "describe your change"
alembic upgrade head
```

Running tests

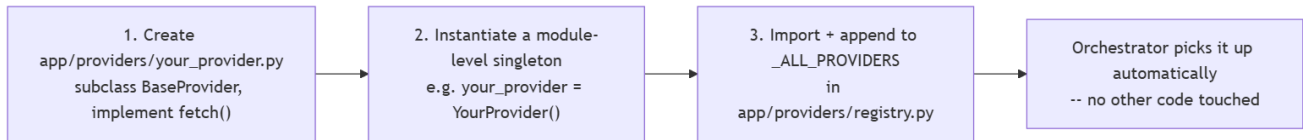
```
docker compose up -d postgres redis # only needed for the two DB/Redis-backed integration tests
cd backend
pytest
```

There is no `pytest.ini` / `pyproject.toml` / `conftest.py` — this is a bare default-discovery run. See [TESTING.md](#) for the full test inventory and Windows-specific `bcrypt/asyncpg` gotchas (two drifted local venvs, `backend/.venv` and `backend/.venv_test`, exist in this repo).

3. Adding a new provider connector

Every intelligence source — real or stub — implements `BaseProvider` (`backend/app/providers/base.py`) and is registered as a module-level singleton in `backend/app/providers/registry.py`. The orchestrator (`backend/app/providers/orchestrator.py`) never imports a concrete provider; it only calls

`BaseProvider.run()`. This is what makes adding a provider a two-file change that touches no other code.



Step 1 — subclass `BaseProvider`

```
# backend/app/providers/your_provider.py
import httpx

from app.core.config import get_settings
from app.ioc.types import IOType
from app.providers.base import BaseProvider, ProviderCategory, ProviderResult, ProviderStatus

class YourProvider(BaseProvider):
    provider_id = "your_provider"
    provider_name = "Your Provider"
    category = ProviderCategory.THREAT_INTEL # or SANDBOX, PASSIVE_DNS, CERTIFICATE_INTEL,
                                           # WHOIS, VULNERABILITY, OSINT
    supported_types = {IOType.IPV4, IOType.DOMAIN}
    requires_key = True
    base_url = "https://api.example.com"

    def __init__(self) -> None:
        super().__init__()
        self.configured = bool(get_settings().your_provider_api_key)

    async def fetch(self, ioc_value: str, ioc_type: IOType, client: httpx.AsyncClient) -> ProviderResult:
        response = await client.get(
            f"{self.base_url}/lookup/{ioc_value}",
            headers={"Authorization": f"Bearer {get_settings().your_provider_api_key}"},
        )
        response.raise_for_status()
        payload = response.json()
        return ProviderResult(
            provider_id=self.provider_id,
            provider_name=self.provider_name,
            category=self.category,
            status=ProviderStatus.OK,
            ioc_value=ioc_value,
            ioc_type=ioc_type,
            data={"verdict": "malicious"}, # whatever shape makes sense
            raw=payload,
            source_url=f"{self.base_url}/lookup/{ioc_value}",
        )

your_provider = YourProvider()
```

What `BaseProvider.run()` (which you do **not** override) already handles for you:

- Short-circuits to `ProviderStatus.UNSUPPORTED_IOC` if `ioc_type` not in `self.supported_types`.
- Short-circuits to `ProviderStatus.NOT_CONFIGURED` if `requires_key=True` and `self.configured` is falsy.
- Catches `httpx.HTTPStatusError`, mapping HTTP 429 / 403 / 509 → `RATE_LIMITED`, everything else → `ERROR`.
- Catches any other exception raised from `fetch()` → `ERROR`.
- Times the call (`latency_ms`).

Do **not** implement your own retry loop or `asyncio.wait_for` timeout inside `fetch()` — that's the orchestrator's job (`asyncio.wait_for` + `tenacity.AsyncRetrying`, retrying only on `httpx.ConnectError` / `ReadTimeout` / `PoolTimeout`). See `backend/app/providers/crtsh.py` for a minimal reference implementation (no-key connector) and `backend/app/providers/virustotal.py` for a keyed one.

Step 2 — add settings + `.env` entries (if the provider needs a key)

Add a field to `Settings` in `backend/app/core/config.py` (e.g. `your_provider_api_key: Optional[str] = None`) and a matching blank line in `.env.example`. `pydantic-settings` maps the field name to an uppercase env var (`YOUR_PROVIDER_API_KEY`) automatically.

Step 3 — register in `registry.py`

```
# backend/app/providers/registry.py
from app.providers.your_provider import your_provider

_ALL_PROVIDERS: list[BaseProvider] = [
    ...,
    your_provider,
]
```

That's it — `run_all_providers()` filters candidates by `p.supports(ioc_type)`, so your provider is automatically included for any lookup whose `ioc_type` is in `supported_types`, and it automatically shows up in `GET /api/v1/providers/health`.

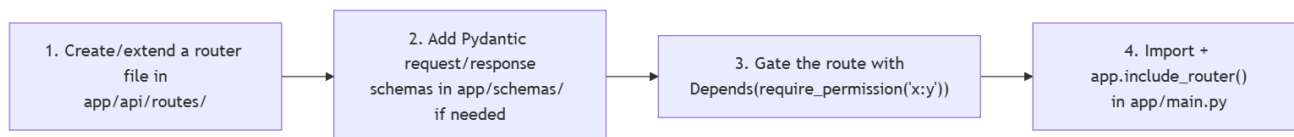
Caching note

Only `ProviderStatus.OK` results are cached in Redis (`orchestrator.py`), keyed as `provider_cache:{provider_id}:{ioc_type}:{sha256(ioc_value)}` for `settings.provider_cache_ttl_seconds` (default 3600s). You don't need to implement caching yourself.

Full connector-by-connector detail (VirusTotal, AbuseIPDB, OTX, URLhaus, ThreatFox, MalwareBazaar, crt.sh, NVD, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, and the stub connectors Hybrid Analysis, Spamhaus, PhishTank, Censys): [PROVIDERS.md](#).

4. Adding a new API route

Routers live in `backend/app/api/routes/` and are mounted directly in `backend/app/main.py` — there is no router auto-discovery and `api/routes/__init__.py` is empty.



Step 1 — write the router

```
# backend/app/api/routes/your_feature.py
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession

from app.auth.rbac import CurrentUser, require_permission
from app.core.db import get_db

router = APIRouter(prefix="/your-feature", tags=["your-feature"])

@router.get("")
async def list_things(
    db: AsyncSession = Depends(get_db),
    user: CurrentUser = Depends(require_permission("lookup:read")),
):
    ...
```

Existing routers all follow this shape — see `backend/app/api/routes/pivot.py` for a short, representative example (path param + `Depends(get_db)` + `Depends(require_permission(...))` + 404 on missing resource).

Step 2 — permissions

`require_permission(permission: str)` (`backend/app/auth/rbac.py`) is a dependency factory: it resolves `get_current_user` from the bearer JWT, then raises `403` with detail `"Role '{role}' lacks permission '{permission}'"` unless `permission` is in `ROLE_PERMISSIONS[user.role]` (`backend/app/models/user.py`). Reuse one of the existing permission strings (`lookup:create`, `lookup:read`, `evidence:read`, `analysis:generate`, `hunting:generate`, `copilot:query`, `basket:manage`, `case:create`, `case:read`, `case:write`, `case:close`) or define a new one and add it to `ROLE_PERMISSIONS` for the roles that should have it. Note: `lookup:export`, `provider:manage`, `user:manage`, and `audit:read` are defined in `ROLE_PERMISSIONS` but currently have **no route anywhere that checks them** — see [API_DOCUMENTATION.md](#) for the full permission matrix and which permissions are configured-but-unused.

Auth uses `HTTPBearer(auto_error=False)` rather than `OAuth2PasswordBearer`, deliberately, because `/auth/login` takes a JSON body, not an OAuth2 form-encoded grant.

Step 3 — mount it

```
# backend/app/main.py
from app.api.routes import analysis, auth, basket, cases, hunting, lookup, pivot, providers, your_feature
...
app.include_router(your_feature.router, prefix=settings.api_v1_prefix)
```

Every router is mounted with `prefix=settings.api_v1_prefix` (`/api/v1`), so `APIRouter(prefix="/your-feature")` becomes `GET /api/v1/your-feature`.

Full endpoint-by-endpoint reference (all 8 routers, request/response shapes, status codes): [API_DOCUMENTATION.md](#).

5. Related docs

- [ARCHITECTURE.md](#) — component diagram and end-to-end lookup flow.
- [DATA_MODEL.md](#) — full schema, ERD, migration history.
- [PROVIDERS.md](#) — every connector's quirks, endpoints, and auth scheme.
- [AI_ENGINE.md](#) — the `AI_BACKEND` abstraction (Ollama/Bedrock/Gemini/Anthropic) and grounding/anti-hallucination rules.
- [API_DOCUMENTATION.md](#) — full REST/SSE route reference.
- [TESTING.md](#) — test inventory, how to run it, Windows-specific gotchas.
- [CONFIGURATION.md](#) — every environment variable / `Settings` field.

API Documentation

API Documentation

Complete REST/SSE reference for the IOC Intelligence Platform backend (FastAPI, `backend/app/`).

Source of truth: `backend/app/api/routes/*.py`, `backend/app/schemas/*.py`, `backend/app/ai/analysis_schemas.py`, `backend/app/auth/rbac.py`, `backend/app/models/*.py`.

See also: [ARCHITECTURE.md](#) for how these routes fit into the overall system, [DATA_MODEL.md](#) for the underlying tables, and [PROVIDERS.md](#) for the provider connectors fanned out to by the lookup endpoint.

1. Base URL, versioning, docs

Item	Value
API prefix	<code>/api/v1</code> (<code>backend/app/core/config.py</code> — <code>api_v1_prefix</code> , all routers mounted with this prefix in <code>backend/app/main.py:37-44</code>)
OpenAPI JSON	<code>/api/v1/openapi.json</code>
Interactive docs (Swagger UI)	<code>/docs</code>
Health check (unversioned, no auth)	<code>GET /health</code> → <code>{ "status": "ok", "service": "<app_name>" }</code>
Metrics (Prometheus, unversioned)	<code>GET /metrics</code>

All request/response bodies below use the app's default base URL of `http://localhost:8000` in examples; substitute your deployment's actual host.

2. Authentication

All endpoints except `/auth/register`, `/auth/login`, `/auth/refresh`, `/health`, and `/metrics` require an `Authorization: Bearer <access_token>` header. Tokens are JWTs (HS256, `python-jose`), signed with `JWT_SECRET_KEY`, containing `sub` (email), `role`, `type` (`access` | `refresh`), `iat`, `exp`. Access tokens expire after `ACCESS_TOKEN_EXPIRE_MINUTES` (default 30); refresh tokens after `REFRESH_TOKEN_EXPIRE_DAYS` (default 7).

`HTTPBearer(auto_error=False)` is used instead of `OAuth2PasswordBearer` because `/auth/login` accepts a JSON body, not OAuth2 form-encoded fields (`backend/app/auth/rbac.py:14-18`).

Role/permission model

Three roles (`backend/app/models/user.py:11-14`): `admin`, `analyst`, `viewer`.

Permission	admin	analyst	viewer
<code>lookup:create</code>	Yes	Yes	No
<code>lookup:read</code>	Yes	Yes	Yes
<code>lookup:export</code>	Yes	Yes	No
<code>evidence:read</code>	Yes	Yes	Yes
<code>analysis:generate</code>	Yes	Yes	No
<code>hunting:generate</code>	Yes	Yes	No
<code>copilot:query</code>	Yes	Yes	No
<code>basket:manage</code>	Yes	Yes	No
<code>case:create</code>	Yes	Yes	No
<code>case:read</code>	Yes	Yes	Yes
<code>case:write</code>	Yes	Yes	No
<code>case:close</code>	Yes	Yes	No
<code>provider:manage</code>	Yes	No	No
<code>user:manage</code>	Yes	No	No
<code>audit:read</code>	Yes	No	No

NOT IMPLEMENTED: `lookup:export`, `provider:manage`, `user:manage`, and `audit:read` are all defined in `ROLE_PERMISSIONS` (`backend/app/models/user.py:31-44`) but **no route anywhere in `backend/app/api/routes/` calls `require_permission()` with any of these four strings**. There is no admin/provider-management endpoint, no user-management endpoint, no audit-log endpoint, and (see §5) no working server-side export endpoint. These permissions are configured but currently unused.

The first user ever registered on a fresh database automatically becomes `admin` ; every subsequent registration becomes `analyst` (bootstrap logic, `backend/app/api/routes/auth.py:26-28`). There is no UI or API path to promote/demote a user afterward.

Error responses common to every authenticated route

Status	Trigger
401 Could not validate credentials	Missing <code>Authorization</code> header, undecodable/expired JWT, wrong token <code>type</code> (e.g. a refresh token used where an access token is required), or the user no longer exists / <code>is_active=False</code> . Response includes header <code>WWW-Authenticate: Bearer</code> .
403 Role ' <code><role></code> ' lacks permission ' <code><permission></code> '	Token is valid but the user's role does not have the specific permission the route requires.

3. Auth routes (`/api/v1/auth`)

POST `/api/v1/auth/register`

Public. No permission required.

Request body:

```
{ "email": "analyst@example.com", "password": "<your-password>", "full_name": "Jane Analyst" }
```

`full_name` is optional (defaults to `""`). The backend enforces **no minimum password length** — the frontend's 8-character minimum (`frontend/app/register/page.tsx:70`) is client-side only.

Response 201:

```
{ "id": "b3f1...-uuid", "email": "analyst@example.com", "full_name": "Jane Analyst", "role": "admin" }
```

```
curl -sS -X POST http://localhost:8000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"analyst@example.com","password":"<your-password>","full_name":"Jane Analyst"}'
```

Errors: 400 Email already registered if the email exists.

POST `/api/v1/auth/login`

Public. No permission required.

```
curl -sS -X POST http://localhost:8000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"analyst@example.com","password":"<your-password>}'
```

Response 200:

```
{ "access_token": "<jwt>", "refresh_token": "<jwt>", "token_type": "bearer" }
```

Errors: 401 Invalid email or password (user not found or bad password); 403 Account disabled (`user.is_active == False`).

POST `/api/v1/auth/refresh`

Public (token-based, no `Authorization` header needed — the refresh token is the body).

```
curl -sS -X POST http://localhost:8000/api/v1/auth/refresh \
-H "Content-Type: application/json" \
-d '{"refresh_token":"<your-refresh-token>}'
```

Response 200: a **new** `TokenResponse` (both access and refresh tokens rotated).

Errors: 401 Invalid refresh token if the token is undecodable, not of `type=refresh` , or the user is missing/inactive.

GET /api/v1/auth/me

Requires a valid access token (Depends(`get_current_user`)), no specific permission string.

```
curl -sS http://localhost:8000/api/v1/auth/me \
-H "Authorization: Bearer <your-access-token>"
```

Response 200 :

```
{ "id": "b3f1...-uuid", "email": "analyst@example.com", "full_name": "", "role": "admin" }
```

Note: `full_name` is **hardcoded to ""** in this response (`backend/app/api/routes/auth.py:73`) regardless of what was stored at registration — the `CurrentUser` dataclass used internally only carries `id / email / role` , not `full_name` . Only the `/auth/register` response contains the real `full_name` .

4. Lookup routes (/api/v1/lookup)

The core investigation pipeline: detect the IOC type, fan out to every configured provider in parallel, stream results over SSE as each provider finishes, then run correlation and AI summarization. See [AI_ENGINE.md](#) for the summarization/assessment pipeline and [PROVIDERS.md](#) for the provider list.

POST /api/v1/lookup/stream

Permission: `lookup:create` .

Request body (`LookupCreateRequest`):

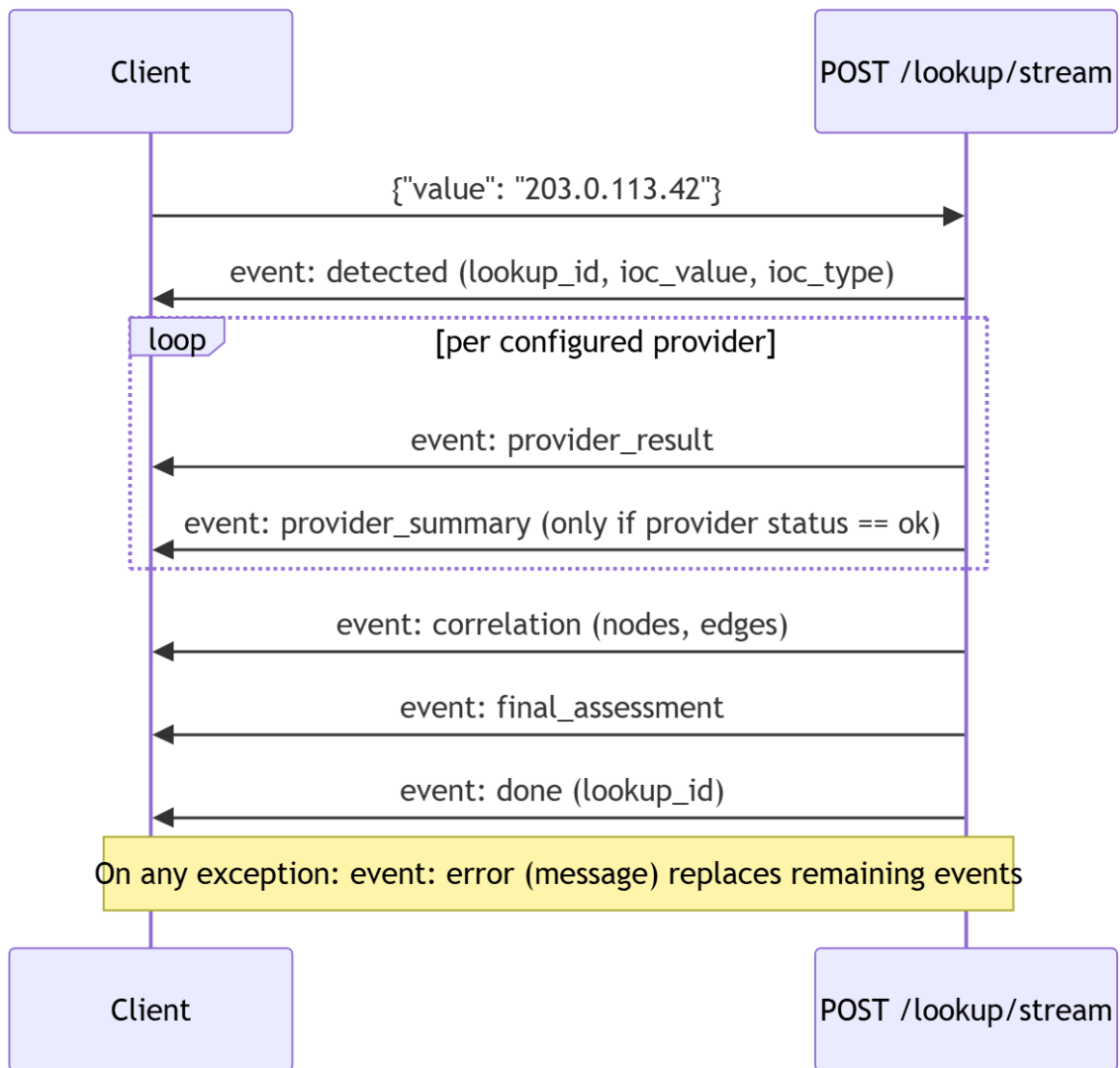
```
{ "value": "203.0.113.42", "ioc_type_hint": null }
```

`ioc_type_hint` is optional; when omitted the backend auto-detects the type. Valid `IOCType` values (`backend/app/ioc/types.py`): `ipv4` , `ipv6` , `domain` , `url` , `hostname` , `email` , `md5` , `sha1` , `sha256` , `sha512` , `tls_certificate` , `ja3` , `ja4` , `asn` , `cidr` , `malware_family` , `threat_actor` , `campaign` , `cve` , `cwe` , `capec` , `mitre_technique` , `file_name` , `registry_key` , `process_name` , `mutex` , `windows_service` , `file_path` , `user_agent` , `crypto_wallet` , `yara_rule` , `sigma_rule` , `unknown` .

Response: `StreamingResponse` , `media_type`: `text/event-stream` . Event sequence (per the route's docstring, `backend/app/api/routes/lookup.py:51-53`):

```
detected -> N x provider_result -> N x provider_summary -> correlation -> final_assessment -> done
```

or an `error` event in place of the remaining sequence if an exception occurs mid-stream.



```
curl -N -X POST http://localhost:8000/api/v1/lookup/stream \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"value": "203.0.113.42"}'
```

Sample `provider_result` event data shape (fields per `ProviderResultResponse`):

```
{
  "provider_id": "abuseipdb",
  "provider_name": "AbuseIPDB",
  "category": "reputation",
  "status": "ok",
  "data": { "...": "..."},
  "source_url": "https://www.abuseipdb.com/check/203.0.113.42",
  "error_message": null,
  "latency_ms": 412
}
```

Errors:

Status	Trigger
429 Rate limit exceeded: max <N> lookups per <W>s	<code>RateLimiter</code> (per user, key <code>lookup_create:{user.id}</code>) rejects the call. Defaults: <code>LOOKUP_RATE_LIMIT_MAX_CALLS=10</code> , <code>LOOKUP_RATE_LIMIT_WINDOW_SECONDS=60</code> .
422 Could not determine IOC type; pass <code>ioc_type_hint</code> .	<code>detect_ioc_type()</code> returns <code>unknown</code> and no <code>ioc_type_hint</code> was supplied.

The generator that streams events opens its **own** database session (`new_session()`) rather than reusing the request-scoped `Depends(get_db)` session — the latter is torn down as soon as the route returns the `StreamingResponse`, before the generator body runs, which would silently drop the final `lookup.status` mutation (comment, `lookup.py:87-94`).

GET /api/v1/lookup/{lookup_id}

Permission: `lookup:read`.

```
curl -sS http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6 \
-H "Authorization: Bearer <your-access-token>"
```

Response `200` (`LookupDetailResponse` shape): `id`, `ioc_value`, `ioc_type`, `status`, `final_verdict`, `risk_score`, `confidence_score`, `final_assessment`, `provider_results[]` (`provider_id`, `provider_name`, `category`, `status`, `data`, `source_url`, `error_message`, `latency_ms`), `ai_summaries[]`, `created_at`.

Errors: `404` Lookup not found.

GET /api/v1/lookup

Permission: `lookup:read`. Lookups are **shared across the whole SOC team** — any user with `lookup:read` sees every lookup, not just their own (comment, `lookup.py:250-252`).

Query params: `limit` (int, default `50`, silently clamped to `[1, 200]`).

```
curl -sS "http://localhost:8000/api/v1/lookup?limit=25" \
-H "Authorization: Bearer <your-access-token>"
```

Response `200`: array of `LookupSummary` (`id`, `ioc_value`, `ioc_type`, `status`, `final_verdict`, `risk_score`, `confidence_score`, `created_at`).

Errors: none beyond the common 401/403.

5. Export — NOT IMPLEMENTED

The frontend (`frontend/components/dashboard/ExportMenu.tsx:136`) issues:

```
POST /api/v1/lookup/{lookup_id}/export?format=pdf|csv
```

This route does not exist in the backend. A repository-wide search of `backend/app/api/routes/` for `/export` returns zero matches — no server-side PDF or CSV export was ever implemented, and the `lookup:export` permission (see §2) is defined but never checked by any route.

The frontend already degrades gracefully: `handleServerExport()` treats an HTTP `404` response (and any network/fetch failure) identically, showing the user the message **"Export format not yet available."** (`ExportMenu.tsx:140-152`) rather than crashing. Do not rely on this endpoint in integrations.

What actually works today is two purely client-side exports built from data already fetched into the browser (no backend round-trip):

- `handleExportJson()` — downloads the current `assessment` object as `.json`.
- `handleExportMarkdown()` — downloads a Markdown rendering of the same `assessment` object as `.md`.

If a real server-side export endpoint is added later, it should be documented here under a new `POST /api/v1/lookup/{lookup_id}/export` heading with request/response shapes and gated behind the already-reserved `lookup:export` permission.

6. Analysis routes (/api/v1/lookup/{lookup_id}/analysis)

AI-generated, evidence-grounded explanations over an **already-completed** lookup. Every route except `GET /evidence` requires permission `analysis:generate` (the copilot route requires `copilot:query` instead). All routes in this group call an internal `_load_lookup()` helper that raises:

Status	Trigger
<code>404</code> Lookup not found	<code>lookup_id</code> does not exist.
<code>409</code> Lookup is not completed yet (<code>status=<status></code>)	<code>lookup.status</code> is not completed (i.e. <code>pending</code> , <code>running</code> , or <code>failed</code>).

This 404/409 pair applies to **every** route in this section.

GET /api/v1/lookup/{lookup_id}/analysis/evidence

Permission: `evidence:read`. No AI involved — powers "Show Receipts" / the Evidence tab.

```
curl -sS http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6/analysis/evidence \
-H "Authorization: Bearer <your-access-token>"
```

Response `200`: array of evidence items — `id`, `evidence_type` (`detection`, `reputation`, `relationship`, `malware_association`, `threat_actor_association`, `campaign_association`, `mitre_technique`, `infrastructure`, `other`), `source_label`, `provider_id`, `claim`, `interpretation`, `confidence` (0-100), `related_ioc_type`, `related_ioc_value`, `source_url`, `observed_at`, `created_at`.

POST /api/v1/lookup/{lookup_id}/analysis/why

Permission: `analysis:generate`. Response model `WhyMaliciousExplanation`: `verdict_restated`, `reasons[]` (`{reason, evidence_ids[]}`), `caveat`.

```
curl -sS -X POST http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6/analysis/why \
-H "Authorization: Bearer <your-access-token>"
```

POST /api/v1/lookup/{lookup_id}/analysis/what-is-this

Permission: `analysis:generate`. Response model `WhatIsThisIOCI`: `plain_language_summary`, `technical_explanation`, `evidence_ids[]`, `confidence_narrative`, `related_infrastructure[]`.

POST /api/v1/lookup/{lookup_id}/analysis/disagreement

Permission: `analysis:generate`. Response model `DisagreementSummary`: `agreement`, `conflict`, `missing_data`, `most_reliable_evidence`, `evidence_ids[]`.

POST /api/v1/lookup/{lookup_id}/analysis/false-positive

Permission: `analysis:generate`. Response model `FalsePositiveAssessment`: `likely_false_positive` (bool), `candidate_categories[]` (One of `cdn`, `cloud_provider`, `shared_hosting`, `nat`, `vpn`, `proxy`, `security_scanner`, `search_crawler`, `monitoring_system`, `legitimate_business_infrastructure`, `none`), `explanation`, `evidence_ids[]`.

POST /api/v1/lookup/{lookup_id}/analysis/challenge

Permission: `analysis:generate`. Response model `ChallengeVerdict`: `supporting_evidence[]`, `contradictory_evidence[]` (both `{reason, evidence_ids[]}`), `missing_evidence[]`, `alternative_explanation`, `final_confidence` (`low` | `medium` | `high`), `final_confidence_rationale`.

POST /api/v1/lookup/{lookup_id}/analysis/next-actions

Permission: `analysis:generate`. Also loads the lookup's correlation edges. Response model `SmartNextActions`: `actions[]`, each `{action, target_ioc_value, target_ioc_type, rationale, priority (low|medium|high)}`.

POST /api/v1/lookup/{lookup_id}/analysis/gaps

Permission: `analysis:generate`. Splits the lookup's `ProviderResultRecord` rows into providers that returned data (`status == "ok"`) vs. those that didn't. Response model `IntelligenceGaps`: `gaps[]`, each `{gap, how_to_close}`.

POST /api/v1/lookup/{lookup_id}/analysis/score-explanation

Permission: `analysis:generate`. Response model `ScoreExplanation`: `components[]` (`{component, contribution, evidence_ids[]}`), `summary`.

POST /api/v1/lookup/{lookup_id}/analysis/copilot

Permission: `copilot:query`. Also loads correlation edges. Request body (raw dict, not a formal schema):

```
{ "question": "Is this IP linked to any known malware family?", "notes": ["optional analyst note"] }
```

```
curl -sS -X POST http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6/analysis/copilot \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"question": "Is this IP linked to any known malware family?"}'
```

Response model `CopilotAnswer`: `answer`, `evidence_ids[]`, `suggested_follow_ups[]` (max 4).

Errors (in addition to the 404/409 above): `422 question is required` if `question` is missing or blank after stripping whitespace.

7. Hunting routes (/api/v1/lookup/{lookup_id})

Permission: `hunting:generate` for both routes. Unlike the analysis routes, the local `_load_lookup` helper here (`backend/app/api/routes/hunting.py:26-30`) **only checks existence, not `status == completed`** — it raises `404 Lookup not found` if the lookup doesn't exist, but does not return `409` for an incomplete lookup.

POST /api/v1/lookup/{lookup_id}/hunt

Query param: `formats` (repeatable, default list = `sigma`, `splunk_spl`, `sentinel_kql`, `elastic`, `qradar_aql`, `chronicle_yara_l`, `suricata`, `snort`, `zeek`).

```
curl -sS -X POST "http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6/hunt?formats=sigma&formats=splunk_spl" \
-H "Authorization: Bearer <your-access-token>"
```

Response model `HuntingPackage`: `exact_match_queries[]` (`{format, query, detects}`), `expansion_targets[]` (`{related_ioc_value, related_ioc_type, rationale}`), drawn only from real correlation edges), `broader_queries[]` (same shape as `exact_match_queries`).

POST /api/v1/lookup/{lookup_id}/detection

Query param: `format` (required, no default — must be one of `sigma`, `yara`, `splunk_spl`, `sentinel_kql`, `elastic`, `qradar_aql`, `chronicle_yara_l`, `suricata`, `snort`, `zeek`).

```
curl -sS -X POST "http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6/detection?format=sigma" \
-H "Authorization: Bearer <your-access-token>"
```

Response model `DetectionRuleDraft`: `format`, `title`, `rule`, `detection_objective`, `data_source`, `logic_explanation`, `false_positive_considerations`, `severity` (`none` | `low` | `medium` | `high` | `critical`), `mitre_technique_ids[]` .

Errors: `404 Lookup not found` . (`422 Unprocessable Entity` from FastAPI's own query-param validation if `format` is omitted, since it has no default.)

8. Pivot route (/api/v1/lookup/{lookup_id}/pivots)

Permission: `lookup:read` . Deliberately **not** AI-generated — a deterministic sort over real correlation-graph edges, so it can never hallucinate a pivot target (module docstring, `backend/app/api/routes/pivot.py:1-8`). Complementary to the AI-generated `/analysis/next-actions` .

GET /api/v1/lookup/{lookup_id}/pivots

Query param: `limit` (int, default `10` ; internally clamped to `[1, 50]` inside `rank_pivots`).

```
curl -sS "http://localhost:8000/api/v1/lookup/3fa85f64-5717-4562-b3fc-2c963f66afa6/pivots?limit=5" \
-H "Authorization: Bearer <your-access-token>"
```

Response `200`: array of pivot candidates, each `{ioc_value, ioc_type, relationship, confidence (0-100), corroborating_providers, provenance, relevance (low|medium|high)}` , sorted by corroborating-provider count then confidence, descending.

Errors: `404 Lookup not found` .

9. Providers route (/api/v1/providers)

GET /api/v1/providers/health

Permission: `lookup:read` . No request body/params.

```
curl -sS http://localhost:8000/api/v1/providers/health \
-H "Authorization: Bearer <your-access-token>"
```

Response `200`: array of `{provider_id, provider_name, category, configured, requires_key, supported_types[]}` — one entry per registered connector (see [PROVIDERS.md](#) for the full connector list and their configuration).

Errors: none beyond the common 401/403 (no explicit error handling in this route).

10. Basket routes (/api/v1/basket)

Permission `basket:manage` on every route. The basket is **per-analyst private scratch space** (scoped to `BasketItem.owner_id == user.id`), unlike lookups/cases, which are shared team-wide.

GET /api/v1/basket

```
curl -sS http://localhost:8000/api/v1/basket \
-H "Authorization: Bearer <your-access-token>"
```

Response 200: array of {id, ioc_value, ioc_type, note, latest_lookup_id, created_at} , scoped to the caller.

POST /api/v1/basket

```
curl -sS -X POST http://localhost:8000/api/v1/basket \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"ioc_value":"203.0.113.42","note":"seen in phishing email"}'
```

Request body (BasketAddRequest): ioc_value (str), ioc_type_hint (optional IOCType), note (optional str). Response 201 : the created (or, if already present for this owner+ioc_value, the pre-existing) basket item — adding a duplicate is idempotent, returns 200 -shaped body with status 201 , and does **not** raise a 409 . If a completed lookup already exists for that exact ioc_value , its id is best-effort attached as latest_lookup_id .

Errors: 422 Could not determine IOC type; pass ioc_type_hint. if type detection fails and no hint was supplied.

DELETE /api/v1/basket/{item_id}

```
curl -sS -X DELETE http://localhost:8000/api/v1/basket/3fa85f64-5717-4562-b3fc-2c963f66afa6 \
-H "Authorization: Bearer <your-access-token>"
```

Response 204. Errors: 404 Basket item not found if no such item owned by the caller exists.

DELETE /api/v1/basket

Clears the caller's entire basket. No body/params.

```
curl -sS -X DELETE http://localhost:8000/api/v1/basket \
-H "Authorization: Bearer <your-access-token>"
```

Response 204. No documented error codes.

POST /api/v1/basket/compare

Request body (raw dict): {"lookup_ids": ["<uuid>", "<uuid>", ...]} — each id must reference an already-completed lookup.

```
curl -sS -X POST http://localhost:8000/api/v1/basket/compare \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"lookup_ids":["3fa85f64-5717-4562-b3fc-2c963f66afa6","7c9e6679-7425-40de-944b-e07fc1f90ae7"]}'
```

Response 200: {"rows": [...], "narrative": {...}}. Each row: ioc_value, ioc_type, verdict, risk_score, confidence_score, asn[], malware_families[], threat_actors[], related_domains[], first_seen. narrative is IOCTComparisonNarrative: most_dangerous_ioc_value, narrative, key_differences[] .

Errors:

Status	Trigger
422 Provide at least 2 lookup_ids to compare	Fewer than 2 ids supplied.
422 Cannot compare more than 10 IOCs at once	More than 10 ids supplied.
422 Fewer than 2 of the given lookup_ids resolved to completed lookups	After resolving ids to lookups, fewer than 2 valid/completed rows remained.

11. Case routes (/api/v1/cases)

Case visibility mirrors lookups — **shared across the whole SOC team** for anyone with the relevant permission, unlike the private basket (module docstring, backend/app/api/routes/cases.py:1-6).

CaseStatus values (backend/app/models/case.py:16-22): open, investigating, contained, resolved, false_positive, closed . CaseSeverity values (case.py:25-29): low, medium, high, critical .

GET /api/v1/cases

Permission: case:read . Query params: status_filter (optional str — compared directly against Case.status as a raw string, **not validated against the CaseStatus enum** before the query runs), limit (default 50 , clamped to [1, 200]).

```
curl -sS "http://localhost:8000/api/v1/cases?status_filter=open&limit=25" \
-H "Authorization: Bearer <your-access-token>"
```

Response 200: array of {id, title, severity, status, tags, created_at}.

POST /api/v1/cases

Permission: case:create.

```
curl -sS -X POST http://localhost:8000/api/v1/cases \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"title":"Phishing campaign - Aug 2026","severity":"high","tags":["phishing"]}'
```

Request body (CaseCreateRequest): title (1-255 chars), description (optional), severity (default medium), tags (default []). analyst_id is set to the caller automatically. Response 201: full serialized case (see shape under GET /{case_id} below).

GET /api/v1/cases/{case_id}

Permission: case:read. **Errors:** 404 Case not found.

```
curl -sS http://localhost:8000/api/v1/cases/3fa85f64-5717-4562-b3fc-2c963f66afa6 \
-H "Authorization: Bearer <your-access-token>"
```

Response 200:

```
{
  "id": "...", "title": "...", "description": null, "analyst_id": "...",
  "severity": "high", "status": "open", "tags": ["phishing"],
  "created_at": "...", "updated_at": "...",
  "iocs": [{"id": "...", "ioc_value": "...", "ioc_type": "...", "lookup_id": null, "added_by": "...", "created_at": "...}],
  "notes": [{"id": "...", "author_id": "...", "body": "...", "anchor_type": null, "anchor_ref": null, "created_at": "...}],
  "reports": [{"id": "...", "report_type": "...", "title": "...", "generated_by": "...", "created_at": "...}]
}
```

PATCH /api/v1/cases/{case_id}

Permission: case:write. Body (CaseUpdateRequest , all fields optional): title, description, severity, status, tags. Only fields actually supplied are applied (model_dump(exclude_unset=True)). **Errors:** 404 Case not found.

```
curl -sS -X PATCH http://localhost:8000/api/v1/cases/3fa85f64-5717-4562-b3fc-2c963f66afa6 \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"status":"investigating"}
```

POST /api/v1/cases/{case_id}/close

Permission: case:close. No body. Sets status = closed. **Errors:** 404 Case not found.

```
curl -sS -X POST http://localhost:8000/api/v1/cases/3fa85f64-5717-4562-b3fc-2c963f66afa6/close \
-H "Authorization: Bearer <your-access-token>"
```

POST /api/v1/cases/{case_id}/iocs

Permission: case:write. Body (CaseIOCAddRequest): ioc_value, ioc_type (plain str, not validated against IOCType), lookup_id (optional str). added_by set to caller. **Errors:** 404 Case not found.

```
curl -sS -X POST http://localhost:8000/api/v1/cases/3fa85f64-5717-4562-b3fc-2c963f66afa6/iocs \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"ioc_value":"203.0.113.42","ioc_type":"ipv4"}
```

DELETE /api/v1/cases/{case_id}/iocs/{ioc_id}

Permission: case:write. **Errors:** 404 Case IOC not found if no matching row for that case_id + ioc_id.

```
curl -sS -X DELETE http://localhost:8000/api/v1/cases/3fa85f64-5717-4562-b3fc-2c963f66afa6/iocs/7c9e6679-7425-40de-944b-e07fc1f90ae7 \
-H "Authorization: Bearer <your-access-token>"
```

POST /api/v1/cases/{case_id}/notes

Permission: `case:write`. Body (`CaseNoteCreateRequest`): `body` (min length 1), `anchor_type` (optional), `anchor_ref` (optional). `author_id` set to caller. **Errors:**
404 Case not found.

```
curl -sS -X POST http://localhost:8000/api/v1/cases/3fa85f64-5717-4562-b3fc-2c963f66afa6/notes \
-H "Authorization: Bearer <your-access-token>" \
-H "Content-Type: application/json" \
-d '{"body":"Escalated to IR team."}'
```

12. Consolidated error reference

Status	Meaning	Where it fires
400	Email already registered	POST /auth/register
401	Invalid email or password / Could not validate credentials / Invalid refresh token	Login, any protected route with a bad/missing/expired token, refresh
403	Account disabled / Role ' <code><role></code> ' lacks permission ' <code><permission></code> '	Login for inactive users; any route when the caller's role lacks the required permission
404	Lookup not found / Case not found / Basket item not found / Case IOC not found	Any route keyed on a path-param id that doesn't resolve to a row (also applies to any row not owned by the caller, for basket routes)
409	Lookup is not completed yet (<code>status=<status></code>)	Any <code>/analysis/*</code> route when the target lookup's <code>status</code> is not <code>completed</code>
422	Could not determine IOC type; pass <code>ioc_type_hint</code> .	POST /lookup/stream, POST /basket when IOC-type auto-detection fails
422	question is required	POST /analysis/{id}/copilot with an empty/missing <code>question</code>
422	Provide at least 2 <code>lookup_ids</code> to compare / Cannot compare more than 10 IOCs at once / Fewer than 2 of the given <code>lookup_ids</code> resolved to completed lookups	POST /basket/compare
422	FastAPI request-validation errors	Any route, for malformed JSON bodies or missing required query params (e.g. <code>format</code> on POST /detection)
429	Rate limit exceeded: max <code><N></code> lookups per <code><W></code> s	POST /lookup/stream, per-user, per <code>LOOKUP_RATE_LIMIT_MAX_CALLS</code> / <code>LOOKUP_RATE_LIMIT_WINDOW_SECONDS</code>

13. Route inventory (quick reference)

Method	Path	Permission
POST	/api/v1/auth/register	public
POST	/api/v1/auth/login	public
POST	/api/v1/auth/refresh	public
GET	/api/v1/auth/me	any valid access token
POST	/api/v1/lookup/stream	lookup:create
GET	/api/v1/lookup/{lookup_id}	lookup:read
GET	/api/v1/lookup	lookup:read
POST	/api/v1/lookup/{lookup_id}/export	NOT IMPLEMENTED
GET	/api/v1/lookup/{lookup_id}/analysis/evidence	evidence:read
POST	/api/v1/lookup/{lookup_id}/analysis/why	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/what-is-this	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/disagreement	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/false-positive	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/challenge	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/next-actions	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/gaps	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/score-explanation	analysis:generate
POST	/api/v1/lookup/{lookup_id}/analysis/copilot	copilot:query
POST	/api/v1/lookup/{lookup_id}/hunt	hunting:generate
POST	/api/v1/lookup/{lookup_id}/detection	hunting:generate
GET	/api/v1/lookup/{lookup_id}/pivots	lookup:read
GET	/api/v1/providers/health	lookup:read
GET	/api/v1/basket	basket:manage
POST	/api/v1/basket	basket:manage
DELETE	/api/v1/basket/{item_id}	basket:manage
DELETE	/api/v1/basket	basket:manage
POST	/api/v1/basket/compare	basket:manage
GET	/api/v1/cases	case:read
POST	/api/v1/cases	case:create
GET	/api/v1/cases/{case_id}	case:read
PATCH	/api/v1/cases/{case_id}	case:write
POST	/api/v1/cases/{case_id}/close	case:close
POST	/api/v1/cases/{case_id}/iocs	case:write
DELETE	/api/v1/cases/{case_id}/iocs/{ioc_id}	case:write
POST	/api/v1/cases/{case_id}/notes	case:write
GET	/health	none
GET	/metrics	none

No route in the codebase exists for `provider:manage`, `user:manage`, or `audit:read` — there is currently no admin/provider-management API, no user-management API, and no audit-log API despite those permission strings being defined on the `admin` role.

Data Model

Data Model

This document describes the persisted schema for the IOC Intelligence Platform: every SQLAlchemy model in `backend/app/models/`, the Alembic migration history that created them, and the relationships between tables. Postgres (via the async `asyncpg` driver) is the single system of record for all tables below.

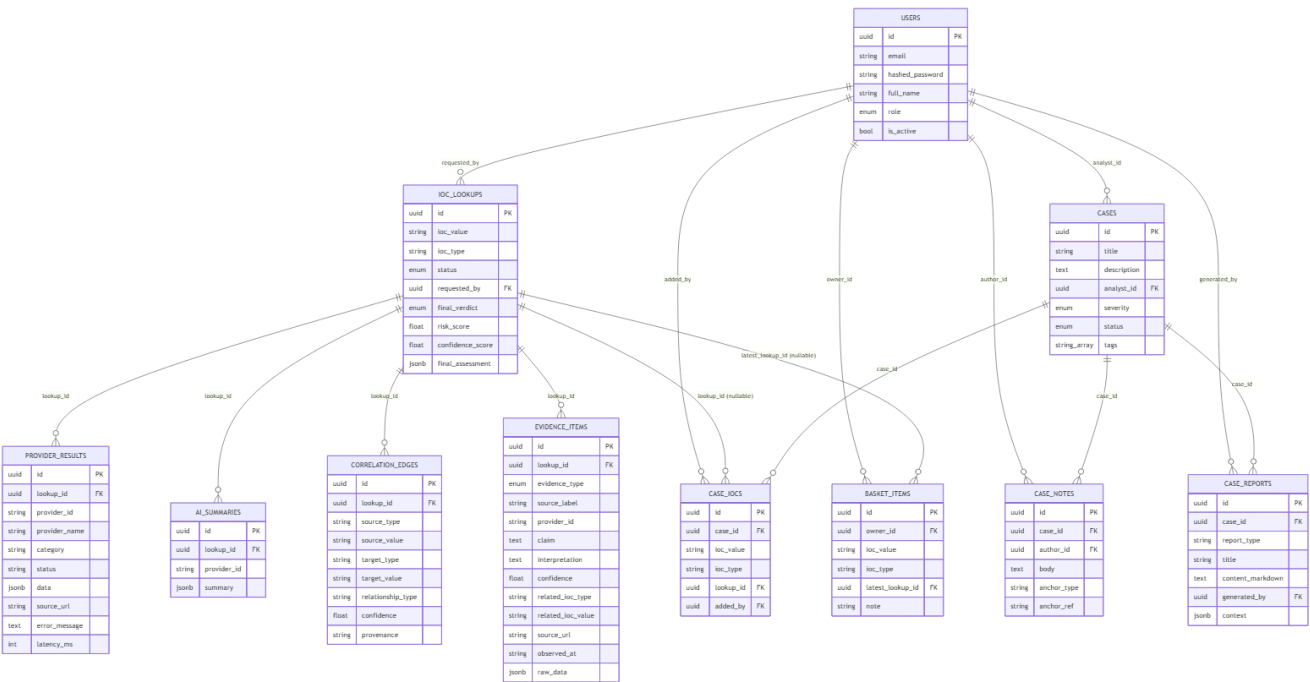
Source of truth: `backend/app/models/*.py` and `backend/alembic/versions/*.py`.

See also: [ARCHITECTURE.md](#) for how these tables fit into the overall system, and [CONFIGURATION.md](#) for `DATABASE_URL` and other environment settings.

Conventions

- Primary keys:** every table inherits `UUIDPrimaryKeyMixin` (`backend/app/models/base.py:14-17`) — a `UUID` column named `id`, `primary_key=True`, Python-side default `uuid.uuid4()`.
- Timestamps:** every table inherits `TimestampMixin` (`base.py:20-26`) — `created_at` and `updated_at`, both `DateTime(timezone=True)` with `server_default=func.now()`; `updated_at` also sets `onupdate=func.now()`.
- Enums:** persisted as Postgres `ENUM` types via SQLAlchemy's `Enum()`, backed by Python `str`, `enum.Enum` classes.
- JSON columns:** use Postgres `JSONB` (`sqlalchemy.dialects.postgresql.JSONB`), not generic JSON.
- Deletes:** there is no soft-delete / `is_deleted` column anywhere in the schema. Deletion is hard-delete, propagated through SQLAlchemy relationship `cascade="all, delete-orphan"` (e.g. deleting an `IOCLookup` cascades to its `provider_results`, `ai_summaries`, `correlation_edges`, and `evidence_items`; deleting a `Case` cascades to its `iocs`, `notes`, and `reports`).

Entity-Relationship Diagram



Notes on the diagram:

- `IOC_LOOKUPS.requested_by`, `CASE_IOC.lookup_id`, and `BASKET_ITEMS.latest_lookup_id` are all **nullable** foreign keys — an unauthenticated/system lookup can have no `requested_by`, a case IOC can be added without an existing lookup, and a basket item can exist before any lookup has run for it.
- `CORRELATION_EDGES` rows reference IOC *values* (`source_value` / `target_value`), not foreign keys into `ioc_lookups` — an edge's endpoints may or may not correspond to an IOC that itself has a lookup row.

Tables

users

Defined in backend/app/models/user.py:17-24 .

Column	Type	Constraints
id	UUID	PK, default uuid.uuid4()
email	String(255)	unique, indexed, not null
hashed_password	String(255)	not null
full_name	String(255)	not null, default ''
role	Enum(Role)	not null, default analyst
is_active	Boolean	not null, default true
created_at	DateTime(timezone=True)	server default now()
updated_at	DateTime(timezone=True)	server default now(), updates now() on change

Role enum (user.py:11-14): admin , analyst , viewer .

Permission matrix — ROLE_PERMISSIONS (user.py:31-44), consumed by the require_role dependency in app/auth/rbac.py :

Permission	admin	analyst	viewer
lookup:create	yes	yes	
lookup:read	yes	yes	yes
lookup:export	yes	yes	
provider:manage	yes		
user:manage	yes		
audit:read	yes		
evidence:read	yes	yes	yes
analysis:generate	yes	yes	
hunting:generate	yes	yes	
copilot:query	yes	yes	
basket:manage	yes	yes	
case:create	yes	yes	
case:read	yes	yes	yes
case:write	yes	yes	
case:close	yes	yes	

ioc_lookups

Defined in backend/app/models/lookup.py:37-66 . The central record of one IOC investigation.

Column	Type	Constraints
id	UUID	PK
ioc_value	String(2048)	not null, indexed
ioc_type	String(64)	not null, indexed
status	Enum(LookupStatus)	not null, default pending
requested_by	UUID	FK -> users.id , nullable
final_verdict	Enum(Verdict)	nullable
risk_score	Float	nullable
confidence_score	Float	nullable
final_assessment	JSONB	nullable — shape defined in app/ai/schemas.py
created_at / updated_at	DateTime(timezone=True)	mixin defaults

LookupStatus enum (lookup.py:15-19): pending , running , completed , failed .

Verdict `enum` (lookup.py:22-34): `highly_malicious`, `malicious`, `suspicious`, `unknown`, `likely_benign`, `benign`, `scanner`, `tor_exit_node`, `vpn`, `cdn`, `cloud_infrastructure`, `dormant_infrastructure`.

Relationships (all `cascade="all, delete-orphan"`, deleting a lookup deletes these children): `provider_results`, `ai_summaries`, `correlation_edges`, `evidence_items`.

`provider_results`

Defined in `backend/app/models/lookup.py:69-84`. One row per provider connector call for a lookup.

Column	Type	Constraints
<code>id</code>	<code>UUID</code>	PK
<code>lookup_id</code>	<code>UUID</code>	FK -> <code>ioc_lookups.id</code> , not null, indexed
<code>provider_id</code>	<code>String(128)</code>	not null, indexed
<code>provider_name</code>	<code>String(255)</code>	not null
<code>category</code>	<code>String(64)</code>	not null
<code>status</code>	<code>String(32)</code>	not null
<code>data</code>	<code>JSONB</code>	default <code>{}</code>
<code>source_url</code>	<code>String(2048)</code>	nullable
<code>error_message</code>	<code>Text</code>	nullable
<code>latency_ms</code>	<code>Integer</code>	nullable
<code>created_at</code> / <code>updated_at</code>	<code>DateTime(timezone=True)</code>	mixin defaults

See [PROVIDERS.md](#) for the provider connectors that populate this table.

`ai_summaries`

Defined in `backend/app/models/lookup.py:87-97`. Holds both per-provider AI summaries and the single consolidated assessment for a lookup.

Column	Type	Constraints
<code>id</code>	<code>UUID</code>	PK
<code>lookup_id</code>	<code>UUID</code>	FK -> <code>ioc_lookups.id</code> , not null, indexed
<code>provider_id</code>	<code>String(128)</code>	nullable, indexed — null means this row is the final consolidated assessment , not a per-provider summary
<code>summary</code>	<code>JSONB</code>	not null
<code>created_at</code> / <code>updated_at</code>	<code>DateTime(timezone=True)</code>	mixin defaults

See [AI_ENGINE.md](#) for how these rows are generated.

`correlation_edges`

Defined in `backend/app/models/lookup.py:100-118`. One row per graph edge discovered by the correlation engine for a lookup.

Column	Type	Constraints
<code>id</code>	<code>UUID</code>	PK
<code>lookup_id</code>	<code>UUID</code>	FK -> <code>ioc_lookups.id</code> , not null, indexed
<code>source_type</code>	<code>String(64)</code>	not null
<code>source_value</code>	<code>String(2048)</code>	not null
<code>target_type</code>	<code>String(64)</code>	not null
<code>target_value</code>	<code>String(2048)</code>	not null
<code>relationship_type</code>	<code>String(64)</code>	not null
<code>confidence</code>	<code>Float</code>	default <code>1.0</code>
<code>provenance</code>	<code>String(128)</code>	not null — which provider asserted this edge
<code>created_at</code> / <code>updated_at</code>	<code>DateTime(timezone=True)</code>	mixin defaults

Note on Neo4j: the model's docstring (`lookup.py:101-103`) says these edges are "mirrored into Neo4j for graph traversal but kept here too so a lookup's graph can be rebuilt from Postgres alone if Neo4j is unavailable." In the current codebase there is **no Neo4j driver usage anywhere — NOT IMPLEMENTED**. `correlation_edges` in Postgres is, in practice, the *only* store for this data; no code path writes to Neo4j. The `neo4j_uri` / `neo4j_user` / `neo4j_password` settings exist in configuration but are not currently exercised for graph writes.

evidence_items

Defined in `backend/app/models/evidence.py:31-56`. The evidence ledger: deterministic, reproducible facts that AI-generated text can cite by `evidence_id`, per the module docstring — rows are built by `app/evidence/builder.py` from provider summaries and correlation edges and are **never written directly by an AI call**.

Column	Type	Constraints
<code>id</code>	UUID	PK
<code>lookup_id</code>	UUID	FK -> <code>ioc_lookups.id</code> , not null, indexed
<code>evidence_type</code>	Enum(EvidenceType)	not null, indexed
<code>source_label</code>	String(255)	not null — human-readable origin, e.g. "VirusTotal", "Correlation Engine"
<code>provider_id</code>	String(128)	nullable, indexed — machine-readable provider id when source is a connector; null when source is the correlation engine
<code>claim</code>	Text	not null
<code>interpretation</code>	Text	nullable
<code>confidence</code>	Float	not null — 0-100 scale, matches <code>RiskAssessment</code>
<code>related_ioc_type</code>	String(64)	nullable
<code>related_ioc_value</code>	String(2048)	nullable
<code>source_url</code>	String(2048)	nullable
<code>observed_at</code>	String(64)	nullable — provider-reported timestamp, kept as raw string (mixed formats across providers)
<code>raw_data</code>	JSONB	nullable
<code>created_at</code> / <code>updated_at</code>	DateTime(timezone=True)	mixin defaults

EvidenceType enum (`evidence.py:19-28`): `detection`, `reputation`, `relationship`, `malware_association`, `threat_actor_association`, `campaign_association`, `mitre_technique`, `infrastructure`, `other`.

basket_items

Defined in `backend/app/models/basket.py:16-29`. Per-analyst scratch space for collecting IOCs before comparing, bulk-investigating, or promoting them into a Case. By explicit design (per the module docstring) this table has **no status/workflow field** — "that's what Case is for." This is an intentional omission, not a gap.

Column	Type	Constraints
<code>id</code>	UUID	PK
<code>owner_id</code>	UUID	FK -> <code>users.id</code> , not null, indexed
<code>ioc_value</code>	String(2048)	not null
<code>ioc_type</code>	String(64)	not null
<code>latest_lookup_id</code>	UUID	FK -> <code>ioc_lookups.id</code> , nullable — most recent completed lookup for this IOC, lets basket actions reuse existing intelligence instead of re-running providers
<code>note</code>	String(1024)	nullable
<code>created_at</code> / <code>updated_at</code>	DateTime(timezone=True)	mixin defaults

Table constraint: `UniqueConstraint("owner_id", "ioc_value", name="uq_basket_owner_ioc")` — an analyst can only have one basket entry per IOC value.

cases

Defined in `backend/app/models/case.py:32-44`. Groups IOCs, notes, and reports under one investigation with a status workflow.

Column	Type	Constraints
id	UUID	PK
title	String(255)	not null
description	Text	nullable
analyst_id	UUID	FK -> users.id , not null, indexed
severity	Enum(CaseSeverity)	not null, default medium
status	Enum(CaseStatus)	not null, default open , indexed
tags	ARRAY(String(64))	default []
created_at / updated_at	DateTime(timezone=True)	mixin defaults

CaseStatus **enum** (case.py:16-22): open , investigating , contained , resolved , false_positive , closed .

CaseSeverity **enum** (case.py:25-29): low , medium , high , critical .

Relationships (all cascade="all, delete-orphan"): iocs -> CaseIOC , notes -> CaseNote , reports -> CaseReport .

case_iocs

Defined in backend/app/models/case.py:47-58 . Join of a Case to an IOC, optionally tied to a specific lookup.

Column	Type	Constraints
id	UUID	PK
case_id	UUID	FK -> cases.id , not null, indexed
ioc_value	String(2048)	not null
ioc_type	String(64)	not null
lookup_id	UUID	FK -> ioc_lookups.id , nullable
added_by	UUID	FK -> users.id , not null (not indexed)
created_at / updated_at	DateTime(timezone=True)	mixin defaults

case_notes

Defined in backend/app/models/case.py:61-75 . Free-text analyst notes, optionally anchored to a target within the case.

Column	Type	Constraints
id	UUID	PK
case_id	UUID	FK -> cases.id , not null, indexed
author_id	UUID	FK -> users.id , not null
body	Text	not null
anchor_type	String(64)	nullable — one of ioc evidence graph_node timeline_event provider_result (by convention, not enforced)
anchor_ref	String(2048)	nullable
created_at / updated_at	DateTime(timezone=True)	mixin defaults

Note: anchor_type / anchor_ref are deliberately loose strings, not foreign keys — per the model docstring, a note can point at a graph node id or timeline event id that has no dedicated table. There is **no database-level referential integrity** validating that an anchor points at a real target; any such validation would have to happen in application code.

case_reports

Defined in backend/app/models/case.py:78-88 . AI- or analyst-generated report documents attached to a case.

Column	Type	Constraints
id	UUID	PK
case_id	UUID	FK -> cases.id, not null, indexed
report_type	String(64)	not null — matches a ReportType literal in app/ai/schemas.py
title	String(255)	not null
content_markdown	Text	not null
generated_by	UUID	FK -> users.id, not null
context	JSONB	nullable
created_at / updated_at	DateTime(timezone=True)	mixin defaults

Migration history

Migrations live in `backend/alembic/versions/`. There are exactly two revisions in the history today:

Order	Revision	File	Down-revision	Creates
1	660d2aa3bc20	660d2aa3bc20_initial_schema.py	None	users, ioc_lookups, ai_summaries, correlation_edges, provider_results (+ their indexes)
2 (HEAD)	a6d3ad2bb63c	a6d3ad2bb63c_add_evidence_basket_and_case_management_.py	660d2aa3bc20	cases, basket_items, case_iocs, case_notes, case_reports, evidence_items (+ their indexes/constraints)

Applying migrations:

```
cd backend
alembic upgrade head
```

The engine URL is resolved at runtime from `Settings.database_url` (`backend/app/core/config.py:29`, default `postgresql+asyncpg://ioc:ioc@postgres:5432/ioc_intel`), which Alembic's `env.py` reads via `get_settings().database_url` — so a `.env` file or `DATABASE_URL` environment variable overrides it for both the app and migrations. See [CONFIGURATION.md](#).

`backend/app/models/__init__.py` imports every model module so `Base.metadata` is fully populated for `alembic revision --autogenerate`.

Session / engine wiring

- `backend/app/core/db.py:8` — `create_async_engine(get_settings().database_url, pool_pre_ping=True, echo=False)`.
- `backend/app/core/db.py:9` — `async_sessionmaker(bind=_engine, expire_on_commit=False, class_=AsyncSession)`.
- `get_db()` (`db.py:12-14`) is the FastAPI dependency that yields a request-scoped session.
- `new_session()` (`db.py:17-22`) returns a session directly for use outside FastAPI request scope (Celery tasks, scripts) — caller owns its lifecycle, used as `async with new_session() as db:`.

Known gaps / not implemented

- NOT IMPLEMENTED:** Neo4j graph mirroring. `correlation_edges` is documented in code as being "mirrored into Neo4j," but no Neo4j driver call exists anywhere in the codebase. Postgres is the sole store for graph edges today.
- Not implemented, by design:** soft deletes. No model has an `is_deleted` / `deleted_at` column; all deletion is a hard delete via ORM cascade.
- Not implemented, by design:** `basket_items` workflow/status fields — intentionally omitted since `cases` covers workflow tracking.
- Not enforced at the database level:** `case_notes.anchor_type` / `anchor_ref` reference targets (IOC, evidence, graph node, timeline event, provider result) by convention only; there is no FK or check constraint tying them to a real row.

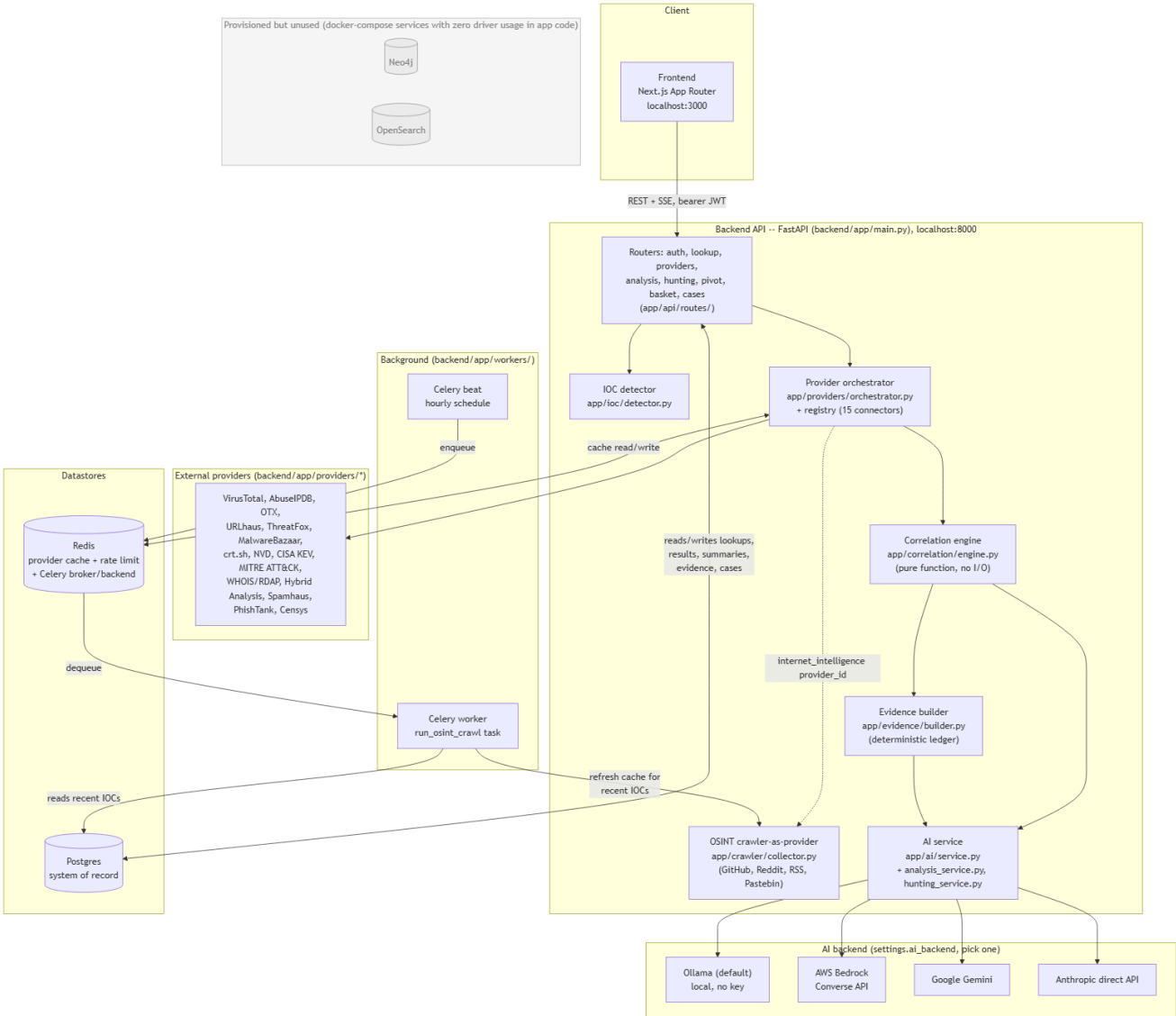
Architecture

Architecture

System design for the IOC Intelligence Platform: a single-search threat-intelligence workbench that fans one indicator of compromise (IOC) out to many providers in parallel, correlates the results, and produces an AI-generated assessment.

Related docs: [PROVIDERS.md](#) (every connector in depth), [DATA_MODEL.md](#) (full schema/ERD), [AI_ENGINE.md](#) (prompting/grounding detail), [API.md](#) (REST/SSE reference).

Component diagram



Neo4j and OpenSearch are provisioned in `docker-compose.yml` but are dead infrastructure today — `neo4j_uri` / `opensearch_url` exist as `Settings` fields (`backend/app/core/config.py`), and both services start as containers, but **no code path anywhere in `backend/app/` calls a Neo4j driver or an OpenSearch client**. `correlation_edges` in Postgres is, in practice, the sole store for the correlation graph, despite a model docstring (`app/models/lookup.py:101-103`) describing edges as "mirrored into Neo4j." This is called out explicitly rather than drawn as an active path — see [DATA_MODEL.md](#) for the model-level detail. **NOT IMPLEMENTED.**

Components

Frontend

Next.js App Router app (`frontend/app/`). A search-box home page (`app/page.tsx`) routes to `/lookup/new?value=...`, which opens the SSE stream via `lib/api.ts` (`streamLookup` — a manual `fetch` + stream reader, **not** the browser `EventSource` API, because `EventSource` cannot send an `Authorization` header and the stream endpoint requires a bearer token) and renders incoming events into state. `app/lookup/[id]/page.tsx` shows a previously-completed lookup via a plain `GET`. Other routes: `app/basket/`, `app/cases/`, `app/login/`, `app/register/`.

Backend API (FastAPI)

`backend/app/main.py` builds the FastAPI app, mounts Prometheus instrumentation at `/metrics`, and includes routers under `settings.api_v1_prefix` (`/api/v1`): `auth`, `lookup`, `providers`, `analysis`, `hunting`, `pivot`, `basket`, `cases` (`backend/app/api/routes/`). A plain `GET /health` exists outside the versioned prefix. OpenAPI/Swagger UI is auto-served at `/docs`. See [API.md](#) for the full route reference.

IOC detector

`backend/app/ioc/detector.py` (`detect_ioc_type`) classifies the raw input string into an `IOCType` enum value, or the caller can override via `ioc_type_hint`.

Provider orchestrator + registry

`backend/app/providers/base.py` defines the `BaseProvider` plugin contract. `backend/app/providers/registry.py` is the single place that imports every concrete connector and lists it in `_ALL_PROVIDERS` (15 connector modules, ordered as declared: VirusTotal, AbuseIPDB, OTX, URLhaus, ThreatFox, MalwareBazaar, crt.sh, NVD, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Hybrid Analysis, Spamhaus, PhishTank, Censys, plus the `internet_intelligence` crawler-as-provider as the 16th/last entry) — the orchestrator never imports a concrete provider directly. `backend/app/providers/orchestrator.py` (`run_all_providers` / `run_all_providers_collected`) fans one IOC out to every provider whose `supported_types` includes the detected type, concurrently, over one shared `httpx.AsyncClient`, with a Redis cache check first, then a per-attempt `asyncio.wait_for` timeout plus `tenacity` retry for connection-level failures. See [PROVIDERS.md](#) for the full fan-out diagram and every connector's detail.

OSINT crawler (`internet_intelligence` provider)

`backend/app/crawler/collector.py`'s `InternetIntelligenceCollector` is itself a `BaseProvider` (category `osint`, no key required) that fans out to 4 source modules (`app/crawler/sources/`: `github.py`, `reddit.py`, `rss_news.py`, `pastebin_search.py`) concurrently via `asyncio.gather(..., return_exceptions=True)`, dedupes findings by URL, and caps the merged result at `crawler_max_results_per_source * 4`. Supports `DOMAIN`, `IPV4`, `MALWARE_FAMILY`, `THREAT_ACTOR`, `CAMPAIGN`, `CVE`, `FILE_NAME` only (raw network atoms like JA3/mutex are excluded — free-text search on them is noise). Only `github.py` and `reddit.py` self-throttle via a process-local `AsyncMinIntervalLimiter` (6.0s / 1.1s minimum interval, not Redis-backed); `rss_news.py` and `pastebin_search.py` have no limiter and never raise `SourceRateLimitedError`.

Correlation engine

`backend/app/correlation/engine.py` (`correlate`) — a **pure function, no I/O** — takes the full list of `ProviderResult`s for one lookup and:

1. Extracts typed relationships (IP resolutions, related hashes/URLs, certificates, ASN, malware families, threat actors, campaigns, MITRE techniques, CVEs) into `GraphEdge`s, driven by the normalized `data` keys (see [PROVIDERS.md](#) for the field → edge mapping).
2. Deduplicates edges asserted identically by multiple providers, merging provenance into one edge (comma-joined provider IDs).
3. Tracks provider agreement/disagreement on verdict/reputation (`provider_agreement`) and deduplicates scalar facts (`detection_ratio`, `malicious_count`, `total_engines`, `first_seen`, `last_seen`).
4. Produces a `CorrelationResult` (`nodes`, `edges`, `deduplicated_facts`, `provider_agreement`) that feeds the SSE `correlation` event, the evidence builder, and the AI service's final-assessment prompt.

Edges are persisted as `CorrelationEdgeRecord` rows in Postgres (not, despite the model docstring, mirrored to Neo4j — see the component diagram above).

Evidence builder

`backend/app/evidence/builder.py` (`build_evidence`) builds `EvidenceItem` rows **deterministically** from `provider_results` + `provider_summaries` + the correlation result — never from a raw AI call. This ledger is what every later AI-generated explanation (WHY malicious, Score Explanation, Challenge Verdict, Copilot answers) cites by `evidence_id`, so the analysis endpoints can strip any `evidence_id` the model invents that doesn't correspond to a real row (`app/ai/analysis_service.py`'s `_strip_invalid_evidence_ids`). See [DATA_MODEL.md](#) for the table.

AI service

`backend/app/ai/service.py` owns the two-step lookup-time prompting flow; `app/ai/analysis_service.py` and `app/ai/hunting_service.py` own the on-demand, evidence-grounded explanation/hunting endpoints. Four interchangeable backends are selected via `settings.ai_backend` (`ollama` default, `bedrock`, `gemin`i, `anthropic`), all exposing an identical `call_claude_json()` method. Full prompting, grounding, and schema-validation detail is in [AI_ENGINE.md](#).

Auth / RBAC

`backend/app/auth/security.py` issues/decodes JWTs; `backend/app/auth/rbac.py`'s `require_permission(<permission>)` dependency factory gates every route by checking the current user's role against `ROLE_PERMISSIONS` (`backend/app/models/user.py`) — three roles: `admin`, `analyst`, `viewer`. See [DATA_MODEL.md](#) for the full permission matrix.

Celery workers

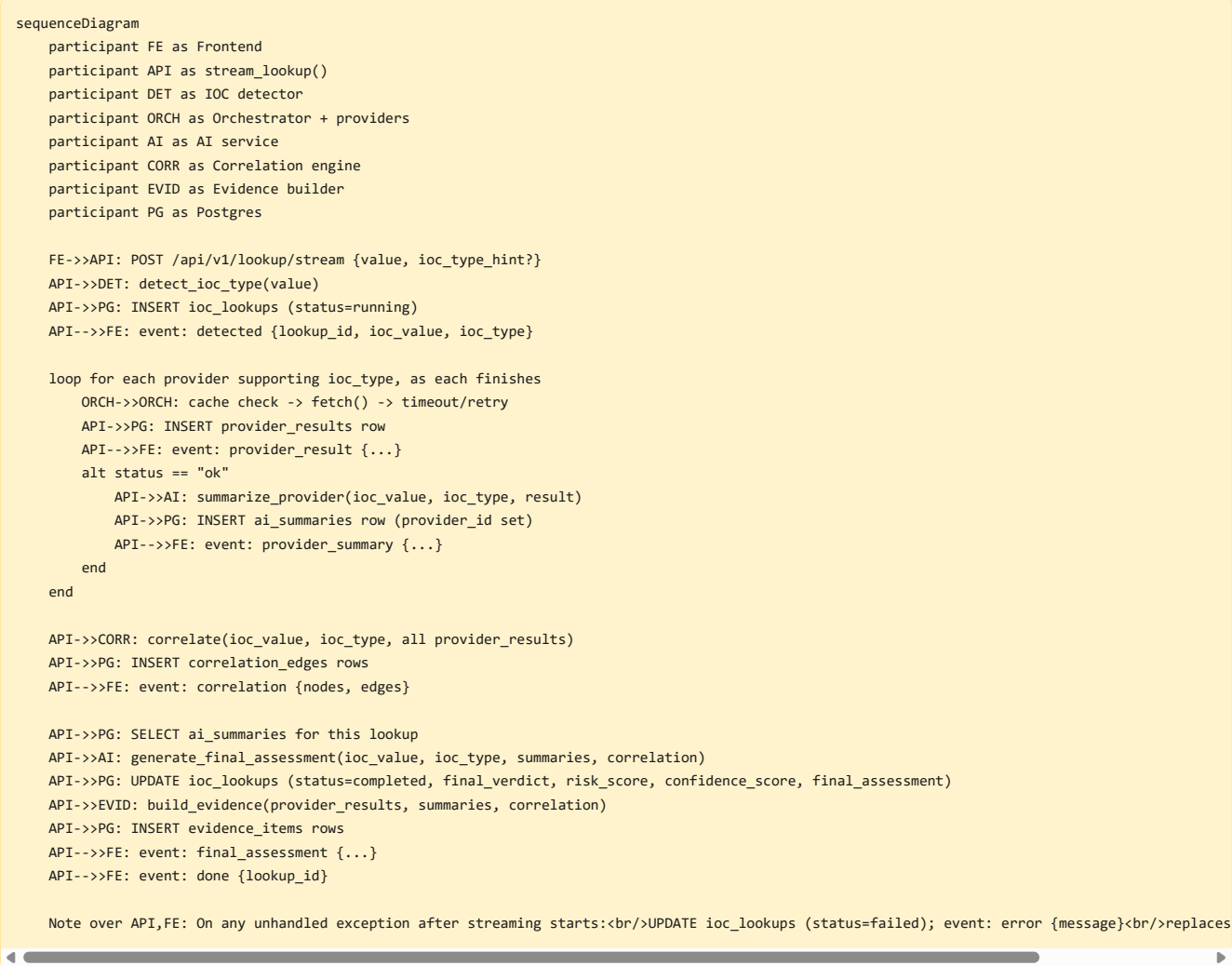
`backend/app/workers/celery_app.py` defines a Celery app (`ioc_intel_platform`, broker `redis://redis:6379/1`, result backend `redis://redis:6379/2`) for **background** work only — today, exclusively the scheduled OSINT crawl refresh (`run_osint_crawl`, `backend/app/workers/tasks.py`). `docker-compose.yml` runs a `celery_worker` service and a `celery_beat` service alongside the API. See "Celery beat: scheduled crawl refresh" below for the full task flow.

Datastore roles

Store	Role	Status
Postgres	System of record: <code>users</code> , <code>ioc_lookups</code> , <code>provider_results</code> , <code>ai_summaries</code> , <code>correlation_edges</code> , <code>evidence_items</code> , <code>basket_items</code> , <code>cases</code> , <code>case_iocs</code> , <code>case_notes</code> , <code>case_reports</code> . Async via SQLAlchemy + <code>asyncpg</code> ; schema managed by Alembic.	Active
Redis	Provider result cache (<code>app/core/cache.py</code> , keyed <code>provider_cache:{provider_id}:{ioc_type}:{sha256(ioc_value)}</code> , TTL <code>provider_cache_ttl_seconds</code> , default 3600s); fixed-window rate limiter for lookup <i>creation</i> (<code>lookup_rate_limit_max_calls</code> , default 10/60s); Celery broker (db 1) and result backend (db 2).	Active
Neo4j	<code>docker-compose.yml</code> runs a <code>neo4j:5-community</code> container and <code>Settings.neo4j_uri</code> / <code>neo4j_user</code> / <code>neo4j_password</code> exist, but no Neo4j driver call exists anywhere in the codebase .	NOT IMPLEMENTED — dead infra
OpenSearch	<code>docker-compose.yml</code> runs an <code>opensearchproject/opensearch:2.17.0</code> container and <code>Settings.opensearch_url</code> exists, but no OpenSearch client call exists anywhere in the codebase read for this platform .	NOT IMPLEMENTED — dead infra

Request lifecycle: a lookup (SSE)

Driven by `POST /api/v1/lookup/stream` (`backend/app/api/routes/lookup.py`, `stream_lookup`, requires permission `lookup:create`). Returns a `text/event-stream` response and emits SSE events in this exact order:



Step by step:

- detected** — the raw input is classified by `detect_ioc_type()` (or overridden via `ioc_type_hint`); an `IOCLookup` row is created with `status=running`. 422 if the type can't be determined and no hint was given.

2. **provider_result** × **N** — `run_all_providers` fans the IOC out to every provider whose `supported_types` includes the detected type, concurrently, over a shared `http.AsyncClient`. Each result streams the instant that provider finishes (`asyncio.as_completed` order, not registration order) and is persisted as a `ProviderResultRecord`.
3. **provider_summary** × **N** — for every `provider_result` with `status == "ok"`, `summarize_provider()` is called synchronously inside the same loop iteration (before moving to the next provider event), persisted as an `AISummaryRecord` (`provider_id` set), and streamed. Providers that returned anything other than `ok` (error/timeout/rate_limited/not_configured/unsupported_ioc/no_data) get **no** AI summary call at all.
4. **correlation** — once every provider has reported, `correlate()` runs over the full result set; its edges are persisted as `CorrelationEdgeRecord` rows and the node/edge graph is streamed. This event's payload is **not** persisted verbatim — `GET /api/v1/lookup/{id}` does not return nodes/edges at all.
5. **final_assessment** — all persisted `AISummaryRecord` rows for the lookup are re-loaded and re-validated into `ProviderSummary` objects (unparseable rows are skipped, logged, and don't sink the whole assessment), then `generate_final_assessment()` runs once over that list plus the correlation output (never over raw provider JSON directly — bounds prompt size). The `IOClookup` row is updated to `completed` with `final_verdict`, `risk_score`, `confidence_score`, and the full `FinalAssessment` JSON. The deterministic `evidence_items` ledger is built and persisted in the same step, from `provider_results` / `provider_summaries` / `correlation` only — never from the final assessment's own AI output.
6. **done** — `{lookup_id}`, terminates the stream successfully.

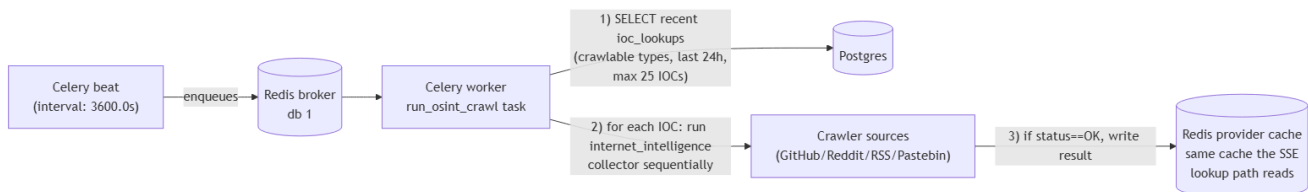
On any unhandled exception during the stream, the lookup row is marked `failed` and an `error` event (`{message}`) is emitted instead of `final_assessment` / `done`.

Full event JSON shapes are documented in [API.md](#).

Important: this SSE request path deliberately does **not** go through Celery — it runs in-process `async` inside the FastAPI worker so the UI gets live per-provider updates. The generator opens its **own** `AsyncSession` via `new_session()` rather than reusing the route's `Depends(get_db)` session, because that session is torn down as soon as the route function returns the `StreamingResponse` (before the generator body actually runs).

Celery beat: scheduled crawl refresh

The only currently-scheduled background job. `celery_app.py`'s `beat_schedule` defines `crawl-osint-sources-hourly` as a plain interval (`3600.0` seconds — **not** a cron expression) running `app.workers.tasks.run_osint_crawl`.



Flow (`backend/app/workers/tasks.py`):

1. `_recent_crawlable_iocs()` selects (`ioc_value`, `ioc_type`) from `ioc_lookups` where `ioc_type` is one of the crawlable types (`domain`, `ipv4`, `malware_family`, `threat_actor`, `campaign`, `cve`, `file_name` — mirrors the crawler's own `_SUPPORTED_TYPES`) and `created_at` is within the last 24 hours, ordered most-recent-first, deduplicated in Python, capped at 25 IOCs per run.
2. `_run_osint_crawl_async()` opens one shared `http.AsyncClient` and calls `_crawl_one()` **sequentially** per target (not concurrently); any per-target exception is caught and logged so one bad target doesn't abort the run.
3. `_crawl_one()` runs the `internet_intelligence` provider for that IOC; if the result status is `OK`, it writes the result into the **same Redis provider-result cache** the interactive SSE lookup path reads from (`set_cached_result`) — so a fresh interactive lookup for a recently-seen IOC can serve warmed OSINT findings from cache instead of re-crawling.
4. `run_osint_crawl()` is the actual Celery task (sync entrypoint, wraps the async flow in `asyncio.run(...)` since Celery workers are sync processes); its return value is the count of IOCs refreshed.

This path is entirely separate from the interactive SSE lookup — it exists to keep the provider cache warm with OSINT findings for IOCs that have already been looked up recently, not to run new lookups from scratch.

Plugin contract: `BaseProvider`

Defined in `backend/app/providers/base.py`. Every connector subclasses `BaseProvider` and sets `provider_id`, `provider_name`, `category` (`ProviderCategory`: `threat_intel`, `sandbox`, `passive_dns`, `certificate_intel`, `whois`, `vulnerability`, `osint`), `supported_types`: `set[IOType]`, `requires_key`: `bool`, a `configured`: `bool` computed from settings, and implements `async def fetch(self, ioc_value, ioc_type, client) -> ProviderResult`.

`BaseProvider.run()` (not overridden by connectors) wraps `fetch()` with timing and short-circuits: unsupported IOC types return `UNSUPPORTED_IOC` without calling `fetch`; not-configured providers return `NOT_CONFIGURED` without calling `fetch`. **Connectors must not implement their own retry/timeout loop** — the orchestrator wraps every `run()` call in `asyncio.wait_for` (`provider_timeout_seconds`, default 20s) plus a `tenacity` `retry` (`provider_max_retries`, default 2, exponential backoff) for connection-level failures, and owns Redis result caching. Full connector-by- connector detail, the fan-out diagram, and "how to add a provider" steps are in [PROVIDERS.md](#).

Deployment topology

`docker-compose.yml` services: `postgres` (5433→5432 host mapping), `redis` (6379), `neo4j` (7475/7688, provisioned but unused — see above), `opensearch` (9200, provisioned but unused), `backend` (8000, runs `alembic upgrade head` then `uvicorn --reload`), `celery_worker`, `celery_beat`, `frontend` (3000, `npm run dev`). The `backend` service's `environment`: block explicitly sets `OLLAMA_BASE_URL` to `http://host.docker.internal:11434` — the default AI backend runs on the **host**, not in a container, per a `docker-compose.yml` comment explaining that a large local model's mmap load is dramatically slower through Docker Desktop's WSL2 volume-mount layer than direct host filesystem access. (`celery_worker` / `celery_beat` don't override `OLLAMA_BASE_URL` in `docker-compose.yml`, but land on the same value via `Settings`' own default.)

See [DATA_MODEL.md](#) for the full schema/ERD and migration history, and [AI_ENGINE.md](#) for how the AI backends, grounding, and schema validation work.

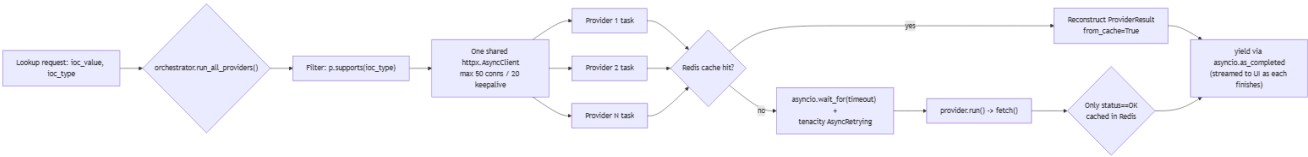
Providers

Providers Reference

Every intelligence source in the platform — real connector or stub — implements the same `BaseProvider` interface (`backend/app/providers/base.py`) and is registered as a module-level singleton in `backend/app/providers/registry.py` . The orchestrator (`backend/app/providers/orchestrator.py`) never imports a concrete provider; it only calls `BaseProvider.run()` , which is what makes adding a new source a two-line change (import + append to `_ALL_PROVIDERS`) with no other code touched.

For the overall request flow, correlation, and AI summarization stages that sit downstream of this fan-out, see [ARCHITECTURE.md](#). For the IOC type enum referenced throughout this document, see [DATA_MODEL.md](#).

How a lookup fans out



`run_all_providers_collected()` is a non-streaming wrapper over the same generator, used only by callers that need the whole batch before proceeding (the correlation engine and the AI service, both of which run after every provider has reported back).

Shared building blocks

Component	Location	Behavior
ProviderCategory enum	base.py:21-28	THREAT_INTEL , SANDBOX , PASSIVE_DNS , CERTIFICATE_INTEL , WHOIS , VULNERABILITY , OSINT
ProviderStatus enum	base.py:31-38	OK , ERROR , TIMEOUT , RATE_LIMITED , NOT_CONFIGURED , UNSUPPORTED_IOC , NO_DATA
ProviderResult dataclass	base.py:41-73	Normalized envelope every provider returns: <code>provider_id</code> , <code>provider_name</code> , <code>category</code> , <code>status</code> , <code>ioc_value</code> , <code>ioc_type</code> , <code>data</code> , <code>raw</code> , <code>source_url</code> , <code>error_message</code> , <code>latency_ms</code> , <code>fetch_at</code> , <code>from_cache</code> . <code>.to_dict()</code> serializes enums via <code>.value</code> .
BaseProvider.run()	base.py:93-153	Short-circuits to <code>UNSUPPORTED_IOC</code> if not <code>self.supports(ioc_type)</code> ; to <code>NOT_CONFIGURED</code> if <code>requires_key</code> and not <code>configured</code> . Catches <code>httpx.HTTPStatusError : status 429/403/509 -> RATE_LIMITED</code> (509 is PhishTank's documented over-limit code), any other status -> <code>ERROR</code> . Any other exception -> <code>ERROR</code> .
Retry/timeout policy	orchestrator.py:45-58	Per attempt: <code>asyncio.wait_for(timeout=settings.provider_timeout_seconds)</code> (default 20s). Retries only on <code>httpx.ConnectError</code> , <code>httpx.ReadTimeout</code> , <code>httpx.PoolTimeout</code> , via <code>tenacity.AsyncRetrying(stop_after_attempt(provider_max_retries + 1), wait_exponential(multiplier=0.5, max=4))</code> (default 2 retries). Individual connectors implement no retry logic themselves — this is enforced solely by the orchestrator, per <code>base.py</code> 's own docstring.
Redis result cache	core/cache.py:24-39 , used in orchestrator.py:32-43,80-83	Key: <code>provider_cache:{provider_id}:{ioc_type}:{sha256(ioc_value)}</code> — the IOC value itself is SHA-256-hashed before being placed in the key. Only <code>status == OK</code> results are cached, with <code>EX=settings.provider_cache_ttl_seconds</code> (default 3600s).

Not implemented: no per-provider `RateLimiter` (the Redis-backed fixed-window limiter in `core/cache.py`) is instantiated anywhere in `providers/*.py` or `providers/stubs/*.py` . That class is only used in `api/routes/lookup.py` to rate-limit lookup *creation* per user (`lookup_rate_limit_max_calls=10` per 60s window), not to self-throttle any individual connector. There is also no per-provider circuit breaker — all resilience logic lives in the orchestrator as described above.

VirusTotal

Field	Value
Provider Name	VirusTotal
provider_id	virustotal
Purpose	Multi-engine reputation lookup (public API v3)
Supported IOC Types	IPV4 , IPV6 , DOMAIN , URL , and all HASH_TYPES (MD5 , SHA1 , SHA256 , SHA512)
Authentication	Header x-apikey ; env var VIRUSTOTAL_API_KEY
Free or Paid	Free tier documented in source docstring: "4 req/min, 500/day". No paid-tier code path found in the connector.
Data Returned	Multi-engine detection data from the VT v3 API, normalized into ProviderResult.data
Failure Behavior	404 -> NO_DATA ; HTTP 429/403/509 -> RATE_LIMITED (via BaseProvider.run); other HTTP errors -> ERROR
Code quirk	URL IOCs are base64 urlsafe-encoded with trailing = stripped to build the /urls/{id} endpoint path, per VT's URL-identifier scheme (virustotal.py:35-38).

AbuseIPDB

Field	Value
Provider Name	AbuseIPDB
provider_id	abuseipdb
Purpose	Crowd-sourced IP abuse reputation
Supported IOC Types	IPV4 , IPV6
Authentication	Header Key (not Bearer / apikey); env var ABUSEIPDB_API_KEY
Free or Paid	Free tier documented in source docstring: "v2 API, free tier: 1000 checks/day". No paid-tier code path found in the connector.
Data Returned	Abuse score, report count, and derived verdict
Failure Behavior	Empty payload["data"] maps to NO_DATA (distinct from an HTTP 404)
Code quirk	Verdict logic (abuseipdb.py:61-66): score > 25 -> malicious ; elif total_reports == 0 -> unknown ; else -> clean .

AlienVault OTX

Field	Value
Provider Name	AlienVault OTX
provider_id	otx
Purpose	Community threat-pulse correlation
Supported IOC Types	IPV4 , IPV6 , DOMAIN , HOSTNAME , URL , plus all HASH_TYPES
Authentication	Header X-OTX-API-KEY ; env var OTX_API_KEY
Free or Paid	Docstring: "Requires a free OTX API key".
Data Returned	Pulse count and related pulse metadata
Failure Behavior	pulse_count == 0 maps to NO_DATA even on an HTTP 200
Code quirk	Maps IOCType to a URL-path "section" via a _SECTION_BY_TYPE dict; all hash types are hardcoded to the file section (otx.py:37-41). Verdict = malicious if pulse_count > 0 else unknown — OTX never returns clean .

URLhaus

Field	Value
Provider Name	URLhaus
provider_id	urlhaus
Purpose	Malicious URL / malware-hosting-host database (abuse.ch)
Supported IOC Types	URL , DOMAIN , IPV4
Authentication	Header Auth-Key ; env var ABUSECH_AUTH_KEY (shared across URLhaus, ThreatFox, MalwareBazaar)
Free or Paid	Docstring: "now requires a free abuse.ch account Auth-Key".
Data Returned	Host/URL malware-hosting records
Failure Behavior	Non- "ok" query_status delegates to the shared map_query_status() helper (see abuse.ch helper below)
Code quirk	DOMAIN and IPV4 are both routed to the /host/ endpoint as a "host" lookup; only URL uses /url/ (urlhaus.py:32-37).

ThreatFox

Field	Value
Provider Name	ThreatFox
provider_id	threatfox
Purpose	IOC-to-malware-family attribution (abuse.ch)
Supported IOC Types	IPV4 , IPV6 , DOMAIN , URL , MD5 , SHA256 (note: only two of the four hash types — not SHA1 / SHA512)
Authentication	Header Auth-Key ; env var ABUSECH_AUTH_KEY
Free or Paid	Free abuse.ch account (same key family as URLhaus/MalwareBazaar).
Data Returned	Matched IOC entries with malware-family attribution
Failure Behavior	Shared abuse.ch query_status mapping (see below)
Code quirk	An "ok" query_status with zero entries is explicitly downgraded from OK to NO_DATA (threatfox.py:56-60) — code comment: "ok" with zero entries is a genuine empty result, not an error .

MalwareBazaar

Field	Value
Provider Name	MalwareBazaar
provider_id	malwarebazaar
Purpose	Malware-sample hash lookup (abuse.ch)
Supported IOC Types	All HASH_TYPES (MD5 , SHA1 , SHA256 , SHA512)
Authentication	Header Auth-Key ; env var ABUSECH_AUTH_KEY ; request body form={"query": "get_info", "hash": ioc_value}
Free or Paid	Free abuse.ch account (same key family as URLhaus/ThreatFox).
Data Returned	Malware sample metadata
Failure Behavior	Same "ok" + empty-entries -> NO_DATA override as ThreatFox (malwarebazaar.py:49-53)
Code quirk	Only entries[0] is used even if the API returns multiple matching entries (malwarebazaar.py:66).

Shared abuse.ch helper (backend/app/providers/abusech.py)

Used by URLhaus, ThreatFox, and MalwareBazaar. map_query_status(query_status) returns (ProviderStatus, error_message) :

query_status value	Mapped status
"ok"	OK
"no_api_key" , "invalid_api_key" , "unauthorized"	ERROR ("abuse.ch Auth-Key rejected (query_status=...)")
anything else	NO_DATA

Rationale (docstring, abusech.py:5-9): auth failures must map to ERROR , not NO_DATA , because NO_DATA is rendered by the UI as "nothing malicious found" — a misconfigured key must never look like a clean verdict.

crt.sh

Field	Value
Provider Name	crt.sh
provider_id	crtsh
Purpose	Certificate Transparency log search
Supported IOC Types	DOMAIN , TLS_CERTIFICATE
Authentication	None — requires_key=False , configured=True always
Free or Paid	Free community service, no key.
Data Returned	Up to 25 certificates (_MAX_CERTIFICATES=25)
Failure Behavior	JSON parse failures (json.JSONDecodeError / ValueError) are explicitly caught and degraded to NO_DATA (not raised) with error_message="crt.sh returned malformed JSON"
Code quirk	Docstring explicitly documents crt.sh as "sometimes slow/flaky" and "known to emit malformed JSON" — the malformed-JSON handling exists specifically to work around that (crtsh.py:8-13,58-73).

NIST NVD

Field	Value
Provider Name	NIST NVD
provider_id	nvd
Purpose	CVE detail and CVSS scoring lookup
Supported IOC Types	CVE
Authentication	Optional header apiKey ; env var NVD_API_KEY — purely a rate-limit booster, not required for access
Free or Paid	Free. Docstring: works without a key at ~5 req/30s; with NVD_API_KEY set, ~50 req/30s.
Data Returned	CVE detail, best-available CVSS metric, derived severity
Failure Behavior	Standard BaseProvider HTTP-error handling (no CVE-specific NO_DATA override documented)
Code quirk	_best_metric() prefers CVSS v3.1 over v3.0 over v2, and within a metric list prefers the entry with type == "Primary" (nvd.py:78-91); severity -> verdict mapping: CRITICAL / HIGH -> malicious , MEDIUM / LOW -> suspicious , else unknown .

CISA Known Exploited Vulnerabilities (KEV)

Field	Value
Provider Name	CISA Known Exploited Vulnerabilities
provider_id	cisa_kev
Purpose	Confirmed-exploited-in-the-wild CVE catalog
Supported IOC Types	CVE
Authentication	None — fully free/public
Free or Paid	Free, public.
Data Returned	Matched KEV catalog entry
Failure Behavior	Standard BaseProvider HTTP-error handling
Code quirk	The entire ~1MB catalog is cached in a module-level in-memory dict, refreshed at most once per hour (_CACHE_TTL_SECONDS=3600), guarded by an asyncio.Lock with a double-check-after-acquire pattern (cisa_kev.py:20-55). Any matched entry gets verdict="malicious" unconditionally, since presence in KEV means confirmed exploitation.

MITRE ATT&CK

Field	Value
Provider Name	MITRE ATT&CK
provider_id	mitre_attack
Purpose	ATT&CK technique lookup from the Enterprise ATT&CK STIX bundle
Supported IOC Types	MITRE_TECHNIQUE
Authentication	None — free, public STIX bundle
Free or Paid	Free, public.
Data Returned	Technique detail from the STIX bundle
Failure Behavior	Standard <code>BaseProvider</code> HTTP-error handling
Code quirk	Docstring notes the bundle is "~30MB"; uses the same module-level cache + <code>asyncio.Lock</code> pattern as CISA KEV (TTL 3600s), but the fetch call overrides the client timeout to an explicit <code>timeout=60.0</code> (<code>mitre_attack.py:61</code>) since the bundle download can legitimately exceed the orchestrator's default <code>provider_timeout_seconds</code> .

WHOIS/RDAP

Field	Value
Provider Name	WHOIS/RDAP
provider_id	whois_rdap
Purpose	Domain WHOIS and IP/ASN RDAP registration lookup
Supported IOC Types	DOMAIN , IPV4 , IPV6 , ASN
Authentication	None
Free or Paid	Free.
Data Returned	WHOIS record (domains) or RDAP record (IPs/ASNs)
Failure Behavior	Socket failures during the WHOIS call raise real exceptions that map to <code>TIMEOUT</code> / <code>ERROR</code> rather than being silently swallowed
Code quirk	<code>DOMAIN</code> lookups run the blocking <code>python-whois</code> library off the event loop via <code>asyncio.to_thread</code> , deliberately passing <code>ignore_socket_errors=False</code> — overriding the library's own default of <code>True</code> — precisely so socket errors don't get silently absorbed into unparseable text that would otherwise look like a clean <code>NO_DATA</code> result. <code>IPV4</code> / <code>IPV6</code> / <code>ASN</code> instead query RDAP against <code>https://rdap.org</code> , which bootstraps to whichever Regional Internet Registry actually holds the record.

Stub Connectors

These implement the identical `BaseProvider` interface and are fully wired into the registry and orchestrator, but the platform ships without valid credentials for the ones that require a key. Per the top-level README, "adding a new provider is a two-line change" — activating these is just a matter of populating the relevant `.env` variable.

Hybrid Analysis (Falcon Sandbox) — stub

Field	Value
Provider Name	Hybrid Analysis (Falcon Sandbox)
provider_id	hybrid_analysis
Purpose	Sandbox detonation report lookup
Supported IOC Types	SHA256 only
Authentication	Header <code>api-key</code> ; env var <code>HYBRID_ANALYSIS_API_KEY</code> . Also requires header <code>user-agent: Falcon Sandbox</code> (Hybrid Analysis rejects requests without a descriptive user-agent) and <code>accept: application/json</code> .
Free or Paid	Docstring explicitly: "requires a paid/free-tier API key".
Data Returned	Sandbox summary/overview for a SHA256 sample
Failure Behavior	Both HTTP 400 and HTTP 404 map to <code>NO_DATA</code>
Code quirk	Documented in the module docstring: the historical <code>/api/v2/search/hash</code> endpoint is deprecated and now returns HTTP 410 Gone, so the connector was rewritten against <code>GET /api/v2/overview/{sha256}/summary</code> — which only accepts SHA256, which is why <code>MD5</code> / <code>SHA1</code> support was dropped from <code>supported_types</code> . URL support was removed entirely because the documented <code>/api/v2/search/terms</code> parameter contract "could not be confirmed against the live API" — every plausible field name tried returned <code>"Search terms were not given"</code> .

Spamhaus DBL/ZEN — stub

Field	Value
Provider Name	Spamhaus DBL/ZEN
provider_id	spamhaus
Purpose	DNSBL-based IP/domain reputation
Supported IOC Types	IPV4 , DOMAIN
Authentication	None — <code>requires_key=False</code> , <code>configured=True</code>
Free or Paid	Free public DNSBL, no key.
Data Returned	Listing reason decoded from a DNSBL return code
Failure Behavior	Both "listed" and "not listed" outcomes return <code>ProviderStatus.OK</code> — never <code>NO_DATA</code> / <code>ERROR</code> — because a negative DNS lookup is itself a valid, definitive DNSBL answer
Code quirk	Makes no HTTP call at all . Performs a DNS A-record lookup via <code>loop.getaddrinfo()</code> against <code>{reversed-octets}.zen.spamhaus.org</code> (IPs) or <code>{domain}.dbl.spamhaus.org</code> (domains); a returned <code>127.0.0.x</code> / <code>127.0.1.x</code> address is decoded against hardcoded <code>_ZEN_CODES</code> / <code>_DBL_CODES</code> tables to build a human-readable listing reason. <code>socket.gaierror</code> (NXDOMAIN) is treated as "not listed" -> <code>verdict="clean"</code> .

PhishTank — stub

Field	Value
Provider Name	PhishTank
provider_id	phishtank
Purpose	Community-verified phishing URL lookup
Supported IOC Types	URL
Authentication	None required; optional form field <code>app_key</code> , env var <code>PHISHTANK_API_KEY</code> , sent only if set, purely to raise rate limits
Free or Paid	Docstring: "free community-sourced phishing URL verification... works without an API key".
Data Returned	Phishing-verification record from the PhishTank database
Failure Behavior	<code>results.get("in_database")</code> is <code>False</code> -> <code>NO_DATA</code> . HTTP 509 (PhishTank's documented over-limit code) -> <code>RATE_LIMITED</code> via <code>BaseProvider.run()</code> .
Code quirk	Verdict logic (<code>phishtank.py:59-60</code>): <code>valid == True</code> -> <code>malicious</code> ; else -> <code>suspicious</code> — PhishTank never returns <code>clean</code> , since being "in_database" at all implies some level of suspicion even if community-voted not valid.

Censys — stub

Field	Value
Provider Name	Censys
provider_id	censys
Purpose	Internet asset / exposure data for an IP
Supported IOC Types	IPV4 , IPV6
Authentication	Header <code>Authorization: Bearer {token}</code> plus header <code>X-Organization-ID</code> ; env vars <code>CENSYS_PERSONAL_ACCESS_TOKEN</code> and <code>CENSYS_ORGANIZATION_ID</code> — both must be set for <code>configured</code> to be <code>True</code>
Free or Paid	Not verified — requires a Personal Access Token + Organization ID against the Censys Platform API (<code>platform.censys.io</code>), implying a registered/paid account, but no explicit free/paid statement exists in code.
Data Returned	Host asset/exposure data via <code>GET /v3/global/asset/host/{ip}</code>
Failure Behavior	Standard <code>BaseProvider</code> HTTP-error handling
Code quirk	Verdict is hardcoded to <code>"unknown"</code> always (<code>censys.py:76</code>) — Censys supplies asset/exposure data, not a malicious/clean verdict, unlike every other threat-intel connector in this platform.

Note on the `stubs/` naming: the `.env.example` comment groups Hybrid Analysis, Censys, and PhishTank under "Paid providers (stub connectors)", but Spamhaus (also in `stubs/`) requires no key and is always free/configured, and PhishTank itself is explicitly free/no-key in its own docstring. The directory name `stubs/` does not consistently mean "paid" — it groups connectors that ship without a working default credential, not connectors that are inherently paid.

Internet Intelligence Collector (OSINT crawler-as-provider)

Field	Value
Provider Name	Internet Intelligence Collector
provider_id	internet_intelligence
Purpose	Best-effort OSINT crawl across four open-internet sources (GitHub, Reddit, RSS security news, paste-dump search), wrapped behind the standard <code>BaseProvider</code> contract so it flows through the same orchestrator, cache, and API surface as every vendor connector
Supported IOC Types	<code>DOMAIN</code> , <code>IPV4</code> , <code>MALWARE_FAMILY</code> , <code>THREAT_ACTOR</code> , <code>CAMPAIGN</code> , <code>CVE</code> , <code>FILE_NAME</code> (raw network atoms like JA3 hashes or mutexes are excluded — free-text search on them is "nearly always noise" per the module docstring)
Authentication	None — <code>requires_key=False</code> , <code>configured=True</code> ; every underlying source is unauthenticated / best-effort
Free or Paid	Free — all four underlying sources are unauthenticated.
Data Returned	Deduplicated <code>{title, url, snippet, published_at, source}</code> hits merged from all four crawler sources, each retaining its original source URL for attribution
Failure Behavior	Individual source failures are tolerated (the module runs all four sources concurrently and continues if one fails)
Code quirk	Category is <code>OSINT</code> (the only provider using that category); it is registered from <code>app.crawler</code> , not <code>app.providers</code> , and is the 16th and last entry in the registry's <code>_ALL_PROVIDERS</code> list.

The crawler itself uses a separate, non-Redis, process-local `AsyncMinIntervalLimiter` (`backend/app/crawler/sources/rate_limit.py:17-43`) to self-throttle its GitHub (6.0s min interval) and Reddit (1.1s min interval) source calls — this is distinct from the Redis-backed `RateLimiter` in `core/cache.py` and is not shared across worker processes.

Configuration reference

Every provider credential is read once, centrally, in `backend/app/core/config.py` via `pydantic-settings`, and mirrored in `.env.example`:

```
# --- Free-tier providers (real connectors) ---
VIRUSTOTAL_API_KEY=<your-key-here>
ABUSEIPDB_API_KEY=<your-key-here>
OTX_API_KEY=<your-key-here>
NVD_API_KEY=<your-key-here>           # optional: raises NVD rate limit if set
ABUSECH_AUTH_KEY=<your-key-here>     # shared by URLhaus, ThreatFox, MalwareBazaar

# --- Paid providers (stub connectors -- add key to activate) ---
HYBRID_ANALYSIS_API_KEY=<your-key-here>
CENSYS_PERSONAL_ACCESS_TOKEN=<your-key-here>
CENSYS_ORGANIZATION_ID=<your-org-id-here>
PHISHTANK_API_KEY=<your-key-here>    # optional: raises PhishTank rate limits
```

Provider execution tuning (also in `.env / config.py`, defaults shown):

```
provider_timeout_seconds=20      # per-attempt asyncio.wait_for timeout
provider_max_retries=2           # tenacity retries on connect/read/pool timeout
provider_cache_ttl_seconds=3600  # Redis TTL for cached OK results
```

See [CONFIGURATION.md](#) for the full environment-variable reference across the whole platform, and [SECURITY.md](#) for credential-handling guidance. Never commit real key values — use `<configure securely>` placeholders in any shared `.env` file, and keep actual secrets out of version control.

Health endpoint

`GET /providers/health` (prefix `/providers`, gated by `require_permission("lookup:read")`, `backend/app/api/routes/providers.py:9-11`) returns `get_provider_health()` (`backend/app/providers/registry.py:53-64`) — a list of, for every registered provider:

```
{
  "provider_id": "virustotal",
  "provider_name": "VirusTotal",
  "category": "threat_intel",
  "configured": true,
  "requires_key": true,
  "supported_types": ["domain", "ipv4", "ipv6", "md5", "sha1", "sha256", "sha512", "url"]
}
```

`configured` is computed once, at process import time, from `get_settings()` when each provider's module-level singleton is instantiated. Setting an API key in `.env` after the backend process has started will not retroactively flip `configured` for that provider — a process restart is required. See [API_DOCUMENTATION.md](#) for the full REST/SSE API reference this endpoint is part of.

All providers at a glance

provider_id	category	requires_key	configured by default	Notes
virustotal	threat_intel	true	no (needs <code>VIRUSTOTAL_API_KEY</code>)	Free tier: 4 req/min, 500/day
abuseipdb	threat_intel	true	no (needs <code>ABUSEIPDB_API_KEY</code>)	Free tier: 1000 checks/day
otx	threat_intel	true	no (needs <code>OTX_API_KEY</code>)	Free key required
urlhaus	threat_intel	true	no (needs <code>ABUSECH_AUTH_KEY</code>)	Shared abuse.ch key
threatfox	threat_intel	true	no (needs <code>ABUSECH_AUTH_KEY</code>)	Shared abuse.ch key
malwarebazaar	threat_intel	true	no (needs <code>ABUSECH_AUTH_KEY</code>)	Shared abuse.ch key
crtsh	certificate_intel	false	yes	No key, free community service
nvd	vulnerability	false	yes	<code>NVD_API_KEY</code> optional, boosts rate limit only
cisa_kev	vulnerability	false	yes	Free, public catalog
mitre_attack	threat_intel	false	yes	Free, public STIX bundle
whois_rdap	whois	false	yes	No key
hybrid_analysis	sandbox	true	no (needs <code>HYBRID_ANALYSIS_API_KEY</code>)	Paid/free-tier key per docstring
spamhaus	threat_intel	false	yes	DNS-only, no HTTP call
phishtank	threat_intel	false	yes	<code>PHISHTANK_API_KEY</code> optional, boosts rate limit only
censys	passive_dns	true	no (needs <code>PAT</code> + <code>org ID</code>)	Free/paid tier not verified
internet_intelligence	osint	false	yes	Crawler-as-provider (GitHub/Reddit/RSS/paste search)

Adding a new provider

- Subclass `BaseProvider` in a new module under `backend/app/providers/` (or `backend/app/providers/stubs/` if it ships without working default credentials).
- Implement `async def fetch(self, ioc_value, ioc_type, client) -> ProviderResult`. Do not implement your own retry/timeout loop — that is the orchestrator's job.
- Export a module-level singleton instance.
- Import and append that instance to `_ALL_PROVIDERS` in `backend/app/providers/registry.py`.

No other file needs to change — the orchestrator, API routes, and correlation engine all discover providers exclusively through the registry.

AI Engine

AI Engine

How the platform turns raw provider data into an AI-written `FinalAssessment`, which backend actually receives your data, and the specific mechanisms that stop a local or cloud LLM from hallucinating facts into a security verdict.

Source: `backend/app/ai/` (`service.py`, `schemas.py`, `schema_utils.py`, `ollama_client.py`, `anthropic_client.py`, `gemini_client.py`, `bedrock_client.py`, `analysis_service.py`, `hunting_service.py`), plus the call sites in `backend/app/api/routes/lookup.py` and the deterministic `backend/app/correlation/engine.py` and `backend/app/evidence/builder.py` that produce everything the AI is grounded in. See also [ARCHITECTURE.md](#) for the full request lifecycle, [PROVIDERS.md](#) for the connector contract, and [DATA_MODEL.md](#) for the tables mentioned below.

1. Backend selection: `_get_ai_client()`

`backend/app/ai/service.py:41-77`. One setting, `ai_backend`, picks which of four client modules handles every AI call for the whole platform:

```
# backend/app/core/config.py
ai_backend: str = "ollama" # "ollama", "anthropic", "gemini", or "bedrock"
```

ai_backend value	Client module	is_configured check	Notes
"ollama" (default, and any unrecognized value)	<code>app/ai/ollama_client.py</code>	<code>bool(base_url and model)</code> — true out of the box because both have non-empty defaults	Self-hosted, no API key, no cloud cost
"anthropic"	<code>app/ai/anthropic_client.py</code>	<code>bool(settings.anthropic_api_key)</code>	Direct Anthropic Messages API
"gemini"	<code>app/ai/gemini_client.py</code>	<code>bool(settings.gemini_api_key)</code>	Google Gemini API
"bedrock"	<code>app/ai/bedrock_client.py</code>	<code>bool(settings.bedrock_api_key or (aws_access_key_id and aws_secret_access_key))</code>	AWS Bedrock Converse API

If the resolved client's `is_configured` is `False`, `_get_ai_client()` raises `RuntimeError(f"AI backend '{settings.ai_backend}' is not configured (missing API key/URL/model) -- check the matching settings in .env")`. Note the fallback behavior: `_get_ai_client()` only branches on the literal strings `"bedrock"`, `"gemini"`, `"anthropic"` — **any other value (including typos) silently falls through to Ollama**, it does not raise on an unrecognized `ai_backend` string.

Model defaults per backend

Backend	Model setting	Default	Max tokens setting	Default
Ollama	<code>ollama_model</code>	<code>llama3.2:3b</code>	<code>ollama_max_tokens</code>	8192
Anthropic	<code>anthropic_model_id</code>	<code>claude-sonnet-4-5-20250929</code>	<code>anthropic_max_tokens</code>	8192
Gemini	<code>gemini_model_id</code>	<code>gemini-2.0-flash</code>	<code>gemini_max_tokens</code>	8192
Bedrock	<code>bedrock_model_id</code>	<code>global.anthropic.claude-sonnet-4-5-20250929-v1:0</code>	<code>bedrock_max_tokens</code>	4096

Ollama also defaults `ollama_base_url` to `http://host.docker.internal:11434` (reaches the host machine's Ollama server from inside the backend container).

Structured-output mechanism per backend

All four clients expose the same method, `call_claude_json(system_prompt, user_prompt, json_schema, tool_name, max_tokens=...)`, but force valid JSON out of the model in backend-specific ways:

Backend	Mechanism	Schema \$ref / \$defs handling	Retry policy
Ollama	POST {base_url}/api/chat with format: <json schema> — grammar-constrained decoding (server 0.5+)	Flattened first via inline_refs() (llama.cpp does not resolve \$ref)	None — single attempt; httpx.ConnectError → RuntimeError("Could not reach Ollama at {base_url} -- is it running?"); TimeoutException (120s, _TIMEOUT_SECONDS) → RuntimeError("Ollama did not respond within 120s...")
Anthropic	POST /v1/messages with tools=[{name, input_schema}] + forced tool_choice={ "type": "tool", "name": ... }; payload extracted from the matching tool_use block	Sent as-is — Anthropic's Messages API resolves standard JSON Schema \$ref / \$defs natively	None — single attempt
Gemini	generateContent with responseMimeType: application/json, responseSchema: <flat schema>, temperature: 0.1	Flattened via inline_refs() (Gemini's OpenAPI-3.0-style responseSchema doesn't resolve \$ref)	None — single attempt
Bedrock	Converse API with toolConfig forcing toolChoice={ "tool": { "name": ... } }; payload from the toolUse block	Sent as-is (native tool-use)	Yes, but only at the HTTP layer: boto3 client constructed with BotoConfig(retries={ "max_attempts": 3, "mode": "adaptive" }) — this is botocore retrying transport-level failures, not the AI service retrying on validation failure

NOT IMPLEMENTED: app/ai/schemas.py 's module docstring claims "the AI service retries once with the validation error appended to the prompt" for malformed output. No such retry-with-validation-error-appended logic exists anywhere in service.py, analysis_service.py, or hunting_service.py. Every one of those functions goes straight from a validation/call exception to a static fallback object on the **first** failure. Bedrock's automatic retry (above) is a botocore transport retry, unrelated to schema validation.

The Ollama grammar-compiler gotcha (confirmed, live bug)

backend/app/ai/schemas.py:62-71, ON MitreMapping.technique_id:

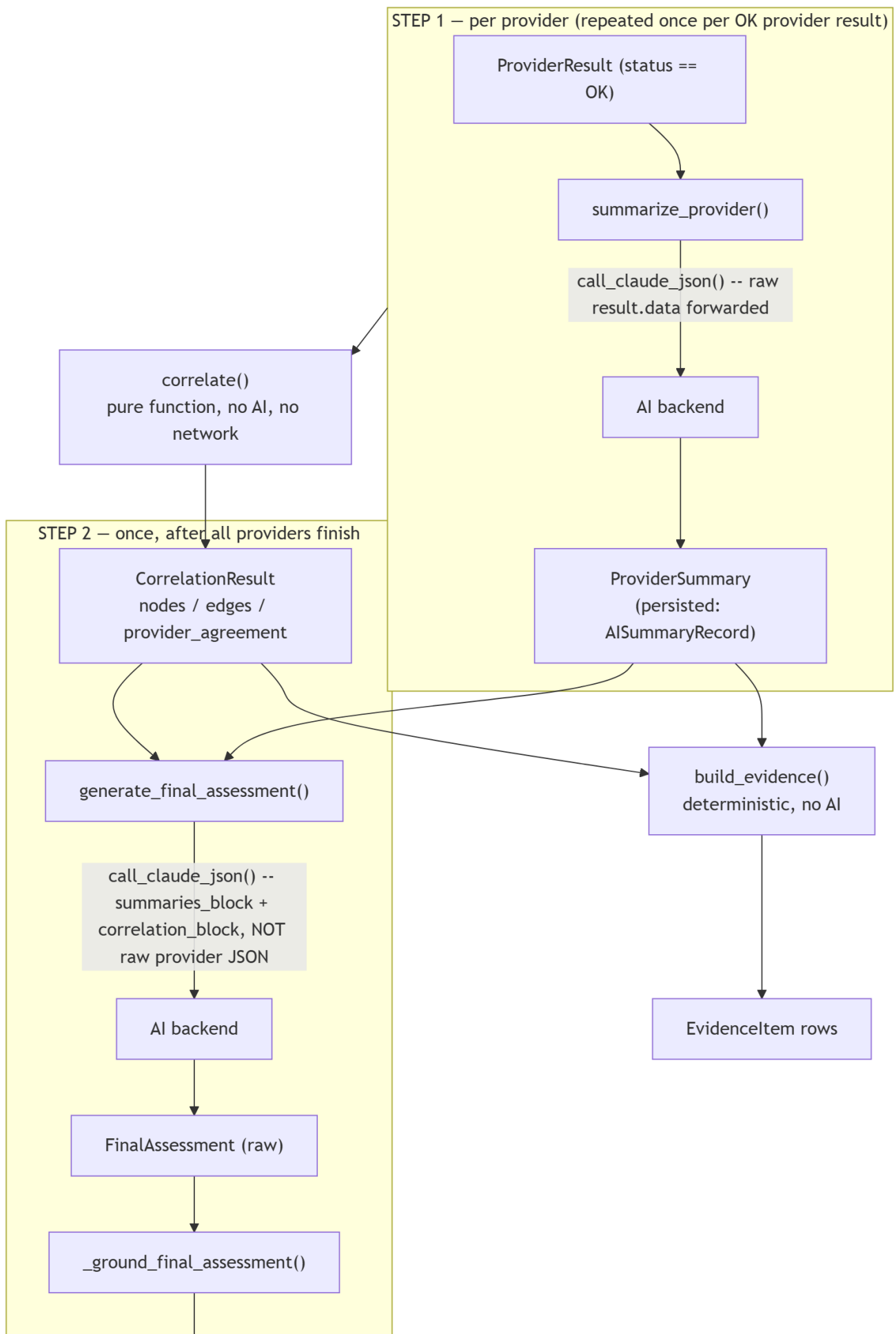
"No pattern= here on purpose: Ollama's grammar compiler (llama.cpp json-schema-to-grammar, observed on server 0.32.6) fails to compile a regex pattern constraint into decoding grammar and returns HTTP 400 'failed to parse grammar', which breaks every generate_final_assessment() call."

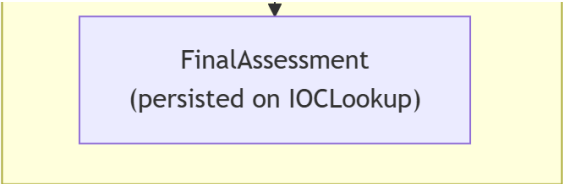
This is not scoped to just that one field — **Ollama compiles the entire JSON Schema into a single decoding grammar**, so one unsupported constraint anywhere in FinalAssessment 's schema fails the whole compile with HTTP 400 and takes down every call using that schema, not just the offending field. The fix in place is to never use Pydantic's pattern= on any field whose schema is sent to Ollama; a malformed technique_id is tolerated because _ground_final_assessment (below) only checks it for membership against real correlation-graph technique IDs, not format, so loose typing costs nothing downstream. Field description= text is kept anyway because it still guides cloud models (Claude/Gemini/Bedrock), which don't compile schemas into a grammar.

Practical rule for anyone editing app/ai/schemas.py : don't add pattern= (or any other constraint llama.cpp 's grammar compiler doesn't support) to any field — it will silently break the whole Ollama backend, not just fail one field, and the failure mode is an opaque HTTP 400 from Ollama's side, not a Pydantic error.

2. The two-call pipeline

app/ai/service.py has exactly two AI entry points, called from the SSE lookup stream (backend/app/api/routes/lookup.py) in this order:





FinalAssessment
(persisted on IOCLookup)

STEP 1 — `summarize_provider(ioc_value, ioc_type, result)`

- **Skips the AI backend entirely** if `result.status != ProviderStatus.OK` or `result.data` is empty — returns a degraded `ProviderSummary` (`what_it_knows="No data returned by this provider for this IOC."`, `reputation="unknown"`, `detection_status="no data"`, `threat_level="none"`, `confidence="low"`, `caveats=result.error_message`) with **zero** network calls.
- Otherwise builds a prompt from `_provider_result_to_prompt(result)` — provider name, id, category, status, `source_url`, and the **full raw** `result.data` dict for that one provider — and calls `call_claude_json(system_prompt=_PROVIDER_SUMMARY_SYSTEM_PROMPT, ..., json_schema=ProviderSummary.model_json_schema(), tool_name="emit_provider_summary")`.
- Sets `payload["provider_id"] = result.provider_id` before validating (the model never has to get the ID right).
- On **any** exception: logs a warning, returns a degraded `ProviderSummary` with `caveats=repr(exc)`.

STEP 2 — `generate_final_assessment(ioc_value, ioc_type, provider_summaries, correlation)`

- Runs **once**, after every provider has reported and `correlate()` has run.
- Prompt is built from two blocks — **not** raw provider JSON, which keeps prompt size bounded as the number of providers grows:
 - `summaries_block`: per-provider `reputation` / `threat_level` / `confidence` / `what_it_knows` / `findings` / `relationships`, from the already-AI-generated `ProviderSummary` objects.
 - `correlation_block`: the `provider_agreement` dict, `deduplicated_facts`, and up to the **first 50** `correlation.edges`, each formatted as `"source -- relationship--> target [provenance]"`.
- Calls `call_claude_json(system_prompt=_FINAL_ASSESSMENT_SYSTEM_PROMPT, ..., json_schema=FinalAssessment.model_json_schema(), tool_name="emit_final_assessment", max_tokens=8192)` — `max_tokens` is hardcoded to 8192 for this call regardless of the backend's configured default.
- Sets `payload["ioc_value"] / payload["ioc_type"]` before validating, then calls `_ground_final_assessment(assessment, correlation)` (§3) before returning.
- On **any** exception (including a `FinalAssessment` schema validation failure — see §3): logs a warning, returns a fallback `FinalAssessment` with every summary/text field stating generation failed, `risk=RiskAssessment(overall_risk_score=0, confidence_score=0, severity="none", reputation="unknown", malicious_probability=0, analyst_confidence="low")`, `final_verdict="unknown"`, and `verdict_rationale` citing the exception's `repr()`.

Both system prompts (`_PROVIDER_SUMMARY_SYSTEM_PROMPT`, `_FINAL_ASSESSMENT_SYSTEM_PROMPT`, `service.py:80-119`) explicitly instruct the model to **never fabricate** and to state that `overall_risk_score` / `confidence_score` / `malicious_probability` are always on a **0-100 scale**, never a 0-1 fraction — with an instruction to multiply by 100 if the model was about to think in probability terms.

3. `FinalAssessment` — full field reference

`backend/app/ai/schemas.py:134-159`. This is the only object persisted onto `IOCLookup` and rendered as the platform's final verdict.

Field	Type	Notes
ioc_value	str	Overwritten by <code>service.py</code> before validation, not trusted from the model
ioc_type	str	Same
executive_summary	str, min length 1	
technical_summary	str, min length 1	
threat_assessment	str, min length 1	
supporting_evidence	list[str]	
agreeing_providers	list[str]	Filtered by <code>_ground_final_assessment</code>
disagreeing_providers	list[str]	Filtered by <code>_ground_final_assessment</code>
relationships_summary	str, min length 1	
mitre_mappings	list[MitreMapping]	<code>grounded</code> field relabeled by <code>_ground_final_assessment</code>
risk	RiskAssessment	See below
detection_rules	list[DetectionRule]	<code>format</code> is one of 9 Literal values: <code>sigma</code> , <code>yara</code> , <code>splunk_spl</code> , <code>sentinel_ql</code> , <code>elastic</code> , <code>qradar_aql</code> , <code>suricata</code> , <code>snort</code> , <code>zeek</code>
recommended_actions	list[str]	
investigation_priorities	list[str]	
incident_response_recommendations	list[str]	
final_verdict	Verdict enum	See values below
verdict_rationale	str	

`RiskAssessment` (`schemas.py:101-131`):

Field	Type	Range/enum	Notes
overall_risk_score	float	0-100	"NOT a 0-1 probability"
confidence_score	float	0-100	"NOT a 0-1 probability"
severity	Literal	<code>none</code> , <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>	
reputation	Literal	<code>malicious</code> , <code>suspicious</code> , <code>clean</code> , <code>unknown</code> , <code>no data</code>	
malicious_probability	float	0-100	e.g. <code>50</code> , not <code>0.5</code>
analyst_confidence	Literal	<code>low</code> , <code>medium</code> , <code>high</code>	

`MitreMapping` (`schemas.py:61-92`): `technique_id`: str (no regex constraint — see the Ollama gotcha above), `technique_name`: str, `tactic` (Literal of the 14 ATT&CK tactic slugs), `kill_chain_stage`: Optional[str], `rationale`: str, `grounded`: bool = True (overwritten by `_ground_final_assessment`, not trusted from the model).

`Verdict enum` (`backend/app/models/lookup.py:22-34`) — the exact set of values `final_verdict` can take: `highly_malicious`, `malicious`, `suspicious`, `unknown`, `likely_benign`, `benign`, `scanner`, `tor_exit_node`, `vpn`, `cdn`, `cloud_infrastructure`, `dormant_infrastructure`.

Verdict/risk consistency enforcement

`FinalAssessment._verdict_must_agree_with_risk` (`model_validator`, `schemas.py:161-182`) raises `ValueError` if:

- `final_verdict` is `malicious` / `highly_malicious` **and** `risk.malicious_probability` < `30`, or
- `final_verdict` is `benign` / `likely_benign` **and** `risk.malicious_probability` > `50`.

This validator raising is the actual enforcement mechanism — the exception propagates straight into `generate_final_assessment`'s `except Exception` block, which is what triggers the degraded fallback described in §2. There is no repair/retry step; a contradictory verdict simply becomes the "generation failed" fallback assessment. Comment in the code notes this targets llama3.2:3b's observed behavior of emitting `final_verdict="malicious"` alongside `malicious_probability=0`.

The 0-1 vs 0-100 rescue

`RiskAssessment._reject_0_to_1_scale` (`field_validator`, applied to `overall_risk_score`, `confidence_score`, `malicious_probability`): if `0 < value < 1`, **rescales** it (`value * 100`) instead of rejecting it. Comment cites the observed bug directly: `llama3.2:3b` returning `0.5` for "50% confidence" despite the field description.

4. Grounding / anti-hallucination mechanisms

Four independent, code-level (not prompt-level) grounding mechanisms exist. None of them ask the model to double-check itself — they all recompute membership against real, already-persisted data and discard or relabel whatever the model claimed that doesn't match.

`_ground_final_assessment(assessment, correlation)` — `service.py:122-150`

Runs after every successful `generate_final_assessment` call, before the `FinalAssessment` is returned:

1. Builds `real_provider_ids` = the union of provider IDs parsed out of `correlation.edges[].provenance` (comma-split strings) and the provider lists in `correlation.provider_agreement.values()`.
2. Filters `assessment.agreeing_providers` and `assessment.disagreeing_providers` to only IDs present in `real_provider_ids`. **Caveat:** if `real_provider_ids` is empty, the model's claimed list is passed through **unfiltered** (a ternary fallback in `_filter_providers`, not a stricter check).
3. Builds `grounded_technique_ids` from `correlation.edges` where `relationship == "uses_technique"`, taking `edge.target.split(":", 1)[1].upper()`.
4. For every entry in `assessment.mitre_mappings`, **overwrites** `mapping.grounded = (mapping.technique_id.upper() in grounded_technique_ids)` — regardless of what the model set. It does **not** remove ungrounded mappings, only relabels the boolean.

It does **not** touch `risk`, `executive_summary`, `supporting_evidence`, `detection_rules`, or `final_verdict` — grounding here is scoped strictly to provider-agreement lists and MITRE mapping labels.

`_strip_invalid_evidence_ids` — `analysis_service.py:55-71`

Used by the 10 analyst-facing "explain the evidence" functions (§5), which are grounded against a real `EvidenceItem` ledger rather than correlation edges:

- Recursively walks a validated Pydantic model. Any field **literally named** `evidence_ids` (the only name checked — `_EVIDENCE_ID_FIELDS = ("evidence_ids",)`) that is a list gets filtered in place (`setattr`, models aren't frozen) to only IDs present in `real_ids`.
- `real_ids` is built from the actual `EvidenceItem` ledger passed into the call (`{str(item.id) for item in evidence}`) — **never** from the AI's own output.
- Recurses into nested `BaseModel` fields and into `BaseModel` items inside any list field, so a nested `evidence_ids` field is also caught.

`_call_and_ground()` (`analysis_service.py:74-97`) wraps every one of these 10 functions: call the model → validate into the function's schema → run `_strip_invalid_evidence_ids` → on **any** exception, return the caller-specific `fallback` model instance for that function (e.g. `WhyMaliciousExplanation(verdict_restated=..., reasons=[], caveat=...)`).

`hunting_service.py` grounding — different mechanism, no shared helper

`generate_hunting_package()` (`hunting_service.py:44-80`) does **not** import or use `_strip_invalid_evidence_ids`. Instead it filters `package.expansion_targets` to only entries whose `related_ioc_value.lower()` is present in `real_values = {node.value.lower() for node in correlation.nodes}` — i.e. grounded against real correlation-graph node values, not an evidence ledger. `HuntingPackage.expansion_targets`'s schema description states these must be "drawn ONLY from the supplied correlation edges — never invented." On failure, returns an empty `HuntingPackage()`.

`generate_detection_rule()` (`hunting_service.py:83-113`) has **no grounding step at all** — it operates on a single seed IOC with no expansion targets to invent. On failure, returns a fallback `DetectionRuleDraft(title=f"Detection rule unavailable for {ioc_value}", rule="# Generation failed -- see platform logs.", severity="none")`.

`analysis_service.compare_iocs()` (lines 345-375) uses yet another bespoke check — `most_dangerous_ioc_value` is restricted to the set of IOC value strings actually supplied for comparison — since it operates on comparison row dicts, not an `EvidenceItem` ledger, so it doesn't go through `_call_and_ground` / `_strip_invalid_evidence_ids` either.

Schema design as a grounding tool

- **Literal /enum instead of free-text description=** for every field the frontend keys off exact string values (`final_verdict`, `reputation`, `threat_level`, `confidence`, `severity`, `detection_rule_format`). Two independent reasons this matters (per `schemas.py`'s module docstring): (1) backends that compile the schema into a decoding grammar (Ollama) only enforce real `enum` constraints, not description hints; (2) the frontend's badge-color logic is a hardcoded switch on exact lowercase values — an off-spec string like `"critical1"` doesn't error, it just silently falls through to a default/muted style.
- `inline_refs()` (`app/ai/schema_utils.py:9-25`) recursively replaces `{"$ref": "#/$defs/X"}` with the resolved definition and strips `$defs` keys. Used only by `ollama_client.py` and `gemini_client.py` — Anthropic and Bedrock resolve standard `$ref` / `$defs` natively via their tool-use APIs and don't need it.

5. Downstream AI-assisted analyst features

Beyond the two-call core pipeline, `analysis_service.py` exposes 10 grounded functions (all via `_get_ai_client()` + `_call_and_ground()` + `_strip_invalid_evidence_ids`, all consuming the real `EvidenceItem` ledger via `_evidence_block()`): `explain_why_malicious`, `explain_what_is_this`, `explain_disagreement`, `assess_false_positive`, `challenge_verdict`, `suggest_next_actions`, `identify_intelligence_gaps`, `explain_score`, `answer_copilot_question`, and `compare_iocs` (bespoke grounding, see §4).

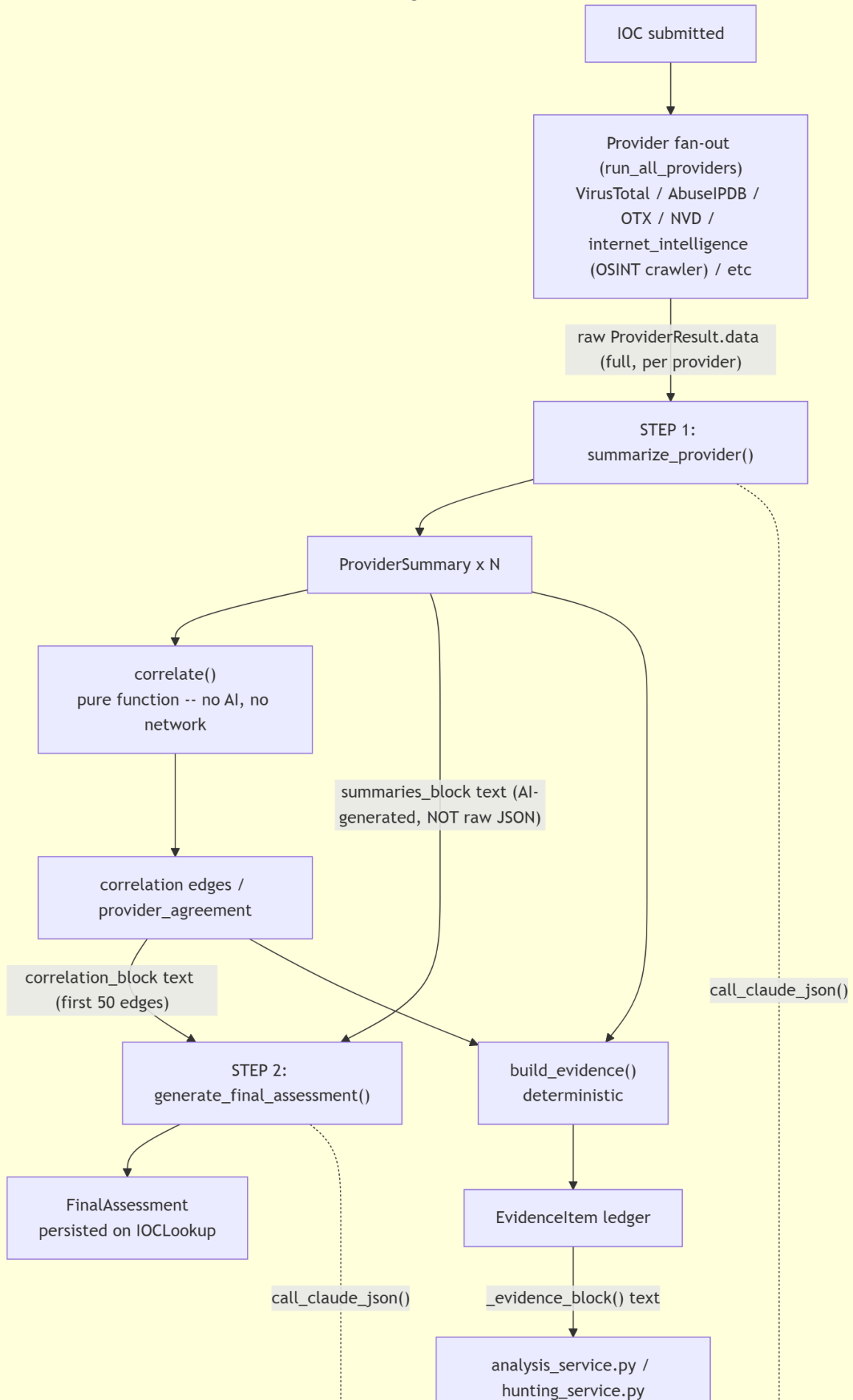
`hunting_service.py` exposes 2 functions: `generate_hunting_package` (hunt queries + correlation-grounded expansion targets, `max_tokens=8192`) and `generate_detection_rule` (single-IOC detection rule draft, `max_tokens=4096`, ungrounded).

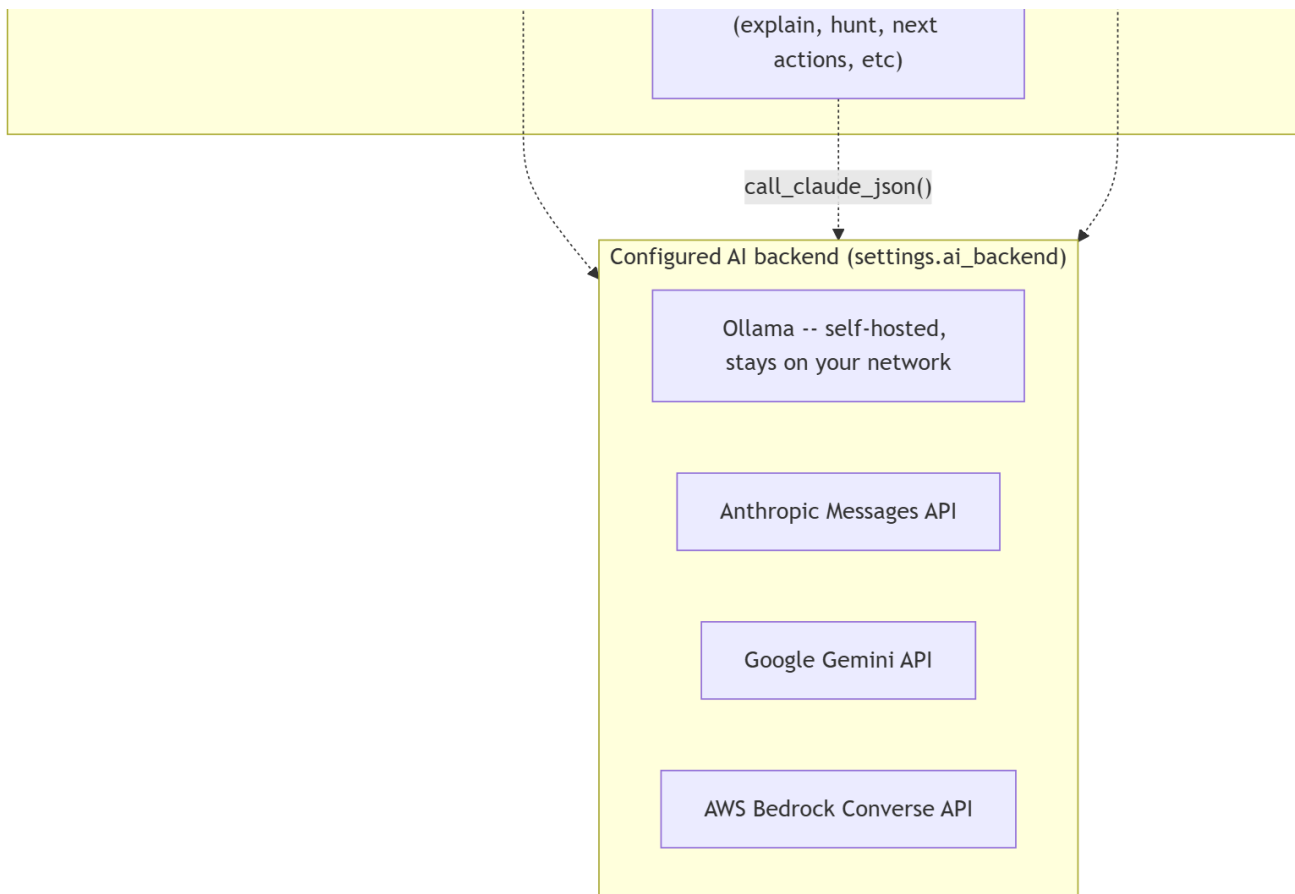
These are all invoked against **already-completed** lookups, rebuilding a `CorrelationResult` from persisted `CorrelationEdgeRecord` rows via `app/evidence/loaders.py`'s `correlation_from_records()` rather than re-running `correlate()` — used by the `analysis` and `hunting` route modules (`backend/app/api/routes/analysis.py`, `hunting.py`).

app/evidence/pivot.py's `rank_pivots()` (behind `GET /api/v1/lookup/{lookup_id}/pivots`, permission `lookup:read`) is worth noting by contrast: it is **deliberately not AI-generated** — a pure sort over real, persisted correlation edges touching the seed IOC, so a pivot suggestion can never be hallucinated. It ranks by corroborating-provider count then confidence, clamps the result count to `[1, 50]`.

6. What data actually leaves the platform (privacy / data-flow)

Your infrastructure -- Postgres / Redis / backend container





Concretely, per call:

- **STEP 1 (`summarize_provider`)** sends the **entire raw `result.data` dict** for one provider — every normalized field it populated (see [PROVIDERS.md](#)) plus any provider-specific extra fields — along with the IOC value/type, provider name/id/category/status, and `source_url`. For the `internet_intelligence` provider specifically, this includes **verbatim third-party text** scraped from GitHub, Reddit, RSS feeds, and Pastebin dumps (titles, snippets, URLs — see §7) that mention the IOC.
- **STEP 2 (`generate_final_assessment`)** sends only the **AI-generated provider summaries** (reputation/threat_level/confidence/findings text) and a **correlation edge digest** (up to 50 edges, plus `provider_agreement` / `deduplicated_facts`) — the raw per-provider JSON is *not* resent, bounding prompt size.
- **Downstream analyst features** (`analysis_service.py`, `hunting_service.py`) send formatted `EvidenceItem` claims/interpretations and, for hunting, correlation node values — derived data, not raw provider payloads.
- **If `ai_backend` is "ollama" (the default):** none of the above leaves the host machine — Ollama is reached over `OLLAMA_BASE_URL`, typically `http://host.docker.internal:11434`.
- **If `ai_backend` is "anthropic", "gemini", or "bedrock":** the IOC value, provider data (including any scraped OSINT text), and AI-generated summaries above are transmitted to that vendor's API over the public internet, subject to that vendor's own data-handling terms. See [SECURITY.md](#) for the platform's own security posture; the vendor's data handling for API traffic is outside this platform's control.

7. The OSINT crawler as an AI data source

`backend/app/crawler/collector.py` implements the `internet_intelligence` provider (`ProviderCategory.OSINT`, no key required, always `configured=True`) — its output flows through the exact same STEP 1/STEP 2 AI pipeline as every other provider, which is why it matters for the privacy picture in §6.

- **Supported IOC types:** domain, IPv4, malware family, threat actor, campaign, CVE, file name (`_SUPPORTED_TYPES`) — raw network atoms like JA3/mutex are excluded as free-text search noise.
- **Four source modules**, fanned out concurrently via `asyncio.gather` (exceptions captured, not raised): `github.py`, `reddit.py`, `rss_news.py`, `pastebin_search.py`.
- **Rate limiting is inconsistent by design**, not a bug in one module — only `github.py` and `reddit.py` use an in-process `AsyncMinIntervallimiter` (process-local, **not** Redis-backed, unlike `app/core/cache.py`'s rate limiter) and raise `SourceRateLimitedError` on HTTP 403/429 (GitHub) or 429 (Reddit). `rss_news.py` (8 fixed security-blog feeds) and `pastebin_search.py` (`psbtmp.ws`, unofficial/unauthenticated) have **no limiter at all** and never raise `SourceRateLimitedError` — any failure there is swallowed and returns an empty list.
- **Result status:** `collector.py` reports `RATE_LIMITED` only if merged findings are empty *and* at least one source was rate-limited (so it can in practice only ever be triggered by GitHub or Reddit); otherwise `OK` if any findings were found, else `NO_DATA`.
- **Findings are capped:** `crawler_max_results_per_source` (default 5) × 4 sources = 20 total findings max per IOC, deduped by URL.
- **Scheduled refresh:** Celery beat (`app/workers/celery_app.py`) runs `crawl-osint-sources-hourly` on a plain **3600-second interval** (not a cron expression — no `crontab(...)` usage exists anywhere in `celery_app.py`), which re-crawls up to 25 recently-looked-up IOCs (`_MAX_IOCS_PER_RUN`, from the last 24 hours,

`_LOOKBACK_HOURS`) and writes results into the same Redis cache the interactive lookup path reads.

Config gotcha: `GITHUB_TOKEN` (raises GitHub's unauthenticated 10 req/min ceiling to 30 req/min) is read via `os.getenv("GITHUB_TOKEN")` in `github.py`, but is **intentionally not a field in `Settings`** (`app/core/config.py`) — it only works if set directly in the process environment, not via the platform's normal `.env` mechanism used for every other provider key.

8. Configuration reference

Env var	Default	Used by
<code>AI_BACKEND</code>	<code>ollama</code>	<code>_get_ai_client()</code> — <code>ollama</code> , <code>anthropic</code> , <code>gemini</code> , or <code>bedrock</code>
<code>OLLAMA_BASE_URL</code>	<code>http://host.docker.internal:11434</code>	<code>ollama_client.py</code>
<code>OLLAMA_MODEL</code>	<code>llama3.2:3b</code>	<code>ollama_client.py</code>
<code>OLLAMA_MAX_TOKENS</code>	<code>8192</code>	<code>ollama_client.py</code>
<code>ANTHROPIC_API_KEY</code>	<i>(unset)</i>	<code>anthropic_client.py</code> — <code><your-key-here></code>
<code>ANTHROPIC_MODEL_ID</code>	<code>claude-sonnet-4-5-20250929</code>	<code>anthropic_client.py</code>
<code>ANTHROPIC_MAX_TOKENS</code>	<code>8192</code>	<code>anthropic_client.py</code>
<code>GEMINI_API_KEY</code>	<i>(unset)</i>	<code>gemini_client.py</code> — <code><your-key-here></code>
<code>GEMINI_MODEL_ID</code>	<code>gemini-2.0-flash</code>	<code>gemini_client.py</code>
<code>GEMINI_MAX_TOKENS</code>	<code>8192</code>	<code>gemini_client.py</code>
<code>BEDROCK_API_KEY</code>	<i>(unset)</i>	<code>bedrock_client.py</code> — bearer token, preferred over IAM keys; <code><configure securely></code>
<code>AWS_ACCESS_KEY_ID</code> / <code>AWS_SECRET_ACCESS_KEY</code>	<i>(unset)</i>	<code>bedrock_client.py</code> — alternative to <code>BEDROCK_API_KEY</code> ; <code><configure securely></code>
<code>AWS_REGION</code>	<code>us-east-1</code>	<code>bedrock_client.py</code>
<code>BEDROCK_MODEL_ID</code>	<code>global.anthropic.claude-sonnet-4-5-20250929-v1:0</code>	<code>bedrock_client.py</code>
<code>BEDROCK_MAX_TOKENS</code>	<code>4096</code>	<code>bedrock_client.py</code>
<code>GITHUB_TOKEN</code>	<i>(unset, process env only — not in <code>Settings</code>)</i>	<code>crawler/sources/github.py</code> , raises GitHub API rate limit from 10 to 30 req/min
<code>CRAWLER_USER_AGENT</code>	<code>IOC-Intel-Platform/1.0 (+https://github.com/your-org/ioc-intel-platform)</code>	all 4 crawler source modules
<code>CRAWLER_REQUEST_TIMEOUT_SECONDS</code>	<code>15</code>	crawler HTTP calls
<code>CRAWLER_MAX_RESULTS_PER_SOURCE</code>	<code>5</code>	<code>collector.py</code>

Full settings list: `backend/app/core/config.py`. See [CONFIGURATION.md](#) for the platform-wide env var reference and [DEPLOYMENT.md](#) for `docker-compose.yml` wiring.

9. Known gaps (explicitly NOT IMPLEMENTED)

- **Retry-with-validation-error-appended**, as claimed in `schemas.py`'s module docstring, does not exist in any AI service module. Every function falls back to a static object on the first failure. (§1)
- **No retry/backoff inside `ollama_client.py`, `anthropic_client.py`, or `gemini_client.py`'s `call_claude_json()`** — each makes exactly one HTTP attempt. Only Bedrock has any retry, and it's `botocore`'s transport-level retry, not application logic. (§1)
- **`rss_news.py` and `pastebin_search.py` have no rate limiter** and never raise `SourceRateLimitedError`, despite `collector.py`'s status logic being written generically for all 4 sources. (§7)
- **Celery beat's schedule is a plain interval (3600.0 seconds), not a cron expression** — there is no `crontab(...)` usage in `celery_app.py`. (§7)
- **`GITHUB_TOKEN` is not part of `Settings`** — cannot be configured via the platform's standard `.env` mechanism like every other API key; it must be present in the actual process environment. (§7)
- **`GraphNode.labels`** (`app/correlation/engine.py`) is defined but never populated anywhere in `correlate()` — always an empty list in practice. Not an AI-service field, but it is part of the correlation output the AI and evidence builder consume.
- **Neo4j mirroring of correlation edges**, described in code comments (`engine.py`'s module docstring, `CorrelationEdgeRecord`'s docstring) as intended, has **no actual implementation** — no Neo4j driver import, session, or write/query call exists anywhere in the codebase read for this document. `CorrelationEdgeRecord` rows in Postgres are the sole persisted source that `_ground_final_assessment`, `correlation_from_records()`, and every downstream analyst feature actually read.

Related documentation

- [ARCHITECTURE.md](#) — full SSE request lifecycle, plugin contract, datastore roles.
- [PROVIDERS.md](#) — connector contract and the normalized `ProviderResult.data` fields the AI and correlation engine depend on.
- [DATA_MODEL.md](#) — `AISummaryRecord`, `CorrelationEdgeRecord`, `EvidenceItem`, `IOClookup` table shapes.
- [CONFIGURATION.md](#) — full environment variable reference.
- [SECURITY.md](#) — platform security posture; vendor-side data handling for cloud AI backends is outside this platform's control.

Testing

Testing

This document describes the actual test suite in the repo: what exists, how to run it, and the Windows-specific gotchas you will hit if you run it locally rather than in CI/Docker.

There is **no** `pytest.ini`, `pyproject.toml`, `setup.cfg`, `tox.ini`, or `conftest.py` anywhere in the repo. Pytest runs with default discovery and configuration. There is also no frontend test config file (no `vitest.config.*` / `jest.config.*`).

Backend test suite

Tests live under `backend/app/tests/`, structured as a proper Python package (`app/tests/__init__.py`, `app/tests/unit/__init__.py`, `app/tests/integration/__init__.py`).

```
backend/app/tests/
├─ unit/           # 9 test files, no network/DB/Redis required
└─ integration/    # 3 test files, 2 of which need Docker services
```

Unit tests (`backend/app/tests/unit/`)

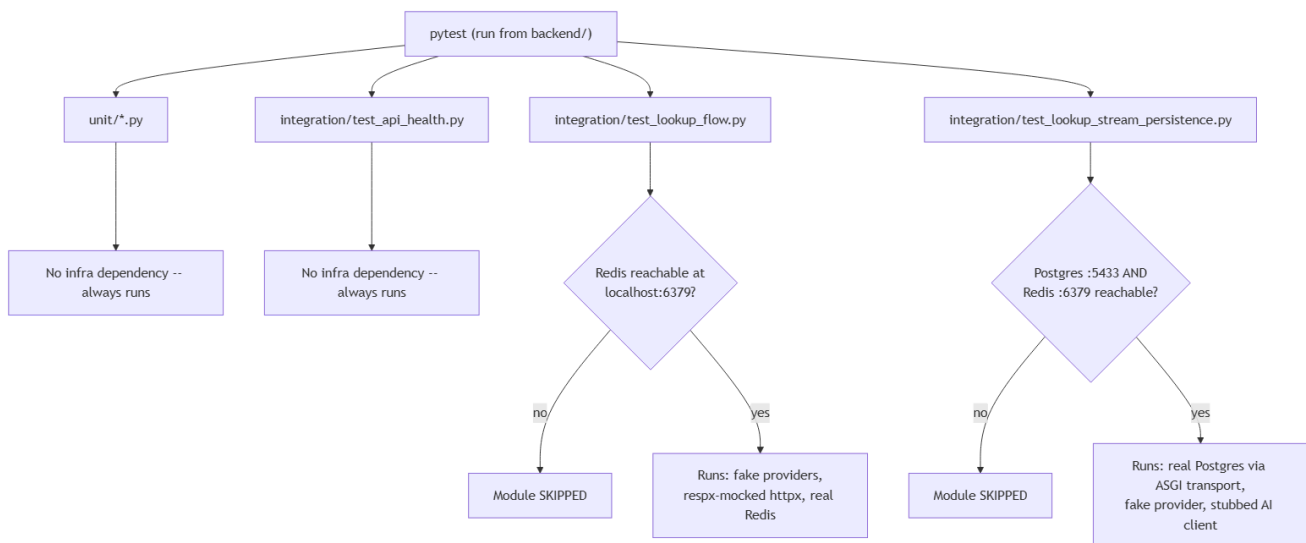
No database, Redis, or provider credentials required. Run unconditionally.

File	Covers
<code>test_abusech.py</code>	<code>app.providers.abusech.map_query_status</code> -- shared <code>abuse.ch</code> <code>query_status</code> mapping used by URLhaus/ThreatFox/MalwareBazaar; pins a fix where <code>auth/config</code> failures collapsed into <code>ProviderStatus.NO_DATA</code>
<code>test_ai_schemas.py</code>	<code>app.ai.schemas.MitreMapping</code> (<code>ATT&CK</code> tactic enum; <code>technique_id</code> is a plain <code>str</code> with no regex, because Ollama's grammar compiler can't compile that constraint) and <code>FinalAssessment</code> 's grounding validator
<code>test_analysis_service.py</code>	<code>app.ai.analysis_service._strip_invalid_evidence_ids</code> -- the guardrail that prevents the AI from citing <code>evidence_id</code> 's not present in the supplied evidence, using the real <code>WhyMaliciousExplanation</code> / <code>ReasonWithEvidence</code> schemas
<code>test_correlation_engine.py</code>	<code>app.correlation.engine.correlate</code> -- pure function building the correlation graph node/edge structure, edge dedup/merge, and provider-agreement bucketing from fake <code>ProviderResult</code> envelopes (no network I/O)
<code>test_crawler_collector.py</code>	<code>InternetIntelligenceCollector.fetch</code> (<code>app/crawler/collector.py</code>); pins a fix where rate-limited crawler sources were indistinguishable from "found nothing" (both previously collapsed to <code>NO_DATA</code>); exercises <code>app/crawler/sources/errors.py</code> 's <code>SourceRateLimitedError</code>
<code>test_crawler_rate_limit.py</code>	<code>app.crawler.sources.rate_limit.AsyncMinIntervallimiter</code> timing behavior
<code>test_evidence_builder.py</code>	<code>app.evidence.builder</code> (<code>build_evidence</code> , <code>build_evidence_from_correlation</code> , <code>build_evidence_from_providers</code>) -- the deterministic, non-AI evidence extraction that WHY/Challenge/Score Explanation/Copilot AI features must cite
<code>test_ioc_detector.py</code>	<code>app.ioc.detector.detect_ioc_type</code> contract across many IOC string patterns (IPs, hashes, CVEs, wallets, Windows file paths/registry keys, user agents, MITRE technique IDs, etc.)
<code>test_pivot.py</code>	<code>app.evidence.pivot.rank_pivots</code> -- the pure ranking logic behind <code>GET /lookup/{id}/pivots</code> and the "Recommended pivots" panel
<code>test_provider_base.py</code>	<code>app.providers.base.BaseProvider</code> contract (unsupported IOC types, missing credentials, error normalization) shared by every provider connector
<code>test_whois_rdap.py</code>	WHOIS half of <code>app/providers/whois_rdap.py</code> 's <code>WhoisRdapProvider</code> ; pins a fix where socket-level failures (timeout, connection refused) were mapped to <code>NO_DATA</code> identically to a genuine "no WHOIS record" response

Integration tests (`backend/app/tests/integration/`)

File	Covers	Infra required
<code>test_api_health.py</code>	Smoke test for the FastAPI app object: <code>GET /health</code> and <code>GET /docs</code> . Also proves the OpenAPI schema (including DB-backed lookup/auth/providers routers) serializes correctly at app-construction time	None
<code>test_lookup_flow.py</code>	<code>app.providers.orchestrator.run_all_providers</code> / <code>run_all_providers_collected</code> fan-out pipeline, using fake <code>BaseProvider</code> subclasses registered via the <code>providers=</code> override param (the real registry is never imported). <code>httpx</code> calls are mocked with <code>respx</code>	Redis (<code>localhost:6379</code>)
<code>test_lookup_stream_persistence.py</code>	Regression test for a bug where every lookup was stuck at <code>status=RUNNING</code> forever because the route's <code>Depends(get_db)</code> session was torn down before the lazy <code>event_stream()</code> generator ran. Fix: <code>app/api/routes/lookup.py</code> 's <code>event_stream()</code> now opens its own session via <code>app.core.db.new_session()</code> and re-fetches the <code>IOCLookup</code> row through it. Uses real Postgres over ASGI transport, a fake provider, and a stubbed AI client	Postgres (<code>localhost:5433</code>) + Redis (<code>localhost:6379</code>)

`test_lookup_flow.py` and `test_lookup_stream_persistence.py` each do a live TCP reachability probe at import time and use `pytest.mark.skipif` to skip the whole module (not fail) if the required service isn't up. There is no shared `conftest.py`, so each file reimplements very similar Postgres/Redis-override and connection-pool-disposal fixture logic independently.



There is no test for auth's HTTP endpoints

There is no dedicated unit or integration test that hits `POST /api/v1/auth/register` or `POST /api/v1/auth/login` directly. Auth is only exercised indirectly, by constructing `User` rows and tokens directly inside `test_lookup_stream_persistence.py`'s fixtures. **NOT IMPLEMENTED.**

Running the backend tests

Start the infra the integration tests need (skip this if you only want unit tests + the health-check integration test):

```
docker compose up -d postgres redis
```

Run the full suite from `backend/`:

```
cd backend
pytest
```

Or explicitly target a subdirectory:

```
cd backend
pytest app/tests/unit
pytest app/tests/integration
```

There is no pytest config file, so this is a bare default-discovery run -- no custom markers, no coverage gate, no `asyncio_mode=auto`. Async tests are marked explicitly with `@pytest.mark.asyncio` throughout (pytest-asyncio's default "strict" mode requires this on every async test function).

Test-tooling dependencies, pinned in `backend/requirements.txt`:

Package	Version
pytest	8.3.3
pytest-asyncio	0.24.0
pytest-cov	5.0.0
respx	0.21.1

`pytest-cov` is pinned in `requirements.txt`, but there is no documented `--cov` invocation anywhere in the repo (no CI config, no Makefile, no doc referencing it) -- coverage reporting is available but unused. **NOT IMPLEMENTED** as a documented workflow.

Frontend test suite

There is no frontend test suite. `frontend/package.json` defines `"test": "vitest run"` and lists `vitest@2.1.1` as a `devDependency`, but there are zero `*.test.*` / `*.spec.*` files anywhere under `frontend/app`, `frontend/components`, or `frontend/lib`, and no `vitest.config.*` exists. Running `npm test` invokes vitest against an empty test suite -- this is configured-but-unused test infrastructure, **NOT IMPLEMENTED** as an actual test suite.

Things to know (Windows-specific gotchas)

Two separate backend virtualenvs exist in this repo (`backend/.venv` and `backend/.venv_test`) with drifted dependency versions relative to what `backend/requirements.txt` pins. Which one is the "intended" environment for running tests is not documented anywhere in the repo; the drift itself is what causes gotcha 1 below.

Venv	pytest	pytest-asyncio	bcrypt	asyncpg
<code>backend/requirements.txt</code> (pinned)	8.3.3	0.24.0	4.0.1	0.29.0
<code>backend/.venv</code>	9.1.1	1.4.0	4.0.1	0.31.0
<code>backend/.venv_test</code>	8.3.3	0.24.0	5.0.0	0.30.0

Gotcha 1: bcrypt/passlib version mismatch breaks password hashing

`app/auth/security.py` builds its password context with `CryptContext(schemes=["bcrypt"], deprecated="auto")` via `passlib 1.7.4`. If your environment has `bcrypt>=4.1` (confirmed reproduced with `bcrypt==5.0.0` in `backend/.venv_test`), calling `ctx.hash()` fails:

- passlib's bcrypt handler reads `__bcrypt__.__about__.__version__` to detect the bcrypt version. `bcrypt>=4.1` no longer exposes `__about__`, so this raises `AttributeError: module 'bcrypt' has no attribute '__about__'` (passlib logs this as "(trapped) error reading bcrypt version" and falls through).
- passlib's subsequent internal wrap-bug self-test then hits bcrypt 5.x's stricter 72-byte check and fails with `ValueError: password cannot be longer than 72 bytes, truncate manually if necessary`.

This was reproduced by directly invoking passlib's `CryptContext.hash()` inside `.venv_test`'s Python -- it does **not** reproduce in `backend/.venv`, which pins `bcrypt==4.0.1`. If any test in your environment starts failing with either error above, check `pip show bcrypt` and pin it to `4.0.1` to match `requirements.txt`.

Gotcha 2: "Event loop is closed" from Postgres/Redis connection pools

Both `app.core.db`'s SQLAlchemy engine/pool and `app.core.cache`'s Redis pool are bound to whichever `asyncio` event loop was running when their connections were first opened. `pytest-asyncio` (in strict/function mode) gives each test function its own event loop, so a pool created in one test would otherwise be reused - against a now-closed loop -- by the next test, raising `RuntimeError: Event loop is closed` deep inside `asyncpg` or `redis-py`.

Both integration test files that touch real infra work around this explicitly:

- `test_lookup_flow.py`'s `clean_redis_cache` fixture clears `app.core.cache`'s pool so each test gets a fresh Redis client.
- `test_lookup_stream_persistence.py`'s `_dispose_pools_after_each_test` fixture (`autouse=True`) disposes both `app.core.db`'s engine and `app.core.cache`'s pool after every test.

If you write a new integration test against real Postgres/Redis without this pattern, expect intermittent `Event loop is closed` failures on the second test in a module, not the first.

Related docs

- [ARCHITECTURE.md](#) -- system design, including the orchestrator and SSE lookup flow exercised by the integration tests
- [API.md](#) -- REST/SSE endpoints referenced by `test_api_health.py` and `test_lookup_stream_persistence.py`
- [INSTALL.md](#) -- how to run `docker compose` locally (the same services the integration tests probe for)

Security

Security

This document describes the actual, implemented security model of the IOC Intelligence Platform: authentication, authorization, rate limiting, CORS, and secrets handling — plus a factual list of gaps an operator must address before running this in production. It is derived from the backend source (`backend/app/auth/` , `backend/app/models/user.py` , `backend/app/core/config.py` , `backend/app/main.py`) and the frontend token-handling code (`frontend/lib/api.ts` , `frontend/app/login/` , `frontend/app/register/`).

For endpoint-by-endpoint request/response shapes, see [API_DOCUMENTATION.md](#). For environment variable defaults and how `.env` is loaded, see [CONFIGURATION.md](#). For container/network layout, see [DEPLOYMENT.md](#).

1. Authentication (JWT)

Auth endpoints are mounted under `/api/v1/auth` (`backend/app/api/routes/auth.py` , registered in `backend/app/main.py`):

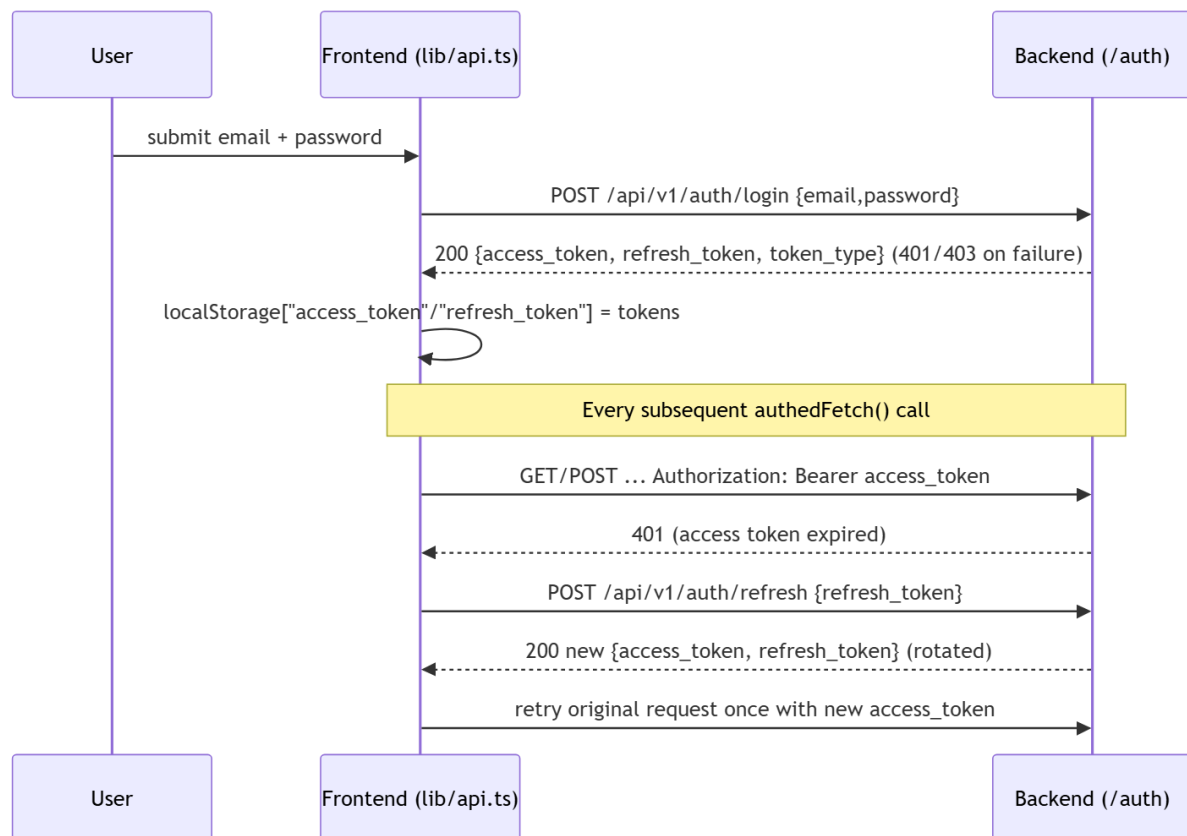
Method	Path	Auth required	Notes
POST	<code>/api/v1/auth/register</code>	No	First-ever user becomes <code>admin</code> ; all others become <code>analyst</code>
POST	<code>/api/v1/auth/login</code>	No	Returns access + refresh token pair
POST	<code>/api/v1/auth/refresh</code>	No (bearer refresh token in body)	Rotates both tokens
GET	<code>/api/v1/auth/me</code>	Yes	Returns current user; <code>full_name</code> is hardcoded to <code>" "</code> in the response, regardless of the stored value

Token mechanics

- Library: `python-jose[cryptography]` for signing/verification, `passlib + bcrypt` for password hashing (`CryptContext(schemes=["bcrypt"])`).
- Every token payload: `{"sub": <email>, "role": <role value>, "type": "access"|"refresh", "iat": <issued>, "exp": <expiry>}` .
- Signed with a single symmetric secret (`JWT_SECRET_KEY`) using `HS256` (default `jwt_algorithm`).
- `decode_token()` swallows all `JWTError` s (expired, bad signature, malformed) and returns `None` ; callers treat `None` as "unauthenticated" rather than raising directly.

Setting	Default	Meaning
<code>jwt_algorithm</code>	<code>HS256</code>	Signing algorithm
<code>access_token_expire_minutes</code>	<code>30</code>	Access token lifetime
<code>refresh_token_expire_days</code>	<code>7</code>	Refresh token lifetime

Login / refresh flow



Implementation details worth knowing:

- `require_permission()` failures and any 401 use `HTTPBearer(auto_error=False)`, chosen deliberately over `OAuth2PasswordBearer` because `/auth/login` takes a JSON body, not OAuth2 form-encoded credentials (see code comment in `backend/app/auth/rbac.py`).
- The frontend stores tokens in `localStorage` under the literal keys `access_token` and `refresh_token` — not cookies. This means tokens are reachable by any JS running on the page (standard XSS-exposure tradeoff of `localStorage` vs. `httpOnly` cookies).
- `authedFetch()` in `frontend/lib/api.ts` retries a request **exactly once** after a transparent refresh-on-401.
- The SSE lookup stream (`streamLookup()`) uses `fetch` with a manual stream reader instead of `EventSource`, specifically because `EventSource` cannot send an `Authorization` header; it has its own separate 401-retry-once logic.
- `isLoggedInIn()` on the frontend only checks whether `access_token` exists in `localStorage` — it does **not** validate signature or expiry client-side.

2. Role-Based Access Control (RBAC)

Three roles, defined in `backend/app/models/user.py`:

```

class Role(str, enum.Enum):
    ADMIN = "admin"
    ANALYST = "analyst"
    VIEWER = "viewer"
  
```

Every protected route depends on `require_permission("<permission-string>")` (`backend/app/auth/rbac.py`), which checks the current user's role against a static `ROLE_PERMISSIONS` dict and raises 403 with `detail="Role '<role>' lacks permission '<permission>'"` if absent.

Permission matrix (verbatim from `ROLE_PERMISSIONS`)

Permission	Admin	Analyst	Viewer
lookup:create	✓	✓	✗
lookup:read	✓	✓	✓
lookup:export	✓	✓	✗
provider:manage	✓	✗	✗
user:manage	✓	✗	✗
audit:read	✓	✗	✗
evidence:read	✓	✓	✓
analysis:generate	✓	✓	✗
hunting:generate	✓	✓	✗
copilot:query	✓	✓	✗
basket:manage	✓	✓	✗
case:create	✓	✓	✗
case:read	✓	✓	✓
case:write	✓	✓	✗
case:close	✓	✓	✗

Admin has all 15 permissions. Analyst has 12 (everything except `provider:manage`, `user:manage`, `audit:read`). Viewer has 3 (read-only: `lookup:read`, `evidence:read`, `case:read`).

NOT IMPLEMENTED: no route in `backend/app/api/routes/` was found calling `require_permission("provider:manage")`, `require_permission("user:manage")`, or `require_permission("audit:read")`. The permissions exist in the matrix and are enforceable by `require_permission()`, but there is currently no admin route (e.g. user management, audit log viewing, provider configuration) that actually consumes them — an admin-only permission with no gated route behind it is effectively unused today.

Route → permission reference

Route file	Permissions enforced
lookup.py	lookup:create, lookup:read (×2)
providers.py	lookup:read (on <code>/providers/health</code>)
analysis.py	evidence:read, analysis:generate (×7), copilot:query
hunting.py	hunting:generate (×2)
pivot.py	lookup:read
basket.py	basket:manage (×5)
cases.py	case:read (×2), case:create, case:write (×4), case:close
auth.py	none (register/login are unauthenticated; <code>/me</code> requires only a valid access token, no specific permission)

3. Rate Limiting

There is **one** rate limiter in the codebase, and it applies to **one** endpoint: `POST /api/v1/lookup/stream` (`backend/app/api/routes/lookup.py`).

- Implementation: fixed-window counter in Redis (`backend/app/core/cache.py`, `RateLimiter` class), keyed as `rate_limit:lookup_create:<user_id>` — scoped per authenticated user, enforced across all backend workers (not per-process).
- Rationale (from code comment): each lookup fans out to every provider plus the crawler plus multiple AI calls, so this bounds cost/load per user.
- Exceeding the limit returns `429` with a descriptive `detail` message.

Setting	Default	Meaning
lookup_rate_limit_max_calls	10	Max lookups per window
lookup_rate_limit_window_seconds	60	Window length, seconds

NOT IMPLEMENTED: no rate limiting or account-lockout exists on `/auth/login` or `/auth/register` — repeated failed login attempts are not throttled or locked out anywhere in the code.

4. CORS

Configured once, in `backend/app/main.py` :

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"] if settings.debug else [],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- When `DEBUG=true` (the default — see `debug: bool = True` in `backend/app/core/config.py`), the only allowed origin is `http://localhost:3000` (the local Next.js dev server).
- When `DEBUG=false`, `allow_origins` becomes an **empty list** — meaning no browser-based cross-origin requests are permitted at all until this is changed. There is no `CORS_ORIGINS` /allow-list environment variable; the production origin list is not externally configurable today and would require a code change to add real frontend origin(s).

5. Secrets Handling

Settings are loaded via `pydantic-settings` (`backend/app/core/config.py`), reading from a `.env` file (`env_file=".env"`) and/or process environment variables. `.env` is listed in `.gitignore` (`.env`, `.env.*`, with `!.env.example` explicitly un-ignored so the template stays tracked). Never commit a real `.env`; use `.env.example` as the template and fill in real values only in your local/deployed `.env`.

Env var **names** referencing credentials/secrets (values intentionally omitted — see `.env.example` for the full template):

Category	Env var names
JWT signing	<code>JWT_SECRET_KEY</code>
Database	<code>DATABASE_URL</code> (embeds Postgres credentials)
Graph DB	<code>NEO4J_USER</code> , <code>NEO4J_PASSWORD</code>
AI backend — Bedrock	<code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , <code>BEDROCK_API_KEY</code>
AI backend — Gemini	<code>GEMINI_API_KEY</code>
AI backend — Anthropic direct	<code>ANTHROPIC_API_KEY</code>
Threat-intel providers	<code>VIRUSTOTAL_API_KEY</code> , <code>ABUSEIPDB_API_KEY</code> , <code>OTX_API_KEY</code> , <code>NVD_API_KEY</code> , <code>ABUSECH_AUTH_KEY</code>
Stub/paid providers	<code>HYBRID_ANALYSIS_API_KEY</code> , <code>CENSYS_PERSONAL_ACCESS_TOKEN</code> , <code>CENSYS_ORGANIZATION_ID</code> , <code>PHISHTANK_API_KEY</code>

Use `<configure securely>` as a placeholder in any shared `.env` template or secrets manager entry — never write a real value into version control or documentation.

6. Known Limitations

These are factual gaps identified in the current source. They are framed as things to address before production use, not defects introduced carelessly — this is a threat-intel workbench, and the auth model is intentionally minimal.

- **Default JWT secret in code:** `jwt_secret_key` defaults to the literal string `"change-me-in-production"` (`backend/app/core/config.py`) if `JWT_SECRET_KEY` is not set. If this default ships to a real deployment, anyone can forge valid access/refresh tokens for any user/role. **Action:** always set a long, random `JWT_SECRET_KEY` in `.env` before any non-local deployment.
- **No refresh-token revocation:** `/auth/refresh` issues a new token pair on every call but there is no server-side token store or denylist. A refresh token remains valid and reusable until it naturally expires (`refresh_token_expire_days`, default 7 days) — client-side `logout()` only clears `localStorage` and does not invalidate the token server-side. There is no "log out everywhere" capability.
- **No login rate limiting / brute-force protection:** `/auth/login` and `/auth/register` have no per-IP or per-account throttling or lockout.
- **No backend password strength validation:** `RegisterRequest.password` is an unconstrained `str` server-side. The 8-character minimum enforced in the register page (`frontend/app/register/page.tsx`) is client-side only and can be bypassed by calling the API directly.
- **No MFA/2FA:** no TOTP, SMS, or other second factor exists anywhere in the backend or frontend.
- **No password reset / account recovery:** no forgot-password endpoint, reset-token field, or UI link exists. A locked-out user has no self-service recovery path.
- **No email verification:** the `User` model has no verified/unverified state; any syntactically valid email is accepted at registration.
- **Empty-list CORS in production mode:** with `DEBUG=false`, `allow_origins` is `[]` (no allowed origins at all) rather than a configurable production origin list — the frontend origin must be added directly in `backend/app/main.py` before deploying with `DEBUG=false`.
- **Admin-only permissions with no enforcing route:** `provider:manage`, `user:manage`, and `audit:read` are defined in the permission matrix but no route currently calls `require_permission()` with any of them — there is no user-management or audit-log endpoint yet (see Section 2).
- **GET `/auth/me` returns an empty `full_name`:** the endpoint hardcodes `full_name=""` in its response regardless of the stored value on the user record, so any client relying on `/me` for display-name purposes will see a blank string.

Related Documentation

- [API_DOCUMENTATION.md](#) — full endpoint reference
- [CONFIGURATION.md](#) — environment variables and defaults
- [DEPLOYMENT.md](#) — docker-compose / network topology
- [ADMIN_GUIDE.md](#) — operational guidance for administrators

Troubleshooting

Troubleshooting

Symptom -> Cause -> Fix, for failure modes confirmed in the actual source code and tests. If your symptom isn't here, check the relevant reference doc first: [PROVIDERS.md](#), [AI_ENGINE.md](#), [SECURITY.md](#), [TESTING.md](#), [CONFIGURATION.md](#).

1. AI assessment fails with "failed to parse grammar" / every `generate_final_assessment()` call breaks

Symptom

Final AI assessment (and/or provider summaries) come back as the degraded fallback object, or backend logs show:

```
Ollama invocation failed: HTTP 400: ... failed to parse grammar ...
```

Cause

This is a real, documented gotcha in `backend/app/ai/schemas.py` (comment at `schemas.py:62-71`). Ollama's structured-output mode (`format: on /api/chat`) compiles the *entire* JSON schema into one llama.cpp decoding grammar (`json-schema-to-grammar`, observed on server 0.32.6). If **any** field in that schema has a regex `pattern=` constraint, the whole grammar fails to compile and Ollama returns HTTP 400 "failed to parse grammar" — which breaks the call for *every* field, not just the one with the pattern.

This is why `MitreMapping.technique_id` (`backend/app/ai/schemas.py`) is deliberately typed as a plain `str` with no `pattern=`, even though it looks like it should be constrained to something like `^T\d{4}(\.\d{3})?%`. Malformed technique IDs aren't a safety issue here: `app/ai/service.py`'s `_ground_final_assessment()` only checks `technique_id` membership against real correlation-graph technique IDs and sets `grounded=False` on a mismatch — it doesn't care about ID format. `test_ai_schemas.py` (`backend/app/tests/unit/test_ai_schemas.py`) pins this contract.

Fix

- If you see this error, you (or a change you made) added a `pattern=` (or similar grammar-incompatible constraint) to a Pydantic field on a schema that gets sent to Ollama (`ProviderSummary`, `FinalAssessment`, or anything under `backend/app/ai/analysis_schemas.py`). Remove the regex constraint and rely on `Literal`/`enum` fields plus the description text instead — enums compile fine into llama.cpp grammars; free-form regex does not.
 - Cloud backends (Anthropic, Gemini, Bedrock) don't compile schemas into a decoding grammar, so a `pattern=` constraint would work fine there — it only breaks Ollama.
- There is **no retry** for this failure. Every AI client (`ollama_client.py`, `anthropic_client.py`, `gemini_client.py`) makes exactly one HTTP attempt and raises `RuntimeError` on failure; `bedrock_client.py` is the only backend with retries, and those are botocore-level (`BotoConfig(retries={"max_attempts": 3, "mode": "adaptive"})`), not application logic. On any exception, `generate_final_assessment()` / `summarize_provider()` (`backend/app/ai/service.py`) return a static degraded object instead of retrying — the lookup itself still completes.
- See [AI_ENGINE.md](#) for the full grounding/fallback pipeline.

2. Ollama unreachable / times out

Symptom

```
Could not reach Ollama at http://host.docker.internal:11434 -- is it running?
```

or

```
Ollama did not respond within 120s (model=llama3.2:3b) -- likely too slow for available hardware
```

Cause

`backend/app/ai/ollama_client.py` maps `httpx.ConnectError` and `httpx.TimeoutException` to these exact `RuntimeError` messages. `AI_BACKEND` defaults to `ollama` (`backend/app/core/config.py`), and `OLLAMA_BASE_URL` defaults to `http://host.docker.internal:11434` — Ollama is expected to run **on the host**, not inside a container. The 120-second timeout (`_TIMEOUT_SECONDS` in `ollama_client.py`) is sized for a model that fits fully in VRAM; a model that spills to CPU/mmap can legitimately take minutes and will trip this.

Fix

- Confirm Ollama is running on the host and reachable: `ollama list`, then from inside the backend container `curl http://host.docker.internal:11434`.
- Confirm `OLLAMA_MODEL` in `.env` matches a model you've actually pulled (`ollama pull llama3.2:3b`).
- If your hardware is slow, use a smaller model rather than raising the timeout — `_TIMEOUT_SECONDS` is a hardcoded constant in `ollama_client.py`, not a `.env` setting.

- Either way, the lookup does not fail outright: `summarize_provider()` returns a degraded `ProviderSummary` and `generate_final_assessment()` returns a degraded `FinalAssessment` (verdict `unknown`, all scores `0`) rather than raising to the caller.

3. A provider shows `not_configured`

Symptom

`GET /api/v1/providers/health` (or a provider card in the UI) shows `"configured": false` / status `not_configured` for a specific provider.

Cause

`BaseProvider.run()` (`backend/app/providers/base.py:112-123`) short-circuits to `ProviderStatus.NOT_CONFIGURED` whenever `requires_key=True` and the connector's `configured` flag is falsy. Each connector computes `configured` once, from `get_settings()`, at **module import time** (the module-level singletons in `backend/app/providers/registry.py` are constructed when the process starts) — so adding a key to a running container's `.env` will not retroactively flip this without a restart.

Common causes per provider (env var -> connector):

Provider	Required env var(s)	Notes
VirusTotal	<code>VIRUSTOTAL_API_KEY</code>	
AbuseIPDB	<code>ABUSEIPDB_API_KEY</code>	
OTX	<code>OTX_API_KEY</code>	
URLhaus / ThreatFox / MalwareBazaar	<code>ABUSECH_AUTH_KEY</code>	one shared key across all three abuse.ch connectors
Hybrid Analysis	<code>HYBRID_ANALYSIS_API_KEY</code>	
Censys	<code>CENSYS_PERSONAL_ACCESS_TOKEN</code> and <code>CENSYS_ORGANIZATION_ID</code>	both must be set — either one alone leaves <code>configured=False</code>

Providers that never show `not_configured` (no key required, `configured=True` unconditionally): crt.sh, NVD (key optional, only raises the rate limit), CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, PhishTank, and the internet-intelligence crawler.

Fix

- Set the correct env var(s) in `.env` (see table above and [PROVIDERS.md](#) for the full per-provider reference) and **restart the backend container** — `docker compose up -d --build backend` (or `docker compose restart backend` if only `.env` changed and no image rebuild is needed) — since `configured` is fixed at import time.
- `not_configured` is a normal, expected state for any provider you haven't set a key for — it does not fail the lookup or block other providers.

4. 429 Rate limit exceeded on `POST /api/v1/lookup/stream`

Symptom

```
{"detail": "Rate limit exceeded: max 10 lookups per 60s. Each lookup fans out to every provider plus the crawler and multiple AI calls, so this bound is tight."}
```

Cause

`backend/app/api/routes/lookup.py` enforces a Redis-backed fixed-window `RateLimiter` (`backend/app/core/cache.py`) keyed `lookup_create:{user.id}`, scoped per authenticated user and enforced across all backend workers (not per-process, since it's Redis-backed). This is the **only** rate limiter in the codebase — it applies to lookup *creation* only, not to individual providers, and not to any other endpoint (login/register have no rate limiting at all — see [SECURITY.md](#)).

Defaults (`backend/app/core/config.py`):

Setting	Default
<code>lookup_rate_limit_max_calls</code>	10
<code>lookup_rate_limit_window_seconds</code>	60

Fix

- Wait for the window to elapse (fixed window, so worst case up to 60s), or raise `LOOKUP_RATE_LIMIT_MAX_CALLS` / `LOOKUP_RATE_LIMIT_WINDOW_SECONDS` in `.env` and restart the backend.
- If you're load-testing or scripting bulk lookups, throttle client-side to stay under the configured window rather than raising the limit in production.

5. Lookup stuck at `status=RUNNING` forever

Historical bug, now fixed — included here because it's exactly the kind of symptom worth recognizing if you're on an older checkout or debugging something similar.

Symptom (historical)

A lookup's SSE stream would open, and the `IOCLookup` row in Postgres would remain at `status=RUNNING` indefinitely, even after all providers had finished.

Cause (historical, fixed)

The route's `Depends(get_db)` session was torn down (FastAPI closes the dependency's session when the request handler returns) before the lazy `event_stream()` async generator actually ran its body — so by the time the generator tried to update the lookup's final status, its DB session was already closed.

Fix (already applied)

`app/api/routes/lookup.py` 's `event_stream()` now opens its **own** session via `app.core.db.new_session()` and re-fetches the `IOCLookup` row through it, instead of relying on the route-level `Depends(get_db)` session. `test_lookup_stream_persistence.py` (`backend/app/tests/integration/test_lookup_stream_persistence.py`) is a regression test for exactly this bug — run it against real Postgres/Redis (see [TESTING.md](#)) if you suspect a regression:

```
cd backend
docker compose up -d postgres redis
pytest app/tests/integration/test_lookup_stream_persistence.py
```

If you see `status=RUNNING` stuck lookups on the current code, it is **not** this bug — check backend logs for an unhandled exception inside the streaming generator instead.

6. Local `pytest` run fails with a `bcrypt/passlib` error

Symptom

```
AttributeError: module 'bcrypt' has no attribute '__about__'
```

or

```
ValueError: password cannot be longer than 72 bytes, truncate manually if necessary
```

Cause

`backend/app/auth/security.py` builds `CryptContext(schemes=["bcrypt"], deprecated="auto")` via `passlib 1.7.4`. `Passlib`'s `bcrypt` handler reads `__bcrypt__.__about__.__version__` to detect the `bcrypt` version; `bcrypt>=4.1` (confirmed with `bcrypt==5.0.0`) no longer exposes `__about__`, so this raises `AttributeError` (`passlib` logs it as "(trapped) error reading bcrypt version" and falls through to an internal self-test that then hits `bcrypt 5.x`'s stricter 72-byte check, raising `ValueError`).

This repo has **two** backend venvs with drifted versions:

Venv	bcrypt	Fails?
<code>backend/.venv</code>	4.0.1 (matches <code>requirements.txt</code>)	No
<code>backend/.venv_test</code>	5.0.0	Yes, reproduced directly

Fix

```
pip show bcrypt
pip install "bcrypt==4.0.1"
```

Pin to `bcrypt==4.0.1` to match `backend/requirements.txt`. Confirm which venv your shell is actually activated into before debugging further — `python -c "import sys; print(sys.executable)"`. See [TESTING.md](#) for the full venv/version-drift table.

7. Local `pytest` run fails with `RuntimeError: Event loop is closed`

Symptom

Intermittent `RuntimeError: Event loop is closed` raised deep inside `asyncpg` or `redis-py`, typically on the second (or later) test in an integration test module, not the first.

Cause

`app.core.db` 's `engine/pool` and `app.core.cache` 's Redis pool are created lazily and bound to whichever `asyncio` event loop was running when their connections were first opened. `pytest-asyncio` (strict mode — no `asyncio_mode=auto` is configured anywhere, so every `async` test needs an explicit `@pytest.mark.asyncio` marker) gives each test function its own event loop, so a pool created in one test gets reused — against a now-closed loop — by the next test.

Fix

Already handled by fixtures local to each integration test file (there is no shared `conftest.py`):

- `test_lookup_flow.py` 's `clean_redis_cache` fixture clears the Redis client between tests.
- `test_lookup_stream_persistence.py` 's `_dispose_pools_after_each_test` (autouse) fixture disposes both `app.core.db` 's `_engine` and `app.core.cache` 's `_pool` after every test.

If you add a **new** integration test that touches Postgres or Redis directly, copy one of these fixtures rather than assuming a shared one exists — see [TESTING.md](#).

8. Integration tests silently skip

Symptom

`pytest app/tests/integration` reports tests as `SKIPPED`, not `PASSED` or `FAILED`.

Cause

`test_lookup_flow.py` and `test_lookup_stream_persistence.py` each do a live TCP reachability probe at import time and use `pytest.mark.skipif` to skip the whole module if required infrastructure isn't up:

Test file	Requires	Skip reason string
<code>test_lookup_flow.py</code>	Redis at <code>localhost:6379</code>	Redis not reachable at {REDIS_HOST}:{REDIS_PORT} -- run \ docker compose up -d redis` first.`
<code>test_lookup_stream_persistence.py</code>	Postgres at <code>localhost:5433</code> + Redis at <code>localhost:6379</code>	Postgres/Redis not reachable -- run \ docker compose up -d postgres redis` first.`

`test_api_health.py` and everything under `app/tests/unit/` have no infrastructure dependency and run unconditionally.

Fix

```
docker compose up -d postgres redis
cd backend
pytest app/tests
```

Note the Postgres host port is `5433` (docker-compose maps container `5432` -> host `5433`), not the default `5432`.

9. Crawler / OSINT source shows rate-limited instead of "no results"

Symptom

A lookup's internet-intelligence provider (`internet_intelligence`) result has `status=RATE_LIMITED` and `error_message` like `Rate-limited by: github, reddit`, instead of a clean empty result.

Cause

This is the intended, fixed behavior (`backend/app/crawler/collector.py:112-133`) — not a bug. Before this fix, a rate-limited source and a source that genuinely found nothing both collapsed into `ProviderStatus.NO_DATA`, which is misleading (`NO_DATA` is rendered as "nothing found" rather than "we couldn't check"). Now:

- If `merged` findings are non-empty -> `OK`.
- If `merged` is empty **and** at least one source raised `SourceRateLimitedError` -> `RATE_LIMITED`, with `error_message` listing which source(s).
- Otherwise -> `NO_DATA`.

Only `github.py` (403/429 -> `SourceRateLimitedError`) and `reddit.py` (429 -> `SourceRateLimitedError`) implement this; `rss_news.py` and `pastebin_search.py` have no rate limiter and never raise it, so in practice this status can only come from GitHub or Reddit throttling. `test_crawler_collector.py` (`backend/app/tests/unit/test_crawler_collector.py`) pins this contract.

Fix

- GitHub's unauthenticated search API allows ~10 req/min; the crawler self-throttles to one call every 6s via `AsyncMinIntervalLimiter` (`backend/app/crawler/sources/rate_limit.py`), but this is process-local, not Redis-backed, so multiple backend workers each keep their own clock.

- Setting a `GITHUB_TOKEN` environment variable (read via `os.getenv`, **not** a Settings field in `backend/app/core/config.py`, so it only works if present in the actual process environment, not via `.env`) raises the GitHub ceiling from 10 to 30 req/min.
- This status is expected to appear occasionally under normal use; it is not something to "fix" beyond waiting or adding a `GITHUB_TOKEN`.

10. WHOIS/RDAP provider returns `NO_DATA` on what looks like a network failure

Symptom

A domain lookup's `whois_rdap` provider result is `NO_DATA` when you'd expect `ERROR` or `TIMEOUT` (e.g., you know the WHOIS server was unreachable).

Cause (fixed, pinned by test)

`backend/app/providers/whois_rdap.py` runs the blocking `python-whois` library via `asyncio.to_thread` with `ignore_socket_errors=False` — deliberately overriding that library's own default of `True`. The library's default silently swallows socket-level failures (timeout, connection refused) into unparseable text that looks identical to a genuine "no WHOIS record" response. `test_whois_rdap.py` (`backend/app/tests/unit/test_whois_rdap.py`) pins this fix — if you see a real `NO_DATA` result for a domain you believe should have a WHOIS record and the `ignore_socket_errors` override has since been removed or altered, that's a regression of this fix, not a new issue.

Fix

No action needed for a correctly-behaving system — a true "no WHOIS record" result is a valid `NO_DATA`. If you suspect the override regressed, check `backend/app/providers/whois_rdap.py:28-36,62-94` for `ignore_socket_errors=False` and run `test_whois_rdap.py`.

11. Frontend `npm test` — nothing to run

Symptom

`vitest run` reports zero test files (or the command appears to do nothing useful).

Cause

NOT IMPLEMENTED: `frontend/package.json` wires up `"test": "vitest run"` and lists `vitest` as a devDependency, but there are zero `*.test.* / *.spec.*` files anywhere under `frontend/app`, `frontend/components`, or `frontend/lib`, and no `vitest.config.*` exists. This is configured-but-unused test infrastructure, not a broken test suite — there is nothing to fix, because there is nothing written yet.

Fix

Not applicable. If you're adding frontend tests for the first time, you'll also need to add a `vitest.config.*` — none exists to model against.

Quick reference: HTTP status -> cause

HTTP status	Where	Meaning
429 on <code>/api/v1/lookup/stream</code>	<code>lookup.py</code> <code>RateLimiter</code>	Per-user lookup creation limit (§4 above)
400 from Ollama (<code>/api/chat</code>)	<code>ollama_client.py</code>	Grammar compile failure — check for a <code>pattern=</code> constraint (§1 above)
429 / 403 / 509 from a provider's own API	<code>BaseProvider.run()</code>	Normalized to <code>ProviderStatus.RATE_LIMITED</code> (509 is PhishTank's documented over-limit code)
Any other non-2xx from a provider	<code>BaseProvider.run()</code>	Normalized to <code>ProviderStatus.ERROR</code>

See also

- [PROVIDERS.md](#) — per-provider auth, status mapping, and quirks
- [AI_ENGINE.md](#) — AI backend selection, grounding, and fallback behavior
- [SECURITY.md](#) — rate limiting, permissions, auth
- [TESTING.md](#) — full test suite layout and `venv` gotchas
- [CONFIGURATION.md](#) — every `.env` variable and its default

Changelog

Changelog

No git history was available to build this changelog from. The repository at `C:\Users\User\ioc-intel-platform` is not (or is no longer) tracked as a git repository with retrievable commit history, so nothing below is derived from commit logs, tags, or PRs. This is a **snapshot changelog**: a single "Unreleased" entry describing the feature set as it exists in the current source tree, compiled by reading the backend and frontend code directly. It will not show you *when* something was added or *why* it changed — only *what is present today*. Future entries should be appended above this one once real commit/release history exists.

For endpoint-level detail see [API_DOCUMENTATION.md](#); for schema detail see [DATA_MODEL.md](#); for the AI subsystem see [AI_ENGINE.md](#); for provider integrations see [PROVIDERS.md](#).

[Unreleased] — current snapshot

Snapshot date basis: the most recent Alembic migration in the tree (`a6d3ad2bb63c_add_evidence_basket_and_case_management_.py`) carries a create date of 2026-08-08, so the schema/feature set below reflects that point in the codebase's life.

Subsystems present in this snapshot

Subsystem	Status	Entry point(s)
Authentication & RBAC	Implemented	<code>POST /api/v1/auth/register</code> , <code>/login</code> , <code>/refresh</code> , <code>GET /me</code>
IOC lookup pipeline (SSE)	Implemented	<code>POST /api/v1/lookup/stream</code> , <code>GET /api/v1/lookup/{id}</code> , <code>GET /api/v1/lookup</code>
Provider health	Implemented	<code>GET /api/v1/providers/health</code>
Evidence ledger	Implemented	<code>GET /api/v1/lookup/{id}/analysis/evidence</code>
AI verdict analysis	Implemented	<code>POST /api/v1/lookup/{id}/analysis/*</code> (8 sub-routes)
Pivoting (deterministic)	Implemented	<code>GET /api/v1/lookup/{id}/pivots</code>
Hunting / detection content	Implemented	<code>POST /api/v1/lookup/{id}/hunt</code> , <code>/detection</code>
Basket (per-analyst scratch space)	Implemented	<code>GET/POST/DELETE /api/v1/basket</code> , <code>/basket/compare</code>
Case management	Implemented	<code>GET/POST/PATCH /api/v1/cases</code> , <code>notes</code> , <code>IOCs</code> , <code>close</code>
Server-side export (PDF/CSV)	NOT IMPLEMENTED	frontend calls a route that does not exist on the backend
Provider/user/audit admin API	NOT IMPLEMENTED	permission strings exist, no routes use them

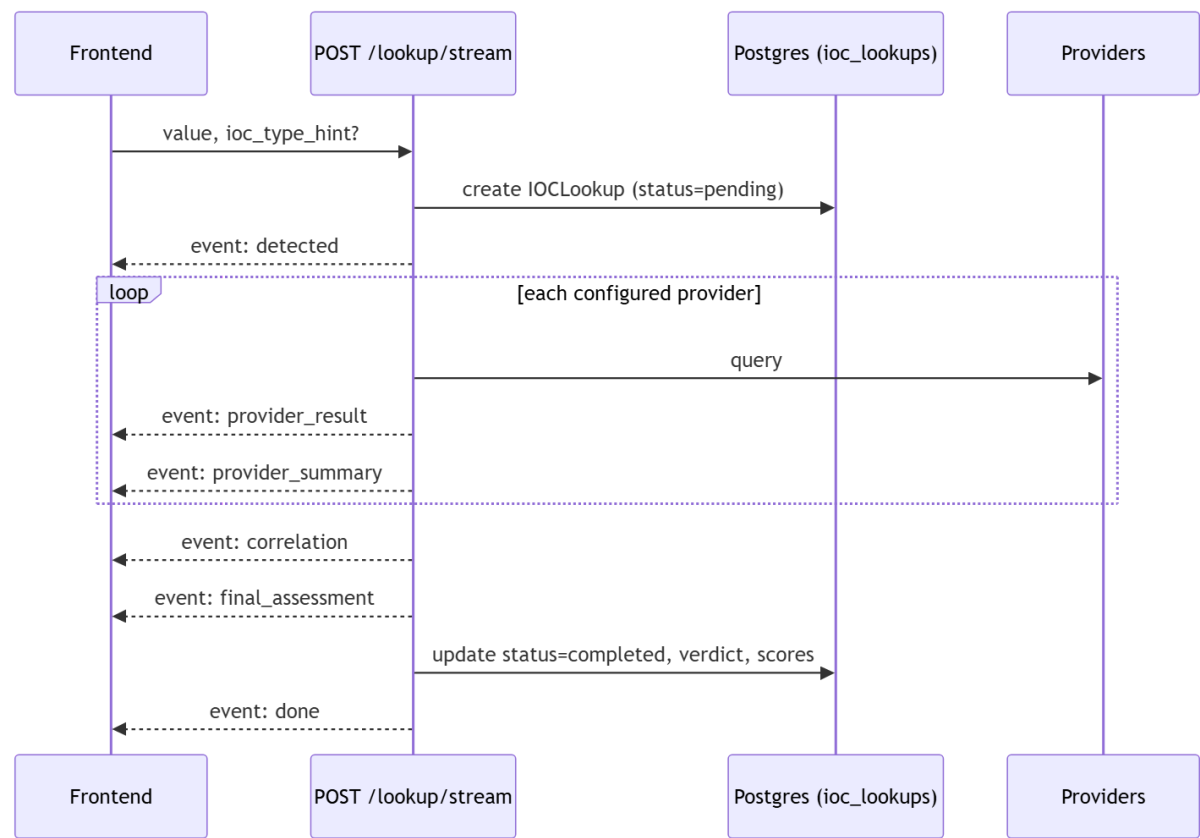
Added — Authentication & RBAC

- Self-service registration at `POST /api/v1/auth/register`. The **first user ever registered becomes admin**; every subsequent registration defaults to `analyst`. This is a deliberate bootstrap mechanism (no manual DB edit needed to get an initial admin) — see comment in `auth.py`.
- JSON login (`POST /api/v1/auth/login`) issuing a JWT access + refresh token pair (`TokenResponse`), and `POST /api/v1/auth/refresh` to rotate the access token from a refresh token.
- `GET /api/v1/auth/me` returns the current user, but note: it currently hardcodes `full_name` to an empty string in the response rather than returning the stored value.
- Three roles: `admin` , `analyst` , `viewer` , enforced via a permission-string RBAC layer (`require_permission("<resource>:<action>")`) rather than role-name checks directly on routes.
 - `viewer` is read-only: `lookup:read` , `evidence:read` , `case:read` .
 - `analyst` has all working operational permissions (create lookups, generate AI analysis, manage basket, manage cases, run hunts).
 - `admin` has everything `analyst` has, plus three permission strings — `provider:manage` , `user:manage` , `audit:read` — that are defined but **not wired to any route** (see Known Gaps below).
- Bearer-token auth via `HTTPEnder` (not OAuth2 password flow), matching the JSON-body login contract.

Added — IOC lookup pipeline

- `POST /api/v1/lookup/stream` accepts an IOC value (+ optional `ioc_type_hint`) and streams progress as **Server-Sent Events**: `detected` → `N × provider_result` → `N × provider_summary` → `correlation` → `final_assessment` → `done` (or `error`).
- Per-user rate limiting on lookup creation (default: 10 calls / 60s window, configurable — see [CONFIGURATION.md](#)).
- Returns `422` if the IOC type cannot be auto-detected and no `ioc_type_hint` was supplied.

- `GET /api/v1/lookup/{id}` returns the full record: provider results, AI summaries, correlation-derived assessment, verdict, risk/confidence scores.
- `GET /api/v1/lookup` lists recent lookups (default 50, capped at 200). Lookups are **shared across the whole SOC team** — any user with `lookup:read` sees everyone's lookups, unlike the basket (see below).



Added — Provider health

- `GET /api/v1/providers/health` reports live provider status from the in-process provider registry. No request body/params.

Added — Evidence ledger

- `GET /api/v1/lookup/{id}/analysis/evidence` returns a deterministic, **non-AI** list of evidence items (claim, interpretation, confidence, source, related IOC) built by `app/evidence/builder.py` from provider summaries and correlation edges. This powers the "Show Receipts" / Evidence tab in the UI and is explicitly the layer AI outputs are meant to be checkable against.
- Both this route and every AI analysis route below require the target lookup to exist (404 Lookup not found) **and** to have `status == completed` (409 Lookup is not completed yet).

Added — AI verdict analysis

Eight generation endpoints under `POST /api/v1/lookup/{id}/analysis/*`, each gated by permission `analysis:generate` (except `copilot`, gated by `copilot:query`):

Route	Response model
<code>/why</code>	<code>WhyMaliciousExplanation</code>
<code>/what-is-this</code>	<code>WhatIsThisIOC</code>
<code>/disagreement</code>	<code>DisagreementSummary</code>
<code>/false-positive</code>	<code>FalsePositiveAssessment</code>
<code>/challenge</code>	<code>ChallengeVerdict</code>
<code>/next-actions</code>	<code>SmartNextActions</code> (also reads correlation edges)
<code>/gaps</code>	<code>IntelligenceGaps</code> (splits providers into with/without data)
<code>/score-explanation</code>	<code>ScoreExplanation</code>
<code>/copilot</code>	<code>CopilotAnswer</code> — free-form <code>{ "question": str, "notes": [str]? }</code> , 422 if question is blank

Added — Pivoting

- `GET /api/v1/lookup/{id}/pivots` — a deliberately **non-AI**, deterministic ranking of related IOCs from stored correlation edges (`limit` query param, default 10). Positioned as the factual complement to the AI-generated `/next-actions` route.

Added — Hunting & detection content generation

- `POST /api/v1/lookup/{id}/hunt` generates a `HuntingPackage` across multiple formats via a `formats` query param, defaulting to all nine supported: `sigma`, `splunk_spl`, `sentinel_kql`, `elastic`, `qradar_aql`, `chronicle_yara_l`, `suricata`, `snort`, `zeek`.
- `POST /api/v1/lookup/{id}/detection` generates a single `DetectionRuleDraft` for one required `format` query param.
- Both require permission `hunting:generate` and only check that the lookup *exists* (404) — unlike the analysis routes, there is **no 409 check** for lookup completion status on these two routes.

Added — Basket (per-analyst scratch space)

- Private to each analyst: every route scopes to `owner_id == current user`. Contrast with cases and lookups, which are shared team-wide.
- `GET /api/v1/basket`, `POST /api/v1/basket` (idempotent — re-adding an existing (`owner`, `ioc_value`) pair returns the existing item, no 409), `DELETE /api/v1/basket/{item_id}`, `DELETE /api/v1/basket` (clear all).
- `POST /api/v1/basket/compare` compares 2–10 lookups at once (422 outside that range, or if fewer than 2 resolve to completed lookups), returning per-IOC rows plus an AI narrative.
- By design, `BasketItem` has no status/workflow field — that responsibility belongs to `Case`.

Added — Case management

- Cases are shared team-wide (any `case:read` holder sees all cases), mirroring lookup visibility, not basket visibility.
- `GET /api/v1/cases` (filterable by `status_filter`, limit capped at 200 — note: the filter is a raw string compare, not validated against the `CaseStatus` enum), `POST /api/v1/cases`, `GET /api/v1/cases/{id}`, `PATCH /api/v1/cases/{id}` (partial update via `exclude_unset`), `POST /api/v1/cases/{id}/close`.
- IOC attachment: `POST /api/v1/cases/{id}/iocs`, `DELETE /api/v1/cases/{id}/iocs/{ioc_id}`.
- Notes: `POST /api/v1/cases/{id}/notes`, with a deliberately loose `anchor_type` / `anchor_ref` string pair (not a foreign key) so a note can point at a graph node, timeline event, evidence item, IOC, or provider result without a dedicated join table per type.
- `CaseSeverity`: `low`, `medium`, `high`, `critical`. `CaseStatus`: `open`, `investigating`, `contained`, `resolved`, `false_positive`, `closed`.

Added — Data model & migrations

Two Alembic revisions make up the full schema history in this snapshot:

```
660d2aa3bc20  initial_schema
└─ users, ioc_lookups, provider_results, ai_summaries, correlation_edges

a6d3ad2bb63c  add_evidence_basket_and_case_management
└─ evidence_items, basket_items, cases, case_iocs, case_notes, case_reports
```

Run migrations with:

```
cd backend
alembic upgrade head
```

See [DATA_MODEL.md](#) for full table/column detail.

Added — Operational endpoints

- `GET /health` — plain health check, not under the `/api/v1` prefix.
- `GET /metrics` — Prometheus metrics via `Instrumentator`.
- `GET /api/v1/openapi.json`, interactive docs at `GET /docs`.

Known gaps in this snapshot

Documented explicitly per project convention — these are confirmed absent from the codebase, not omissions from this changelog:

- **NOT IMPLEMENTED: server-side export.** The frontend (`ExportMenu.tsx`) calls `POST /api/v1/lookup/{id}/export?format=pdf|csv`, but no backend route registers this path anywhere. The frontend already defensively handles the failure with "Export format not yet available." Only **client-side** JSON and Markdown export (built from data already loaded in the browser) currently work.
- **NOT IMPLEMENTED: admin/provider-management API.** The permission strings `provider:manage`, `user:manage`, and `audit:read` exist in the `admin` role's permission set, and `lookup:export` exists for `admin` / `analyst`, but none of these four permission strings are referenced by any route in the backend. There is currently no admin console API, no user-management API, and no audit-log API — only the permission scaffolding for them.

- **NOT IMPLEMENTED (by design): basket workflow state.** `BasketItem` intentionally has no status/workflow fields; use a `Case` if you need to track investigation state for an IOC.
- **NOT IMPLEMENTED: Neo4j graph mirror.** The `correlation_edges` table docstring states edges are "mirrored into Neo4j for graph traversal," but there is no Neo4j driver usage anywhere in the codebase. Postgres is currently the sole store for correlation/graph data.
- **NOT IMPLEMENTED: soft deletes.** No model has an `is_deleted` column; all deletes are hard deletes via SQLAlchemy `cascade="all, delete-orphan"`.

Template for future entries

Once real commit/release history is available, prefer this format going forward instead of a single snapshot block:

```
## [x.y.z] - YYYY-MM-DD

### Added
### Changed
### Fixed
### Removed
```